



웹 퍼블리셔를 위한 자바스크립트

코딩웍스 JAVASCRIPT 핵심 이론 가이드북(Guidebook)

JavaScript Contents



웹 퍼블리셔를 위한 코딩웍스 JAVASCRIPT 핵심 이론 가이드북(Guidebook)

01) 자바스크립트 기본 문법

- 자바스크립트 언어 연결
- 주석 달기
- 데이터 출력
- 데이터 유형
- 사칙 연산자와 비교 연산자

02) 자바스크립트 변수

- 변수 선언, 할당, 초기화, 브라우저 출력
- 변수 이름 규칙
- 변수에 담는 데이터 유형
- 변수의 재선언
- 변수의 사용 이유
- 변수와 함수를 이용한 프로그램 만들기
- 백틱(`)과 템플릿 문자열

03) 자바스크립트 조건문 - if문

- 기본 if else 조건문
- 다중 if 조건문
- 논리 연산자
- 중첩 if 조건문
- switch 문 / 삼항 연산자
- if와 '?'를 사용한 조건 처리

04) 반복문 - while문, do while문, for문

- 복합 대입 연산자
- 증감 연산자
- while 반복문
- do while 반복문
- for 반복문
- 중첩 for 반복문

05) 자바스크립트 배열과 for in 문

- 배열(Array)
- 배열과 for 문
- for in 문
- break 키워드
- continue 키워드

06) 자바스크립트 함수(function)

- 함수 선언과 호출
- 매개 변수와 리턴 키워드
- 변수식 함수 선언
- 내장 함수
- 즉시 실행 함수
- 화살표 함수

07) 자바스크립트 객체와 생성자함수

- 객체의 종류 / 객체의 선언과 호출
- 변수, 배열, 객체 개념 비교
- this 키워드
- 생성자 함수와 프로토타입(Prototype)

08) 자바스크립트 기본 내장 객체

- String 문자열
- Number 숫자 & Math 수학
- Array 배열의 속성과 메서드
- Date 날짜 / Date 객체의 Getter Setter 메서드

09) 자바스크립트 - 브라우저 객체(BOM)

- BOM(Browser Object Model)
- window 객체 속성과 메서드

10) 자바스크립트 - 문서 객체(DOM)

- DOM 문서 객체 가져오는 메서드
- DOM 문서 객체 준비 구문 메서드(onload)
- 문서 객체의 속성과 메서드
- 이벤트 객체의 속성과 메서드
- 마우스 & 키보드 이벤트 객체의 속성과 메서드
- 자바스크립트 주요 구문 무지성으로 연습 방법 with Code Runner

11) 자바스크립트 - HTML DOM Node

- HTML DOM Node 란?
- HTML DOM Node 간의 탐색

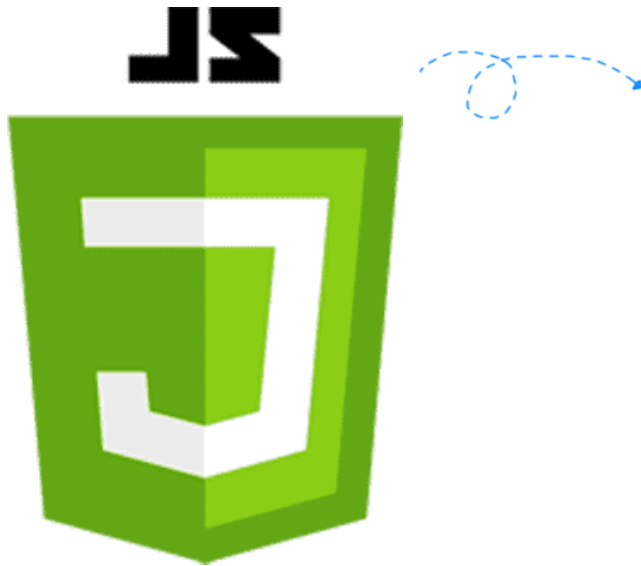
12) 자바스크립트 - 문서 객체 속성

- 문서 객체 - 폼(Form)
- 문서 객체 - 클래스 속성
- 문서 객체 - 영역 속성
- 문서 객체 - dataset 속성

13) 자바스크립트 - 실전예제

JAVASCRIPT 핵심이론 커리큘럼

[웹 퍼블리셔를 위한 자바스크립트] 01. 자바스크립트 기본 문법



1. 자바스크립트 언어 연결
2. 주석 달기
3. 데이터 출력
4. 데이터 유형
5. 사칙 연산자와 비교 연산자

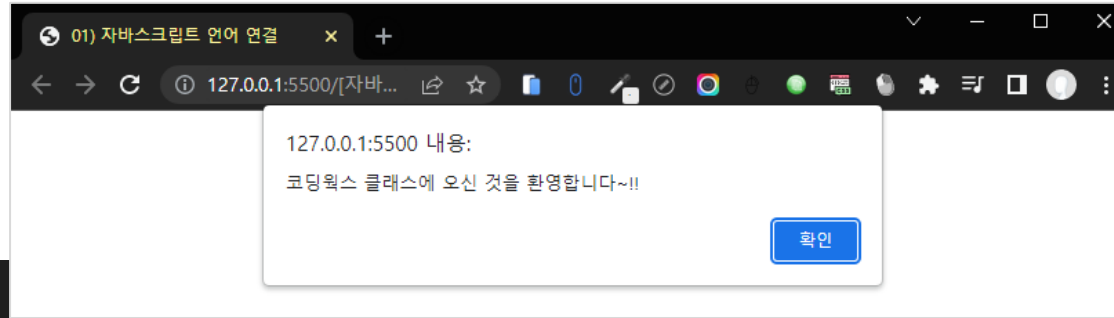
01) 자바스크립트 언어 연결 내부 자바스크립트와 외부 자바스크립트로 나뉩니다.

{ 내부 자바스크립트 }

- ① 내부 자바스크립트는 문서 내에 script 태그를 사용해서 head 내에 사용하거나 body 내에 사용
- ② 외부 자바스크립트는 JS 파일을 만들어서 링크하는 방식

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>01) 자바스크립트 언어 연결</title>
  <script>
    alert('코딩웍스 클래스에 오신 것을 환영합니다~!!')
  </script>
</head>
<body>
  <script>
    alert('코딩웍스 클래스에 오신 것을 환영합니다~!!')
  </script>
</body>
</html>
```

▲ 내부 자바스크립트



```
3 <head>
4   <meta charset="UTF-8">
5   <title>01) 자바스크립트 언어 연결</title>
6   <script src="main.js"></script>
7 </head>
8 <body>
9
10  <script src="main.js"></script>
11
12 </body>
13 </html>
```

```
JS main.js
[자바스크립트] 01. 자바스크립트 기본문법 > JS main.js
1 alert('코딩웍스 클래스에 오신 것을 환영합니다~!!')
2
```

▲ 외부 자바스크립트

02) 자바스크립트 주석 달기 ~ 주석 사용은 웹 퍼블리셔 및 개발자의 기본 중에 기본입니다.(실무에서는 한글 주석 사용 금지)

{ HTML+CSS 주석 사용 방식 }

- ① HTML에서 사용하는 주석 방식 : `<!-- 주석 내용 -->`
- ② CSS에서 사용하는 주석 방식 : `/* 주석 내용 */`

```
<style>
  section h1 {
    color: crimson;
    /* font-size: 20px; */
  }
</style>
```

{ JAVASCRIPT 주석 사용 방식 }

- ① 한 줄 주석 방식 : `// 주석 내용`
- ② 블록 주석 방식 : `/* 주석 내용 */`

- 자바스크립트에서 한 줄 주석은 슬래시(/)를 2개 사용합니다.
- 아래에 있는 자바스크립트에 대한 설명을 하는 용도 또는 주석 처리 하고 싶은 줄을 줄 단위로 주석할 경우 사용합니다.

- 자바스크립트에서 블록 주석은 CSS에서 주석 방식과 동일합니다.
- 2줄 이상의 코드를 블록 단위로 주석 처리할 경우 사용합니다.

```
<section>
  <h1>코딩웍스 클래스</h1>
  <!-- <h2>A부터 Z까지, 웹 퍼블리셔 취업을 위한 코딩웍스 유튜브 퍼블리싱 채널</h2> -->
  <p>
    코딩웍스 퍼블리싱 강의가 여러분의 HTML+CSS+JQUERY 퍼블리싱 실력을 높여서 퍼블리셔
    취업이라는 목표를 꼭! 이루실 수 있도록 응원하겠습니다. 화이팅~!!
  </p>
</section>

<script>
  // alert('코딩웍스 클래스입니다.')
  alert('홍길동 님 환영합니다~!!')

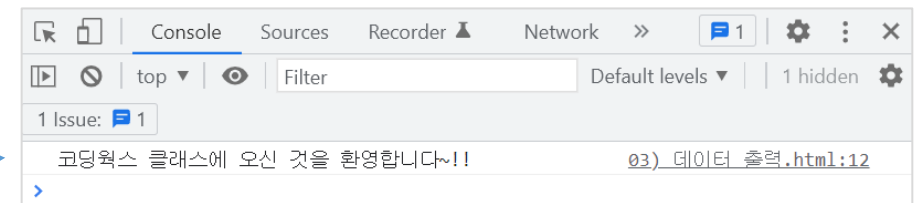
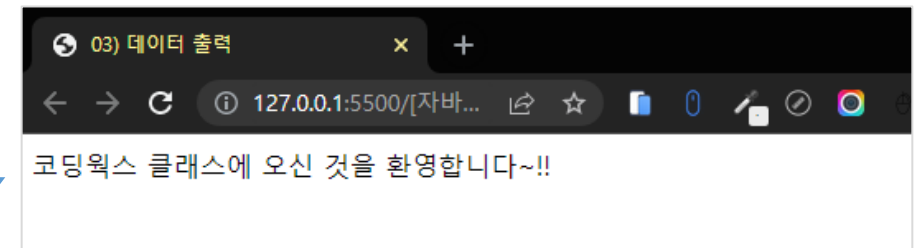
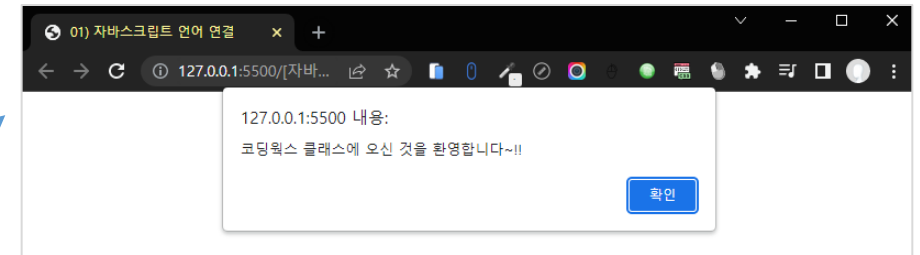
  // 3초마다 텍스트 출력하는 함수
  /* setInterval(function() {
    alert("Hello~ World!");
  }, 3000); */
</script>
```

03) 데이터 출력 HTML+CSS+JS에서 '출력'이란 브라우저에 보여지는 것을 말함

{ 자바스크립트에서 데이터 출력 }

- ① alert(출력할 내용);
 - 브라우저에 경고창으로 출력
 - 자바스크립트 기초 학습에서 사용
- ② document.write(출력할 내용);
 - 브라우저의 body 내에 출력
 - 대부분의 실무 작업에서 브라우저 출력이 목적이므로 대표적으로 사용됨
- ③ console.log(출력할 내용);
 - 코드 오류 및 실행 여부를 체크하기 위해서 콘솔창에 출력
 - 자바스크립트 기초 학습에서 사용 또는 실무에서 주로 사용됨

```
<script>
  alert('코딩웍스 클래스에 오신 것을 환영합니다~!!') ①
  document.write('코딩웍스 클래스에 오신 것을 환영합니다~!!') ②
  console.log('코딩웍스 클래스에 오신 것을 환영합니다~!!') ③
</script>
```



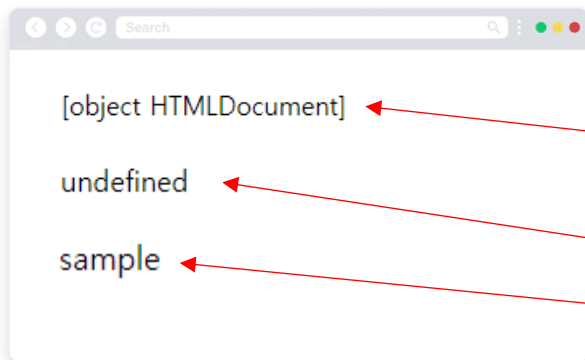
▲ 브라우저에서 F12를 누르면 콘솔창이 나옴

04) 데이터 유형

숫자, 문자열, 불(Boolean), 함수, 객체, undefined

{ 자바스크립트에서 데이터 유형 }

- ① 숫자 : 일반적인 숫자 표현
- ② 문자열 : 쌍 따옴표 또는 외 따옴표로 작성하는 일반적인 문자 또는 문자열
- ③ 불(Boolean) : 참(true)과 거짓(false)로만 표현되는 값
- ④ 함수(function) : 자바스크립트 코드의 집합
- ⑤ 객체 : 자바스크립트가 제어 가능한 모든 요소
- ⑥ undefined : 변수에 값이 할당 되지 않은 경우 출력되는 값



▲ 브라우저 출력 결과

```
<script>
// 숫자 : 일반적인 숫자 표현
document.write(77);
document.write('77');

// 문자열 : 쌍 따옴표 또는 외 따옴표로 작성하는 일반적인 문자 또는 문자열
document.write('안녕하세요.');
```

숫자 데이터는 따옴표로 묶으면 안됩니다. 만약 따옴표로 묶으면 브라우저에서 보여지는 건 77이지만 숫자 데이터가 아니라 77이라는 문자열 데이터가 됩니다.

```
document.write('<br>');
document.write('코딩웍스 클래스에 오신 것을 환영합니다~!!');
```

문자열은 쌍 따옴표 또는 외 따옴표로 묶습니다.
HTML 태그 사용 가능
쌍 따옴표와 외 따옴표를 동시에 쓸 경우 반드시 구분해서 사용해야 합니다.

```
document.write('<br>');
document.write('<a href="http://www.infllearn.com">인프런 바로가기</a>');
```

※ 따옴표 오류 : document.write(' ... ');

```
// 불(Boolean) : 참(true)과 거짓(false)로만 표현되는 값
document.write(77 > 20);
document.write(77 < 20);
document.write(true);
document.write(false);
document.write('true');
document.write('false');
```

true, false는 키워드(예약어)이기 때문에 따옴표 없이 사용해야 합니다.

```
// 함수(function) : 자바스크립트 코드의 집합
alert('안녕하세요.');
```

객체 : 자바스크립트가 제어 가능한 모든 요소

```
prompt('나이를 입력해주세요.', '숫자로만 입력');
```

```
document.write(document);
```

```
// undefined : 변수에 값이 할당 되지 않은 경우 출력되는 값
var sample;
document.write(sample);
document.write('sample');
```

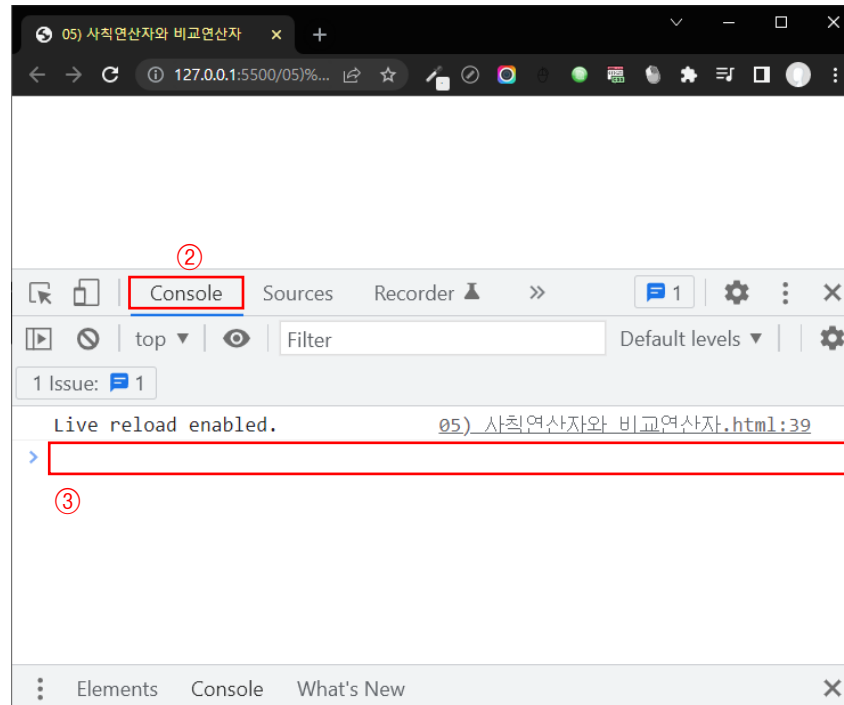
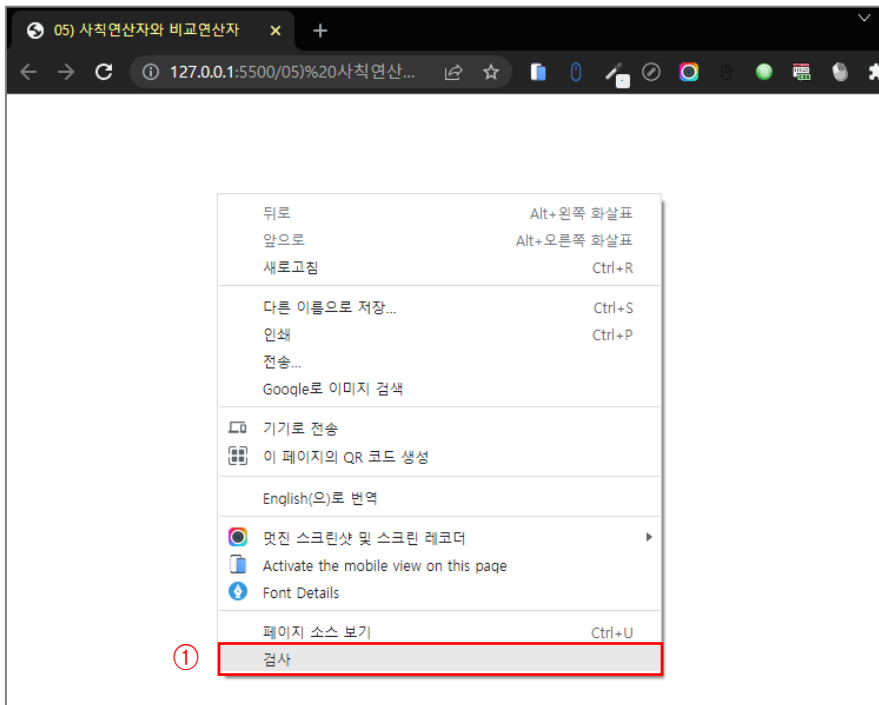
변수를 선언하면 값을 주지 않으면 오류(undefined)가 납니다.
변수를 따옴표 없이 사용해야 합니다. 따옴표를 사용하면 변수가 그냥 문자로 인식됩니다.

```
</script>
```

05) 사칙 연산자와 비교 연산자 - 사칙 연산자(1)

더하고(+) 빼고(-) 곱하고(*) 나누고(/) 나머지(%) 계산

{ 콘솔 로그(console.log) 사용하기 }



- ① 브라우저에서 마우스 우측 버튼 클릭 후
검사 클릭 또는 F12 클릭
- ② Console 탭 클릭
- ③ 자바스크립트 테스트 코드 입력 후 결과
확인(콘솔 로그에서 확인하는 결과는 실
제 브라우저에 반영되지 않습니다.)

05) 사칙 연산자와 비교 연산자 - 사칙 연산자(2)

```
<script>
// 더하기
console.log(77 + 22);
// 빼기
console.log(77 - 22);
// 곱하기
console.log(77 * 2);
// 나누기
console.log(77 / 7);
// 나머지
console.log(77 % 7);
console.log(77 % 5);
</script>
```

- 사칙 연산자는 좌변을 우변을 더하거나 빼거나 곱하거나 나눕니다.
- 나머지(%)는 좌변을 우변으로 나누고 남는 값을 출력합니다.
- 77을 7로 나누면 나머지는 0이 됩니다.
- 77을 5로 나누면 나머지는 2가 됩니다.

```
<script>
// 숫자와 문자를 사칙 연산하기
console.log(77 - '22');
console.log(77 * 2);
console.log(77 / 7);
console.log(77 + '22');

// 더하기 사칙 연산자 활용 예시
console.log('코딩웍스' + '님 결제 금액은 ' + 10000 + '원 입니다. ');
</script>
```

- '22' 라는 문자열로 숫자를 빼고 곱하고 나뉘도 정상적인 사칙 연산 가능
- 더하기의 경우 77이라는 숫자에 22라는 문자를 더하기 때문에 결과는 7722라는 문자열이 됩니다.
- 이유는 77이라는 숫자에서 22라는 문자를 더할 수 없기 때문입니다.

일반적으로 코딩웍스와 10000 이라는 곳에 변수를 선언하고 사용자에게 따른 이름과 가격을 변화시켜 브라우저에 출력합니다.

Console	Sources	Recorder	Network	1	Settings	Close
top	Filter	Default levels	1 Issue: 1			
99	05) 사칙연산자와 비교연산자.html:11					
55	05) 사칙연산자와 비교연산자.html:13					
154	05) 사칙연산자와 비교연산자.html:15					
11	05) 사칙연산자와 비교연산자.html:17					
0	05) 사칙연산자와 비교연산자.html:19					
2	05) 사칙연산자와 비교연산자.html:20					

Console	Sources	Recorder	Network	1	Settings	Close
top	Filter	Default levels	1 Issue: 1			
55	05) 사칙연산자와 비교연산자.html:25					
154	05) 사칙연산자와 비교연산자.html:26					
11	05) 사칙연산자와 비교연산자.html:27					
7722	05) 사칙연산자와 비교연산자.html:28					
코딩웍스님 결제 금액은 10000원 입니다.	05) 사칙연산자와 비교연산자.html:31					

05) 사칙 연산자와 비교 연산자 - 비교 연산자

- 좌변과 우변을 비교해서 결과 값을 참(true)과 거짓(false)로 출력
- $a = b$ 는 a와 b가 같다가 아니라 a에 b를 대입시킨다는 말입니다.
- 이퀄(=)은 항상 우측에 있고 달라지는 $> < !$ 와 같은 것은 항상 좌측에 있음

{ 비교 연산자 종류(1) }

- ① $>$: 좌변이 우변보다 크다
- ② $<$: 좌변이 우변보다 작다
- ③ $>=$: 좌변이 우변보다 크거나 같다(이상)
- ④ $<=$: 좌변이 우변보다 작거나 같다(이하)
- ⑤ $==$: 좌변과 우변이 같다
- ⑥ $!=$: 좌변과 우변이 같지 않다

```
<script>
// > : 좌변이 우변보다 크다
console.log(77 > 7);
// < : 좌변이 우변보다 작다
console.log(77 < 7);
// >= : 좌변이 우변보다 크거나 같다(이상)
console.log(77 >= 77);
console.log(77 >= 7);
// <= : 좌변이 우변보다 작거나 같다(이하)
console.log(77 <= 77);
console.log(77 <= 7);
// == : 좌변과 우변이 같다
console.log(77 == 77);
// != : 좌변과 우변이 같지 않다
console.log(77 != 55);
</script>
```

→ 이퀄(=)은 항상 뒤에 있고 달라지는 것은 항상 앞에 있음
 $>= <= == !=$

Console	Sources	Recorder	1
top	Filter	Default levels	
1 Issue: 1			
true	05) 사칙연산자와 비교연산자-02.html:11		
false	05) 사칙연산자와 비교연산자-02.html:13		
true	05) 사칙연산자와 비교연산자-02.html:15		
true	05) 사칙연산자와 비교연산자-02.html:16		
true	05) 사칙연산자와 비교연산자-02.html:18		
false	05) 사칙연산자와 비교연산자-02.html:19		
true	05) 사칙연산자와 비교연산자-02.html:21		
true	05) 사칙연산자와 비교연산자-02.html:23		

{ 비교 연산자 종류(2) }

- ① $=$: 대입 연산자(우변을 좌변에 넣는다)
- ② $==$: 항등 연산자(좌변과 우변이 같다)
- ③ $===$: 완전 항등 연산자
좌변과 우변이 데이터 유형까지 정확히 일치

```
<script>
// = : 대입 연산자(우변을 좌변에 넣는다)
var sample = 77;
console.log(sample);
// == : 항등 연산자(좌변과 우변이 같다)
console.log(77 == 77);
console.log(77 == '77');
// === : 완전 항등 연산자
console.log(77 === 77);
console.log(77 === '77');
</script>
```

→ 데이터 유형은 체크하지 않기 때문에 모두 true 결과 값 출력

→ 데이터 유형은 체크하기 때문에 77이라는 숫자와 77이라는 문자는 같지 않기 때문에 false 출력

Console	Sources	Recorder	1
top	Filter	Default levels	
1 Issue: 1			
77	05) 사칙연산자와 비교연산자-02.html:29		
true	05) 사칙연산자와 비교연산자-02.html:31		
true	05) 사칙연산자와 비교연산자-02.html:32		
true	05) 사칙연산자와 비교연산자-02.html:34		
false	05) 사칙연산자와 비교연산자-02.html:35		

※ 표현식인 세미콜론(;)은 반드시 사용해야 하는 것은 아닙니다. C와 Java는 꼭 써야 하지만 자바스크립트는 엔터를 치면 해당 라인의 구문이 종료되었다고 여깁니다. 그래도 세미콜론(;)을 사용하는 것을 추천 드립니다.

Jump Up) 데이터 출력 alert() prompt() confirm() 앞으로 이론과 예제를 만들 때 매우 자주 사용됩니다.

alert(표시할 내용); → 알려줌

```
// alert() : 알려줌  
alert('비밀번호가 틀립니다.');
```

127.0.0.1:5500 내용:
비밀번호가 틀립니다.

확인

confirm(표시할 내용); → 확인 받음

```
// confirm() : 확인 받음  
confirm('구독을 취소하시겠습니까?');
```

127.0.0.1:5500 내용:
구독을 취소하시겠습니까?

확인 취소

prompt('인자1', '인자2'); → 입력 받음

▲ 인자1은 입력창 위의 안내 문구, 인자2는 입력창의 기본값

```
// prompt() : 입력 받음  
var name = prompt('이름을 입력하세요.');
```

```
alert('환영합니다.' + name + '님');
```

127.0.0.1:5500 내용:
이름을 입력하세요.

코딩웍스

확인 취소

127.0.0.1:5500 내용:
환영합니다.코딩웍스님

확인

127.0.0.1:5500 내용:
상담 날짜를 입력하세요.

2023-02-

확인

취소

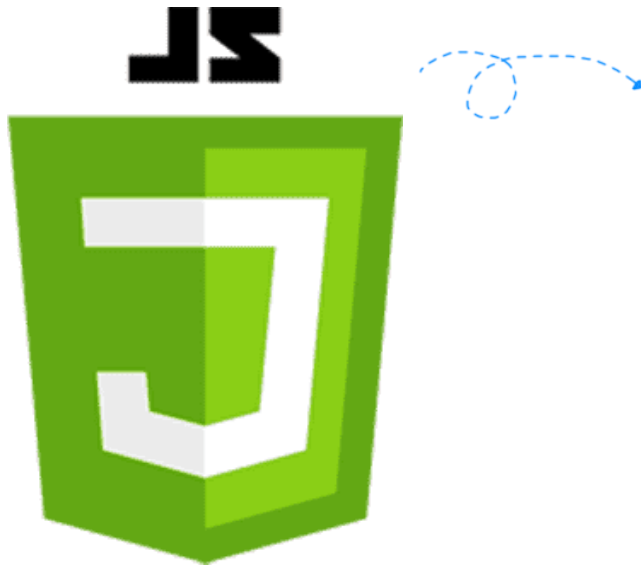
127.0.0.1:5500 내용:
상담 날짜는 2023-02-23입니다.

확인

```
// prompt(인자1, 인자2)  
var bookDate = prompt('상담 날짜를 입력하세요.', '2023-02-');  
alert('상담 날짜는 ' + bookDate + '입니다.');
```

JAVASCRIPT 핵심이론 커리큘럼

[웹 퍼블리셔를 위한 자바스크립트] 02. 자바스크립트 변수



1. 변수 선언, 할당, 초기화, 브라우저 출력
2. 변수 이름 규칙
3. 변수에 담는 데이터 유형
4. 변수의 재 선언
5. 변수의 사용 이유
6. 변수와 함수를 이용해서 프로그램 만들기(평균 구하기)
7. ES2015 백틱(`)과 템플릿 문자열

1) 자바스크립트 변수 - 변수 선언, 변수 값 할당, 변수 초기화, 브라우저 출력, 다중 변수 선언

↪ 변할 수도 있는 값을 변수라 부르고, 변할 수 없는 값을 상수라고 부릅니다.

{ 자바스크립트 변수 선언 방식 }

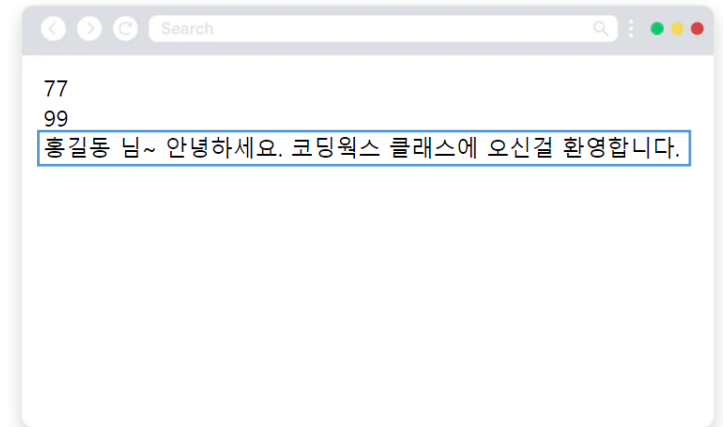
- ① var(variables)
- ② let(let)
- ③ const(constant)

- 자바스크립트 학습에서는 일단 var 또는 let을 사용합니다.
- let과 const는 ES6부터 생긴 변수 선언 방식입니다.
- 처음에는 var을 사용하면서 학습하고 자바스크립트를 더 공부하시게 되면서 자연스럽게 let과 const를 사용하면 됩니다.

```
<script>
// 변수 선언
var num1;
// 변수 값 할당
num1 = 77;
// 변수 초기화
var num2 = 99;
// 브라우저 출력
document.write(num1);
document.write('<br>');
document.write(num2);
document.write('<br>');
// 다중 변수 선언
var string1, string2, string3;
string1 = '홍길동 님~ ';
string2 = '안녕하세요. ';
string3 = '코딩웍스 클래스에 오신걸 환영합니다.';
// 다중 변수 브라우저 출력
document.write(string1 + string2 + string3);
</script>
```

일반적으로 변수 선언과 값 할당은 한번에 합니다.

다중 변수 선언할 경우 변수를 개별적으로 선언할 수도 있지만 효율적으로는 한번에 선언하는 것이 좋습니다.



```
// 다중 변수 선언
var string1;
var string2;
var string3;
```

1) 자바스크립트 변수 - 참고하기

앞으로 [참고하기] 라고 나오는 부분은 말그대로 참고입니다. 당장 몰라도 크게 문제되지 않습니다. 참고 사항은 나중에 자연스럽게 알게 되거나 추후 학습을 하시면 됩니다.

{ 자바스크립트 변수 선언 방식 차이 }

```
// var 선언방식 : 변수 재선언
var user = 'CodingWorks';
console.log(user);

var user = 'Andrew';
console.log(user);
```

CodingWorks

Andrew

▲ 변수 선언 방식 var은 변수를 재 선언해도 정상적으로 출력됨.

```
// let 선언방식 : 변수 재선언
let user = 'CodingWorks';
console.log(user);

let user = 'Andrew';
console.log(user);
```

Uncaught SyntaxError: Identifier 'user' has already been declared

▲ 변수 선언 방식 let은 변수를 재 선언하면 이미 선언된 변수와 이름이 동일하다고 오류를 출력합니다. 그래서 let은 개발자끼리 공동 작업이나 프로그램의 복잡성이 있는 경우에 사용합니다. 학습을 하는 경우는 간단한 프로그램을 하는거라 굳이 let으로 하실 필요까지는 없습니다.

{ 자바스크립트 호이스팅(Hoisting) }

웹 퍼블리셔 입장에서 호이스팅(Hoisting)이란 개념을 깊게 이해할 필요는 없습니다. 호이스팅이 이런 것 정도만 살짝 아시면 됩니다.

```
var user;

console.log(user);

user = 'CodingWorks';
```

undefined

▲ 변수가 선언은 되었지만 값은 할당되지 않은 상태에서 user를 콘솔에 출력하시니까 오류는 나지 않지만 undefined 곧, 값이 할당되지 않았다고 나옵니다.

var로 선언된 모든 변수는 코드가 실제로 이동하지는 않지만 최상위로 끌어올려진 것처럼 동작합니다. 이것 호이스팅이라고 합니다.

호이스팅 개념을 깊게 아실 필요는 없고..

변수 선언 다음에 값 할당 다음에 할당된 값 출력..

이라는 정상적인 코드 순서로 만들면 됩니다.

1) 자바스크립트 변수 - 더 쉽게 이해하기(var let const) let const 개념만 아시고 학습하는 동안은 var 또는 let을 사용합니다.



{ 달콤 커피에서 월~금요일까지 오늘의 커피를 판매 }

```
var todayCoffee = 'Cafe Latte';
```

변수 선언

변수 초기화(변수에 값을 주는 것)

```
todayCoffee = 'Caramel Macchiato';
```

다시 한번 변수 초기화

```
todayCoffee = 'Espresso';
```

다시 한번 변수 초기화

월요일이라 **오늘의 커피**는 카페 라테지만 화요일은 카라멜 마끼야또가 되면서 **오늘의 커피**라는 변수는 다시 카라멜 마끼야또로 초기화 됩니다. 계속 변수의 값을 변경해 주어서 변수를 초기화 할 수 있습니다. (키워드 **var**과 **let**을 사용하는 경우)

```
var todayCoffee = 'Cafe Latte';  
console.log(todayCoffee);
```

Cafe Latte

```
todayCoffee = 'Caramel Macchiato';  
console.log(todayCoffee);
```

Caramel Macchiato

```
todayCoffee = 'Espresso';  
console.log(todayCoffee);
```

Espresso

▼ 키워드 var let const에 대해서

```
let todayCoffee = 'Cafe Latte';  
console.log(todayCoffee);
```

Cafe Latte

```
todayCoffee = 'Caramel Macchiato';  
console.log(todayCoffee);
```

Caramel Macchiato

키워드 let을 사용하는 경우는 값을 변경(초기화)할 수는 있지만 같은 이름으로 변수 선언은 다시 할 수 없습니다.

```
let todayCoffee = 'Espresso';  
console.log(todayCoffee);
```

✖ Uncaught SyntaxError: Identifier 'todayCoffee' has already been declared

```
const coffeeShop = 'Dal.Komm';  
console.log(coffeeShop);
```

✖ ▶ Uncaught ReferenceError: todayCoffee is not defined

상수(변수)인 const의 경우 키워드 let과 다른 점은 값조차 변경할 수 없습니다.

```
coffeeShop = 'Starbucks';  
console.log(coffeeShop);
```

상수(변수)인 const의 경우 키워드 let과 동일하게 같은 이름으로 변수 선언을 다시 할 수 없습니다.

```
const coffeeShop = 'CoffeeBean';  
console.log(coffeeShop);
```

✖ Uncaught SyntaxError: Identifier 'todayCoffee' has already been declared

2) 자바스크립트 변수 - 변수 이름 규칙

키워드 = 자바스크립트 예약어로 대표적으로 alert() prompt() 등
식별자 = 개발자가 임의로 만드는 이름으로 대표적으로 변수 이름, 함수 이름, 객체 이름, 클래스 이름 등



{ 변수 이름 규칙 자바스크립트 규칙 }

- ① 영어 대소문자 철저히 구분
- ② 숫자는 반드시 영문 뒤에 사용. 곧, 숫자가 먼저 나오면 안됨
- ③ 특수문자를 사용할 경우 언더바(_)와 달러(\$)만 사용 가능
- ④ 공백은 포함되면 안됨
- ⑤ 키워드를 변수 이름으로 사용할 수 없음(관례)

{ 단어가 중복된 변수 이름 만드는 관례 }

- ① 두번째 단어의 첫 글자를 대문자로 표시(카멜 표기법, 낙타의 등 모양처럼 표시한다고 해서 카멜 표기법이라고 부름)
- ② '사용자 암호 저장' 이라는 변수를 만든다고 가정하면 user, password, save를 조합해서 아래처럼 변수 이름을 만듦.
userPasswordSave 또는 userPwSave

```
<script>
// 영어 대소문자 철저히 구분
var userPassword;
// 숫자는 반드시 영문 뒤에 사용. 곧, 숫자가 먼저 나오면 안됨
var string1;
var 1string;
// 특수문자를 사용할 경우 언더바(_)와 달러($)만 사용 가능
var _result;
var $result;
var #result;
var @result;
// 공백은 포함되면 안됨
var user password;
// 키워드를 변수 이름으로 사용할 수 없음

var alert;
alert = 77;
console.log(alert);
</script>
```

브라우저에서 오류를 확인 하지 않아도 비주얼 스튜디오 코드 자체에서 붉은 색 언더라인으로 오류 체크함

alert라는 키워드(예약어)를 사용해도 오류는 나지 않습니다. 하지만 키워드를 변수 이름으로 사용하면 나중에 혼란을 겪게 됩니다. 특히 협업에서는 다른 개발자에게 혼돈을 줍니다. 그래서 키워드는 절대 변수 이름으로 사용하지 않습니다.

3) 자바스크립트 변수 - 변수에 담는 데이터 유형

{ 자바스크립트 변수에 담는 데이터 유형 }

- 숫자, 문자열, 불(Boolean), 함수(function), 객체, undefined

127.0.0.1:5500 내용:

나이를 입력하세요.

확인 취소

prompt() 함수

Console	Sources	Recorder	1	Settings	Close
77	03) 변수에 담는 데이터 유형.html:12				
99	03) 변수에 담는 데이터 유형.html:15				
코딩웍스 퍼블리싱 클래스	03) 변수에 담는 데이터 유형.html:17				
true	03) 변수에 담는 데이터 유형.html:20				
false	03) 변수에 담는 데이터 유형.html:22				
32세	03) 변수에 담는 데이터 유형.html:25				
▶ #document	03) 변수에 담는 데이터 유형.html:28				
undefined	03) 변수에 담는 데이터 유형.html:31				

```
<script>
// 숫자 : 일반적인 숫자 표현
var num = 77;
console.log(num);
// 문자열 : 쌍 따옴표 또는 외 따옴표로 작성하는 일반적인 문자 또는 문자열
var str1 = '99';
console.log(str1);
var str2 = "코딩웍스 퍼블리싱 클래스";
console.log(str2);
// 불(Boolean) : 참(true)과 거짓(false)로만 표현되는 값
var bool1 = 99 > 77;
console.log(bool1);
var bool2 = false;
console.log(bool2);
// 함수(function) : 자바스크립트 코드의 집합
var fxTest = prompt('나이를 입력하세요.', '숫자로 입력');
console.log(fxTest + '세');
// 객체 : 자바스크립트가 제어 가능한 모든 요소
var obj = document;
console.log(obj);
// undefined : 변수에 값이 할당 되지 않은 경우 출력되는 값
var sample;
console.log(sample);
</script>
```

- 문자열은 쌍 따옴표 또는 외 따옴표 사용 가능
- 혼용할 경우 순서를 잘 지켜서 사용해야 함

4) 자바스크립트 변수 - 변수 재 선언 vs 변수의 재 사용

{ 변수의 재 선언 }

- 같은 변수 이름을 재 선언해서 값을 바꾸는 것
- 개념만 알고 사용은 하지 않습니다.

```
<script>
// 변수의 재사용
var food = 'noodle';
var food = 'spaghetti';
var food = 'pizza';
console.log(food);
</script>
```

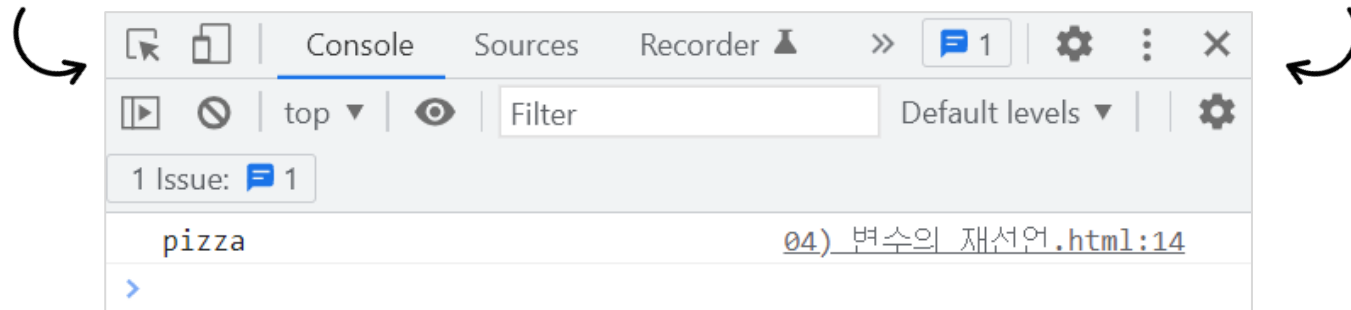
• 결과 값이 pizza인 이유는 같은 변수 선언에 마지막으로 들어가 간 값이 출력되기 때문

• 같은 이름의 변수를 중복적으로 선언하는 것은 좋지 않음

{ 변수의 재 사용 }

- 같은 변수 이름에 값을 바꾸어 다시 사용함
- 개념만 알고 특별한 경우를 제외하고 일반적으로 사용하지 않음

```
<script>
// 변수의 재사용
var food = 'noodle';
food = 'spaghetti';
food = 'pizza';
console.log(food);
</script>
```



5) 자바스크립트 변수 - 변수를 사용하는 이유

{ 값이 변할 수 있는 경우를 대비해서 변수를 사용 }

```
<script>
// 환율 계산
var exchangeRate = 1200;
var korAmount = prompt('환전할 원화를 입력하세요.', '1만원 단위 입력');
alert('현재 환율 기준 ' + '$' + (korAmount / exchangeRate) + '입니다.');
```

개발자가 이 곳의 숫자를 현재 환율에 유동적으로 바뀌게 프로그래밍

127.0.0.1:5500 내용:
환전할 원화를 입력하세요.

1만원 단위 입력

확인 취소

127.0.0.1:5500 내용:
환전할 원화를 입력하세요.

200000

확인 취소

127.0.0.1:5500 내용:
현재 환율 기준으로 \$167입니다.

확인

127.0.0.1:5500 내용:
현재 환율 기준 \$166.66666666666666입니다.

확인

```
// 소수점을 반올림해서 정수로 변환하는 Math.round() 함수 사용
alert('현재 환율 기준으로 ' + '$' + Math.round((korAmount / exchangeRate)) + '입니다.');
```

▲ Math.round() 와 같은 자바스크립트 함수는 나중에 자세히 배우게 됩니다.

{ 코드가 길어지는 것을 변수로 간단하게 사용하기 위함 }

```
<script>
// 산수 능력 테스트
var firstNum = prompt('첫번째 숫자 입력', '정수로 입력');
var secondNum = prompt('두번째 숫자 입력', '정수로 입력');
alert('더하기 결과 : ' + firstNum + secondNum);
```

첫번째 값에 1 입력하고 두번째 값에 1을 입력하면 결과는 2가 아니고 11이 됩니다.
prompt() 함수는 문자열을 반환하는 함수이므로 입력 받은 1이라는 문자열을 숫자로 변환하는 Number() 함수를 사용해서 숫자로 변환해야 정확한 결과인 2가 나옵니다.

```
alert('더하기 결과 : ' + (Number(firstNum) + Number(secondNum)));
```

127.0.0.1:5500 내용:
더하기 결과 : 11

확인

127.0.0.1:5500 내용:
더하기 결과 : 2

확인

6) 자바스크립트 변수 - 변수와 함수를 이용해서 간단한 프로그램 만들기(평균 구하기)

Small Mission) 사용자의 과목별 점수를 받아서 평균 구하기

- ① 입력 받을 4가지 과목별 점수를 prompt() 함수로 받기
- ② 입력 받은 4가지 점수를 숫자로 변환하기
- ③ 변환한 값을 평균 내기
- ④ 평균 값을 소수점 없이 정수로 alert() 함수로 출력하기
 - alert() 출력 시 아래처럼 출력하기
 - 평균 점수는 : ??입니다.

- 자바스크립트 함수를 사용할 때 괄호를 사용을 잘 해야 합니다.
- 입력 받은 4가지 값을 4로 나눠야 하니까 입력 받은 4가지 값을 괄호로 묶어서 먼저 계산해야 합니다.

```
<script>
// 과목별로 점수 입력 받아 변수에 담기
var EnglishScore = prompt('점수를 입력하세요.', '정수로 입력');
var MathScore = prompt('점수를 입력하세요.', '정수로 입력');
var HistoryScore = prompt('점수를 입력하세요.', '정수로 입력');
var KoreanScore = prompt('점수를 입력하세요.', '정수로 입력');
// 입력 받은 값들의 평균 내서 새로운 변수에 담기(과목별 점수 예시는 87, 77, 92, 63)
var examAvg = (Number(EnglishScore) + Number(MathScore) + Number(HistoryScore) + Number(KoreanScore)) / 4;
alert('평균 점수는 : ' + examAvg + '입니다.')
</script>
```

```
alert('평균 점수는 : ' + Math.round(examAvg) + '입니다.');
```

127.0.0.1:5500 내용:
평균 점수는 : 79.75입니다.

확인

127.0.0.1:5500 내용:
평균 점수는 : 80입니다.

확인

7) ES2015 백틱(`)과 템플릿 문자열

백틱(`) 문자는 키보드 탭 키 위에 있는 키입니다.



{ 변수에 대입하는 값에서 백틱(`) 사용하기 }

- 변수 값인 문자열에 어퍼스트로피 유무 관계 없이 사용 가능
- 변수 값 내에 다른 변수를 사용 가능
백틱(`)에서 변수를 사용할 경우 `${변수명}`으로 사용합니다.
- 변수 값 내에서 연산 가능
- 변수 값인 문자열에서 줄 바꿈을 하려면 `\n`를 줘야 하는데 백틱 안에서는 엔터를 치면 곧 줄 바꿈으로 표시(잘 안씀)

```
// 변수 값에 문자열은 반드시 따옴표로, 숫자는 따옴표 없이  
var name = 'Andrew';  
var age = 35;
```

```
// 문자열은 3가지 가능  
var name1 = "Andrew";  
var name2 = 'Andrew';  
var name3 = `Andrew`;
```

```
console.log(name1);  
console.log(name2);  
console.log(name3);
```

Andrew
Andrew
Andrew

```
// 문자열에서 어퍼스트로피(') 출력하기
```

```
var msg1 = "I'm Andrew.";  
var msg2 = 'I\'m Andrew.'; → 역 슬래시를 사용하면 어퍼스트로피를  
var msg3 = `I'm Andrew.`; → 특수 문자로 인식  
                           (역 슬래시는 키보드 \ 표시 키보드)  
                           ex) \n은 줄바꿈
```

```
console.log(msg1);  
console.log(msg2);  
console.log(msg3);
```

I'm Andrew.
I'm Andrew.
I'm Andrew.

```
// 백틱(`)을 사용할 경우 내부에 변수 사용가능
```

```
var whoIam1 = `저의 이름은 ${name}이고 나이는 ${age}입니다.`;  
console.log(whoIam1);
```

저의 이름은 Andrew이고 나이는 35입니다.

```
var whoIam2 = `내년에 ${age + 1}이 되고, 내후년이면 ${36+1}이 됩니다.`;  
console.log(whoIam2);
```

내년에 36이 되고, 내후년이면 37이 됩니다.

```
var num1 = 20;  
var num2 = 30;
```

```
var sum = `${num1} + ${num2} = ${num1 + num2} 입니다.`;  
console.log(sum);
```

20 + 30 = 50 입니다.

```
// 경고창에서 줄바꿈
```

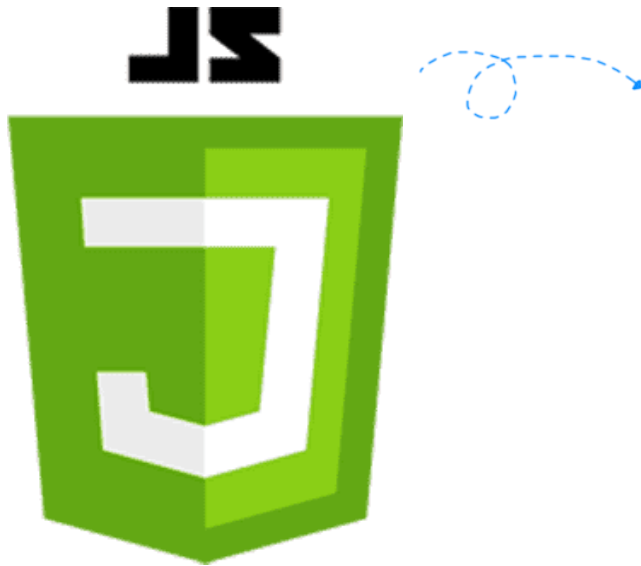
```
alert('환영합니다.\n코딩웍스 온라인 클래스입니다.');
```

127.0.0.1:5500 내용:

환영합니다.
코딩웍스 온라인 클래스입니다.

JAVASCRIPT 핵심이론 커리큘럼

[웹 퍼블리셔를 위한 자바스크립트] 03. 자바스크립트 조건문 if, switch 문

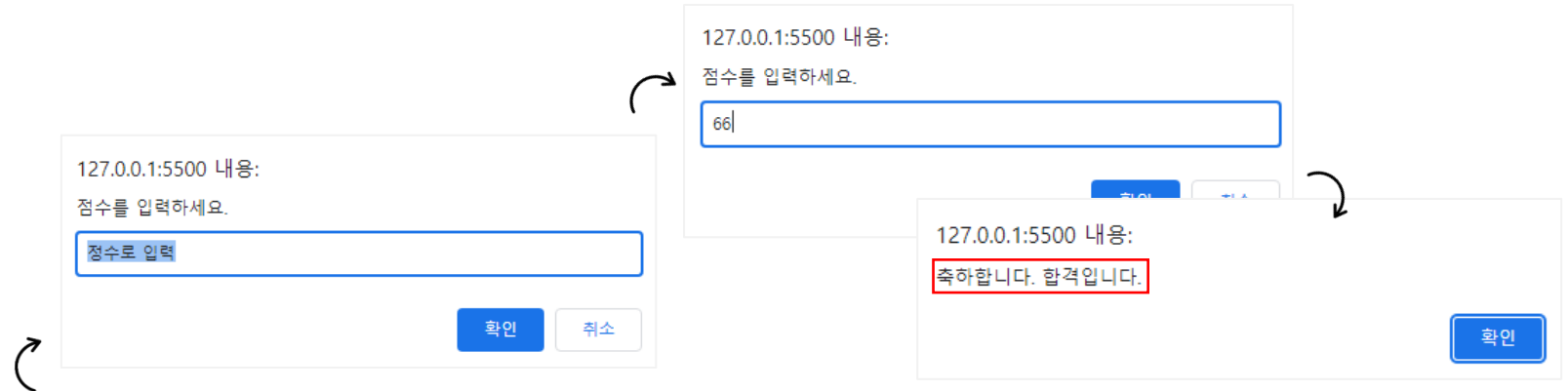


1. 기본 if else 조건문
2. 다중 if 조건문
3. 논리 연산자
4. 중첩 if 조건문
5. switch 문
6. 삼항 연산자
7. if와 '?'를 사용한 조건 처리



01) 기본 if 조건문

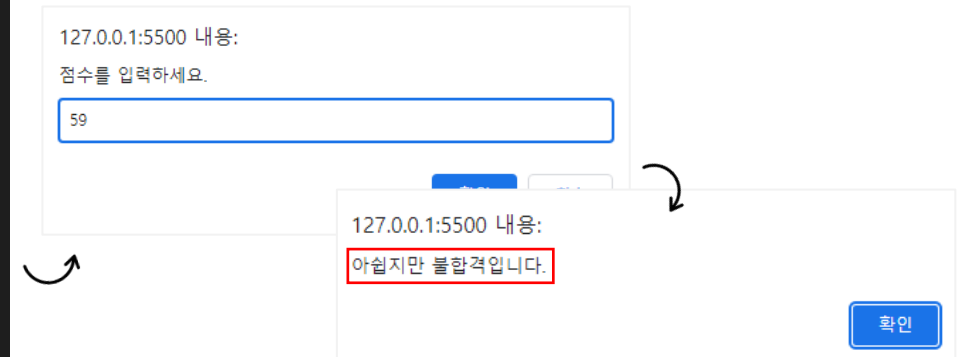
```
if(조건식) {  
    조건이 참(true)일 경우 실행 코드  
}  
  
else {  
    조건이 거짓(false)일 경우 실행 코드  
}
```



```
<script>  
// Q) 사용자에게 점수를 입력 받아 60점 이상이면 합격, 60점 미만이면 불합격 출력  
  
// 사용자에게 점수 입력 받아 변수에 담기  
var scoreResult = prompt('점수를 입력하세요.', '정수로 입력')  
  
if(scoreResult >= 60) {  
    alert('축하합니다. 합격입니다.');}  
else {  
    alert('아쉽지만 불합격입니다.');}  
</script>
```

if 조건문 괄호 안의 내용이 참일 경우 중괄호 {} 내의 코드를 실행

if 조건문 괄호 안의 내용이 참이 아닌 경우 else 중괄호 {} 내의 코드를 실행



◀ 숫자가 아닌 경우, 정수가 아닌 경우는 괄호 안의 조건식이 참이 아니기 때문에 중괄호 {} 안의 코드를 실행하지 않고 else if 내 코드를 실행하고 if 조건문을 빠져 나오게 됩니다.



02) 다중 if 조건문

```
if(조건식1) {
    조건식1이 참(true)일 경우 실행 코드
}
else if(조건식2) {
    조건식2가 참(true)일 경우 실행 코드
}
else if(조건식3) {
    조건식3이 참(true)일 경우 실행 코드
}
else {
    조건식1,2,3에 해당되지 않을 경우 실행 코드
}
```

Small Mission 01)

사용자에게 나이를 입력받아 20세 이상이면 '환영합니다.' 20세 미만이면 '안녕히 가세요.' 그리고 19세라면 '수능 잘 보세요.' 라고 출력하기

```
<script>
// 사용자에게 나이 입력 받아 변수에 담기
var age = prompt('나이를 입력하세요.', '정수 입력');

if(age >= 20) { → age > 19 라고 조건을 써도 동일합니다.
    alert('환영합니다.');
```

나이를 입력하세요.

정수 입력

확인 취소

127.0.0.1:5500 내용: 환영합니다.

확인

Small Mission 02)

사용자에게 점수를 입력 받아 90점 이상이면 A학점, 80점 이상이면 B학점, 70점 이상이면 C학점, 60점 이상이면 D학점, 60점 미만이면 F학점 출력

```
<script>
// 사용자에게 점수 입력 받아 변수에 담기
var scoreResult = prompt('점수를 입력하세요.', '정수로 입력');
if(scoreResult >= 90) {
    alert('A학점');
}
else if(scoreResult >= 80) {
    alert('B학점');
}
else if(scoreResult >= 70) {
    alert('C학점');
}
else if(scoreResult >= 60) {
    alert('D학점');
}
else {
    alert('F학점'); → 조건식1,2,3 어디에도 해당되지 않을 경우 실행 코드
}
</script>
```



03) 논리 연산자 - 조건식을 비교해서 참(true)과 거짓(false)을 판단

{ 논리 연산자 종류 }

- ① && **논리곱 연산자** : &(앰퍼샌드)를 사용해서 참과 거짓을 판단(and 연산으로 모든 조건식이 참이어야 참으로 출력)

[사용법] 조건식1 && 조건식2 && 조건식3

ex) 코딩웍스는 미국인이고 남자이다. ==> false (뒤에 조건이 여러 개라도 처음이 아니면 뒤는 체크 안함)

- ② || **논리합 연산자** : |를 사용해서 참과 거짓을 판단(or 연산으로 조건식 중 한 개라도 참이면 참으로 출력)

[사용법] 조건식1 || 조건식2 || 조건식3

ex) 코딩웍스는 미국인이거나 남자이다. ==> true (뒤에 모든 조건을 체크함)

- ③ ! **부정 연산자** : 조건식을 참이면 거짓으로, 거짓이면 참으로 바꾸어 출력

[사용법] !조건식

ex) 코딩웍스는 미국인이 아니다. ==> true

Self Mission) 셀프 미션은 혼자서 해보세요!

학력은 고졸 이상이고 직장 경력 2년 이상이고 웹 디자인 기능사 자격증 소지자..를 뽑는다면?

- 큰 부분을 걸러낼 조건을 먼저 사용하는 것이 프로그래밍 퍼포먼스를 위해 좋습니다.
- 전체 표본에서 고졸이상(90%) 경력 2년 이상(60%) 웹 디자인 자격증 소지자(5%)라면...
- 웹 디자인 기능사 자격증 && 경력 2년 이상 && 고졸 이상 ==> 첫번째 조건에서 대부분 걸러냄

```
<script>
  // 조건을 비교할 값을 변수에 담기
  var test1 = 20 > 10;
  var test2 = 20 < 10;
  var test3 = 20 == 10;

  // 논리곱연산자 : &&
  console.log(test1 && test2 && test3);

  // 논리합연산자 : ||
  console.log(test1 || test2 || test3);

  // 부정연산자 : !
  console.log(!test1);
</script>
```

현재 변수에 담긴 값의 참과 거짓

- test1은 참
- test2은 거짓
- test3은 거짓

test1,2,3가 모두 참이어야 참을 출력하는데 test2,3가 거짓이므로 false 출력

test1,2,3 중 한 개라도 참이면 참을 출력하는데 test1이 참이므로 true 출력

test1의 최초 값이 참이므로 반대인 false 출력

자바스크립트에서의 같다는
== 또는 === 입니다.

false
true
false



03) 논리 연산자 - 조건식을 비교해서 참(true)과 거짓(false)를 판단

Small Mission 01) 논리곱연산자(&&) 과락을 체크해서 합격 불합격 출력하기(과락은 60점 미만)

- ① 사용자에게 입력 받은 3가지 과목별 점수를 prompt() 함수로 받아서 변수에 담기
- ② 조건문과 논리연산자를 사용해서 변수를 비교해 합격 여부를 alert()로 출력하기

※ 일반적인 국가 시험은 평균이 60점 이상이면 합격 하지만 과목 하나라도 60점 미만인 과락이 있으면 불합격

```
<script>
// 사용자에게 입력 받을 3가지 과목별 점수를 prompt() 함수로 받아서 변수에 담기
// 테스트 입력 값으로 90, 70, 50을 입력
var scoreMath = prompt('수학 점수를 입력하세요.', '정수로 입력');
var scoreHistory = prompt('역사 점수를 입력하세요.', '정수로 입력');
var scoreEnglish = prompt('영어 점수를 입력하세요.', '정수로 입력');

// 조건문과 논리연산자를 사용해서 변수를 비교해 합격 여부를 alert()로 출력하기
if(scoreMath >= 60 && scoreHistory >= 60 && scoreEnglish >= 60) {
    alert('합격입니다.')
}
else {
    alert('불합격입니다.')
}
}</script>
```

3과목 평균은 70점이지만 영어 점수가 50점 과락이므로 논리곱 연산자는 하나라도 거짓이면 거짓으로 처리하기 때문에 else 내의 코드를 실행

127.0.0.1:5500 내용:
불합격입니다.

확인

127.0.0.1:5500 내용:
합격입니다.

확인

▲ 60점 미만의 과락이 없는 경우 합격 출력



03) 논리 연산자 - 조건식을 비교해서 참(true)과 거짓(false)를 판단

Small Mission 02) 논리합연산자(||) 주민등록번호 뒷 번호의 앞자리 숫자를 체크해서 남녀 출력

- ① 사용자에게 1~4 사이의 정수를 입력 받아 prompt() 함수로 받아서 변수에 담기
- ② 조건문과 논리연산자를 사용해서 변수를 비교해 남녀를 판단해서 alert()로 출력하기
- ③ 1~4 사이의 정수가 아니거나 문자이거나 하는 경우는 1~4사이의 정수를 입력하라고 alert()로 출력하기

```
<script>
// 사용자에게 1~4 사이의 정수를 입력 받아 prompt() 함수로 받아서 변수에 담기
var birthNum = prompt('주민등록번호 뒷 번호의 앞자리 숫자를 입력하세요.', '정수로 입력')

// 조건문과 논리연산자를 사용해서 변수를 비교해 남녀를 판단해서 alert()로 출력하기
if(birthNum == 1 || birthNum == 3) {
    alert('남자');
}
else if(birthNum == 2 || birthNum == 4) {
    alert('여자');
}
else {
    alert('1~4사이의 정수를 입력하세요.');
```

논리합연산자는 또는(or)으로 하나라도 참이면 참을 출력

127.0.0.1:5500 내용:

남자

확인

127.0.0.1:5500 내용:

여자

확인

127.0.0.1:5500 내용:

1~4사이의 정수를 입력하세요.

확인

▲ 1~4 사이의 정수가 아니거나 문자이거나 하는 경우



03) 논리 연산자 - 조건식을 비교해서 참(true)과 거짓(false)을 판단

{ confirm 함수 }

```
<script>
// confirm 함수는 확인을 누르면 true, 취소를 누르면 false 출력
var payOk = confirm('결제하시겠습니까?');
alert(payOk);
</script>
```

127.0.0.1:5500 내용:

결제하시겠습니까?

true 출력 false 출력

확인

취소

127.0.0.1:5500 내용:

결제하시겠습니까?

확인

취소

127.0.0.1:5500 내용:

결제가 취소되었습니다.

확인

Small Mission 03) 부정연산자(!) 확인(true)과 취소(false) 판단하기

- ① 사용자에게 confirm() 함수로 확인과 취소를 받아 변수에 담기
- ② 변수에 담긴 값(true 또는 false)을 조건문으로 판단해서 결제 취소 alert() 출력하기
- ③ 출력 결과는 같지만 부정연산자(!)를 사용해서 간결하게 코딩하기

```
<script>
// 사용자에게 confirm() 함수로 확인과 취소를 받아 변수에 담기
var payOk = confirm('결제하시겠습니까?');

// 변수에 담긴 값(true 또는 false)을 조건문으로 판단해서 alert() 출력하기
if(payOk == true) {

}
else {
  alert('결제가 취소되었습니다.');
```

부정연산자를 사용해서 간결하게 코딩하기

```
// 출력 결과는 같지만 부정연산자(!)를 사용해서 간결하게 코딩하기
if(!payOk) {
  alert('결제가 취소되었습니다.');
```




04) 중첩 if 조건문 with 비교 연산자

{ 중첩 if 조건문 기본 형식 }

```
if(조건식1) {
    if(조건식2) {
        조건식2가 참(true)일 경우 실행 코드
    } else {
        조건식2가 거짓(false)일 경우 실행 코드
    }
}
else {
    조건식1이 거짓(false)일 경우 실행 코드
}
```

- 조건식1이 참일 경우 중첩된 if의 조건식2가 참인지 체크
- 조건식1이 거짓인 경우 중첩된 if를 넘어서 else {} 실행

Small Mission 01) 60~100점 사이면 합격, 60점 미만이면 불합격 출력

- ① 사용자에게 prompt() 함수로 점수 입력 받아 변수에 담기
- ② 60점 미만이면 '불합격' 출력, 60점~100점 사이면 '합격' 출력
- ③ 변수에 담긴 값이 1~100 사이 정수가 아니면 '0~100 사이의 정수를 입력하세요'라고 alert() 출력

```
<script>
// 사용자에게 prompt() 함수로 점수 입력 받아 변수에 담기
var examScore = prompt('점수를 입력하세요.', '0~100 사이의 정수를 입력하세요. ');

// 60점 미만이면 '불합격' 출력, 60점~100점 사이면 '합격' 출력
if(examScore >= 0 && examScore <= 100) {
    if(examScore >= 60) {
        alert('합격');
    }
    else {
        alert('불합격');
    }
}
// 변수에 담긴 값이 1~100 사이 정수가 아니면 '0~100 사이의 정수를 입력하세요'라고 alert() 출력
else {
    alert('0~100 사이의 정수를 입력하세요. ');
}
</script>
```



04) if 조건문 with 사칙 연산자

Small Mission 02) 입력 받은 정수가 홀수인지 짝수인지 출력하고 정수가 아니 경우 '정수를 입력하세요'라고 출력하세요.

```
<script>
// 입력 받은 정수가 홀수인지 짝수인지 출력
// 정수가 아니 경우 '정수를 입력하세요'라고 출력
var num = prompt('홀수 짝수를 체크합니다.', '정수를 입력하세요. ');
if(num % 2 == 1) {
    alert('홀수');
}
else if(num % 2 == 0) {
    alert('짝수');
}
else {
    alert('정수를 입력하세요. ');
}
</script>
```

1. 입력 받은 정수를 2로 나누어 나머지 값을 계산
2. 나머지 값이 1이면 홀수
3. 나머지 값이 2면 짝수

- 나누어 1 또는 2가 나오지 않는다면 정수가 아닌 경우 ex) 22.5
- 나눌 수 없는 문자인 경우 ex) 코딩웍스



05) switch - 조건당 하나의 표현식을 배당 switch문은 if문과 동일하지만 결과가 많은 경우 사용하기 좋음

{ switch 기본 형식 }

```
switch(데이터) {  데이터 : 수식 또는 변수
  case '값1' :  세미콜론(:)이 아니고 콜론(:)
    데이터와 값1이 맞을 때 표현식
    break;
  case '값2' :
    데이터와 값1이 맞을 때 표현식
    break;
  default:
    위의 case 들에 해당 되지 않은 경우 표현식
}
```

Small Mission 01) switch 문 사용해서 원하는 과일의 가격을 콜솔창에 출력하기

- ① 사용자에게 과일 이름을 입력 받아 prompt() 함수로 받아서 변수에 담기
- ② switch 문을 사용해서 변수를 비교해 과일 이름과 맞는 가격을 출력
- ③ 목록에 있는 과일이 아닌 경우 '판매하는 과일 이 아닙니다. 다시 입력해주세요.' 출력하기

3000원

망고라는 입력 값과 같은 case의 실행 구문을 출력하고 switch 문을 빠져나옴.

3000원
4000원
5000원
판매하는 과일이 아닙니다. 다시 입력해주세요.

▲ break가 없다면 망고 case 체크 후 switch 문을 바로 빠져 나오지 못하고 아래에 있는 모든 case 를 출력함.

```
<script>
  // 사용자에게 과일 이름을 입력 받아 prompt() 함수로 받아서 변수에 담기
  var fruit = prompt('가격 체크를 원하는 과일 이름을 입력하세요. ');

  // switch 문을 사용해서 변수를 비교해 과일 이름과 맞는 가격을 출력
  switch(fruit) {
    case '레몬':
      console.log('1000원');
      break;
    case '자몽':
      console.log('2000원');
      break;
    case '망고':
      console.log('3000원');
      break;
    case '포도':
      console.log('4000원');
      break;
    case '딸기':
      console.log('5000원');
      break;
    default:
      alert('판매하는 과일이 아닙니다. 다시 입력해주세요. ');
      break;
  }
</script>
```



05) switch - 조건당 하나의 표현식을 배당 switch문은 if문과 동일하지만 결과가 많은 경우 사용하기 좋음

Small Mission 02) switch 문 사용해서 주민등록번호 뒷 번호의 앞자리 숫자를 체크해서 남녀 출력

- ① 사용자에게 1~4 사이의 정수를 입력 받아 prompt() 함수로 받아서 변수에 담기
- ② switch 문을 사용해서 변수를 비교해 남녀를 판단해서 alert()로 출력하기
- ③ 1~4 사이의 정수가 아니거나 문자이거나 하는 경우는 1~4사이의 정수를 입력하라고 alert()로 출력하기

```
<script>
/* switch 문을 사용해서 주민등록번호 뒷 번호의 앞자리 숫자를 체크해서 남녀 출력 */

// 사용자에게 1~4 사이의 정수를 입력 받아 prompt() 함수로 받아서 변수에 담기
var birthNum = prompt('주민등록번호 뒷 번호의 앞자리 숫자를 입력하세요.', '정수로 입력');

// switch 문을 사용해서 변수를 비교해 남녀를 판단해서 alert()로 출력하기
switch(birthNum) {
    case '1':
        alert('남자');
        break;
    case '2':
        alert('여자');
        break;
    case '3':
        alert('남자');
        break;
    case '4':
        alert('여자');
        break;
    // 1~4 사이의 정수가 아니거나 문자이거나 하는 경우는 1~4사이의 정수를 입력하라고 alert()로 출력하기
    default:
        alert('1~4사이의 정수를 입력하세요.');
```

코드를 간결하게 할 수도 있음

```
case '1':
case '3':
    alert('남자');
    break;
case '2':
case '4':
    alert('여자');
```



06) switch - 조건당 하나의 표현식을 배당

```
<script>
/* switch 문 사용해서 오늘 요일을 출력하기 ex) 오늘은 월요일입니다. */

// 시스템에서 요일 받아와서 변수에 담기
var day = new Date().getDay(); // 시스템에서 현재 요일을 받아오기(숫자로 받아옴)

// 변수의 숫자(index)의 경우(case)에 따라 요일로 출력하기
switch(day) {
  case 0:
    day = '일요일';
    break;
  case 1:
    day = '월요일';
    break;
  case 2:
    day = '화요일';
    break;
  case 3:
    day = '수요일';
    break;
  case 4:
    day = '목요일';
    break;
  case 5:
    day = '금요일';
    break;
  case 6:
    day = '토요일';
    break;
}

alert('오늘은 ' + day + '입니다.');
```

Small Mission 02) switch 문 사용해서 오늘 요일을 출력하기

- ① 시스템에서 요일 받아와서 변수에 담기
- ② 변수의 숫자(index)의 경우(case)에 따라 요일로 출력하기 ex) 오늘은 월요일입니다.

127.0.0.1:5500 내용:
오늘은 일요일입니다.

확인

경우(case)가 0일 경우 변수 day에 일요일 이라는 문자를 넣고 switch 문을 빠져 (break) 나와서 일요일 이라는 문자를 alert() 내에 변수 day에 넣습니다.



07) 일항 연산자, 이항 연산자, 삼항 연산자

{ 연산자의 종류 }

- ① **일항 연산자** : 연산을 하는 피연산자가 1개인 경우
ex) +10, -20
- ② **이항 연산자** : 연산을 하는 피연산자가 2개인 경우(대부분의 연산에 사용)
ex) 10 <= 20, 30 + 40
- ③ **삼항 연산자** : 연산을 하는 피연산자가 3개인 경우
ex) 조건식 ? 참표현식 : 거짓표현식

Small Mission) 삼항 연산자를 사용해서 합격, 불합격 출력하기

- ① 사용자에게 입력 받은 정수를 변수에 담기
- ② 담긴 변수를 삼항 연산자를 사용해서 60점 이상이면 합격, 60점 미만이면 불합격 출력하기

```
<script>
/* Small Mission 01) 삼항 연산자를 사용해서 합격, 불합격 출력하기 */

// 사용자에게 입력 받은 정수를 변수에 담기
var examScore = prompt('점수를 입력하세요.', '정수로 입력');

// 담긴 변수를 삼항 연산자를 사용해서 합격 불합격 출력하기
examScore >= 60 ? alert('합격') : alert('불합격');
</script>
```

조건문에서 if에 해당

조건문에서 else에 해당

127.0.0.1:5500 내용:

합격

127.0.0.1:5500 내용:

불합격

확인



08) if와 조건부 연산자 ?와 :을 사용한 조건 처리 - 삼항 연산자 조건식

조건문을 이전에 학습한 if와 else if를 사용해도 전혀 문제는 없습니다. 더 효율적인 작업을 위해 삼항 연산자 조건식을 활용합니다.

{ 기본 if 조건문 → 단축형 조건식 }

```
var age = prompt('나이를 입력하세요.', '정수로 입력');
```

```
if(age >= 20) {
  alert('성인입니다.');
```

동일한 조건문 코드

```
if(age >= 20) alert('성인입니다.');
```

(조건) 참일 경우 실행할 구문 → 중괄호 생략 가능

```
if(age >= 20) {
  alert('성인입니다.');
```

동일한 조건문 코드

```
age >= 20 ? alert('성인입니다.') : alert('미성인입니다.');
```

(조건) ? 참일 경우 실행할 구문 : 거짓일 경우 실행할 구문

조건을 감싸는 괄호는 생략 가능

{ 다중 if 조건문 → 단축형 조건식 }

```
var scoreResult = prompt('점수를 입력하세요.', '정수로 입력');
```

```
if(scoreResult >= 90) {
  alert('A학점');
```

```
else if(scoreResult >= 80) {
  alert('B학점');
```

```
else if(scoreResult >= 70) {
  alert('C학점');
```

```
else if(scoreResult >= 60) {
  alert('D학점');
```

```
else {
  alert('F학점');
```

if 생략

중괄호 {}

else if

동일한 조건문 코드

```
(scoreResult >= 90) ? alert('A학점') : (scoreResult >= 80) ?
alert('B학점') : (scoreResult >= 70) ? alert('C학점') :
(scoreResult >= 60) ? alert('D학점') : alert('F학점')
```

조건이 여러 개라도 감싸는 괄호는 생략 가능 마지막 콜론(:)은 else



Small Mission)

오늘의 요일을 받아와서 월요일~목요일이면 [평일 특가 : 99000원], 금요일이면 [금요일 특가 : 88000원], 토요일~일요일이면 [주말 특가 : 66000원] 라고 출력하세요.
단, 조건문 if와 switch 문을 사용해서 개별적으로 2번 만드세요.

{ 조건문 if 사용 }

```
<script>
// 시스템에서 오늘 날짜 받아오기
// 인덱스 번호(0123456 : 일월화수목금토)
var now = new Date().getDay();

// 변수가 1~3이면 평일 특가, 4면 금요일 특가, 5~6이면 주말 특가
if(now == 0 || now == 6) {
    alert('주말 특가 : 66,000원');
}
else if(now == 5) {
    alert('금요일 특가 : 88,000원');
}
else {
    alert('평일 특가 : 99,000원');
}
</script>
```

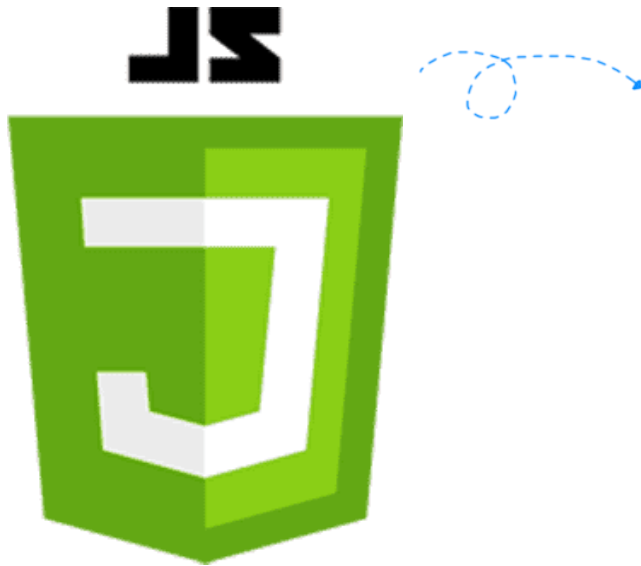
{ switch 사용 }

```
<script>
// 시스템에서 오늘 날짜 받아오기
// 인덱스 번호(0123456 : 일월화수목금토)
var now = new Date().getDay();

// 변수가 1~3이면 평일 특가, 4면 금요일 특가, 5~6이면 주말 특가
switch(now) {
    case 0:
    case 6:
        alert('주말 특가 : 66000원');
        break;
    case 5:
        alert('금요일 특가 : 88000원');
        break;
    default:
        alert('평일 특가 : 99000원');
}
</script>
```


JAVASCRIPT 핵심이론 커리큘럼

[웹 퍼블리셔를 위한 자바스크립트] 04. 자바스크립트 반복문 - while문, do while문, for문



1. 복합 대입 연산자
2. 증감 연산자
3. while 반복문
4. do while 반복문
5. for 반복문
6. 중첩 for 반복문



01) 복합 대입 연산자 사칙 연산자(+, -, *, /)와 대입 연산자(=)를 혼합해서 사용하는 연산자

{ 복합 대입 연산자 기본 형식 }

- $a = a + 1$
a에 1을 더해서 다시 a에 대입해라
- $a = a - 1$
a에 1을 빼서 다시 a에 대입해라
- 축약형으로 $a += 1$ $a -= 1$

{ 복합 대입 연산자의 종류 }

- ① $+=$: 더하기 대입 연산자
- ② $-=$: 빼기 대입 연산자
- ③ $*=$: 곱하기 대입 연산자
- ④ $/=$: 나누기 대입 연산자
- ⑤ $\%=$: 나머지 대입 연산자

이퀄(=)은 항상 우측에 있고 달라지는 + - 와 같은 것은 항상 좌측에 있음

$a = a + 1$ $\rightarrow$ $a = + 1$ $\rightarrow$ $a += 1$

삭제 좌우 바꾸기

```
<script>
a = 1;
a = a + 1;
console.log(a);
</script>
```

▲ 최초의 a라는 변수에 1을 대입하고 그 a에 1을 더해서 다시 a라는 변수에 넣으면 결과는 2

```
<script>
num = 5;
// += : 더하기 대입 연산자
num += 5;
console.log(num);

// -= : 빼기 대입 연산자
num -= 5;
console.log(num);

// *= : 곱하기 대입 연산자
num *= 5;
console.log(num);

// /= : 나누기 대입 연산자
num /= 5;
console.log(num);

// %= : 나머지 대입 연산자
num %= 5;
console.log(num);
</script>
```

현재 num에 5가 들어가 있고 그 num에 5를 나눠서 나머지 값을 다시 num에 대입하면 0

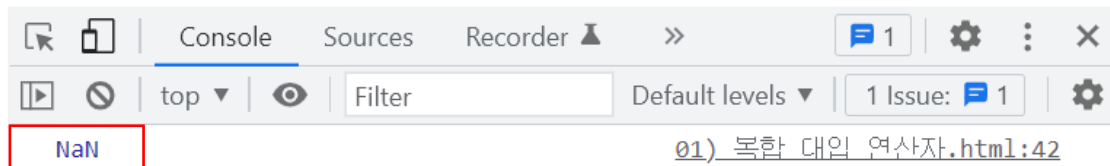
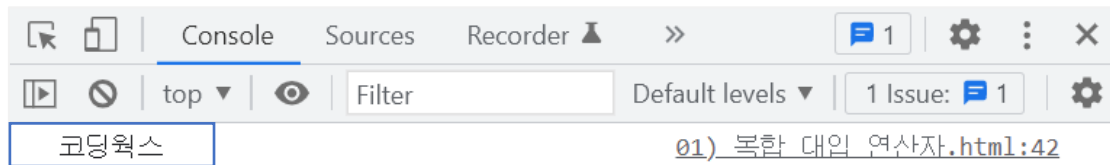
Console	Sources	Recorder
10		
5		
25		
5		
0		



01) 복합 대입 연산자 사칙 연산자(+-/*)와 대입 연산자(=)를 혼합해서 사용하는 연산자

{ 복합 대입 연산자로 문자 결합 }

```
<script>
// 문자1+문자2는 문자1문자2로 출력
var str = '코딩';
str += '웍스';
console.log(str);
</script>
```



▲ 만약 `str += '웍스';` 라고 하면 '코딩'이라는 문자에서 '웍스'라는 문자를 뺄 수 없으므로 오류(NaN)가 출력됨.

{ 복합 대입 연산자로 HTML 태그 만들기 }

```
<script>
var htmlList = '<ul>';
htmlList += '<li>목록 태그 01</li>';
htmlList += '<li>목록 태그 02</li>';
htmlList += '<li>목록 태그 03</li>';
htmlList += '</ul>';

// body에 출력하기
document.write(htmlList);
</script>
```

→ 이런 부분이 계속 반복되므로 반복문을 사용해서 코드의 중복을 하지 않고 처리



- 목록 태그 01
- 목록 태그 02
- 목록 태그 03

▲ html 태그 역시 따옴표로 묶여 문자가 되므로 하나씩 더해져서 결국엔 완전한 html 코드가 됨.



02) 증감 연산자 ↪ 1씩 증가하거나 1씩 감소하는 연산자

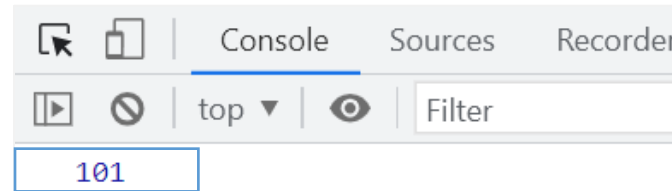
{ 증감 연산자의 종류 }

- ① 변수++ : 기존 변수에 1씩 증가 시키고 다음 줄부터 실행
- ② ++변수 : 기존 변수에 1씩 증가 시키고 해당 줄부터 실행
- ③ 변수-- : 기존 변수에 1씩 감소 시키고 다음 줄부터 실행
- ④ --변수 : 기존 변수에 1씩 감소 시키고 해당 줄부터 실행

증감 연산자를 앞에 사용하는 경우가 대부분이지만 경우에 따라 뒤에 사용해야 하는 경우도 있음

```
<script>
// 최초의 변수 num
var num = 100;
console.log(++num);
</script>
```

↪ 증감 연산자가 앞에 있는 경우 바로 실행

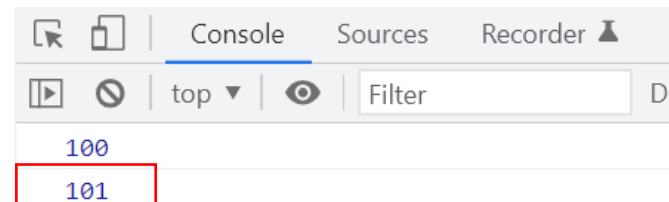


```
<script>
// 최초의 변수 num
var num = 100;
console.log(num++);
</script>
```

↪ 증감 연산자가 뒤에 있는 경우 바로 실행하지 않음

```
<script>
// 최초의 변수 num
var num = 100;
console.log(num++);
console.log(num);
</script>
```

↪ 증감 연산자가 뒤에 있는 경우 다음 줄에서 실행





03) 반복문 while

{ while 문 기본 형식 }

```
while(조건식) {
    반복할 표현식
}
```

Small Mission) while 문 사용해서 1부터 10까지 콘솔창에 출력하기

- ① 최초에 변수 num에 1을 대입하기
- ② num을 1씩 증가 시키되 10이 되면 증가를 멈추고 while 문 빠져나오기

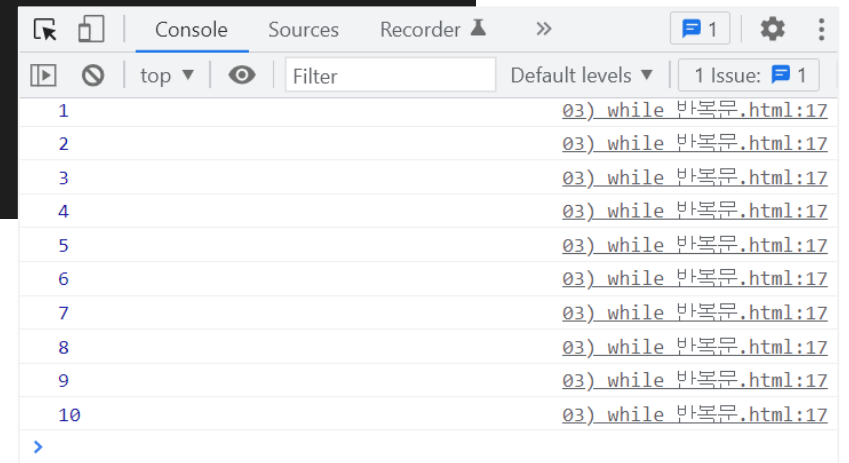
```
<script>
/* while 문 사용해서 1부터 10까지 콘솔창에 출력하기 */

// 최초에 변수 num에 1을 대입하기
var num = 1;

// num을 1씩 증가 시키되 10이 되면 증가를 멈추고 while 문 빠져나오기
while(num <= 10) {
    console.log(num);
    num++;
}
```

조건식에서 num이 10보다 작거나 같을 때까지 while 반복문을 실행

num을 1씩 증가시키지 않으면 num에 1만 계속 찍힐 테니까 while의 조건에서 항상 참이 되므로 반복문은 영원히 종료되지 않음.



※ 앞으로 반복문을 학습할 경우 비주얼 스튜디오 코드를 사용할 때 Live Server가 작동되면 반복문이 완성이 되지 않아도 바로 브라우저에 적용하기 때문에 작성 중인 상태에서 브라우저가 무한 반복하거나 멈추는 현상이 생깁니다.

반복문을 연습하는 경우 일반적인 현상이니 Live Server 끄고 다시 브라우저를 띄우시기 바랍니다.

이런 부분이 번거로운 경우 Live Server를 사용하지 않고 브라우저를 새로고침(F5)를 해도 됩니다.



04) 반복문 do while while 문과 다른 점은 while의 조건식이 맞지 않아도(false) 최소 한번은 실행(do) 합니다.

{ do while 문 기본 형식 }

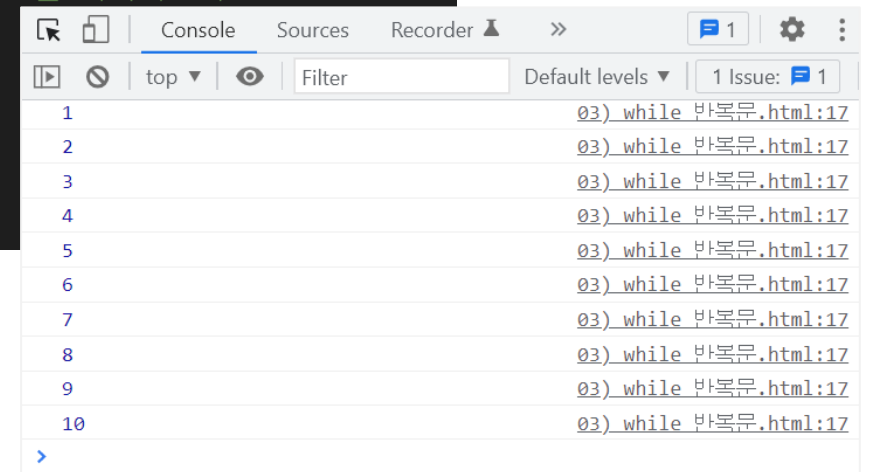
```
do {  
    반복할 표현식  
}  
while(조건식);
```

Small Mission) do while 문 사용해서 1부터 10까지 콘솔창에 출력하기

- ① 최초에 변수 num에 1을 대입하기
- ② num을 1씩 증가 시키되 10이 되면 증가를 멈추고 while 문 빠져나오기

```
<script>  
    /* do while 문 사용해서 1부터 10까지 콘솔창에 출력하기 */  
  
    // 최초에 변수 num에 1을 대입하기  
    var num = 1;  
  
    // num을 1씩 증가 시키되 10이 되면 증가를 멈추고 while 문 빠져나오기  
    do {  
        console.log(num);  
        num++;  
    }  
    while(num <= 10);  
</script>
```

num++; → num을 1씩 증가시키지 않으면 num에 1만 계속 찍힐 테니까 do while의 조건에서 항상 참이 되므로 반복문은 영원히 종료되지 않음.
while(num <= 10); → 조건식에서 num이 10보다 작거나 같을 때까지 while 반복문을 실행



1	03) while 반복문.html:17
2	03) while 반복문.html:17
3	03) while 반복문.html:17
4	03) while 반복문.html:17
5	03) while 반복문.html:17
6	03) while 반복문.html:17
7	03) while 반복문.html:17
8	03) while 반복문.html:17
9	03) while 반복문.html:17
10	03) while 반복문.html:17
>	



05) 반복문 for ↪ 반복문 중에서 가장 많이 사용되는 형식

초기 값 변수 이름은 일반적으로 for 반복문에서 사용하는 변수는 i를 사용

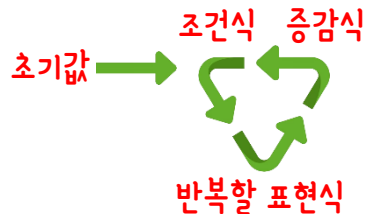
{ for 반복문 기본 형식 }

for(초기값; 조건식; 증감식) {
반복할 표현식
}

초기값: 최초에 한번만 실행
조건식: false일 때 멈춤
증감식: 증감식에는 세미콜론(;) 없음
반복할 표현식: 반복할 표현식 실행 후 실행

for 반복문 실행 순서를 말로 설명하면..

초기값을 한번만 실행하고 조건식이 참일 경우 반복할 표현식을 실행하고 증감식을 실행하고 다시 조건식을 비교해서 참이면 반복할 표현식을 실행하고 증감식을 실행하고 조건식이 거짓일 경우 반복할 표현식을 실행하지 않고 for 반복문을 빠져 나옵니다.



Small Mission 1) for 문으로 1부터 10까지 콘솔 출력하기

- ① 최초에 변수 i에 1을 대입하기
- ② i를 1씩 증가 시키되 10이 되면 증가를 멈추고 for 문 빠져나오기

```
<script>
/* for 문 사용해서 1부터 10까지 콘솔창에 출력하기 */

// 최초에 변수 num에 1을 대입하기
// num을 1씩 증가 시키되 10이 되면 증가를 멈추고 for 문 빠져나오기

for(① i = 1; ② i <= 10; ③ i++) {
  console.log(i);
}
</script>
```

- ① 최초 값 i에 1을 대입
- ② 조건식에 맞으면 곧, true이면 반복할 표현식 실행 (i < 11로 조건식 만들어도 동일함)
- ③ i라는 변수에 1을 증가시키고 다시 조건식과 비교해서 조건식이 맞으면 반복할 표현식 실행 (i가 10일 때까지는 조건식에 맞으므로 실행하지만 i가 11이 되면 조건식에 맞지 않으므로 i를 1씩 증가시키는 것을 하지 않고 for 반복문을 빠져 나옴

Small Mission 2) for 문으로 10부터 1까지 콘솔 출력하기

- ① 최초에 변수 i에 10을 대입하기
- ② i를 1씩 증가 시키되 10이 되면 증가를 멈추고 for 문 빠져나오기

```
<script>
/* for 문 사용해서 1부터 10까지 콘솔창에 출력하기 */

// 최초에 변수 num에 10을 대입하기
// num을 1씩 감소시켜 0이 되면 멈추고 for 문 빠져나오기

for(① i = 10; ② i > 0; ③ i--) {
  console.log(i);
}
</script>
```

- ① 최초 값 i에 10을 대입
- ② 조건식에 맞으면 곧, true이면 반복할 표현식 실행
- ③ i라는 변수에 1을 감소시키고 다시 조건식과 비교해서 조건식이 맞으면 반복할 표현식 실행 (i가 0보다 클 때까지는 조건식에 맞으므로 실행하지만 i가 0이 되면 조건식에 맞지 않으므로 i를 1씩 감소시키는 것을 하지 않고 for 반복문을 빠져 나옴



05) 반복문 for - 스몰 미션

{ for 반복문 없이 HTML 태그를 직접 입력한 경우 }

```
<select>
  <option>1월</option>
  <option>2월</option>
  <option>3월</option>
  <option>4월</option>
  <option>5월</option>
  <option>6월</option>
  <option>7월</option>
  <option>8월</option>
  <option>9월</option>
  <option>10월</option>
  <option>11월</option>
  <option>12월</option>
</select>
```

▲ HTML 태그를 직접 입력해서 작성하는 것을 정적 방식이라고 하고, 자바 스크립트로 반복을 시켜서 작성하는 것을 동적 방식이라고 합니다.

1월~12월도 그렇지만 1일~31일, 1950년~2023년 과 같이 반복되는 개수가 더 많은 경우 반복문을 사용하는 것이 효율적입니다.

Small Mission) for 문 사용해서 1월부터 12월까지 select 태그로 body에 출력하기

- ① 최초에 변수 htmlSelect에 1을 대입하기
- ② i를 1씩 증가 시키되 13이 되면 실행문을 돌리지 않고 for 문 빠져나오기
- ③ document.write로 body에 HTML 태그 출력하기

```
<script>
  /* for 문 사용해서 1월부터 12월까지 select 태그로 body에 출력하기 */

  // 최초에 변수 i에 1을 대입하기
  // i를 1씩 증가 시키되 13이 되면 실행문을 돌리지 않고 for 문 빠져나오기

  var htmlSelect = '<select>';

  // option 태그를 반복할 for 문
  for(i = 1; i <= 12; i++) {
    htmlSelect += '<option>' + i + '월</option>';
  }

  htmlSelect += '</select>';

  // document.write로 body에 HTML 태그 출력하기
  document.write(htmlSelect);
</script>
```

반복해서 option 태그 만들기

i에 1을 최초 값으로 넣고 조건을 비교해서 맞으면 실행문을 한번 돌리고, 위로 올라가서 i를 1증가 시키 조건과 맞으면 다시 한번 실행문을 돌립니다. 계속 이렇게 반복하다가 13이 되는 경우 12보다 크므로 조건에 맞지 않으므로 실행문을 돌리지 않고 for 문을 빠져 나옴.
(참고로 var i = 1; 로 해도 됩니다.)

1월 ▼

1월

2월

3월

4월

5월

6월

7월

8월

9월

10월

11월

12월



05) 반복문 for - 스몰 미션

Small Mission) for 문 사용해서 2023년~1950년까지 Select로 body에 태그를 출력

스몰 미션 제작 노트

- ① 변수를 선언하고 변수에 select 시작 태그 출력
- ② option 태그를 반복할 for 문에서 최초에 변수 i에 2023을 대입하기
- ③ i을 1씩 감소 시키되 1950보다 작으면 되면 실행문을 돌리지 않고 for 문 빠져나오기
- ④ select 마감 태그 출력
- ⑤ 최종적으로 document.write로 body에 HTML 태그 출력하기

i에 제일 먼저 2023이 들어가므로 <option>2023년</option>이라는 태그가 출력되고, 반복해서 2023-1 = 2022가 들어가면 i는 1950보다 여전히 크거나 같기 때문에 실행문을 실행해서 <option>2022년</option>라는 태그가 출력되고, 마지막에 실행문을 실행해서 <option>1950년</option>을 출력하고 1950-1 = 1949가 되면 i는 1950보다 크거나 같지 않기 때문에 option 태그를 만드는 실행문을 실행하지 않고 반복문을 빠져 나옴

```
<script>
/* for 문 사용해서 2023년~1950년까지 select로 body에 태그를 출력 */
// select 시작 태그 출력
var ageSelect = '<select>';

// option 태그를 반복할 for 문
// 최초에 변수 i에 2023을 대입하기
// i을 1씩 증가 시키되 1950보다 작으면 실행문을 돌리지 않고 for 문 빠져나오기
for(var i = 2023; i >= 1950; i--) {
    ageSelect += '<option>' + i + '년</option>'
}

// select 마감 태그 출력
ageSelect += '</select>';

// document.write로 body에 HTML 태그 출력하기
document.write(ageSelect);
</script>
```

2023년 ▼

2023년 ▲

2022년

2021년

2020년

2019년

2018년

2017년

2016년

2015년

2014년

2013년

2012년

2011년

2010년

2009년

2008년

2007년

2006년

2005년

2004년 ▼



05) 반복문 for - 스몰 미션(짝수 홀수 반환)

Small Mission) for 문 사용해서 1~10까지 숫자 중에 짝수만 또는 홀수만 반환해서 콘솔창에 출력하기

스몰 미션 제작 노트

- ① 짝수를 반환할 경우 변수를 2부터 시작
- ② 짝수를 반환할 경우 변수를 1부터 시작
- ③ 증감식에서 2씩 증가 시킴

```
<script>
  /* for 문 사용해서 1~10까지 숫자 중에 짝수만 또는 홀수만 반환해서 콘솔창에 출력하기 */

  // 짝수만 반환할 경우 변수 i를 2부터 시작하기
  // 홀수만 반환할 경우 변수 i를 1부터 시작하기
  for(i = 2; i <= 10; i+=2) {
    console.log(i);
  }
</script>
```

```
for(i = 1; i <= 10; i+=2) {
  console.log(i);
}
```

Console	Sources	Recorder	1	Settings	Close
top	Filter	Default levels	1 Issue: 1	Settings	
2	05) for 반복문-04(복합대입연산자-짝수반환).html:20				
4	05) for 반복문-04(복합대입연산자-짝수반환).html:20				
6	05) for 반복문-04(복합대입연산자-짝수반환).html:20				
8	05) for 반복문-04(복합대입연산자-짝수반환).html:20				
10	05) for 반복문-04(복합대입연산자-짝수반환).html:20				

Console	Sources	Recorder	1	Settings	Close
top	Filter	Default levels	1 Issue: 1	Settings	
1	05) for 반복문-04(복합대입연산자-홀수반환).html:20				
3	05) for 반복문-04(복합대입연산자-홀수반환).html:20				
5	05) for 반복문-04(복합대입연산자-홀수반환).html:20				
7	05) for 반복문-04(복합대입연산자-홀수반환).html:20				
9	05) for 반복문-04(복합대입연산자-홀수반환).html:20				



06) 중첩 for 반복문 중첩 for 문은 반복할 것이 2개 이상인 경우에 사용 ex) 학년과 반 반복, 구구단 반복

{ 중첩 for 반복문 기본 형식 }

Small Mission) 중첩 for 문 사용해서 코딩웍스 초등학교 1학년 1반~10반, 6학년 1반~10반을 body에 출력하기

스몰 미션 제작 노트

- ① 2개의 중첩된 for 문 구조를 먼저 만들면 실수 할 수 있으니 단수 먼저 실행 후 확인
- ② 학년 반복이 확인되면 반 반복 하기

```
for(초기값; 조건식; 증감식) {
    반복할 표현식
}

for(초기값; 조건식; 증감식) {
    반복할 표현식
}
```

▲ 중첩된 for 문 내에 초기 값 변수는 i, j, k 로 작성하는 것이 일반적입니다.

undefined
코딩웍스 초등학교 1학년
1학년 1반
1학년 2반
1학년 3반
1학년 4반
1학년 5반

```
<script>
/* 중첩 for 문 사용해서 코딩웍스 초등학교 1학년 1반~5반, 6학년 1반~5반을 body에 출력하기 */

var outputList = '';
// 학년 반복
for(var i = 1; i <= 6; i++) {
    outputList += '<h3>' + '코딩웍스 초등학교 ' + i + '학년</h3>';
    // 반 반복
    for(var j = 1; j <= 5; j++) {
        outputList += i + '학년 ' + j + '반<br>';
    }
}
document.write(outputList);
</script>
```

`var outputList;` → 만약 변수 outputList에 최초에 어떤 값도 할당하지 않으면 undefined(변수에 값이 할당 되지 않은 경우 출력되는 값)가 출력됩니다. 그래서 값이 없는 빈 공간이지만 꼭 위의 예시처럼 하세요.

코딩웍스 초등학교 1학년
1학년 1반
1학년 2반
1학년 3반
1학년 4반
1학년 5반

코딩웍스 초등학교 2학년
2학년 1반
2학년 2반
2학년 3반
2학년 4반
2학년 5반

코딩웍스 초등학교 3학년
3학년 1반
3학년 2반
3학년 3반
3학년 4반
3학년 5반

코딩웍스 초등학교 4학년
4학년 1반
4학년 2반
4학년 3반
4학년 4반
4학년 5반

코딩웍스 초등학교 5학년
5학년 1반
5학년 2반
5학년 3반
5학년 4반
5학년 5반

코딩웍스 초등학교 6학년
6학년 1반
6학년 2반
6학년 3반
6학년 4반
6학년 5반



06) 중첩 반복문 for - 스몰 미션(구구단 출력)

Small Mission) 중첩 for 문 사용해서 구구단을 body에 출력하기

스몰 미션 제작 노트

- ① 2개의 중첩된 for 문 구조를 먼저 만들면 실수 할 수 있으니 단수 먼저 실행 후 확인
- ② 단수 반복이 확인되면 곱해지는 값 반복 하기

```
<script>
/* 중첩 for 문 사용해서 구구단을 body에 출력하기 */

// 최초 변수 선언
googoodan = '';

for(var i = 2; i <= 9; i++) {
    // 단수 출력
    googoodan += '<h3>----- 구구단 ' + i + '단 -----</h3>';
    // 곱해지는 값 출력
    for(var j = 1; j <= 9; j++) {
        googoodan += i + 'x' + j + ' = ' + i * j + '<br>'
    }
}

// 브라우저에 출력
document.write(googoodan);
</script>
```

----- 구구단 2단 -----

2x1 = 2
2x2 = 4
2x3 = 6
2x4 = 8
2x5 = 10
2x6 = 12
2x7 = 14
2x8 = 16
2x9 = 18

----- 구구단 3단 -----

3x1 = 3
3x2 = 6
3x3 = 9
3x4 = 12
3x5 = 15
3x6 = 18
3x7 = 21
3x8 = 24
3x9 = 27

----- 구구단 4단 -----

4x1 = 4
4x2 = 8
4x3 = 12
4x4 = 16
4x5 = 20
4x6 = 24
4x7 = 28
4x8 = 32
4x9 = 36

----- 구구단 5단 -----

5x1 = 5
5x2 = 10
5x3 = 15
5x4 = 20
5x5 = 25
5x6 = 30
5x7 = 35
5x8 = 40
5x9 = 45

----- 구구단 6단 -----

6x1 = 6
6x2 = 12
6x3 = 18
6x4 = 24
6x5 = 30
6x6 = 36
6x7 = 42
6x8 = 48
6x9 = 54

----- 구구단 7단 -----

7x1 = 7
7x2 = 14
7x3 = 21
7x4 = 28
7x5 = 35
7x6 = 42
7x7 = 49
7x8 = 56
7x9 = 63

----- 구구단 8단 -----

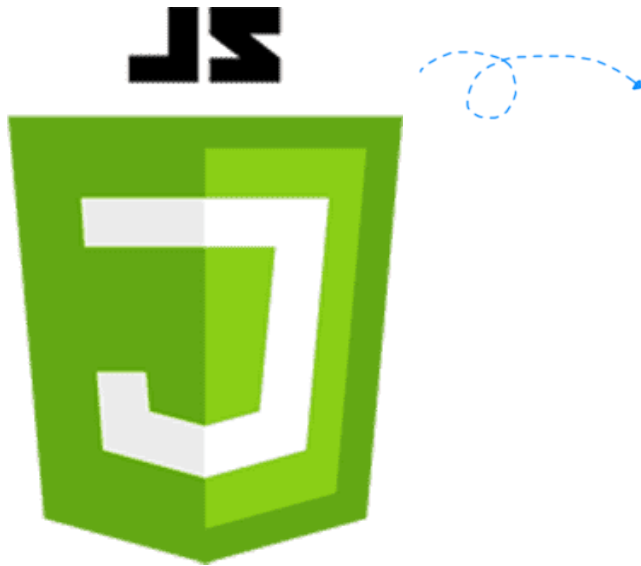
8x1 = 8
8x2 = 16
8x3 = 24
8x4 = 32
8x5 = 40
8x6 = 48
8x7 = 56
8x8 = 64
8x9 = 72

----- 구구단 9단 -----

9x1 = 9
9x2 = 18
9x3 = 27
9x4 = 36
9x5 = 45
9x6 = 54
9x7 = 63
9x8 = 72
9x9 = 81

JAVASCRIPT 핵심이론 커리큘럼

[웹 퍼블리셔를 위한 자바스크립트] 05. 자바스크립트 배열과 for in 문



1. 배열(Array)
2. 배열과 for 문
3. for in 문
4. break 키워드
5. continue 키워드

01) 배열(Array) - 더 쉽게 이해하기 ↪ 배열(array)이란 여러 개의 값을 저장하는 공간 & 규칙이 없는 값에 인덱스 번호로 규칙을 주는 것

{ 배열(array)의 필요성 }



우리 반 학생들 각각을 변수로 만든다면?

```
var Andrew = 'Andrew';  
var Sally = 'Sally';  
var Joel = 'Joel';  
var Ellie = 'Ellie';  
...  
var Peter = 'Peter';
```

- 굉장히 많은 변수를 선언해야 합니다.
- 일일이 변수 이름을 기억해야 합니다.
- 순서도 일정하지 않고 몇 명인지 파악도 어렵습니다.

↑
앞으로 배열 관련 명령어를 통해서 배열을 순서 변경, 추가, 삭제를 할 수 있습니다.

▼ 배열을 탐색할 때는 고유번호를 사용하며, **인덱스(index)**라고 부르고 0부터 시작합니다.

0 1 2 3 9

```
var classMate = ['Andrew', 'Sally', 'Joel', 'Ellie', ... 'Peter'];
```

Code로 표현하면

1번에 Andrew
2번에 Sally
3번에 Joel
4번에 Ellie
...
9번에 Peter

배열에는 문자열 뿐만 아니라 다양한
데이터 유형을 넣을 수 있습니다.

```
var classArray = [  
  '코딩웍스', → 문자열  
  35, → 숫자  
  true, → 불린(Boolean)  
  { → 객체  
    name: 'Andrew',  
    location: 'Seoul'  
  },  
  function(){ → 함수  
    alert('오류가 발생했습니다.')  
  },  
];
```

01) 배열(Array)

{ 배열(array) 기본 형식 }

array

1. 진열하다 2. 다수의 3. 다양한 4. 배치하다 5. 일련의

```
var 배열 이름 = ['값1', '값2', '값3']
```

coma(,) 로 이루어진 부분을 인덱스(index) 라고 하고 자바스크립트에서 인덱스 번호는 항상 0 부터 시작합니다.
곧, 위의 예시에서 값이 3개니까 인덱스 번호는 0, 1, 2 가 됩니다.

- 문자의 경우 따옴표로 묶고 문자가 아닌 경우는 따옴표로 묶지 않음
- 배열을 여러 개 나열한 개수를 반환(확인)하는 length 속성을 활용 (위의 예시에서 값이 3개니까 length는 3이 됩니다.)

Small Mission) 배열을 사용해서 2023년 출시될 AAA 게임 목록을 콘솔창에 출력하기

```
<script>
/* 배열을 사용해서 2023년 출시될 AAA 게임 목록을 콘솔창에 출력하기 */

// 배열 변수 선언
aaaGame = ['칼리스토프로토클', '데드스페이스', '라스트 오브 어스', '바이오하자드4'];

// 배열 전체 출력
console.log(aaaGame); ①

// 배열 중에 특정 인덱스 번호만 출력 ex) 데드스페이스 출력할 경우 인덱스 번호 1
// 배열[인덱스번호]
console.log(aaaGame[0]); ②
console.log(aaaGame[1]); ③
console.log(aaaGame[2]); ④
console.log(aaaGame[3]); ⑤

// 배열의 개수 출력(length 속성)
console.log('2023년 출시될 AAA 게임 목록 개수 : ' + aaaGame.length + '개 타이틀'); ⑥
</script>
```

① ▶ (4) ['칼리스토프로토클', '데드스페이스', '라스트 오브 어스', '바이오하자드4']	01) 자바스크립트 배열-01.html:68
② 칼리스토프로토클	01) 자바스크립트 배열-01.html:72
③ 데드스페이스	01) 자바스크립트 배열-01.html:73
④ 라스트 오브 어스	01) 자바스크립트 배열-01.html:74
⑤ 바이오하자드4	01) 자바스크립트 배열-01.html:75
⑥ 2023년 출시될 AAA 게임 목록 개수 : 4개 타이틀	01) 자바스크립트 배열-01.html:78

02) 배열(Array)과 for 반복문 반복되는 규칙이 없는 것을 반복하기 위해 배열과 for 반복문 사용

{ 배열(array)에서 for 반복문을 사용하는 이유 }

```
<script>
/* 배열을 사용해서 과일 목록을 콘솔창에 출력하기 */

// 배열 변수 선언
fruitList = ['사과', '딸기', '포도', '망고', '오렌지', '바나나', '수박', '파인애플'];

console.log(fruitList[0]);
console.log(fruitList[1]);
console.log(fruitList[2]);
console.log(fruitList[3]);
console.log(fruitList[4]);
console.log(fruitList[5]);
console.log(fruitList[6]);
console.log(fruitList[7]);
</script>
```

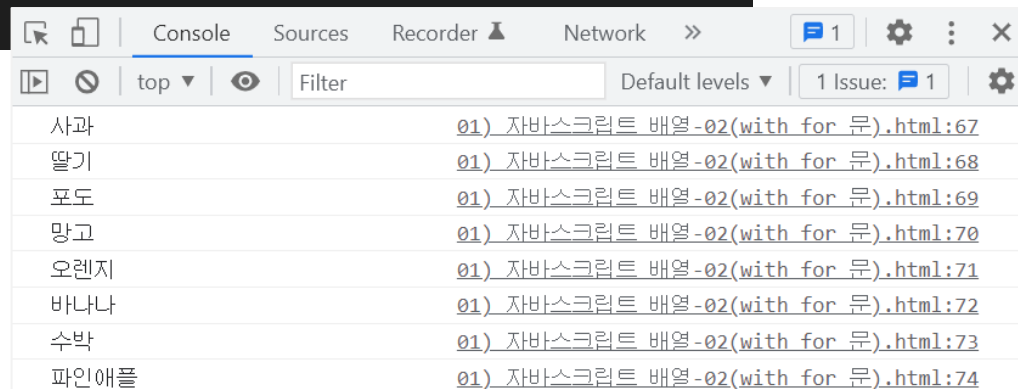
배열 내 인덱스 번호를 수동으로 반복하는건 코드를 중복해서 사용하므로 효율적이지 못합니다. 그래서 배열 인덱스 번호인 0~7까지를 for 반복문을 사용해서 반복하면 효율적이라서 배열+반복문을 사용합니다.
해당 예시는 8개지만 100개 200개 라면 반드시 배열 내 인덱스 번호를 반복해줘야 합니다.

만약 파인애플이라는 값이 없게되면 배열의 개수가 달라 지기 때문에 배열의 개수를 length 속성을 사용해서 유동적으로 받아와야 합니다.

```
// for 반복문을 사용해서 인덱스 번호 호출하기(length 속성 사용)
for(var i = 0; i < fruitList.length; i++) {
    console.log(fruitList[i]);
}
```

```
// for 반복문을 사용해서 인덱스 번호 호출하기
for(var i = 0; i < 8; i++) {
    console.log(fruitList[i]);
}
```

배열에서 for 반복문을 사용해서 인덱스 번호를 호출하면 효율적인 코드가 됩니다.



사과	01) 자바스크립트 배열-02(with for 문).html:67
딸기	01) 자바스크립트 배열-02(with for 문).html:68
포도	01) 자바스크립트 배열-02(with for 문).html:69
망고	01) 자바스크립트 배열-02(with for 문).html:70
오렌지	01) 자바스크립트 배열-02(with for 문).html:71
바나나	01) 자바스크립트 배열-02(with for 문).html:72
수박	01) 자바스크립트 배열-02(with for 문).html:73
파인애플	01) 자바스크립트 배열-02(with for 문).html:74

03) 배열(Array)과 for in 문

{ for in 문 기본 형식 }

```
for(var i in 배열이름) {
    반복할 표현식
}
```

배열이 가지고 있는 인덱스 값들을 변수 i 에
몸땅 넣겠다(in) 라는 의미

Small Mission 01) 배열과 for in 문을 사용해서 휴대폰 번호 앞자리를 select 태그로 body에 출력하기

스몰 미션 제작 노트

- ① select 태그 시작과 마감 태그를 먼저 만들고 브라우저에서 select 태그 출력 확인
- ② 배열과 for in 문으로 option 반복하기

010 ▾

010

011

016

017

018

019

```
<script>
/* 배열과 for in 문을 사용해서 과일 목록을 콘솔창에 출력하기 */

// 배열 변수 선언
fruitList = ['사과', '딸기', '포도', '망고', '오렌지', '바나나', '수박', '파인애플'];

// for in 문을 사용해서 인덱스 번호 호출하기
for(var i in fruitList) {
    console.log(fruitList[i])
}
</script>
```

for in 문을 사용한 경우가 for 반복문을
사용한 경우보다 훨씬 더 간결해진 코드

```
// for 반복문을 사용해서 인덱스 번호 호출하기(length 속성 사용)
for(var i = 0; i < fruitList.length; i++) {
    console.log(fruitList[i]);
}
```

```
<script>
/* 배열과 for in 문을 사용해서 휴대폰 번호 앞자리를 body에 출력하기 */

// 배열 변수 선언(규칙성이 없는 배열)
firstNum = ['010', '011', '016', '017', '018', '019'];

// select 시작 태그 만들기 ①
cellOutput = '<select>'

// option 반복하기 ②
for(var i in firstNum) {
    cellOutput += '<option>' + firstNum[i] + '</option>';
}

// select 마감 태그 만들기 ①
cellOutput += '</select>'

// 문서에 최종 결과물 출력하기 ①
document.write(cellOutput)
</script>
```

firstNum이라는 배열 내에 인덱스 번호를
변수 i로 순서대로 6번 반복함.

firstNum[0] ~ firstNum[5]
<option>010</option> ...

03) 배열(Array)과 for in 문

Small Mission 02) 배열과 for in 문을 사용해서 전국 지역번호를 select 태그로 body에 출력하기

스몰 미션 제작 노트

- ① select 태그 시작과 마감 태그를 먼저 만들고 브라우저에서 select 태그 출력 확인
- ② 배열과 for in 문으로 option 반복하기

서울 02 ▾

서울 02

경기 031

인천 032

강원 033

충남 041

대전 042

충북 043

세종 044

부산 051

울산 052

대구 053

경북 054

경남 055

전남 061

광주 062

전북 063

제주 064

```
<script>
/* 배열과 for in 문을 사용해서 전국 지역번호를 select 태그로 body에 출력하기 */

// 전국 지역번호 배열 변수 선언(규칙성이 없는 배열)
areaCode = ['서울 02', '경기 031', '인천 032', '강원 033', '충남 041', '대전 042', '충북 043', '세종 044',
            '부산 051', '울산 052', '대구 053', '경북 054', '경남 055', '전남 061', '광주 062', '전북 063', '제주 064'];

// select 시작 태그 만들기
areaCodeOutput = '<select>'

// option 반복하기
for(var i in areaCode) {
    areaCodeOutput += '<option>' + areaCode[i] + '</option>';
}

// select 마감 태그 만들기
areaCodeOutput += '</select>'

// 문서에 최종 결과물 출력하기
document.write(areaCodeOutput);
</script>
```

서울 02
경기 031
인천 032
강원 033
충남 041
대전 042
충북 043
세종 044
부산 051
울산 052
대구 053
경북 054
경남 055
전남 061
광주 062
전북 063
제주 064

04) break 키워드 ↪ switch 문과 반복문을 벗어나는 키워드

Small Mission) break 키워드로 반복문 벗어나기 (해당 예시는 break 키워드에 대한 이해도만 가지면 됩니다.)

현재 true는 break 키워드에 대한 예시로 넣은 것이지만, 개발할 경우 조건에 해당 하는 부분이므로 이 부분의 true false에 따라 for 반복문 내의 실행문을 실행할지 여부를 결정합니다.

```
<script>
/* break 키워드로 반복문 벗어나기 */

for(var i = 0; true; i++) {
  var userConfirm = confirm('반복하시겠습니까?'); ①

  if(userConfirm) {
    alert(i + '번째 반복입니다. '); ②
  }
  else {
    break; ③
  }
}
</script>
```

127.0.0.1:5500 내용:
반복하시겠습니까?

확인

취소

127.0.0.1:5500 내용:
0번째 반복입니다.

확인

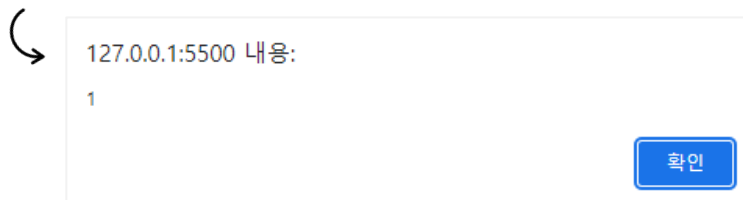
- ① confirm에 의해 true false를 선택합니다. 확인을 누르면 true, 취소를 누르면 false.
 - 확인을 누르면 true가 되므로 if 조건문 실행
 - 취소를 누르면 false가 되므로 if 조건문 실행 안함
- ② 확인을 누르면 true가 되므로 alert 실행 후 위로 올라가서 confirm 반복
- ③ 취소를 누르면 false가 되므로 위로 올라가지 않고 break에 의해 반복문 빠져나옴

05) continue 키워드 ↪ 반복문과 같이 사용되며 여러가지 반복 실행문에서 특정 부분을 스킵하고 이어서(continue) 실행하게 하는 키워드

{ for 반복문 기본 형식 }

```
<script>
/* continue 키워드를 사용해서 경고창을 1~10까지 출력(반환) 하기 */

for(var i =1; i <= 10; i++) {
    alert(i);
}
</script>
```



▲ 경고창에 1~10까지 출력 후 for 반복문 종료

{ continue 키워드 사용한 for 반복문 }

Small Mission)

continue 키워드를 사용해서 경고창을 1~10까지 3의 배수를 제외하고 출력(반환) 하기

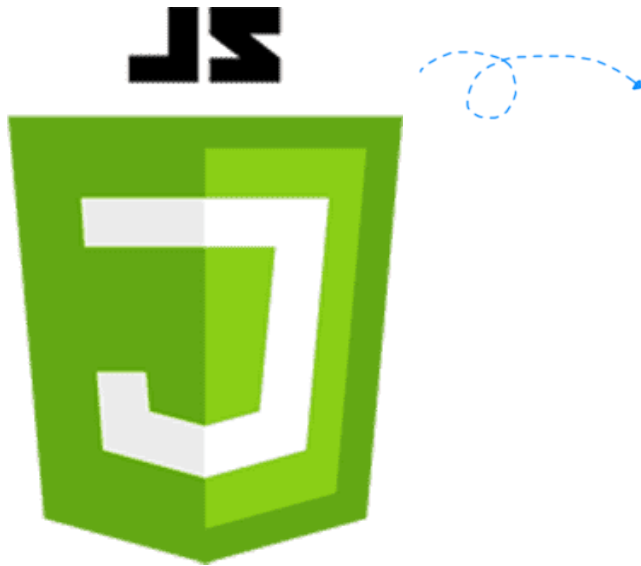
```
<script>
/* continue 키워드를 사용해서 경고창을 1~10까지 3의 배수를 제외하고 출력(반환) 하기 */

for(var i =1; i <= 10; i++) {
    if(i % 3 == 0) { ① → • 홀수만 출력할 경우 i % 2 == 0
        continue; ② • 짝수만 출력할 경우 i % 2 == 1
    }
    else {
        alert(i); ③
    }
}
</script>
```

- ① 3의 배수 확인하는 조건(3으로 나누어 0이 되면 3의 배수인 3, 6, 9가 됩니다.)이 참이면
- ② 3의 배수 확인하는 조건에서 3으로 나누어 0이 되지 않으면 곧, false이면 3) else 구문 실행 후 반복
- ③ 3의 배수 확인하는 조건에서 3으로 나누어 0이 되면 곧, 3의 배수이면 참이 되므로 continue 키워드에 의해서 else로 넘어가지 않고(alert 실행하지 않고) 반복을 다시 합니다.

JAVASCRIPT 핵심이론 커리큘럼

[웹 퍼블리셔를 위한 자바스크립트] 06. 자바스크립트 함수(function)



1. 함수 선언과 호출
2. 매개 변수와 리턴 키워드
3. 변수식 함수 선언
4. 내장 함수
5. 즉시 실행 함수
6. 함수 표현식, 화살표 함수

01) 함수(function)의 선언과 호출



함수(function)란 코드의 집합으로 선언하고 선언된 함수를 호출해서 사용합니다.

함수를 만드는 첫번째 이유는 반복해서 사용되는 코드의 집합을 만들고 필요할 때 호출해서 사용하기 위함.

{ 함수 선언 기본 형식 }

```
function 함수이름() {  
    실행할 코드 집합(code block)  
}
```

{ 함수 호출 기본 형식 }

```
함수이름();
```

```
<script>  
    // 함수 선언  
    function hi() {  
        alert('코딩웍스 온라인 클래스에 오신걸 환영합니다~!!');  
    }  
  
    // 함수 호출  
    hi();  
</script>
```

→ 함수 선언만으로 작동하지는 않고
선언된 함수를 호출하면서 작동

→ 위에 선언된 함수를 즉시 호출해서 alert 를 작동시킴.
즉시 호출이란 브라우저가 로딩되면서 자동으로 함수가
실행되는 것



127.0.0.1:5500 내용:
코딩웍스 온라인 클래스에 오신걸 환영합니다~!!

확인

현재 함수를 아주 단순한 함수이지만 아이디 오류, 비번 오류 등 각종 오류에
대해 웹사이트 전체에서 사용된다고 가정할 때 문구를 수정하는 경우 함수 내
에 실행 구문을 바꿔주면 해당 함수를 사용하는 모든 곳에 반영됩니다.

[참고] 함수 이름 만들기(Naming)

- 함수 이름은 읽기 쉽고 어떤 동작인지 짐작할 수 있게 네이밍합니다.
- ex) `showError` // 에러 출력, `createUserData` // 유저 정보를 생성, `checkLogin` // 로그인 여부 체크

```
<script>  
    // 함수 선언  
    function errorPrint(){  
        alert('에러가 발생했습니다. 다시 시도해주세요.~!!');  
    }  
    // 함수 호출  
    errorPrint();  
</script>
```

01) 함수(function)의 선언과 호출

Small Mission) 함수를 사용해서 단계별 경고창 띄우기

스몰 미션 제작 노트

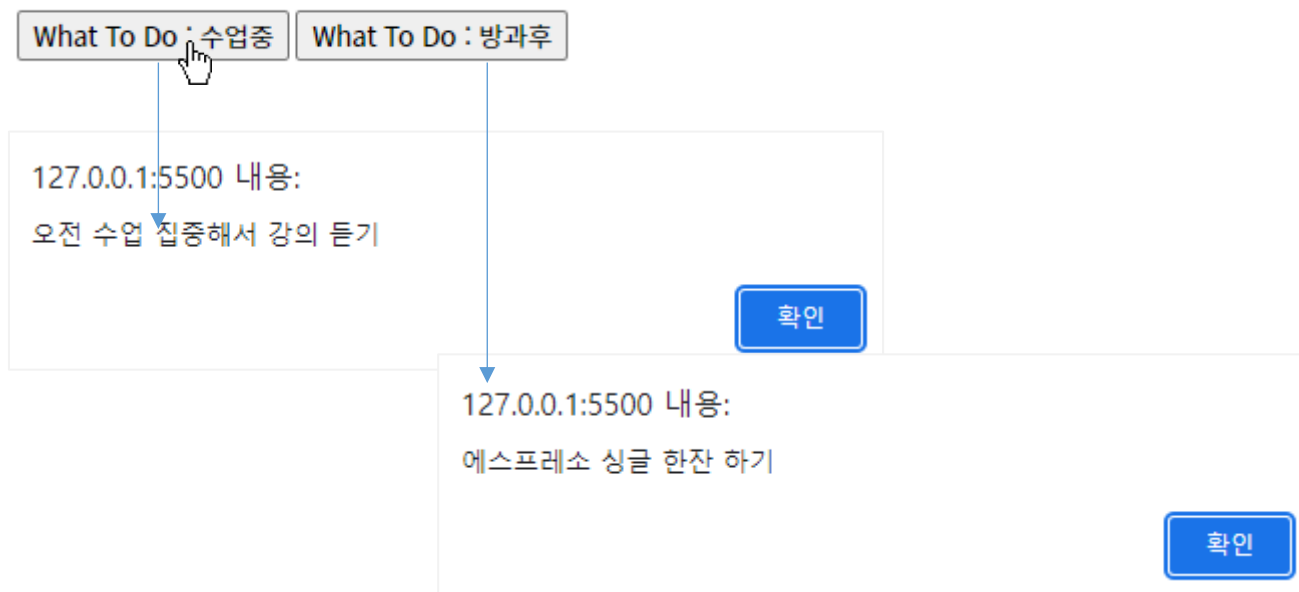
- ① script 안에 함수 선언
- ② 선언된 함수를 버튼을 누를 경우 실행 시킴(버튼 클릭은 **onclick** 으로 함수를 호출함)

```
<script>
// 함수 선언
function atSchool() {
  alert('오전 수업 집중해서 강의 듣기');
  alert('점심 맛있게 먹기');
  alert('오후 수업 집중해서 강의 듣기');
}
function afterSchool() {
  alert('에스프레소 싱글 한잔 하기');
  alert('알바가서 열심히 일하기');
  alert('집에가서 꿈 안꾸고 폭 자기');
}
</script>
```

```
<button onclick="atSchool()">What To Do : 수업중</button>
<button onclick="afterSchool();">What To Do : 방과후</button>
```

▲ 버튼을 누르면 경고창이 위에서 아래로 순차적으로 출력됩니다.

자바스크립트 구문처럼 onclick에서 함수이름 다음에 세미콜론(:)은 필수는 아닙니다.



02) 매개 변수와 리턴 키워드 ↪ 함수(function)를 선언하는 부분과 호출하는 부분의 매개(연결고리)가 되는 변수이며, 파라미터(parameter)라고도 부름

{ 매개변수 함수 선언 기본 형식 }

```
function 함수이름(e) {  
    실행할 코드 집합(code block)  
}
```

{ 매개변수 함수 호출 기본 형식 }

```
함수이름(e);
```

▲ 괄호 안의 문자 e가 매개 변수

매개³ 媒介 +

1. 명사 둘 사이에서 양편의 관계를 맺어 줌.

```
<button value="에스프레소" onclick="order(this.value)">에스프레소</button>  
<button value="아메리카노" onclick="order(this.value)">아메리카노</button>  
<button value="카페라테" onclick="order(this.value)">카페라테</button>  
<button value="카라멜마키야토" onclick="order(this.value)">카라멜마키야토</button>  
  
<script>  
    /* 커피 주문하기 */  
  
    // 커피 주문 함수 선언  
    function order(menu) {  
        alert(menu + '주문이 완료되었습니다.');    }  
</script>
```

this.value란 버튼 자신(this)이 가지고 있는 value 값. 곧, 버튼마다 각자 가지고 있는 value가 다르기 때문에 선언된 함수의 매개 변수 menu와 연결되어 버튼의 value 값을 각각 출력합니다.

menu가 매개변수입니다. 선언된 함수의 매개변수 menu와 함수 호출하는 곳을 매개(연결)하는 변수

```
function plus(x, y) {  
    return x + y;  
}  
  
plus(x, y);
```

▲ 매개 변수는 x y에 어떤 값을 적을지 모르는 말그대로 변수입니다. 곧, x y에 적는 값에 따라 결과가 달라집니다.

127.0.0.1:5500 내용:

에스프레소 주문이 완료되었습니다.

확인

127.0.0.1:5500 내용:

카라멜마키야토 주문이 완료되었습니다.

확인

02) 매개 변수와 리턴 키워드 리턴(return) 키워드는 2가지 경우에 사용되는데, 첫번째는 매개 변수 값을 새롭게 반환하라는 의미로 사용, 두번째는 함수 내 코드 블록을 벗어나라는 의미로 사용됩니다.

{ 리턴 키워드 함수 선언 기본 형식 }

```
function plus(x, y) {
```

```
    return x + y;
```

```
}
```

```
plus(x, y);
```

→ 리턴 뒤에 매개 변수가 있는 경우 매개 변수 x y 를 합해서 새로운 값으로 리턴(반환)하라는 의미

```
function plus(x, y) {
```

```
    return;
```

```
}
```

```
plus(x, y);
```

→ 리턴 뒤에 매개 변수가 없는 경우 함수내 코드 블록을 벗어나라는 의미 곧, 함수를 벗어나서 함수 명령 실행을 멈추라는 의미

Small Mission) 매개 변수와 리턴 키워드를 사용해서 사용자가 입력한 값의 제곱을 구하기

```
<script>
```

```
/* 매개 변수와 리턴 키워드를 사용해서 사용자가 입력한 값의 제곱을 구하기 */
```

```
// 제곱 구하는 함수 선언
```

```
function square(e) {
```

```
    return e * e;
```

```
}
```

```
// 사용자에게 제곱 구할 값 받기
```

```
var sum = prompt('제곱을 구할 값을 입력하세요.', '정수로 입력');
```

```
// 제곱한 결과 출력
```

```
alert('결과 : ' + square(sum));
```

```
</script>
```

매개 변수 e 를 사용해서 e * e 라는 새로운 값을 반환(return) 하기
[참고] return 다음 줄부터 나오는 코드는 절대 실행되지 않습니다.

반환(return)에 의해 새롭게 만들어진 값인 e * e를 sum에 입력됨.
선언된 함수에 return e * e 가 없다면 선언된 함수에 e라는 매개변수는 있는데 호출하는 곳에서는 매개변수 형태로 사용하고 있으므로 square(sum)에 undefined가 출력됨

127.0.0.1:5500 내용:

제곱을 구할 값을 입력하세요.

3

확인

취소

127.0.0.1:5500 내용:

결과 : 9

확인

02) 함수(function) 매개 변수와 리턴 키워드 - 더 쉽게 이해하기

{ 커피 레시피를 만들고 손님마다 다른 커피를 제공하는 함수식 }

```
function coffeeRecipe() {  
    함수가 실행할 실행 구문  
}
```

함수 선언



```
console.log('앤드류 커피 레시피 : ' + andrewCoffee);  
console.log('샐리 커피 레시피 : ' + sallyCoffee);
```

앤드류 커피 레시피 : 1샷, 설탕 없음, 크림 없음
샐리 커피 레시피 : 2샷, 설탕 2스푼, 크림 1스푼

```
function coffeeRecipe(x, y, z) {  
    var yourCoffee = x + ',' + y + ',' + z;  
    return yourCoffee;  
}
```

▲ 예시로 커피 레시피라는 함수를 만들었다고 할 경우 레시피만 만들고 주문하는 사람에 따라 조금씩 다르게 레시피를 만들 수 있습니다. 이런걸 하기 위해 함수를 만들어 사용합니다..

```
var andrewCoffee = coffeeRecipe('1샷', '설탕 없음', '크림 없음');  
var sallyCoffee = coffeeRecipe('2샷', '설탕 2스푼', '크림 1스푼');
```

▲ 1샷, 설탕 없음, 크림 없음, 2샷, 설탕 2스푼 이런걸 **인자(argument)**라고 하고 손님에 따라 커피 레시피 함수를 기본으로 인자를 변화시켜서 다른 결과를 만듭니다.

왼쪽 함수 식을 코드를 말로 풀어보면..

- coffeeRecipe라는 함수를 만들고 나중에 활용하기 위해 **매개변수(Parameter)**라는 것을 괄호 안에 넣어둡니다.
- 매개변수를 활용해서 만든 수식을 yourCoffee라는 변수에 담아 둡니다.
- yourCoffee라는 변수를 반환합니다.

02) 함수(function) 매개 변수와 리턴 키워드 - 더 쉽게 이해하기 스몰미션

Small Mission) 함수와 매개변수를 사용해서 사용자 이름을 로그인 메시지 콘솔창에 출력하기

```
// 함수 선언
function userGreeting(name) {
  var msg = `반갑습니다. ${name}`;
  console.log(msg);
}
// 함수 호출
userGreeting('Andrew'); → 반갑습니다. Andrew
```

```
// 함수 호출
userGreeting(); → 반갑습니다. undefined
```

▲ 매개 변수 없이 함수를 호출하면 name에 undefined 출력

```
// 함수 선언
function userGreeting(name) {
  var msg = '반갑습니다.';
  if(name) {
    msg += ` ${name}`;
  }
  console.log(msg);
}
// 함수 호출
userGreeting(); → 반갑습니다.
userGreeting('Andrew'); → 반갑습니다. Andrew
```

▲ 매개 변수 없이 함수를 호출해도 undefined 출력 안됨

로그인이 안된 상태라면 사용자 이름을 알 수 없으니 사용자 이름을 '고객님'이라는 기본값으로 출력하기

```
function userGreeting(name = '고객님')
```

▲ 매개 변수에 기본값을 넣어두면 매개 변수가 없어도 최소한 고객님이라는 문자열은 출력합니다.

```
// 함수 호출
userGreeting(); → 반갑습니다. 고객님
userGreeting('Andrew'); → 반갑습니다. Andrew
```

[참고] 전역변수(global variable) vs 지역변수(local variable)

- 전역 변수 : 전체 코드에서 어디서든 사용할 수 있는 변수
- 지역 변수 : 해당 함수 내에서만 사용할 수 있는 변수

```
var userName = 'CodingWorks';

function callUserName() {
  var message = 'Welcome'
  console.log(`${message} ${userName}`);
}
```

```
callUserName(); → Welcome CodingWorks
console.log(message);
```

✖ ▶ Uncaught ReferenceError: message is not defined

▲ userName은 전역변수라 어디서든 사용할 수 있지만, message는 함수 내에 선언된 지역변수이기 때문에 해당 함수 밖에서 사용하면 오류를 출력합니다.

03) 변수식 함수 선언

예를 들어 setInterval 이라는 내장 함수를 사용할 경우 구분 해주기 위해 내장 함수를 변수에 담아서 사용 경우
[참고] 변수식 함수 선언은 함수 표현식 선언이라고도 부름

{ 변수식 함수 선언 기본 형식 }

{ 일반 함수 선언 기본 형식 }

```
var 변수이름 = function() {  
    실행할 코드 집합(code block)  
}  
변수이름();
```

VS

```
function 함수이름() {  
    실행할 코드 집합(code block)  
}  
함수이름();
```

◀ 일반적인 함수 선언과 유사하지만 다른 점은 함수의 이름이 없고 함수 전체를 변수에 담아서 변수를 호출해서 사용합니다.
곧, 함수 선언이 변수 형태로 선언되었기 때문에 호출하는 방식도 변수이름으로 호출합니다.
function(e)와 같이 괄호 안에 매개 변수를 사용하는 부분은 동일합니다.

※ 매개 변수를 사용하는 경우 함수 괄호 안에 매개 변수 개수, 리턴의 매개변수 개수, 호출하는 함수 매개변수 개수가 동일해야 합니다. 개수가 맞지 않으면 아무것도 출력하지 못하거나 잘못된 입력으로 인해 계산을 할 수 없음을 나타내는 NaN(Not-A-Number)이 출력됩니다.

Small Mission)

변수식 함수, 매개 변수, 리턴 키워드를 사용해서 사용자가 입력한 값의 곱을 구하기

127.0.0.1:5500 내용:
곱하기 결과 : 25

확인

▲ 사용자가 입력한 첫번째 5, 두번째 5를 곱한 값 출력

```
<script>  
    /* 변수식 함수, 매개 변수, 리턴 키워드를 사용해서 사용자가 입력한 값의 곱을 구하기 */  
  
    // 곱하기 함수 선언  
    var square = function(x, y) {  
        return x * y;  
    }  
    // 사용자가 입력한 값 2개 받기  
    var firstNum = prompt('첫번째 숫자 입력', '정수로 입력');  
    var secondNum = prompt('두번째 숫자 입력', '정수로 입력');  
    // 곱한 결과 출력하기  
    alert('곱하기 결과 : ' + square(firstNum, secondNum));  
</script>
```

매개변수 x 매개변수 y

04) 내장 함수(built-in function) 내장 함수는 자바스크립트가 미리 만들어 놓은 함수로 호출만 해서 사용하는 함수. 대표적으로 alert() prompt() confirm() 등 엄청 많습니다.

{ 자바스크립트 내장함수 : 타이머 함수 종류 }

- `setTimeout()` : 지정된 시간이 지난 후 한번 실행하는 함수
- `setInterval()` : 지정된 시간마다 계속 실행하는 함수
- `clearTimeout()` : `setTimeout()` 함수를 중지시키는 함수
- `clearInterval()` : `setInterval()` 함수를 중지시키는 함수

ex) 3초라면 3000으로 표기

`setTimeout(콜백 함수, 시간, 인수);`
`setInterval(콜백 함수, 시간, 인수);`

▲ Call Back 함수란 다른 함수가 실행을 끝낸 뒤 실행되는 callback 되는 함수를 말한다.

다른 의미로 함수 내부에 함수를 사용하는 경우를 콜백 함수라고 부름.

쉽게 생각해서 메서드 괄호 안에 function으로 선언되는 함수를 콜백 함수라고 부릅니다.

함수이름(function() {
실행할 코드 집합(code block)
});

`setTimeout(function() {
실행할 코드 집합(code block)
}, 시간);`

Small Mission)

내장 함수 `setTimeout`을 사용해서 2초가 지나면 body에 '2초가 지났습니다' 라고 출력하기

```
<script>  
/*  
내장 함수 setTimeout을 사용해서 2초가 지나면  
body에 '2초가 지났습니다' 라고 출력하기  
*/  
  
// 기본형식 : setTimeout(함수, 시간);  
setTimeout(function(){  
    document.write('2초가 지났습니다.')}, 2000)  
</script>
```

```
function showMsg(msg) {  
    document.write(msg);  
}  
setTimeout(showMsg, 3000, '2초가 지났습니다.');
```

▲ showMsg라는 특정 함수를 만들고 필요할 때 호출하는 형태로도 사용 가능

04) 내장 함수(built-in function) - setInterval & setTimeout

Small Mission) 사용자가 접속하고 얼마나 시간이 경과했는지 보여주기(단, 5초 후 시간 경과 보여주기 멈추기)

스몰 미션 제작 노트

- ① 변수 선언
- ② 메시지 출력 함수 만들기
- ③ setInterval로 1초 마다 메시지 출력 함수 반복
- ④ clearInterval로 5초 후 메시지 출력 함수 종료하고 자동 로그아웃 되었다는 경고창 출력

```
// 변수 선언
var num = 1;

// 메시지 출력 함수 만들기
function showTime() {
  console.log(`로그인 상태가 ${num++}초 경과했습니다.`);
}

// setInterval로 1초 마다 메시지 출력 함수 반복
setInterval(showTime, 1000);
```

로그인 상태가 0초 경과했습니다.
로그인 상태가 1초 경과했습니다.
로그인 상태가 2초 경과했습니다.
로그인 상태가 3초 경과했습니다.
로그인 상태가 4초 경과했습니다.
로그인 상태가 5초 경과했습니다.
로그인 상태가 6초 경과했습니다.

```
// 변수 선언
var num = 1;

// 메시지 출력 함수 만들기
function showTime() {
  console.log(`로그인 상태가 ${num++}초 경과했습니다.`);
  if(num > 5) {
    clearInterval(timeOut);
    alert('자동 로그아웃 되었습니다.');
```

로그인 상태가 1초 경과했습니다.
로그인 상태가 2초 경과했습니다.
로그인 상태가 3초 경과했습니다.
로그인 상태가 4초 경과했습니다.
로그인 상태가 5초 경과했습니다.

127.0.0.1:5500 내용:
자동 로그아웃 되었습니다.

확인

04) 내장 함수(built-in function) - setInterval & setTimeout

Small Mission) 내장 함수 `setInterval`을 사용해서 1초마다 body에 '1초가 경과했습니다' 라고 1초 단위로 body에 계속 출력하기

스몰 미션 제작 노트

- ① `setInterval` 기본 형식을 먼저 만들기
- ② 실행 구문 작성하기

```
<script>
/* 내장 함수 setInterval을 사용해서 1초마다 body에 '1초가 지났습니다' 라고
1초단위로 body에 계속 출력하기 */

// 초기값 변수
var secondPass = 1;

// 기본형식 : setInterval(함수, 시간);
setInterval(function(){
    document.write(secondPass + '초가 경과했습니다.<br>');
    // 초기값을 1씩 증가
    secondPass ++;
}, 1000)
</script>
```

1초가 경과했습니다.
2초가 경과했습니다.
3초가 경과했습니다.
4초가 경과했습니다.
5초가 경과했습니다.
6초가 경과했습니다.
7초가 경과했습니다.
8초가 경과했습니다.
9초가 경과했습니다.
10초가 경과했습니다.
11초가 경과했습니다.
12초가 경과했습니다.

{ 2초 경과로 출력하는 경우 }

```
// 초기값 변수
var secondPass = 2;
```

```
// 기본형식 : setInterval(함수, 시간);
setInterval(function(){
    document.write(secondPass + '초가 경과했습니다.<br>');
    // 초기값을 2씩 증가
    secondPass += 2;
}, 2000)
```

▲ 초기값은 2로 시작하고, setInterval 함수 내 시간은 2000으로 하고, 변수에 2를 계속 더해 나갑니다. 10초 마다 하는 경우도 동일한 원리로 하면 됩니다..

04) 내장 함수(built-in function) - setInterval & setTimeout

Small Mission)

내장 함수 `setInterval`을 변수식 함수로 사용해서 1초마다 body에 '1초가 경과했습니다' 라고 1초 단위로 body에 계속 출력하기하고 10초 후 `setInterval` 함수를 멈추기

스몰 미션 제작 노트

- ① 초기값 변수 선언하기
- ② `setInterval` 기본 형식을 먼저 만들기
- ③ `setInterval` 함수를 변수에 담기
- ④ 10초 지난 후 `setInterval` 함수를 멈추는 `clearInterval` 사용하기

`setInterval` 내장 함수를 변수에 담아서 사용하는 이유는 HTML 문서에 `setInterval` 내장 함수를 많이 사용하기 때문에 중복을 피하기 위해서 입니다.
(ex. 슬라이드 제작에서 `setInterval` 내장 함수 사용)

`setInterval` 내장 함수를 변수에 담았기 때문에 타이머 함수를 멈추는 `clearInterval` 도 변수를 사용해야 합니다.

```
// 타이머 함수 멈추기
setTimeout(function(){
    clearInterval(timer);
    alert('10초가 경과했습니다.')
}, 10000);
```

→ 10초 경과를 alert로 출력할 경우

```
<script>
/*
내장 함수 setInterval을 변수식 함수로 사용해서 1초마다 body에 '1초가 경과했습니다' 라고
1초 단위로 body에 계속 출력하기하고 10초 후 setInterval 함수를 멈추기
*/

// 초기값 변수 선언하기
var secondPass = 1;

// 타이머 함수 선언하고 변수에 담기
var timer = setInterval(function(){
    document.write(secondPass + '초가 지났습니다.<br>');
    // 1초씩 증가시키기
    secondPass++;
}, 1000);

// 타이머 함수 멈추기
setTimeout(function(){
    clearInterval(timer);
}, 10000);
</script>
```

- 10초(10000) 경과 후 타이머함수를 멈추는 `setTimeout` 함
- `setTimeout` 함수 내에 실행할 내용은 위에 선언된 타이머 함수를 `clearInterval` 함

04) 내장 함수(built-in function) - parseInt(문자열) ↪ 문자열에서 숫자만 정수로 반환하는 내장 함수 ex) 100달러.. 라고 입력해도 100이라는 숫자만 가져옴

Small Mission) 내장 함수 parseInt를 사용해서 환전 금액을 출력하기(1)

```
<script>
/* 내장 함수 parseInt를 사용해서 환전 금액을 출력하기 */

// 달러 환율 변수
var exchangeRate = 1247;

// 사용자에게 달러 입력 받기
var dollar = prompt('환율을 계산할 달러를 입력하세요.', '정수로 입력');

// 계산된 환율 출력하기
alert('환전 금액은 ' + (dollar * exchangeRate));
</script>
```

127.0.0.1:5500 내용:
환전 금액은 62350원입니다.

확인

만약, 사용자가 50달러 또는 \$50이라고 입력하는 경우 해당 NaN 오류가 납니다.

127.0.0.1:5500 내용:
환전 금액은 NaN원입니다.

확인

▲ 문자열은 계산을 할 수 없으므로 NaN(Not-A-Number)이 출력됩니다. 그래서 parseInt 함수로 문자열 내에 숫자만 가져와야 합니다.

```
// 계산된 환율 출력하기
alert('환전 금액은 ' + (parseInt(dollar) * exchangeRate));
```

50달러 또는 \$50... 이라고 입력해도 parseInt 함수가 문자열에서 숫자인 50이라는 숫자만 가져와서 계산함

04) 내장 함수(built-in function) - isNaN(수식) ↪ 수식이 NaN이면 참(true) 그렇지 않으면 거짓(false)

Small Mission) 내장 함수 `parseInt`를 사용해서 환전 금액을 출력하기(2)

```
<script>
/* 내장 함수 parseInt를 사용해서 환전 금액을 출력하기 */

// 달러 환율 변수
var exchangeRate = 1247;

// 사용자에게 달러 입력 받기
var dollar = prompt('환율을 계산할 달러를 입력하세요.', '정수로 입력');

// NaN인지 아닌지 if 조건문으로 체크하기
if(isNaN(dollar)) { → dollar가 NaN인지 isNaN 함수가 체크해서 NaN이면 참(true)이니까 if 실행문 실행
    alert('환전할 금액을 정수로 다시 입력하세요. ');
}
else { → dollar가 NaN이 아니라면 곧, 정상적으로 입력했다면 거짓(false)이니까 else 실행문 실행
    // 계산된 환율 출력하기
    alert('환전 금액은 ' + (dollar * exchangeRate) + '원입니다. ');
}
</script>
```

127.0.0.1:5500 내용:
환전 금액은 62350원입니다.

확인

만약, 사용자가 **오십달러**라고 입력하는 경우 해당 **NaN** 오류가 납니다.

127.0.0.1:5500 내용:
환전 금액은 NaN원입니다.

확인

▲ 문자열은 계산을 할 수 없으므로 **NaN**(Not-A-Number)이 출력됩니다. 그래서 `isNaN` 함수로 NaN인지 체크해서 만약 NaN이면 경고창으로 다시 입력하라고 출력

127.0.0.1:5500 내용:
환율을 계산할 달러를 입력하세요.

오십달러

127.0.0.1:5500 내용:
환전할 금액을 정수로 다시 입력하세요.

확인

05) 즉시 실행 함수(function) 즉시 실행 함수는 선언과 호출을 동시에 가지고 있어서 선언과 실행이 즉시 이루어집니다.

{ 즉시 실행 함수 선언 기본 형식 } { 일반 함수 선언 기본 형식 }

```
(function(){
```

실행할 코드 집합(code block)

```
})(...);
```

VS

```
function(){
```

실행할 코드 집합(code block)

```
};
```

(함수 선언)(함수 호출);

127.0.0.1:5500 내용:

경고창2

확인

```
<script>
```

```
// 일반 함수 선언
```

```
function sample(){  
  alert('경고창1');  
}
```

일반 함수로 선언만 되어있고 호출이 없기 때문에 자동으로 실행되지 않음

```
// 즉시 실행 함수 선언 & 호출
```

```
(function(){  
  alert('경고창2');  
})();
```

호출하지 않아도 자동으로 실행되는 즉시 실행 함수

```
</script>
```

06) 화살표 함수 화살표 함수(Arrow Function) 현재까지 작성하던 함수 방식을 좀 더 간결하게 작성하는 함수 작성

{ 기존 함수 선언 기본 형식 }

```
var 변수이름 = function(num1, num2) {  
    return num1 + num2;  
}  
  
변수이름();
```

```
// 실행 구문이 한 줄인 경우  
var add = (num1, num2) => {  
    return num1 + num2;  
}
```

함수 내 코드 구문이 한 줄인 경우 중괄호 {} 를 소괄호 ()로 변경 가능하고, 또 실행 구문의 괄호도 생략 가능

```
var add = (num1, num2) => (  
    num1 + num2  
)
```

```
var add = (num1, num2) => num1 + num2;
```

{ 화살표 함수 선언 기본 형식 }

```
var 변수이름 = (num1, num2) => {  
    return num1 + num2;  
}  
  
변수이름();
```

기존 함수 선언에서 function을 =>로 바꿈

```
// 실행 구문이 한 줄이고 매개변수가 한 개인 경우  
var userData = (name) => {  
    return name;  
}
```

(

```
var userData = (name) => (name);  
  
var userData = name => name;
```

[참고] 화살표 함수를 반드시 사용해야 하는 건 아닙니다. 하지만 학습할 경우에도 학습 자료 코드가 화살표 함수로 되어 있거나, 개발자와 협업할 때도 필요하기 때문에 어떻게 사용하는지에 대한 이해도는 충분히 갖고 있어야 합니다.

```
// 매개변수(인자)가 없는 함수의 경우  
var checkLogin = () => {  
    alert('Error!!');  
}
```

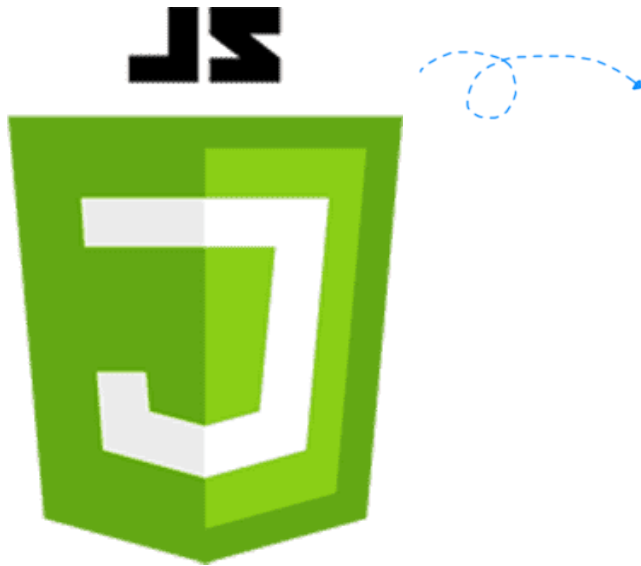
매개변수가 없는 경우 소괄호()를 생략할 수 없음

```
// 실행코드가 여러 줄인 경우  
var checkLogin = (name, age) => {  
    var userInfo = `${name}, ${age}`;  
    return result;  
}
```

여러 줄의 코드가 있는 경우 소괄호() 사용할 수 없음

JAVASCRIPT 핵심이론 커리큘럼

[웹 퍼블리셔를 위한 자바스크립트] 07. 자바스크립트 객체와 생성자 함수



1. 객체의 종류
2. 객체의 선언과 호출
3. 변수, 배열, 객체 개념 비교
4. this 키워드
5. 생성자 함수와 프로토타입(Prototype)

01) 객체의 종류 - 속성 호출, 메서드 호출

기본 내장 객체

- ✓ String : 문자 객체
- ✓ Number : 숫자 객체
- ✓ Math : 수학 객체
- ✓ Array : 배열 객체
- ✓ Date : 날짜 객체
- ✓ object : 개발자 객체

BOM(Browser Object Model)

- ✓ window : 창 객체
- ✓ screen : 화면 객체
- ✓ location : 주소 표시줄 객체
- ✓ history : 주소 내역 객체
- ✓ navigator : 브라우저 객체
- ✓ document : 문서 객체

DOM(Document Object Model)

- ✓ element : HTML 태그 요소
- ✓ text : 글자 요소
- ✓ etc : 그 외

문법이 다른 비슷한 형태로 속성과 값을 나열합니다.

객체 기본 형식 쉽게 이해하기

```

```

▲ HTML 태그의 속성 나열 구조

```
var logoImage = {  
  src: 'images/logo.png',  
  width: 400,  
  height: 200,  
  alt: '로고'  
}
```

▲ JavaScript의 객체 나열 구조

02) 객체의 선언과 호출 - 객체의 키 값에 사용되는 속성과 메서드 선언과 개념

{ 객체 선언 기본 형식 }

속성(Properties)이란?

객체가 가지고 있는 고유한 특징을 기술한 것(함수가 아닌 것들)

메서드(Method)란?

객체가 가지고 있는 동작 또는 객체에 주는 명령(대표적으로 함수)

Small Mission)

자동차 정보와 작동을 예시로 속성과 메세드 개념 구분하기

```
<script>
  var car = {
    name: '쉐보레 말리부 F/L (9세대)',
    fuel: '가솔린',
    efficiency: 14.2,
    color: ['red', 'green', 'blue'],
    starting: function(e){
      if(e == 'on') {
        alert('자동차 시동이 켜져 있습니다.');      }
      else if(e == 'off') {
        alert('자동차 시동이 꺼져 있습니다.');      }
    }
  }
</script>
```

속성 : 값이 함수가 아닌 유형

메서드 : 값이 함수인 유형

함수

name : Scarlett Johansson
age : 38
height : 160
weight : 61



[객체(object) 생성 - 더 쉽게 이해하기]

- 블랙 위도우 배역을 객체로 생성해 봅시다.
- 블랙 위도우라는 객체를 만들고 해당 배역의 정보를 한 곳에 모아놓고 일관성 있게 관리하고 활용합니다.

```
var BlackWidow = {
  name : 'Scarlett Johansson',
  age : 38,
  height : 160,
  weight : 61
}
```

```
var 객체이름 = {
  키이름: 키값,
  키이름: 키값
}
```

키이름과 값이 여러 개 일 경우 반드시 콤마(,)로 이어가야 함

```
var myself = {
  name: 'CodingWorks',
  age: 33,
  gender: 'male',
  location: 'seoul',
  occupation: 'Web Publisher'
}
```



02) 객체의 선언과 호출 - 속성 호출, 메서드 호출 호출이란 객체에 선언된 특정한 키 이름과 키 값을 불러오는 것

```
<script>
var car = {
  name: '쉐보레 말리부 F/L (9세대)',
  fuel: '가솔린',
  efficiency: 14.2,
  color: ['red', 'green', 'blue'],
  starting: function(e){
    if(e == 'on') {
      alert('자동차 시동이 켜져 있습니다.');
```

{ 속성 호출 기본 형식 }

객체.속성이름 → 일반적으로 많이 쓰임
속성이름 => 키이름
객체[속성이름]

{ 메서드 호출 기본 형식 }

객체.메서드이름()

▲ 메서드 이름, 속성 이름은 곧, 객체가 포함하고 있는 키 이름과 같은데 유형에 따라 2가지로 구분해서 다르게 부르는 것 입니다.

```
// 메서드 호출 : 객체.메서드이름()
console.log(car.starting('on'));
```

127.0.0.1:5500 내용:
자동차 시동이 켜져 있습니다.

확인

```
// 속성 호출 : 객체.속성이름
console.log(car.name);
console.log(car.fuel);
console.log(car.efficiency);
console.log(car.color); → 배열 전체 호출
console.log(car.color[0]); → 배열 중 특정한 인덱스 호출
console.log(car.color[1]); →
console.log(car.color[2]); →
```

쉐보레 말리부 F/L (9세대)
가솔린
14.2
▶ (3) ['red', 'green', 'blue']
red
green
blue

02) 객체의 선언과 호출 - 더 쉽게 이해하기

조각난 정보들

```
var head = 1;
var shoulder = 2;
var chest = 1;
var hip = 1;
var arm = 2;
var leg = 2;
var language = true;
```

특정 대상의 특성(속성)이 2개 이상
이면 객체로 만들어 활용합니다.

```
var human = {
  head : 1,
  shoulder : 2,
  chest : 1,
  hip : 1,
  arm : 2,
  leg : 2,
  language : true
}
```

조각난 정보들을 모아서 하나의 이름
을 부여하고 모아 놓으면 이것 객체
(object)라고 합니다.

var andrew = {

```
  gender: 'male',
  cute : true,
  job   : 'instructor',
  age   : 30
}
```

key value
속성(Properties)



```
console.log(andrew);
console.log(andrew.job);
```

```
▶ {gender: 'male', cute: true, job: 'instructor', age: 30}
instructor
```

▲ 객체 전체를 호출하거나 객체 내의 특정 key를 호출할 수도 있습니다.

▼ 객체 안에 또 다른 객체들을 포함시킬 수 있습니다.

```
var student = {
  andrew : {
    gender: 'male',
    cute: true,
    job: 'instructor',
    age: 30
  },
  sally : {
    gender: 'female',
    cute: true,
    job: 'designer',
    age: 27
  }
}
```

```
console.log(student.sally.cute); → true
```

▼ 객체 안에 또 다른 객체의 속성을 추가할 수 있습니다. ex) 지역 추가

```
student.andrew.location = 'seoul';
console.log(student.andrew);
```

```
▶ {gender: 'male', cute: true, job: 'instructor', age: 30, location: 'seoul'}
```



02) 객체의 선언과 호출 - 더 쉽게 이해하기 스몰미션(1)

```
// 블랙위도우 객체 생성
var BlackWidow = {
  name : 'Scarlett Johansson',
  age : 38,
  height : 160,
  weight : 61
}

// 객체 전체 출력
console.log(BlackWidow);

// 객체 내 개별적인 키이름과 키값을 출력
console.log(BlackWidow.name);
console.log(BlackWidow['age']);

// 객체 내 키이름과 키값 추가하고 객체 출력해서 확인하기
BlackWidow.marriageStatus = '기혼';
BlackWidow['nationality'] = '미국';
console.log(BlackWidow);

// 객체 내 키이름과 키값 삭제하고 객체 출력해서 확인하기
delete BlackWidow.weight;
delete BlackWidow.height;
console.log(BlackWidow);
```

Small Mission)

블랙 위도우라는 배역을 객체로 만들고 해당 객체를 콘솔창에 출력하고 추가하고 삭제하기

```
age: 38
height: 160
name: "Scarlett Johansson"
weight: 61
```

```
Scarlett Johansson
38
```

```
age: 38
height: 160
marriageStatus: "기혼"
name: "Scarlett Johansson"
nationality: "미국"
weight: 61
```

```
age: 38
marriageStatus: "기혼"
name: "Scarlett Johansson"
nationality: "미국"
```

02) 객체의 선언과 호출 - 더 쉽게 이해하기 스몰미션(2) 이름과 나이를 받아서 객체를 반환하는 함수 만들기

Small Mission) 마블 영화에 출연하는 배역 별로 정보를 받아서 리턴하는 객체를 만드는 함수를 만들고 배역 별 콘솔창에 출력하기

```
// 마블 영화 출연 배역 함수 만들기
function makeMarvelActor(name, age, height, weight, marriageStatus, nationality) {
  return {
    name,  name: name을 축약형으로 사용할 수 있음
    age,
    height,
    weight,
    marriageStatus,
    nationality
  }
}

var spiderMan = makeMarvelActor('Thomas Holland', 26, 171, 65, '미혼', '영국');
console.log(spiderMan);

var NickFury = makeMarvelActor('Samuel Leroy Jackson', 74, 189, '모름', '기혼', '미국');
console.log(NickFury);
```

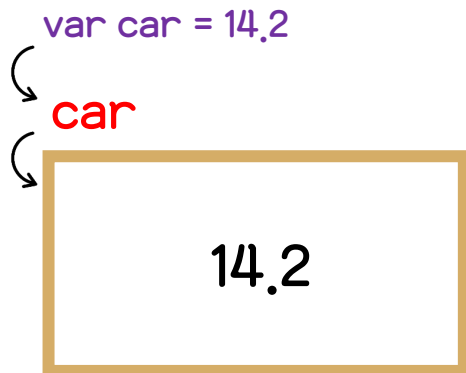
```
age: 26
height: 171
marriageStatus: "미혼"
name: "Thomas Holland"
nationality: "영국"
weight: 65
```

```
age: 74
height: 189
marriageStatus: "기혼"
name: "Samuel Leroy Jackson"
nationality: "미국"
weight: "모름"
```

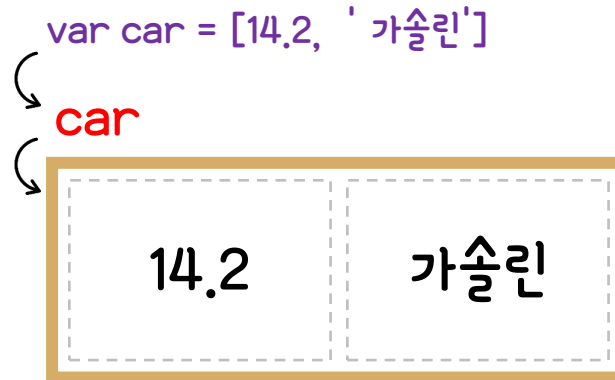
03) 변수, 배열, 객체 개념 비교

{ 데이터를 저장하는 방법 }

- 변수(Variables) : 데이터를 **한 개** 저장하는 공간
- 배열(Array) : 데이터를 **여러 개** 저장하는 공간
- 객체(Object) : 데이터를 **표 형태로 집합적으로** 저장하는 공간

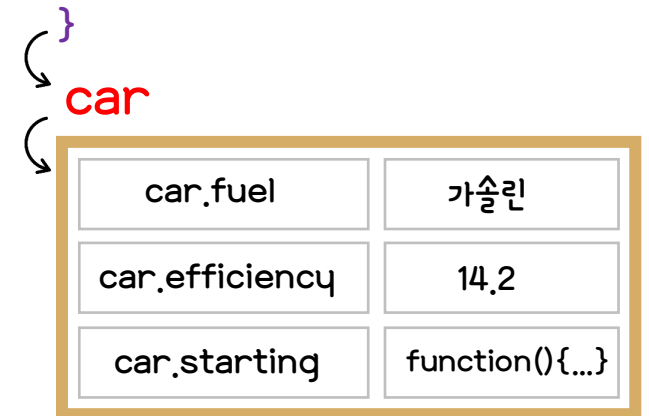


▲ **변수** car에 14.2을 담으면 14.2 한 개가 공간에 저장
변수 num을 호출



▲ **배열** car에 14.2과 가솔린을 담으면 14.2과 가솔린을 저장하는 여러 개 공간을 변수내 가상으로 나누어 저장
배열 num을 호출하거나 num의 인덱스 번호를 호출

```
var car = {  
  fuel: '가솔린',  
  efficiency: 14.2,  
  starting: function(){...}  
}
```



▲ **객체** car에 키 이름과 키 값을 **표(Table) 형태로 집합적**을 저장합니다. 배열과 다른 점은 숫자, 문자 뿐만 아니라 함수와 같은 메서드도 저장하고 호출해서 사용할 수 있습니다.

객체 num을 호출하거나 num의 키 이름을 호출해서 사용. 배열은 인덱스 번호라서 시인성이 떨어지지만 객체는 키 이름을 호출하기 때문에 시인성이 좋습니다.

04) 함수에서 Method & this 키워드 함수에서 메서드(method)는 객체 프로퍼티에 할당된 함수를 말합니다.

Small Mission) 마블 영화에 출연하는 배역 중 블랙위도우의 특수 능력을 출력하기

스몰 미션 제작 노트

- ① 객체를 생성하는 방법은 동일합니다.
- ② 객체 내 키이름과 키값을 함수로 만들고 해당 함수를 메서드로 호출합니다.

```
// 블랙위도우 객체 생성
var BlackWidow = {
  name : 'Scarlett Johansson',
  age : 38,
  height : 160,
  weight : 61,
  marriageStatus: '기혼',
  nationality : '미국',
  debut : function() {
    console.log(`${BlackWidow.name}은 1994년 영화 'North'에서 '롤라 넬슨' 역으로 데뷔`)
  }
}

// 객체에 생성된 메서드 호출
BlackWidow.debut();
```

 `this.name` 으로 사용하는 것이 좋습니다. 혹시라도 BlackWidow라는 객체명이 변경되더라도 this를 사용하면 이 부분을 바꿔줄 필요가 없습니다.

```
debut() {
  // 실행구문
}
```

축약형으로 function 생략하고 작성 가능

Scarlett Johansson은 1994년 영화 'North'에서 '롤라 넬슨' 역으로 데뷔



04) 함수에서 Method & this 키워드 this 키워드는 해당 함수의 '소유자'를 뜻 함.

Small Mission) 객체를 선언하고 선언된 객체의 메서드를 호출하기

```
<script>
// 객체 선언
var user = {
  name: '코딩웍스',
  division: '지식공유자',
  skill: '웹 퍼블리싱 강의',
  rating: 4.95,
  trainee: 7430,
  greeting: function(){
    alert('안녕하세요. ' + user.skill + ' ' + user.division + ' ' + user.name + '입니다.');
```

user.skill 처럼 객체의 속성을 사용할 수 있습니다. 하지만 상황에 따라 객체 이름이 변경되면 user 라고 사용된 부분을 모두 바꿔줘야 합니다. 하지만 this 키워드를 사용하면 이런 경우에도 변경해줄 필요가 없습니다. 그래서 해당 객체를 의미하는 this 키워드를 사용합니다.

this 라는 의미는 객체 내에서 객체에 있는 키 이름을 호출한다.. 입니다.

```
    alert('안녕하세요. ' + this.skill + ' ' + this.division + ' ' + this.name + '입니다.');
```

▲ 변수나 배열 보다 객체로 선언하는 것은 위의 예시처럼 사용자 정보를 일원화 되게 관리할 수 있고, 필요한 경우 호출할 때 키 이름을 사용하므로 기억하기 좋습니다.



127.0.0.1:5500 내용:

안녕하세요. 웹 퍼블리싱 강의 지식공유자 코딩웍스입니다.

확인



04) 함수에서 Method & this 키워드 - 더 쉽게 이해하기 화살표 함수를 사용하는 경우 화살표 함수 내부에 있는 this는 외부에서 값을 가져오는 this

Small Mission 01) 객체 마다 this를 사용해서 객체명이 변경되도 메서드 내에 객체명을 사용하지 않는 효율적인 코드로 만들기

```
// 객체 선언(this를 사용하지 않은 경우)
var userMale = {
  name: 'Andrew',
  welcomeMsg: function(){
    console.log(`반갑습니다. ${userMale.name} 님`);
  }
}

var userFemale = {
  name: 'Selina',
  welcomeMsg: function(){
    console.log(`반갑습니다. ${userFemale.name} 님`);
  }
}

// 객체 키&키값 활용
userMale.welcomeMsg();
userFemale.welcomeMsg();
```

객체명 변경에도 상관없게 하기 위해서 this 사용하면 공통적인 코드를 사용할 수 있습니다.

반갑습니다. Andrew 님
반갑습니다. Selina 님

```
console.log(`반갑습니다. ${this.name} 님`);
```

▲ this는 객체이름인 userMale 또는 userFemale을 의미합니다.

Small Mission 02) 왼쪽 예제에서 함수 welcomeMsg()를 따로 만들어서 메서드를 짧게 호출해서 사용하기

```
// 객체 선언(this를 사용하지 않은 경우)
var userMale = {
  name: 'Andrew',
  welcomeMsg: welcomeMsg
}

var userFemale = {
  name: 'Selina',
  welcomeMsg: welcomeMsg
}

// 객체 내 메서드로 불러올 함수
function welcomeMsg(){
  console.log(`반갑습니다. ${this.name} 님`);
}

// 객체 키&키값 활용
userMale.welcomeMsg();
userFemale.welcomeMsg();
```

welcomeMsg: welcomeMsg 를 축약형으로 welcomeMsg 만 사용할 수 있음

여러 개 객체에서 공통적으로 사용할 함수

반갑습니다. Andrew 님
반갑습니다. Selina 님

05) 생성자 함수(function) - 더 쉽게 이해하기

{ 생성자 함수(function) 란? } → 생성자 함수는 비슷한 객체를 여러 개 만들어야 하는 경우에 사용됩니다.
ex) 회원정보, 상품정보 등

개별적인 객체로 만든다면..

```
var userData = {  
  name: '홍길동',  
  age: 35,  
  userId: 'Andrew777',  
  userPw: '123456789~',  
  grade: 'VIP',  
  joinDay: '2022-10-31'  
}
```

예시로 고객에 따라 몇가지 변경된 객체로 자주 사용되는 '고객 기본정보'라는 객체를 만듭니다. 그런데 고객이 많은데 이런 객체를 고객마다 매번 다시 만들면 매우 번거롭고 관리도 어렵습니다. 그래서 이런 경우같이 재사용이 필요한 경우에 생성자 함수로 만들어 사용합니다.

생성자 함수는 붕어빵 틀이고, 매개변수인 괄호 안의 내용 필요한 재료 목록입니다.
그리고 실제로 만들 때는 재료 목록에 맞게 반죽, 팔 등 실제 재료들을 넣습니다.

생성자 함수로 만든다면..

```
function User(name, age, userId, userPw, grade, joinDay){  
  this.name = name;  
  this.age = age;  
  this.userId = userId;  
  this.userPw = userPw;  
  this.grade = grade;  
  this.joinDay = joinDay;  
}
```

- 생성자 함수의 이름은 보통 첫 글자를 대문자로 합니다. ex) User
- this 는 User 라는 생성자 객체 자신을 의미합니다. ex) this ==> User
- 형식은 this.키명 = 키명;
- new 연산자를 사용해서 생성자 함수를 호출해서 새로운 객체를 생성합니다.

```
var user1 = new User('홍길동', 35, 'Andrew777', '123456789~', 'VIP', '2022-10-31');  
var user2 = new User('향단이', 27, 'sally222', '3434343!!!!', 'Siver', '2021-02-11');
```

```
console.log(user1);  
console.log(user2);
```

```
▶ User {name: '홍길동', age: 35, userId: 'Andrew777', userPw: '123456789~', grade: 'VIP', ...}  
▶ User {name: '향단이', age: 27, userId: 'sally222', userPw: '3434343!!!!', grade: 'Siver', ...}
```





05) 생성자 함수(function)

공통 부분은 유지하면서 변경 항목을 매개변수로 변경시키고 함수 내에 메서드를 통해서 출력(대부분의 값은 같지만 옵션 몇 개만 다를 때)
※ 생성자 함수의 이름은 보통 대문자로 시작하는 것이 관례, 인스턴스(instance)란 복제본 또는 복사본

{ 생성자 함수 선언 기본 형식 }

```
function 생성자함수이름(매개변수, 매개변수, 매개변수){
    // 공통적인 항목
    this.속성 = 값(공통);
    this.속성 = 값(공통);
    // 변경될 항목
    this.속성 = 값(매개변수);
    this.속성 = 값(매개변수);
    this.속성 = 값(매개변수);
    // 메서드
    this.메서드이름 = function(){...}
};
```

new 라는 키워드로 객체를 새로운 인스턴스(instance)를 생성

var 변수이름 = new 생성자함수이름(매개변수, 매개변수, 매개변수);

변하는 부분으로 매개변수의 개수와 이름은 동일하게 해야 함.

Small Mission)

아이템 이름과 가격을 생성자 함수를 선언하고 새로운 생성자 함수 객체를 만들어서 출력하기

```
// 생성자 함수 선언
function Product(title, price, delivery) {
    this.title = title;
    this.price = price;
    this.delivery = delivery;
    this.showInfo = function(){
        console.log(`${title} 가격은 ${price.toLocaleString()}이고 배송은 ${delivery}인 상품`);
    }
}

// 생성자 함수 객체 생성
var item1 = new Product('무드등', 23000, '로켓배송');
var item2 = new Product('블루베리 아몬드', 6690, '묵음배송');
```

▲ toLocaleString() 메서드로 기본 값이 미국으로 설정 되어있어 1,000 단위 표기법 원화에도 적용

// 객체 호출

item1.showInfo();
item2.showInfo();

무드등 가격은 23,000이고 배송은 로켓배송인 상품
블루베리 아몬드 가격은 6,690이고 배송은 묵음배송인 상품



05) 생성자 함수(function)

Small Mission) 지식공유가 공통사항 생성자 함수를 선언하고 새로운 생성자 함수 객체를 만들어서 출력하기

```
// 공통적인 항목
var provider = '인프런';
var division = '웹 개발 분야';
```

→ 변경 사항이 없는 부분이니 생성자 함수에 넣지 않고 전역 변수로 선언

// 생성자 함수 선언 매개변수

```
function Teacher(name, subject, trainee) {
```

// 변경될 항목

```
this.name = name;
this.subject = subject;
this.trainee = trainee;
```

- 매개변수를 위한 함수 속성들
- this.name 에 name으로 정하는 것은 반드시는 아니지만 관례 상 같은 이름으로 사용

```
this.teach = function() {
```

```
  alert(provider + division + '\n' + this.name + '님 ' + this.subject + ' 수강생 수는 ' + this.trainee + '명 입니다.');
```

```
}
```

```
alert(`${provider} ${division}\n${name} 님 ${subject} 수강생 수는 ${trainee}명 입니다`);
```

// 생성자 함수를 사용한 변수 선언

매개변수에 넣을 값

```
var codingworks = new Teacher('코딩웍스', '퍼블리싱 강의', '7,430');
```

```
var sallyclass = new Teacher('샐리클래스', '웹디자인 강의', '1,112');
```

// 생성자 함수를 사용한 변수 호출

```
codingworks.teach()
```

```
sallyclass.teach()
```

→ 지식 공유자 별로 호출해서 사용

127.0.0.1:5500 내용:

인프런웹 개발 분야
코딩웍스님 퍼블리싱 강의 수강생 수는 7,430명 입니다.

127.0.0.1:5500 내용:

인프런웹 개발 분야
샐리클래스님 웹디자인 강의 수강생 수는 1,112명 입니다.

확인

▲ 백틱을 사용한 코드



05) 생성자 함수(function) Small Mission 생성자 함수를 활용해서 자동차 등급별 상세 내역 출력

```
// 생성자 함수 만들기
function OrderMalibu(grade, price, wheel, sunroof) {
  // 해당 모델을 주문할 때 고객의 선택에 의해 변경될 항목
  this.grade = grade;
  this.price = price;
  this.wheel = wheel;
  this.sunroof = sunroof;
}
```

```
// 해당 모델이 가지고 있는 공통적인 항목(prototype)
OrderMalibu.prototype.model = 'Malibu 2.0';
OrderMalibu.prototype.fuel = '가솔린';
OrderMalibu.prototype.engine = '직렬 4기통';
OrderMalibu.prototype.efficiency = 10.8;
```

```
// 선택한 세부사항 미리보기 메서드(prototype)
OrderMalibu.prototype.result = function() {
  // 공통 항목 출력
  alert('공통 사항\n' + this.model + ', ' + this.fuel + ', ' + this.engine + ', ' + this.efficiency + 'km');
  // 선택 항목 출력
  alert('선택 사항\n' + this.grade + ', ' + this.price + ', ' + this.wheel + ', ' + this.sunroof);
}
```

```
// 새로운 인스턴스(instance)를 생성하기
var trim1 = new OrderMalibu('LT 스페셜', '3,022만원', '16인치', '없음');
var trim2 = new OrderMalibu('프리미어 스페셜', '3,298만원', '17인치', '수동 쉐루프');
var trim3 = new OrderMalibu('레드라인', '3,338만원', '18인치', '자동 쉐루프');
```

상세 항목 미리보기

LT 스페셜

프리미어 스페셜

레드라인

선택 사항

LT 스페셜, 3,022만원, 16인치, 없음

공통 사항

Malibu 2.0, 가솔린, 직렬 4기통, 10.8km

```
<div>
  <h1>상세 항목 미리보기</h1>
  <button value="trim1" onclick="trimPreview(this.value)">LT 스페셜</button>
  <button value="trim2" onclick="trimPreview(this.value)">프리미어 스페셜</button>
  <button value="trim3" onclick="trimPreview(this.value)">레드라인</button>
</div>
```

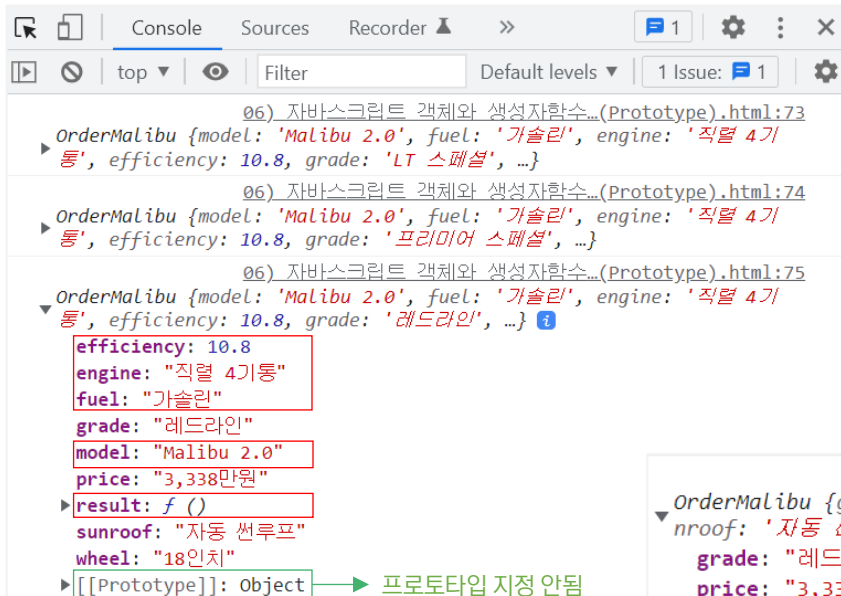
HTML

```
// 등급(trim) 별로 결과 출력
function trimPreview(e) {
  if(e == 'trim1') {
    trim1.result();
  }
  else if(e == 'trim2') {
    trim2.result();
  }
  else if(e == 'trim3') {
    trim3.result();
  }
}
```



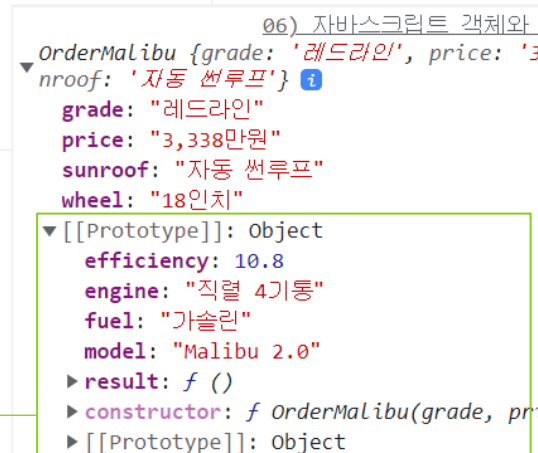
05) 생성자 함수(function) - Prototype → 생성자 함수로 생성된 객체들이 속성과 메서드를 공유하기 위해 사용하는 객체 (메모리 효율을 줄이기 위해 공통된 부분을 프로토타입 키워드로 빼서 모아둠)

▼ 프로토타입으로 변경하지 않은 기존 상태

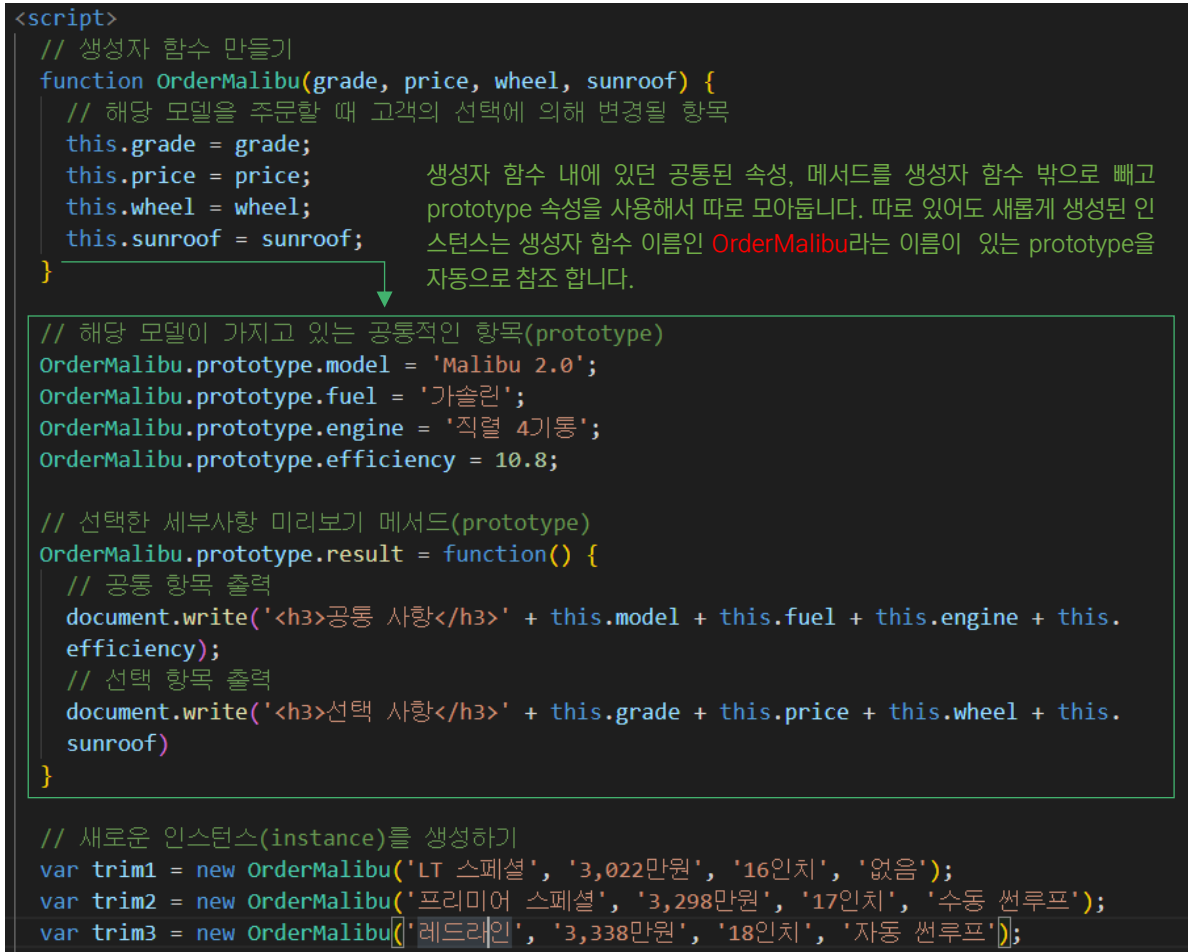


위 콘솔창에서 붉은 색 박스 부분은 공통적이지만 trim1, trim2, trim3라는 새롭게 생성된 인스턴스가 항상 불러오고 있습니다. 곧, 매번 저 부분의 코드를 읽어 오기 때문에 메모리 효율이 떨어집니다. 그래서 **prototype** 속성을 통해서 공통적인 부분을 모아두는 것이 좋습니다.

공통적인 부분을 prototype 속성으로 공간을 만들고 새롭게 생성된 인스턴스들이 참조하는 방식



▲ 프로토타입으로 변경한 상태



05) 생성자 함수(function) - Small Mission

Small Mission)

생성자 함수와 prototype을 활용해서 웹디자인 기능사 필기 평균을 구하기

인스턴스 3개 : 김착한 95 85 80, 이성실 88 77 90, 박노력 66 76 79 / 매개변수 이름 : name, graphic, design, internet

평균을 이용해서 60점 이상이면 합격, 60점 미만이면 불합격을 체크하고 또 출력할 수 있는 if 조건문을 활용할 수도 있습니다.

학생이름 : 김착한 / 평균 : 86.67

학생이름 : 이성실 / 평균 : 85.00

학생이름 : 박노력 / 평균 : 73.67

스몰 미션 제작 노트

- ① 생성자 함수 선언하기(매개변수 4개)
- ② 평균 메서드를 사용해서 문서 출력될 메서드를 prototype으로 만들기(소수점 2자리까지 출력)
- ③ 학생 3명의 인스턴스 3개 만들기
- ④ 3명 학생 평균 처리 결과 body에 출력하기

- 소수점 자리수 처리 메서드 : toFixed(자리수)
- toFixed(0) 라고 하면 소수점 없이 → 86
- toFixed(1) 라고 하면 소수점 한 자리 → 86.5

return 은 반환한다는 의미로, 생성자 함수에서 만들어진 매개변수에는 현재 이름만 있지 값은 없습니다. 그래서 그 값을 return 다음에 나오는 처리 결과에 따른 새로운 값으로 반환(입력)합니다.

홍길동은 문자열이므로 따옴표 사용해야 하고, 점수는 숫자이므로 따옴표 없이 사용해야 합니다.

```
<script>
// 생성자 함수 선언하기(매개변수 4개)
function Score(name, graphic, design, internet) {
    this.name = name;
    this.graphic = graphic;
    this.design = design;
    this.internet = internet;
}

// 평균 메서드를 사용해서 문서 출력될 메서드를 prototype으로 만들기(소수점 2자리까지 출력)
Score.prototype.result = function avg() {
    return '학생이름 : ' + this.name + ' / ' + '평균 : ' + ((this.graphic + this.design + this.internet)/3).toFixed(2) + '<hr>';
}

// 학생 3명의 인스턴스 3개 만들기
var student1 = new Score('김착한', 95, 85, 80);
var student2 = new Score('이성실', 88, 77, 90);
var student3 = new Score('박노력', 66, 76, 79);

// 3명 학생 평균 처리 결과 body에 출력하기
document.write(student1.result());
document.write(student2.result());
document.write(student3.result());
</script>
```

① 생성자 함수의 이름과 값은 동일하게 하는 것이 관례 (this.name => name)

② Score.prototype.result = function avg() { ... } 에서 avg() 는 prototype으로 만들어 새롭게 생성된 인스턴스가 참조(괄호 주의)

③ 새롭게 생성되는 인스턴스에 매개변수 4개의 값을 개별적으로 넣어주기

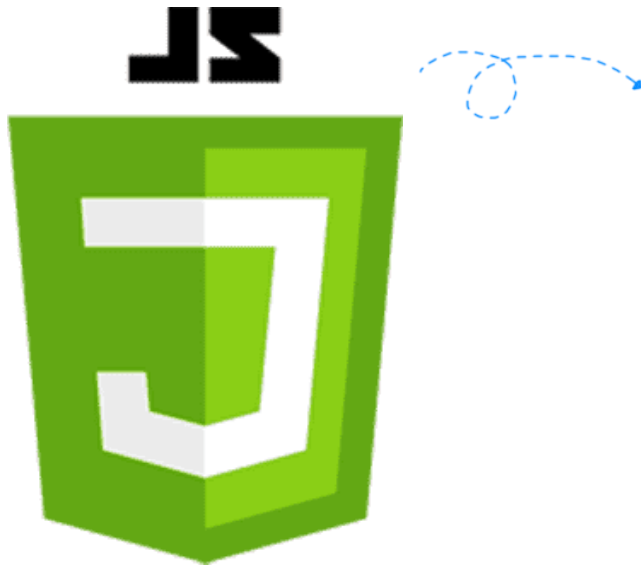
④ document.write : 문서(body)에 써라(출력)
student1 : student1이라는 새롭게 생성된 인스턴스의
result() : 프로토타입으로 만든 메서드인 result를 참조해서

학생 이름 출력하고 3과목 평균을 구하는 메서드를 prototype으로 만들어 새롭게 생성된 인스턴스가 참조(괄호 주의)

- result : prototype이 될 메서드 이름
- avg : 함수 이름(현재는 아무 이름이나)

JAVASCRIPT 핵심이론 커리큘럼

[웹 퍼블리셔를 위한 자바스크립트] 08. 자바스크립트 기본 내장 객체



1. String 문자열
2. Number 숫자 & Math 수학
3. Array 배열의 속성과 메서드
4. Date 날짜
5. Date 객체의 Getter와 Setter 메서드

01) 기본 내장 객체 - String 객체 더 쉽게 이해하기 String 객체 곧, 문자열을 표현하는 방식

{ 자바스크립트에서 문자열을 표현하는 다양한 형태 }

```
var user = 'Andrew';
```

```
var html = '<h1 class="heading">This is H1 Headline</h1>';
```

```
var char = "It's time. 10 o'clock.";
```

```
var msg = `${user} 고객님! 방문을 환영합니다.`;
```

```
var plus = `10 + 10 = ${10+10}`;
```

문자열이 html 태그인 경우 속성에 쌍따옴표로 주로 쓰이니까 문자열은 외따옴표를 사용

어퍼스트로피를 표현하기 좋게 문자열은 쌍따옴표로 사용

백틱으로 변수와 문자열을 동시에 사용

백틱으로 문자열 내에 사칙연산 사용

01) 기본 내장 객체 - String 객체의 선언

{ 변수식 선언 방식 }

```
var 변수이름 = {'문자열'}
```

일반적으로 많이 사용되는 방식

```
var str = {'CodingWorks'};
```

{ 생성자 함수식 선언 방식 }

```
var 변수이름 = new String('문자열')
```

```
var str = new String('CodingWorks');
```

안녕하세요. 코딩웍스입니다.

안녕하세요. 코딩웍스입니다.

▲ 선언 방식에 차이 없이 브라우저 출력 결과는 동일합니다.

```
<script>
  /* 기본 내장 객체 - String 객체의 선언 */

  // 변수식 선언 방식
  var str1 = '안녕하세요. 코딩웍스입니다.';

  document.write(str1);
  console.log(str1);
  console.log(typeof(str1));
  document.write('<hr>');

  // 생성자 함수식 선언 방식
  var str2 = new String('안녕하세요. 코딩웍스입니다.');
```

typeof() : 괄호 안의 데이터 유형이 숫자인지 문자열인지 불린 (Boolean) 값인지 데이터 유형을 체크하는 메서드

안녕하세요. 코딩웍스입니다.	01) 자바스크립트 기본 내장 객체.html:31
string	01) 자바스크립트 기본 내장 객체.html:32
안녕하세요. 코딩웍스입니다.	01) 자바스크립트 기본 내장 객체.html:39
object	01) 자바스크립트 기본 내장 객체.html:40

▲ 콘솔창 출력 결과는 동일합니다. 단, typeof로 데이터 유형을 체크하면 생성자 함수 선언 방식의 문자열은 object로 표시됩니다. 크게 중요한 부분은 아니니 그렇구나 정도로만 알아두세요.

01) 기본 내장 객체 - String 객체의 속성 모든 객체는 속성(Properties)와 메서드(Method)를 가지고 있습니다.

{ String 객체의 속성 }

```
var str = '코딩웍스';
```

객체이름.속성명;

```
str.length;
```

length 라는 속성은 문자열의 개수를 알아내는 속성이므로 결과값은 4입니다.

length 다음에 괄호가 있는 length() 로 쓰지 않는 이유는 괄호가 있는 것들은 메서드라고 부릅니다. **괄호가 없이 length** 사용되는 것은 속성이기 때문입니다.

127.0.0.1:5500 내용:

입력하신 비밀번호는 사용가능합니다.

확인

Small Mission)

사용자 비밀번호가 8~12글자이면 [비밀번호 사용가능], 그렇지 않으면 [비밀번호 사용불가]라고 경고창 띄우기

스몰 미션 제작 노트

- ① 사용자에게 비밀번호 입력 값을 prompt로 받기
- ② 입력 받은 비밀번호의 글자 개수를 비교 연산자를 사용해서 체크해서 글자 개수에 따라 if 조건문으로 분기 시키기

```
<script>
/* 사용자 비밀번호가 8~12글자이면 [비밀번호 사용가능], 그렇지 않으면 [비밀번호 사용불가]라고 경고창 띄우기 */
// 사용자에게 비밀번호 입력 값을 prompt로 받기
var userPw = prompt('사용할 비밀번호를 입력하세요.', '사용할 비밀번호는 8~12자리');

// 입력 받은 비밀번호의 글자 개수를 비교 연산자를 사용해서 체크해서 글자 개수에 따라 if 조건문으로 분기 시키기
if(userPw.length >= 8 && userPw.length <= 12) {
    alert('입력하신 비밀번호는 사용가능합니다.');
```

8자리에서 12자리 사이에 있어야 하므로 논리곱 연산자 && 를 사용합니다.

```
    }
    else {
        alert('입력하신 비밀번호는 사용불가능합니다.');
```

```
        alert('사용할 비밀번호는 8~12자리로 입력하세요.');
```

```
    }
</script>
```

127.0.0.1:5500 내용:

입력하신 비밀번호는 사용불가능합니다.

확인

01) 기본 내장 객체 - 대표적인 String 객체의 메서드

메서드 이름	설명
charAt()	매개변수에 지정한 인덱스 번호에 위치하는 문자를 반환
match()	매개변수의 문자가 문자열에 있는지 확인해서 true, false 반환
substr()	매개변수에 지정한 인덱스 번호부터 지정된 글자수까지 반환
substring()	매개변수의 시작 인덱스부터 종료 인덱스까지 반환
toLowerCase()	문자열을 소문자로 변경해서 반환
toUpperCase()	문자열을 대문자로 변경해서 반환
indexOf()	문자열 앞부터 특정 문자 위치 찾기
replace(a, b)	매개변수에 지정한 a문자열을 b문자열로 변경해서 반환

▲ 자바스크립트 메서드의 종류는 굉장히 많습니다. 모든 메서드의 사용법을 외우거나 할 수 있을 필요는 없습니다. 자주 사용되는 메서드를 위주로 공부하시고 새로운 메서드가 나올 경우 우측 예시처럼 w3schools에서 해당 메서드를 검색해서 사용법을 확인하시기 바랍니다. 기본적으로 영문이지만 구글 번역으로 한글로 볼 수 도 있습니다.

w3schools JavaScript Tutorial : <https://www.w3schools.com/js/>

Google search results for 'w3school'. The top result is 'W3Schools Online Web Tutorials'. A red box highlights the 'JavaScript Tutorial' link. An arrow points from this link to a search bar containing 'substr'. Another arrow points from the search bar to the 'JavaScript String substr() Method' result on the W3Schools website.

JavaScript String **substr()** Method
www.w3schools.com › jsref › jsref_substr
The **substr()** method extracts a part of a string. The **substr()** method begins at a specified position,

01) 기본 내장 객체 - String 객체의 메서드

{ **charAt(위치)** → 위치에 해당하는 문자를 반환 }

```
var str = '코딩웍스';
```

```
str.charAt(2);
```

charAt()은 괄호 안에 숫자가 인덱스 번호
이므로 2를 입력하면 0, 1, 2, 3 중에 2에
해당하는 '웍'이라는 문자열을 반환합니다.

Small Mission)

사용자에게 주민번호 뒷자리 7자리를 입력받아 첫 숫자가 1 또는 3이면 남자, 2 또는 4이면 여자 라고 경고창을 출력, 1~3 또는 2~4를 입력하지 않은 경우 다시 입력이라고 경고창 출력

스몰 미션 제작 노트

- ① 사용자에게 주민번호 뒷자리 7자리 값을 prompt로 받아 변수에 담기
- ② if 조건문으로 주민번호 뒷 번호 첫 글자를 비교해서 남녀 출력하기

```
<script>
// 사용자에게 주민번호 뒷자리 7자리 값을 prompt로 받아 변수에 담기
var inputDigit = prompt('주민등록 번호 뒷번호 7자리를 입력', '정수로 입력');
var residentAfterNum = inputDigit.charAt(0); // 주민번호 뒷번호가 1234567 이라면 인덱스 0번째인 1
                                              //이라는 문자열을 추출함

// if 조건문으로 주민번호 뒷 번호 첫 글자를 비교해서 남녀 출력하기
if(residentAfterNum == 1 || residentAfterNum == 3) {
    alert('남자입니다.');
```

↓
1 또는 3, 2 또는 4라고 또는 이러한 조건이므로 논리합 연산자 || 를 사용

```

}
else if(residentAfterNum == 2 || residentAfterNum == 4) {
    alert('여자입니다.');
```

↓
1~4라는 조건에 맞지 않는 경우(1, 2, 3, 4 외 숫자 입력, 소수점 입력, 문자 입력, 아무것도 입력하지 않은 경우)

```

}
else {
    alert('다시 입력해주세요.');
```

↓
1~4라는 조건에 맞지 않는 경우(1, 2, 3, 4 외 숫자 입력, 소수점 입력, 문자 입력, 아무것도 입력하지 않은 경우)

```

}
</script>
```

127.0.0.1:5500 내용:
주민등록 번호 뒷번호 7자리를 입력

확인 취소

127.0.0.1:5500 내용:
남자입니다.

127.0.0.1:5500 내용:
여자입니다.

127.0.0.1:5500 내용:
다시 입력해주세요.

확인

01) 기본 내장 객체 - String 객체의 메서드

{ **match(문자열)** → 문자열 내에 특정 문자를 반환 }

```
var str = '코딩웍스';
```

`str.match('코딩');` → 문자열 내에 코딩 이라는 문자열이 있으므로 true 반환(출력)

Small Mission

사용자가 사용하는 현재 접속한 기기를 입력 받아 아이폰, 갤럭시 라면 [모바일 사용자입니다.] 태블릿 경우는 [태블릿 사용자입니다.] 그 이외의 경우 [PC 사용자입니다.] 라고 경고창 출력

스몰 미션 제작 노트

- ① 사용자가 사용하는 현재 접속한 기기를 입력 받아 변수에 담기
- ② if 조건문으로 특정 문자열이 있는지 없는지 체크해서 경고창 출력하기

127.0.0.1:5500 내용:
현재 사용하고 있는 기기는 무엇입니까?

확인 취소

127.0.0.1:5500 내용:
모바일 사용자입니다.

127.0.0.1:5500 내용:
태블릿 사용자입니다.
확인

127.0.0.1:5500 내용:
PC 사용자입니다.
확인

```
<script>
// 사용자가 사용하는 현재 접속한 기기를 입력 받아 변수에 담기
var currentDevice = prompt('현재 사용하고 있는 기기는 무엇입니까?', '아이폰/갤럭시/태블릿');

// if 조건문으로 특정 문자열이 있는지 없는지 체크해서 경고창 출력하기
if(currentDevice.match('아이폰') || currentDevice.match('갤럭시')) {
    alert('모바일 사용자입니다.');
```

01) 기본 내장 객체 - String 객체의 메서드

{ **substr**(시작 인덱스 번호, 몇 개까지) }

→ 문자열 내에 특정 인덱스 번호부터 몇 개의 문자를 반환

```
var str = '코딩웍스_퍼블리싱_클래스';
```

str.substr(5, 4); → 시작 번호, 몇 개까지라는 매개변수를 사용해서 문자열 내에 인덱스 번호 시작부터 몇 번째까지의 문자열을 반환. 현재의 경우 인덱스 번호 5인 퍼 라는 글자부터 4개를 선택하므로 퍼블리싱 이라는 문자열 출력 공백(스페이스)도 하나의 문자로 인식합니다.

Small Mission)

사용자에게 주민번호 앞 번호 6자리를 입력 받아 [고객님의 생일은 00월 00일 입니다.] 라고 출력하기(단, 정수 6자리를 입력하지 않은 경우 [숫자 6자리로 다시 입력해주세요.]라고 출력하기

사용자 입력값 : 881204
인덱스 번호 : 012345

스몰 미션 제작 노트

- ① 사용자에게 주민번호 앞 번호 6자리를 입력 받아 변수에 담기
- ② if 조건문으로 입력받은 숫자가 6자리인지 체크해서 경고창 출력하기

127.0.0.1:5500 내용:
주민번호 앞자리를 숫자 6자리로 입력

확인 취소

127.0.0.1:5500 내용:
고객님의 생일은 12월 04일 입니다.

확인

```
<script>
// 사용자에게 주민번호 앞 번호 6자리를 입력 받아 변수에 담기
// ex. 881204(사용자 입력값) ==> 012345(인덱스 번호)
var firstInput = prompt('주민번호 앞자리를 숫자 6자리로 입력', '정수로 입력');
var firstNum = firstInput.length;
var monthBirth = firstInput.substr(2, 2);
var dayBirth = firstInput.substr(4, 2);

console.log(firstInput);
console.log(firstNum);
console.log(monthBirth);
console.log(dayBirth);

// if 조건문으로 입력받은 숫자가 6자리인지 체크해서 경고창 출력하기
// 입력한 숫자가 6자리 정수를 입력했는지 체크
if(firstNum == 6) {
    alert('고객님의 생일은 ' + monthBirth + '월 ' + dayBirth + '일 입니다.');
```

prompt에서 입력 받은 문자의 개수를 length 속성으로 체크해서 새로운 변수에 넣어서 나중에 if 조건문에서 문자의 개수가 6인지 아닌지 체크함.

조건문 작성 전에 콘솔창에서 데이터가 정확히 출력되는지 먼저 확인

881204
6
12
04

01) 기본 내장 객체 - String 객체의 메서드

- { `toUpperCase()` → 문자열 내에 영어를 강제 대문자로 변환 }
- { `toLowerCase()` → 문자열 내에 영어를 강제 소문자로 변환 }

```
var str = 'CodingWorks';
```

```
str.toUpperCase();
```

문자열 내에 영어 문자를 강제로 대문자로 바꾸는
메서드로 결과는 CODINGWORKS

해당 메서드는 괄호 안에 매개변수가 없습니다. 하지만
메서드이기 때문에 반드시 괄호를 사용해야 합니다.

문자열 내에 영어 문자를 강제로 소문자로 바꾸는
메서드로 결과는 codingworks

```
str.toLowerCase();
```

Small Mission

사용자에게 다운로드 쿠폰 코드 6자리를 입력 받아 대문자로 변경해서 경고창 출력 단, 쿠폰 코드가 6자리가 아닌 경우 [쿠폰 코드 6자리를 다시 입력하세요.] 라고 경고창 출력

스몰 미션 제작 노트

- ① 사용자에게 쿠폰 코드 입력 받기 ex) 쿠폰 번호 sgc123
- ② 입력 받은 쿠폰 코드의 문자열 개수 체크 후 6자리면 쿠폰 코드 강제 대문자로 변화, 6자리가 아니면 다시 입력 출력

```
<script>
// 사용자에게 쿠폰 코드 입력 받기 ex) 쿠폰 번호 sgc123
var couponCode = prompt('쿠폰 코드를 입력하세요.', '6자리로 입력');
var codeDigit = couponCode.length; // 사용자가 입력한 쿠폰 코드 문자열의 길이 체크(결과 6)
// length는 속성이므로 괄호가 없습니다.

// 콘솔창에서 정확히 데이터를 받아오는지 체크
console.log(couponCode);
console.log(codeDigit);

// 입력 받은 쿠폰 코드의 문자열 개수 체크 후 6자리면 쿠폰 코드 강제 대문자로 변환
// 6자리가 아니면 다시 입력 출력
if(codeDigit == 6) {
    alert('입력하신 쿠폰 코드는 : ' + couponCode.toUpperCase() + '입니다.');
```

sgc123
6

```
    }
    else {
        alert('쿠폰 코드 6자리를 다시 입력하세요.');
```

강제 대문자로 변환(자바스크립트는 대소문자를 구분하기
때문에 사용자가 대소문자를 혼용해서 입력하더라도 대문
자 또는 소문자로 입력 받아 사용해야 하는 경우 필요함)

```
    }
}</script>
```

01) 기본 내장 객체 - String 객체 스몰 미션

Small Mission) 주민번호 앞 뒤 자리 체크해서 생일 출력하기

사용자에게 주민 등록 번호 앞 뒤 자리를 각각 입력 받아 생일을 출력하기. 주민 번호 문자열 개수가 틀리면 [주민 번호가 틀립니다. 다시 입력하세요.] 라고 출력하기(주민 번호 뒷 번호 첫 글자가 1 또는 2면 1900년대 생이고 3 또는 4면 2000년대 생으로 출력)

- 주민 번호 뒷 번호 첫 글자가 1 또는 2면 1900년대 생이고 3 또는 4면 2000년대 생으로 출력
- 중첩 if 문 사용 / 사용할 문자열 내장 객체: `charAt()` `substr()`

스몰 미션 제작 노트

- ① 사용자에게 주민번호 앞 뒤 자리 입력 받기
- ② 입력받은 주민번호 앞뒤자리의 문자 길이 체크해서 다른 변수에 대입
- ③ 주민번호 앞자리 2개, 뒷자리 2개 반환
- ④ 콘솔창에서 데이터 확인하기
- ⑤ 주민번호 앞 뒤 자리 자리수 체크(앞자리 6, 뒷자리 7)
- ⑥ 주민번호 앞자리 뒷자리 첫 글자 체크

127.0.0.1:5500 내용:
고객님의 생일은 : 1988년 12월 04일 입니다.

127.0.0.1:5500 내용:
고객님의 생일은 : 2002년 07월 11일 입니다.

확인

```
<script>
// 사용자에게 주민번호 앞 뒤 자리 입력 받기
var residentFirst = prompt('주민번호 앞자리 6자리 입력', '정수로 입력');
var residentLast = prompt('주민번호 뒷자리 7자리 입력', '정수로 입력');
// 입력받은 주민번호 앞뒤자리의 문자 길이 체크해서 다른 변수에 대입
var firstNum = residentFirst.length; // 값: 6
var lastNum = residentLast.length; // 값: 7
// 주민번호 앞자리 2개, 뒷자리 2개 반환
var userMonth = residentFirst.substr(2, 2); // 주민번호 앞자리 6자리 중에 인덱스 번호 2번째부터 2개 문자(월)
var userDay = residentFirst.substr(4, 2); // 주민번호 앞자리 6자리 중에 인덱스 번호 4번째부터 2개 문자(일)

// 콘솔창에서 데이터 확인하기
console.log(residentFirst);
console.log(residentLast);
console.log(firstNum);
console.log(lastNum);
console.log(residentLast.charAt(0));
console.log(userMonth);
console.log(userDay);

// 주민번호 앞 뒤 자리 자리수 체크(앞자리 6, 뒷자리 7)
if(firstNum == 6 && lastNum == 7 && residentLast.charAt(0) >= 1 && residentLast.charAt(0) <= 4) {
    // 주민번호 앞자리 뒷자리 첫 글자 체크
    if(residentLast.charAt(0) == 1 || residentLast.charAt(0) == 3) { // 주민번호 뒷자리 첫 글자가 1 이거나 3인 경우
        alert('고객님의 생일은 : 19' + residentFirst.substr(0, 2) + '년 ' + userMonth + '월 ' + userDay + '일 입니다.');
    }
    else if(residentLast.charAt(0) == 2 || residentLast.charAt(0) == 4) {
        alert('고객님의 생일은 : 20' + residentFirst.substr(0, 2) + '년 ' + userMonth + '월 ' + userDay + '일 입니다.');
    }
}
else {
    alert('주민 번호가 틀립니다. 다시 입력하세요.');
}
}

</script>
```

881204	020711
1234567	4567890
6	6
7	7
1	4
12	07
04	11

주민번호 앞자리 6자리, 뒷자리 7자리, 주민번호 뒷자리 첫 글자가 1~4 사이의 정수 인지 체크해서 true면 중첩 if 문을 실행하고, false면 중첩 if 문 실행하지 않고 else로 가서 조건문 빠져 나옴

주민번호 뒷자리가 1 또는 3이면 1900년대 생, 2 또는 4이면 2000년대 생

예제 제작을 위한 주민등록번호 샘플

2000년 이전 출생 : 881204-1234567
2000년 이후 출생 : 020711-4567890

01) 기본 내장 객체 - String 객체의 메서드

{ **indexOf(찾을 문자열)** → 문자열 앞부터 특정 문자 위치의 인덱스 번호 찾기 }

예시) 해당 문자열에서 W라는 단어의 인덱스 번호를 출력하기

```
<script>
/* 해당 문자열에서 w라는 단어의 인덱스 번호를 출력하기 */
var str = 'CodingWorks';

console.log(str.indexOf('W'));
console.log(str.indexOf('w'));
</script>
```

W 라는 문자는 인덱스 번호로 6번째 있으므로 6을 출력합니다.

w 라는 문자는 존재하지 않으므로 -1을 출력합니다. (대소문자 구분)

↳

6
-1

{ **trim()** → 문자열 앞 뒤의 공백 제거 }

```
<script>
/* 문자열의 앞 뒤 공백 제거 */
var str = ' CodingWorks ';

console.log(str.trim());
</script>
```

CodingWorks

{ **replace(a, b)** → 문자열 내에 a 문자열을 b 문자열로 변경 }

예시) 해당 문자열에서 [온라인] 문자열을 [오프라인] 으로 변경하기

```
<script>
/* 해당 문자열에서 [온라인] 문자열을 [오프라인]으로 변경하기 */
var str = '코딩웍스 온라인 클래스';

console.log(str.replace('온라인', '오프라인'));
</script>
```

↳ 코딩웍스 오프라인 클래스

{ **slice(시작점, 몇개까지)** → 문자열에서 특정 범위 문자열만 추출 }

```
<script>
/* 문자열에서 특정 범위 문자열만 추출 */
var msg = '코딩웍스 온라인 클래스';

console.log(msg.slice(2));
console.log(msg.slice(0, 5));
console.log(msg.slice(2, -2));
</script>
```

웍스 온라인 클래스

코딩웍스

웍스 온라인 클

03) 기본 내장 객체 - Number 객체 메서드 선언 숫자를 선언하는 메서드 ex) prompt()로 입력 받은 문자열을 숫자로 변환

{ 변수식 선언 방식 }

```
var 변수이름 = 77;
```

일반적으로 많이 사용되는 방식

```
var num1 = 77; // 문자열  
var num1Change = Number(num1); // 숫자
```

{ 생성자 함수식 선언 방식 }

```
var 변수이름 = new Number(77)
```

```
var num2 = new Number(77);
```

↓
new 는 새로운 객체를 생성하는 키워드

```
<script>  
var num1 = '77';  
var num1Change = Number(num1);  
  
console.log(num1);  
console.log(num1Change);  
  
console.log(typeof(num1));  
console.log(typeof(num1Change));  
</script>
```

num1의 경우 따옴표로 묶여있기 때문에 숫자로 보이지만 문자(string)입니다. 해당 문자를 Number() 함수를 통해서 숫자(number)로 형 변환합니다.

typeof(데이터) : 데이터가 유형(타입)을 체크하는 메서드



Console		Sources	Recorder	Network	1	
77	03) 자바스크립트 - Number ...체 메서드 선언-01.html:40					
77	03) 자바스크립트 - Number ...체 메서드 선언-01.html:41					
string	03) 자바스크립트 - Number ...체 메서드 선언-01.html:43					
number	03) 자바스크립트 - Number ...체 메서드 선언-01.html:44					

03) 기본 내장 객체 - String 형 변환 자바스크립트 데이터 유형 숫자, 문자열, 불, 함수, 객체, undefined ex) 문자 ↔ 숫자

Login Account

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed diam
natummy nibh euismod tincidunt ut laoreet dolore magna aliquam
erat volutpat. Ut wisi enim ad minim veniam, quis nostr

→ `<input type="text" id="user-id">`

→ `<input type="text" id="user-pw">`

☒ Keep me signed in ☐ Already a member?

▲ 사용자가 무엇을 입력하건 type="text" 이기 때문에 숫자를 입력하더라도 문자열로 반환됩니다. 이런 경우 숫자를 문자가 아니라 숫자로 입력 받기 위해서 형 변환이 필요합니다.

Small Mission)

사용자에게 2가지 숫자를 받아서 사칙연산을 하세요.

스몰 미션 제작 노트

- ① 사용자에게 prompt() 함수로 숫자 2개 입력 받기
- ② console.log로 데이터 확인하기
- ③ 입력 받은 숫자같은 문자를 숫자로 형 변환해서 결과값을 경고창에 출력하세요.

127.0.0.1:5500 내용:
결과값 : 10+20 = 30입니다.

확인

```
<script>
/* 사용자에게 2가지 숫자를 받아서 사칙연산을 하세요. */

// 사용자에게 prompt() 함수로 숫자 2개 입력 받기 ex) 10 20
var firstNum = prompt('첫번째 숫자 입력', '정수로 입력');
var secondNum = prompt('두번째 숫자 입력', '정수로 입력');

// console.log로 데이터 확인하기
console.log(firstNum);
console.log(secondNum);
console.log(firstNum + secondNum);
console.log(Number(firstNum) + Number(secondNum));

// 입력 받은 숫자같은 문자를 숫자로 형 변환해서 결과값을 처리하세요.
alert('결과값 : ' + firstNum + '+' + secondNum + ' = ' + (Number(firstNum) + Number(secondNum)) + '입니다.');
```

10
20
1020
30

03) 기본 내장 객체 - Number의 속성과 메서드 Number 메서드 종류가 많지만 사용 빈도가 높은 메서드만 공부하세요.

메서드 이름	설명
toString()	Number() 메서드와 반대로 숫자를 문자로 형 변환
toFixed()	소수점 자리수 지정 ex) (77.7777).toFixed(0) 이라면 소수점 0 번째 자리 끝, 소수점 처리하지 않음

- ① prompt() 로 받은 숫자는 문자열로 받기 때문에 $95+85+80 = 958580$ 이 됩니다. 그래서 $958580 / 3$ 으로 나누면 이상한 숫자가 나오게 됩니다.
- ② 그래서 Number() 메서드로 prompt() 로 받은 문자열을 숫자로 형 변환합니다. 그런 정상적인 평균이 나옵니다.
- ③ 나온 결과에서 toFixed(2) 메서드를 사용해서 소수점 2자리까지만 출력합니다.
- ④ 평균이 계산된 값을 result 변수에 넣고 이걸 활용하는 (4)처럼 하시는게 좋습니다.

```
var result = (Number(graphic) + Number(design) + Number(internet)) / 3;
console.log('3과목 평균은 : ' + result.toFixed(2) + '입니다.');
```

3과목 평균은 : 86.67입니다.

Small Mission)

웹 디자인 기능사 필기 평균을 구하기, 입력할 점수 95 85 80, 변수 이름 : graphic, design, internet

스몰 미션 제작 노트

- ① 사용자에게 prompt() 함수로 점수 3개 입력 받기
- ② 3과목 평균 내기(숫자로 바꿔서 3과목 평균 내기)
- ③ toFixed() 메서드로 소수점 2자리까지 출력하기

```
<script>
/* 웹디자인 기능사 필기 평균을 구하기 */

// 사용자에게 prompt() 함수로 점수 3개 입력 받기
var graphic = prompt('그래픽 점수 입력', '정수로 입력');
var design = prompt('디자인 점수 입력', '정수로 입력');
var internet = prompt('인터넷 점수 입력', '정수로 입력');

// 3과목 평균 내기
var result = graphic + design + internet;
console.log(result);
console.log(result / 3);

// 숫자로 바꿔서 3과목 평균 내기
var result = Number(graphic) + Number(design) + Number(internet);
console.log(result / 3);
// toFixed() 메서드로 소수점 2자리까지 출력하기
console.log((result / 3).toFixed(2));
</script>
```

958580
319526.6666666667 ①

86.66666666666667 ②

86.67 ③

03) 기본 내장 객체 - Math의 속성과 메서드(1) Math 메서드 종류가 많지만 사용 빈도가 높은 메서드만 공부하세요.

{ 숫자를 정수로 만드는 메서드 }

Math.ceil(숫자) → 숫자 보다 크거나 같은 작은 정수로 반환
(자리 올림)

Math.floor(숫자) → 숫자 보다 작거나 같은 작은 정수로 반환
(자리 내림)

Math.round(숫자) → 숫자를 반올림해서 반환
(5미만이면 내림, 5이상이면 올림)

ceil

1. 칠하다 2. 대다 3. **천장**을 치다

floor

1. **바닥** 2. 의원 3. 층 4. 마루 5. 발언권

Small Mission)

변수에 담긴 숫자를 올림, 내림, 반올림 하기 ex) num1 = 88.88, num2 = 88.44

```
<script>
  /* 변수에 담긴 숫자를 올림, 내림, 반올림 하기 ex) num1 = 88.88, num2 = 88.44 */

  // 변수 선언
  var num1 = 88.88;
  var num2 = 88.44;

  // 자리 올림 : ceil()
  console.log(Math.ceil(num1)); → 89
  console.log(Math.ceil(num2)); → 89

  // 자리 내림 : floor()
  console.log(Math.floor(num1)); → 88
  console.log(Math.floor(num2)); → 88

  // 자리 반올림 : round()
  console.log(Math.round(num1)); → 89
  console.log(Math.round(num2)); → 88
</script>
```

03) 기본 내장 객체 - Math의 속성과 메서드(2)

{ 0~1 사이 숫자를 무작위로 소수점까지 반환 }

Math.random() → 숫자 라고 하는 매개변수는 없음

```
<script>
```

```
/* Math.random() 기본 사용법 */
```

```
var testNum = Math.random();
```

```
console.log(testNum);
```

```
</script>
```

랜덤이기 때문에 새로고침 할 때 마다 0~1사이의 소수점을 포함한 숫자를 무작위로 반환

0.08226538577019804

0.4021792344986235

0.705401629992592

Small Mission)

1~10 사이의 랜덤하게 반환되는 숫자를 정수로 변환하기

```
<script>
/* 1~10 사이의 랜덤하게 반환되는 숫자를 정수로 변환하기 */

var int = Math.round(Math.random() * 9 + 1);
console.log(int);
</script>
```

```
var int = Math.round(Math.random() * 10);
console.log(int);
```

랜덤하게 0~1 사이의 숫자를 출력하는데 만약 0이 나오는 경우는 0 * 10을 해도 0이기 됩니다. 이런 경우를 방지 하기 위해서 9를 곱하고 나중에 1을 더해줍니다.

▲ Math.round() Math.ceil() Math.floor()
는 경우에 따라 사용하시면 됩니다.

integer

1. 정수 2. 완전제 3. 완전한 것

04) 기본 내장 객체 - Array 객체의 속성과 메서드(1)

Array 객체는 주로 for 반복문과 함께 자주 사용됩니다.

Array 객체의 속성과 메서드의 종류는 매우 다양합니다. 새로운 속성과 메서드는 필요할 때 검색해서 공부하시면 됩니다.

▼ Array 객체 선언 : 변수식 선언 방식

```
var nums = [10, 15, 20];
```

▼ Array 객체 선언 : 생성자 함수식 선언 방식

```
var nums = new Array(10, 15, 20)
```

속성/메서드 이름	설명
length	배열의 개수를 반환
concat(a, b, c)	다른 배열 합쳐서 새로운 배열 만들기
pop() / shift()	배열 맨 뒤의 요소 제거 / 배열 맨 앞의 요소 제거
push(요소) / unshift(요소)	배열 맨 뒤의 요소 추가 / 배열 맨 앞의 요소 추가
trim()	문자열 내부의 앞 뒤 공백 제거
slice(시작점, 몇개까지)	문자열에서 특정 범위의 문자열 추출(몇 개까지에 숫자가 없으면 문자열 끝까지, 양수면 자신을 뺀 숫자만큼, 음수면 끝에서부터 숫자만큼)
includes(문자열)	해당 문자열이 있는지 체크
forEach()	배열의 값과 인덱스 번호 확인

```
// 배열 선언 : 변수식 선언 방식
var fruit = ['Apple', 'Banana', 'Papaya', 'Peach', 'Melon', 'Pineapple'];

// 배열 선언 : 생성자 함수식 선언 방식
var arr = new Array('아스널', '맨시티', 'Apple', 'Banana', 33, 55, 99);
```

▲ 일반적으로 변수식 선언 방식을 주로 씁니다. 하지만 생성자 함수식 선언 방식도 알아야 하는 두세요.

```
// 배열 선언
var fruit = ['Apple', 'Banana', 'Papaya', 'Peach', 'Melon', 'Pineapple', 'Rambutan', 'Strawberries', 'Watermelon', 'Cherry', 'Mango', 'Melon'];

var eplTeams = ['아스널', '맨시티', '맨유', '뉴캐슬', '토트넘', '브라이턴', '브렌트퍼드', '풀럼', '첼시', '리버풀'];

// 배열 개수(length)
console.log(fruit.length); // 12
console.log(eplTeams.length); // 10
```

Small Mission) Mon~Sun을 배열로 만들고 배열의 길이를 출력하고 Sun을 일요일로 바꾸기

```
// 배열 선언
var days = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun'];

console.log(days.length); // 7
days[6] = '일요일';
console.log(days); // (7) ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', '일요일']
```

04) 기본 내장 객체 - Array 객체의 속성과 메서드(2) 배열 합치기 & 배열 내 요소 제거

{ concat() : 다른 배열 합치기 }

배열A.concat(배열B); → 배열C

Small Mission 1) 두개의 배열을 합쳐서 하나의 배열로 만들기

```
<script>
/* 두개의 배열을 합쳐서 하나의 배열로 만들기 */

var skillBasic = ['HTML', 'CSS'];
var skillAdvaced = ['SCSS', 'Flex', 'Grid', 'JavaScript', 'CSS'];

// 각 배열 출력하기
console.log(skillBasic);
console.log(skillAdvaced);

// 2개의 배열 합치기
console.log(skillBasic.concat(skillAdvaced));
</script>
```

▶ (2) ['HTML', 'CSS']
▶ (5) ['SCSS', 'Flex', 'Grid', 'JavaScript', 'CSS']
▶ (7) ['HTML', 'CSS', 'SCSS', 'Flex', 'Grid', 'JavaScript', 'CSS']

{ pop() : 배열 맨 뒤의 요소 제거 } → 매개변수는 없음

{ shift() : 배열 맨 앞의 요소 제거 } → 매개변수는 없음

Small Mission 2) 배열에서 맨 뒤 요소 제거, 맨 앞 요소 제거하기

```
<script>
/* 배열에서 맨 뒤 요소 제거, 맨 앞 요소 제거하기 */

var skills = ['HTML', 'CSS', 'SCSS', 'Flex', 'Grid', 'JavaScript', 'CSS'];

// 배열 출력하기
console.log(skills);

// 기존 배열에서 맨 뒤 요소 제거하기
skills.pop();
console.log(skills);

// 기존 배열에서 맨 앞 요소 제거하기
skills.shift();
console.log(skills);
</script>
```

▶ (7) ['HTML', 'CSS', 'SCSS', 'Flex', 'Grid', 'JavaScript', 'CSS']
최초의 배열 출력
최초 배열에서 pop()로 맨 뒤의 요소 빼고 배열 출력
▶ (6) ['HTML', 'CSS', 'SCSS', 'Flex', 'Grid', 'JavaScript']
▶ (5) ['CSS', 'SCSS', 'Flex', 'Grid', 'JavaScript']
바로 위의 변경된 배열에서 shift()로 맨 앞의 요소 빼고 배열 출력

04) 기본 내장 객체 - Array 객체의 속성과 메서드(3) 배열 내 요소 추가

{ push(추가할 요소) : 배열 맨 뒤의 요소 추가 }

{ unshift(추가할 요소) : 배열 맨 앞의 요소 추가 }

▶ (4) ['HTML', 'CSS', 'SCSS', 'JavaScript']

▶ (5) ['HTML', 'CSS', 'SCSS', 'JavaScript', 'Vue.js']

▶ (6) ['Flex & Grid', 'HTML', 'CSS', 'SCSS', 'JavaScript', 'Vue.js']

Small Mission 01) 배열의 끝과 앞에 배열 요소를 추가하기

```
<script>
  /* 배열에서 맨 뒤 요소 추가, 맨 앞 요소 추가 */

  var skills = ['HTML', 'CSS', 'SCSS', 'JavaScript'];

  // 배열 출력하기
  console.log(skills);

  // 기존 배열에서 맨 뒤 요소 추가하기
  skills.push('Vue.js');
  console.log(skills);

  // 기존 배열에서 맨 앞 요소 추가하기
  skills.unshift('Flex & Grid');
  console.log(skills);
</script>
```

- push()와 unshift()에서 2개 이상의 요소도 추가 할 수 있습니다.
- ex) push('Vue.js', 'React');
- ex) unshift('Vue.js', 'React');

04) 기본 내장 객체 - Array 객체의 속성과 메서드 includes()

Small Mission 01)

기존 배열에서 앞부분의 숫자를 빼고 제목만 출력하기

배열 텍스트 : 01) 기본 문법, 02) 변수, 03) 조건문, 04) 반복문, 05) 배열

스몰 미션 제작 노트

- 기존 배열에서 번호 부분을 자른 새로운 배열 생성
- for 반복문으로 번호 제외하고 slice()와 push()를 활용해서 새로운 배열에 넣기

```
// 기존 배열
var list = [
  '01) 기본 문법',
  '02) 변수',
  '03) 조건문',
  '04) 반복문',
  '05) 배열'
];

// 새로운 배열 생성
var newList = [];

// for 반복문으로 번호 제외하고 새로운 배열에 넣기
for(var i = 0; i < list.length; i++) {
  newList.push(list[i].slice(4));
}

console.log(newList);
```

03) 조건문
0 1 2 3 4
▲ 문자열 인덱스 번호가 양수 4를 넣으면 자신의 인덱스 번호 4를 뺀 부분부터 곧, 3번부터 출력

▶ (5) ['기본 문법', '변수', '조건문', '반복문', '배열']

Small Mission 02)

사용자가 입력한 검색어에 검색 불가한 문자열이 있는지 체크하기

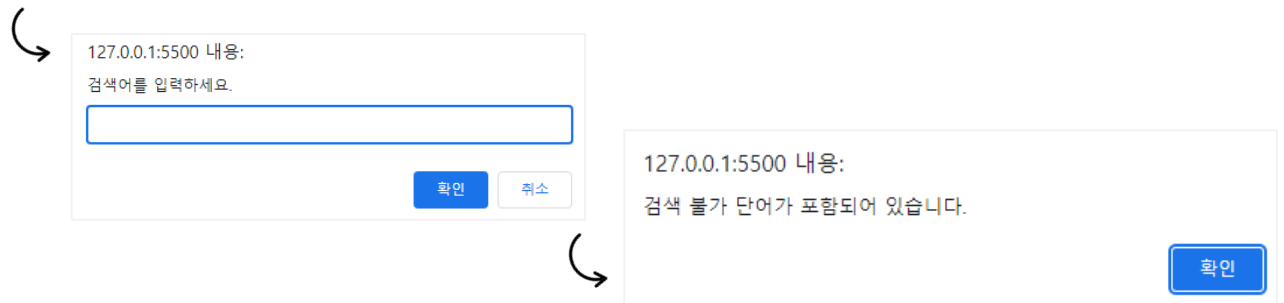
검색 불가 문자열 : 야동, 섹스

```
// 검색어 입력 받기
var hasStr = prompt('검색어를 입력하세요.');
```

```
// 검색어에 검색 불가 단어가 있는지 체크
if(hasStr.includes('섹스') || hasStr.includes('야동')) {
  alert('검색 불가 단어가 포함되어 있습니다.');
```

```
}
else {
  alert('검색 불가 단어 없이 검색되었습니다.');
```

```
}
```



04) 기본 내장 객체 - Array 객체의 속성과 메서드 forEach(), find()

{ 배열이름.forEach(함수이름) } → 배열의 값과 인덱스 번호 확인

```
// 기존 배열 만들기
var arr = ['Apple', 'Banana', 'Papaya', 'Peach', 'Melon'];
```

```
// 매개 변수를 가지 함수
function checkArr(fruit, index) {
  console.log(`${fruit}의 인덱스 번호 : ${index}`);
}
// 인덱스 번호 확인
arr.forEach(checkArr);
```

fruit이라는 이름은 배열 내의 값을 지칭하는 매개 변수라서 아무거나 사용해도 상관없습니다.
단, index는 index로 사용해야 합니다.

Apple의 인덱스 번호 : 0
Banana의 인덱스 번호 : 1
Papaya의 인덱스 번호 : 2
Peach의 인덱스 번호 : 3
Melon의 인덱스 번호 : 4

```
arr.forEach(function(fruit, index){
  console.log(`${fruit}의 인덱스 번호 : ${index}`);
});
```

▲ 왼쪽 코드에서 만든 함수 checkArr은 재사용할 일이 없으므로 함수의 선언과 실행을 한번에 할 수 있습니다.

{ 배열이름.forEach(콜백함수) }

```
numbers.forEach(function(number) {
  console.log(number);
});
```

{ 배열이름.find(함수이름) } → 조건에 맞는 값 찾기

{ 배열이름.findIndex(함수이름) } → 조건에 맞는 인덱스 찾기

```
// 기존 배열 만들기
var arr = [10, 20, 30, 40, 50];
```

```
function checkNum(number) {
  return number > 10;
}
// find(조건) : 조건에 맞는 첫번째 값을 리턴
console.log(arr.find(checkNum));
```

20

화살표 함수를 사용하면 함수 선언 없이 find() 메서드 매개변수를 함수로 해서한 줄로 표현 가능

```
console.log(arr.find((number) => number > 10));
```

▼ 화살표 함수로 표현하는 경우 function은 생략하고 매개변수 number만 사용

```
numbers.forEach((number) => {
  console.log(number);
});
```

04) 기본 내장 객체 - Array 객체의 속성과 메서드 filter(), reduce()

{ 배열이름.filter(함수이름) }

조건에 맞는 모든 값 찾기

```
// 기존 배열 만들기  
var arr = [10, 20, 30, 40, 50];
```

```
function filterNum(number) {  
  return number > 10;  
}  
// filter(조건) : 조건에 맞는 첫번째 값을 리턴  
console.log(arr.filter(filterNum));
```

number라는 이름은 배열 내의 값을 지칭하는 매개 변수라서 아무거나 사용해도 상관없습니다.

▶ (4) [20, 30, 40, 50]

▲ 10보다 큰 수 모두 필터링해서 새로운 배열로 출력

```
console.log(arr.filter((number) => number > 10));
```

화살표 함수를 사용하면 함수 선언 없이 filter() 메서드 매개변수를 함수로 해서 한 줄로 표현 가능

{ 배열이름.reduce(함수이름, 초기값) }

reduce()는 배열의 각 요소를 순회하며 값을 누적하여 하나의 결과값을 반환
초기값을 넣을 경우 초기값으로 일단 시작해서 배열을 돌립니다.
예를 들어 초기값에 10을 넣으면 10+1=11, 11+2=13, 13+3=16...

```
// 기존 배열 만들기  
var arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
```

```
function plusNum(acc, value) {  
  return acc + value;  
}  
function multiplyNum(acc, value) {  
  return acc * value;  
}  
// reduce(조건) : 배열의 각 요소를 돌며 값을 누적하여 하나의 결과값을 반환  
console.log(arr.reduce(plusNum));  
console.log(arr.reduce(multiplyNum));
```

55
3628800

화살표 함수를 사용하면 함수 선언 없이 reduce() 메서드 매개변수를 함수로 해서 한 줄로 표현 가능

```
// 화살표 함수로 표시  
console.log(arr.reduce((acc, value) => acc + value));  
console.log(arr.reduce((acc, value) => acc * value));
```

04) 기본 내장 객체 - Array 객체의 속성과 메서드(4) 배열 내 요소 정렬

오름차순 정렬	내림차순 정렬
• 1 → 9	• 9 → 1
• 가 → 하	• 하 → 가
• A → Z	• Z → A

매개변수는 없음
↓

{ **sort()** : 배열 내 요소들을 오름차순 정렬 }

{ **reverse()** : 배열 내 요소의 순서를 역순으로 정렬 }

Small Mission 2022~2023 프리미어리그 클럽과 과일을 오름차순 정렬, 내림차순 정렬 하기

스몰 미션 제작 노트

- 아스널, 맨시티, 맨유, 뉴캐슬, 토트넘
- Apple, Banana, Papaya, Peach, Melon
- 아스널, 맨시티, Apple, Banana, 33, 55, 99

배열이름.**sort()**;

배열이름.**reverse()**;

```
<script>
// 한글, 영어, 숫자가 혼합된 기존 배열
var mixed = ['아스널', '맨시티', 'Apple', 'Banana', '33', '55', '99'];

console.log(mixed);
console.log(mixed.sort());
console.log(mixed.reverse());
</script>
```

숫자, 한글, 영문이 혼합된 배열(오름차순 : 숫자 → 영어 → 한글)

```
▶ (7) ['아스널', '맨시티', 'Apple', 'Banana', '33', '55', '99']
▶ (7) ['33', '55', '99', 'Apple', 'Banana', '맨시티', '아스널']
▶ (7) ['아스널', '맨시티', 'Banana', 'Apple', '99', '55', '33']
```

```
<script>
/* 2022~2023 프리미어리그 클럽과 과일을 오름차순 정렬, 내림차순 정렬 하기 */

// 기존 정렬
var eplTeam = ['아스널', '맨시티', '맨유', '뉴캐슬', '토트넘'];
var fruit = ['Apple', 'Banana', 'Papaya', 'Peach', 'Melon'];

// 오름차순 정렬
console.log('오름차순 정렬 : ' + eplTeam.sort());
console.log('오름차순 정렬 : ' + fruit.sort());

// 내림차순 정렬
console.log('내림차순 정렬 : ' + eplTeam.reverse());
console.log('내림차순 정렬 : ' + fruit.reverse());
</script>
```

04) 기본 내장 객체 - Array 객체의 속성과 메서드(5) 콜백 함수(function)로 배열 내 숫자 요소 정렬

{ 콜백 함수로 배열 내 요소들을 오름차순 정렬 }

```
배열객체.sort(function(a, b) {
```

```
    return a - b;
```

```
}); -는 빼기 개념이 아니라 부터 라는 개념 곧, a 부터 b까지
```

{ 콜백 함수로 배열 내 요소들을 내림차순 정렬 }

```
배열객체.sort(function(a, b) {
```

```
    return b - a;
```

```
});
```

▲ a, b 라는 매개변수 이름은 다른 이름으로 사용해도 상관 없습니다. start, end 라고 해도 상관없습니다.

다만 함수에 사용한 매개변수의 이름과 return에 반환할 이름이 같으면 됩니다.

Small Mission)

문자이지만 숫자로 구성된 배열을 숫자로 변형해서 정렬하기. (예시 숫자) 22, 7, 3, 9, 1

▼ Call Back 함수란 메서드 괄호 안에
function 으로 선언되는 함수

```
<script>
// 배열 선언
var mixedNum = [22, 7, 3, 9, 1];
// 기존 배열
console.log(mixedNum);
// 콜백 함수 오름차순 정렬
mixedNum.sort(function(a, b){
    return a - b;
});
console.log(mixedNum);
// 콜백 함수 내림차순 정렬
mixedNum.reverse(function(a, b){
    return b - a;
});
console.log(mixedNum);
</script>
```

```
<script>
// 배열 선언
var mixedNum = [22, 7, 3, 9, 1];
// 기존 배열
console.log(mixedNum);
// 오름차순 정렬
console.log(mixedNum.sort());
// 내림차순 정렬
console.log(mixedNum.reverse());
</script>
```

▲ 오름차순과 내림차순이지만 정렬이 이상합니다. 정상적이라면 1, 3, 7, 9, 22 또는 22, 9, 7, 3, 1 이어야 합니다. (문자처럼 배열되기 때문에 숫자의 크기를 인식하지 못함)

04) 기본 내장 객체 - Array 객체 스몰미션 (행운의 로또 번호 생성기)

이번주 행운 로또 번호는 : 1,9,24,41입니다.

스몰 미션 제작 노트

- ① 로또 번호 6개를 담을 값이 없는 배열 선언
- ② for 반복문을 사용해서 1~45까지 숫자 6개 정수를 랜덤하게 반환
- ③ 6개 숫자 중 같은 숫자가 출력되는 것을 방지하기 위해 for in 문으로 처리
- ④ for 반복문으로 출력된 숫자를 배열에 하나씩 마지막에 추가
- ⑤ 최종적으로 6개 숫자를 보기 좋게 오름차순으로 정렬해서 body에 출력

※ 사용할 구문 및 메서드

중첩 for문, if 문, for in 문, Math.random() floor() push() sort()

lottoCreate 배열의 값(j)가 위에 for 문으로 만들어진 lottoNum과 같은지 if 조건문으로 비교해서 중복 숫자가 나오면 안되니까 같으면 [0~1 사이 숫자를 랜덤하게 출력하는데 1~45 정수 숫자 출력]을 중복되지 않을 때까지 for in 문을 반복하고 같지 않은 숫자를 만들면 push하고 다시 위로 올라가 for 반복문을 다시 돌립니다. 곧, for 문은 6번 돌아갑니다. 하지만 for in 문은 6번 돌아갈 수도 있고 같은 숫자가 발생하면 6번 이상 돌아갈 수도 있습니다.

★ 행운의 숫자 (lucky number):

4, 12, 21, 34, 47, 49

> 추첨시간 (lottery time) : 2023. 2. 10. 오전 6:26:41
[Fri Feb 10 2023 06:26:41 GMT+0900 (한국 표준시)]

다운로드

▲ 구글에서 숫자 랜덤 추출 또는 로또 번호 추출 이런 식으로 검색하면 나오는 로또 번호 생성하는 웹사이트 예시입니다. 이걸 스몰 미션으로 만들도록 합니다.

```
<script>
/* 행운의 로또 번호 생성기 */

// 로또 번호 6개를 담을 값이 없는 배열 선언
var lottoCreate = []; // 배열 내에 들어갈 값의 최초에 정할 수 없습니다. for 반복문을 통해서 랜덤하게 반환되는 숫자가 채워져야 하기 때문에 값이 없는 빈 배열을 선언합니다.

// for 반복문을 사용해서 1~45까지 숫자 6개 정수를 랜덤하게 반환
for(var i = 0; i < 6; i++) { // 초기값이 0이고 6보다 작을 때까지 해야 0, 1, 2, 3, 4, 5 까지 6번 반복(만약 1부터 시작하면 7보다 작게)
    var lottoNum = Math.floor(Math.random() * 44) + 1; // 0~1 사이 숫자를 랜덤하게 출력하는데 1~45 정수 숫자 출력
    // 6개 숫자 중 같은 숫자가 출력되는 것을 방지하기 위해 for in 문으로 처리
    for(var j in lottoCreate) { // lottoCreate 배열이 가지고 있는 값들을 변수 j에 몽땅 넣겠다(in)
        if(lottoNum == lottoCreate[j]) {
            lottoNum = Math.floor(Math.random() * 44) + 1;
        }
    }
    // for 반복문으로 출력된 숫자를 배열에 하나씩 마지막에 추가
    lottoCreate.push(lottoNum); // for 문이 한번 돌 때 마다 lottoNum 배열에 한번씩 push 해줌 (i < 6 경우이기 때문에 5번 반복함)
}

// 최종적으로 6개 숫자를 보기 좋게 오름차순으로 정렬
lottoCreate.sort(function(a, b){
    return a - b; // 콜백 함수로 배열 내 요소들을 오름차순 정렬
});

// 콘솔창에 body에 출력
document.write('이번주 행운 로또 번호는 : ' + lottoCreate + '입니다.');
```

05) 기본 내장 객체 - Date 객체의 선언 & to??() 메서드 Date 객체는 날짜, 시간을 출력하는 메서드

{ Date 객체의 선언 }

```
var day = new Date();
```

Small Mission)

Date 객체로 오늘 날짜 시간 출력, 특정 날짜 시간 출력하기

```
<script>
// 오늘 날짜 시간 출력
var today = new Date();
console.log(today);

// 특정 날짜 시간 출력(올해 크리스마스)
var xMas = new Date(2023, 12, 5);
console.log(xMas);

// 특정 날짜 시간 출력(올해 크리스마스)
var xMas = new Date(2023, 11, 5);
console.log(xMas);
</script>
```

Sun Feb 12 2023 18:27:02 GMT+0900 (한국 표준시)

Fri Jan 05 2024 00:00:00 GMT+0900 (한국 표준시)

Tue Dec 05 2023 00:00:00 GMT+0900 (한국 표준시)

월의 경우 january, february 이렇게 되는데 january 의 인덱스 번호가 0입니다. 그래서 12 라고 입력하면 1월이 됩니다. 그래서 년도가 2024년으로 바뀌고 1월로도 바뀌게 됩니다.

Small Mission)

Date 객체와 to??() 메서드로 오늘 날짜만 출력, 시간만 출력하기

```
<script>
// 오늘 날짜 시간 출력
var today = new Date();
console.log(today);

// 오늘 날짜만 출력
console.log(today.toLocaleDateString());

// 오늘 시간만 출력
console.log(today.toLocaleTimeString());
</script>
```

Sun Feb 12 2023 18:40:11 GMT+0900 (한국 표준시)

2023. 2. 12.

오후 6:40:11

▲ Date 객체의 to??() 메서드는 여러가지가 있습니다. 모두 아실 필요는 없고 필요한 경우에 검색해서 사용하시면 됩니다.

05) 기본 내장 객체 - Date 객체의 Getter와 Setter 메서드 값을 받아 올 때 Getter, 값을 변경할 때 Setter

{ Date 객체 Getter }

메서드 이름	설명
getFullYear()	년도를 4자리 숫자로 가져오기 ex) 2023
getMonth()	월을 숫자로 가져오기(0~11) 인덱스 번호 11이 12월, 0이 1월
getDate()	일을 숫자로 가져옴(1~31)
getHour()	시간을 숫자로 가져옴(0~23)
getMinutes()	분을 숫자로 가져옴(0~59)
getSeconds()	초를 숫자로 가져옴(0~59)
getMilliseconds()	밀리 초를 숫자로 가져옴(0~999)
getTime()	1970년 1월 1일 이후 밀리 초 숫자를 반환함.
getDay()	요일 을 숫자로 가져오기(0~6) 인덱스 번호 0이 일요일, 6이 토요일

▲ getDate()는 1일~31일을 가져오고, getDay() 일요일 ~토요일이라는 요일을 가지고 옵니다.

Mon Feb 13 2023 18:06:04 GMT+0900 (한국 표준시)
2023년 2월 13일
18시 6분 4초

Small Mission)

현재 년도, 월, 일, 시간(시분초) 출력하기

```
<script>
  /* 현재 년도, 월, 일, 시간(시분초) 출력 */

  // Date 객체의 선언 후 변수에 담기
  var today = new Date();
  document.write(today + '<hr>');

  // 년도, 월, 일 변수에 담기
  var yyyy = today.getFullYear();
  var mm = today.getMonth() + 1;
  var dd = today.getDate();

  // 년도, 월, 일 출력
  document.write(yyyy + '년 ' + mm + '월 ' + dd + '일' + '<hr>');

  // 시간, 분, 초 변수에 담기
  var nowHour = today.getHours();
  var nowMinutes = today.getMinutes();
  var nowSeconds = today.getSeconds();

  // 시간, 분, 초 출력
  document.write(nowHour + '시 ' + nowMinutes + '분 ' + nowSeconds + '초');
</script>
```

→ 월의 경우 인덱스 번호 0~11이므로 0이 1월이므로 현재는 2월이므로 +1을 해줘야 정상적인 월이 출력됩니다.

05) 기본 내장 객체 - Date 객체의 Getter와 Setter 메서드

{ Date 객체 Setter }

메서드 이름	설명
setFullYear()	년도를 4자리 숫자로 설정하기 ex) 2023
setMonth()	월을 숫자로 설정하기(0~11) 인덱스 번호 11이 12월, 0이 1월
setDate()	일을 숫자로 설정하기(1~31)
setHour()	시간을 숫자로 설정하기(0~23)
setMinutes()	분을 숫자로 설정하기(0~59)
setSeconds()	초를 숫자로 설정하기(0~59)
setMilliseconds()	밀리 초를 숫자로 설정하기(0~999)
setTime()	1970년 1월 1일 이후 밀리 초 숫자를 설정하기
setDay()	요일 을 숫자로 설정하기(0~6) 인덱스 번호 0이 일요일, 6이 토요일

▲ getDate()는 1일~31일을 변경하고, getDay() 일요일~토요일이라는 요일을 변경합니다.

Small Mission)

그녀를 만난지... 그를 만난지 100일 째 되는 날 계산하기

```

<script>
  /* 그녀를 만난지... 그를 만난지 100일 째 되는 날 계산하기 */

  // Date 객체의 선언 후 변수에 담기
  var today = new Date();

  // 현재 날짜 출력하기
  document.write(today + '<hr>');
  document.write(today.toLocaleDateString() + '<hr>');

  // 현재 날짜 가져와서 변수에 담기
  var dd = today.getDate();
  today.setDate(dd + 99); // 오늘을 포함해서 99일을 더하면 100일째 날

  // 100째 되는 날 출력하기
  document.write('100일 기념일 : ' + today.toLocaleDateString() + '<hr>');
</script>

```

Mon Feb 13 2023 18:28:54 GMT+0900 (한국 표준시)

2023. 2. 13.

100일 기념일 : 2023. 5. 23.

05) 기본 내장 객체 - 스몰 미션

Small Mission) 오늘 날짜와 요일을 문서에 출력하기 ex) 2023년 02월 09일 화요일

2023년 02월 13일 월요일

스몰 미션 제작 노트

- ① Date 객체 선언하기
- ② 날짜 관련 데이터 받아와 각 변수에 담기
- ③ 요일명 배열 선언하기
- ④ 월, 일이 10보다 작은 경우 0을 붙여서 출력하기

요일을 담은 배열 변수 week 만 출력하는 경우 아래처럼 배열의 모든 요소가 출력됩니다.

일,월,화,수,목,금,토

그렇기 때문에 배열 중에 특정 인덱스 번호를 가져와야 합니다.

풀어서 쓰면 week라는 배열 중에서 특정 인덱스 번호를 가져오는데 현재 요일을 담은 day라는 변수의 숫자를 가져옵니다. day라는 변수에는 0~6 중에 하나가 들어가 있습니다.

교재 제작하는 요일이 화요일이므로 현재 day 들어 있는 인덱스 번호는 안보이지만 2가 들어 있습니다.

```
document.write(week[현재요일]);  
document.write(week[2]);  
document.write(week[day]);
```

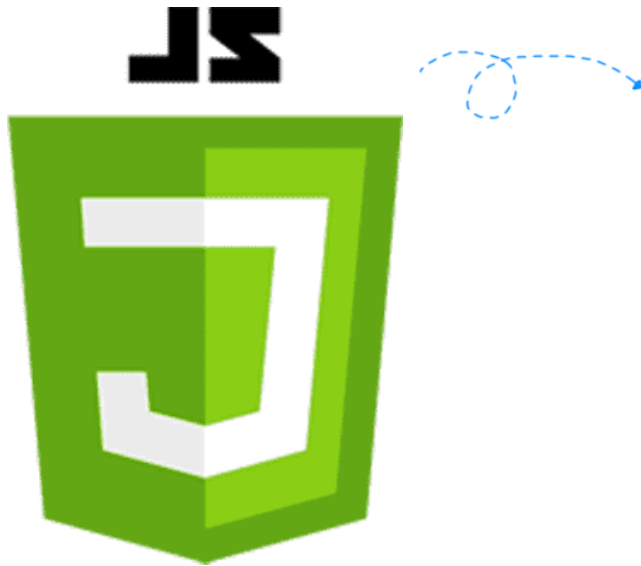
월

```
<script>  
  // Date 객체 선언하기  
  var today = new Date();  
  
  // 날짜 관련 데이터 받아와 각 변수에 담기  
  var yyyy = today.getFullYear();  
  var mm = today.getMonth() + 1; → 0~6이니까 1을 더해서 정상적인 월 출력  
  var dd = today.getDate();  
  var day = today.getDay();  
  
  // 요일명 배열 선언하기  
  var week = ['일', '월', '화', '수', '목', '금', '토']; → 인덱스 번호가 0~6이니까 0이 일요일.. 그래서 일요일부터 문자열 입력  
  ↓  
  // 월, 일이 10보다 작은 경우 0을 붙이기 [0, 1, 2, 3, 4, 5, 6]  
  if(mm < 10) {  
    mm = '0' + mm; → 현재의 mm 앞에 '0'이라는 문자열을 더해서 출력  
  }  
  if(dd < 10) {  
    dd = '0' + dd; → 현재의 dd 앞에 '0'이라는 문자열을 더해서 출력  
  }  
  
  // 결과 출력하기  
  document.write(yyyy + '년 ' + mm + '월 ' + dd + '일 ' + week[day] + '요일');  
</script>
```

JAVASCRIPT 핵심이론 커리큘럼

[웹 퍼블리셔를 위한 자바스크립트] 09. BOM(Browser Object Model) : 브라우저 객체

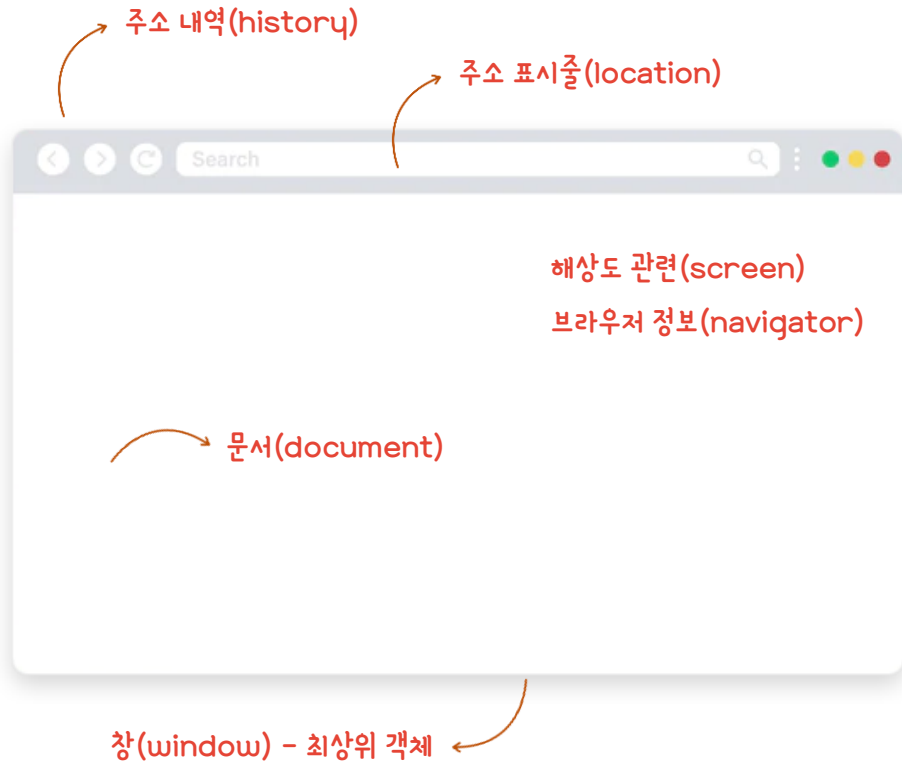
1. BOM(Browser Object Model)
2. window 객체 속성과 메서드



01) BOM BOM은 브라우저 객체 또는 윈도우 객체라고 부릅니다.

윈도우 객체라고 해서 windows, Mac, Linux 이런 컴퓨터 운영체제를 말하는게 아니라 윈도우 = 브라우저 입니다.

{ BOM(Browser Object Model) 구성 } { window 객체의 속성 }



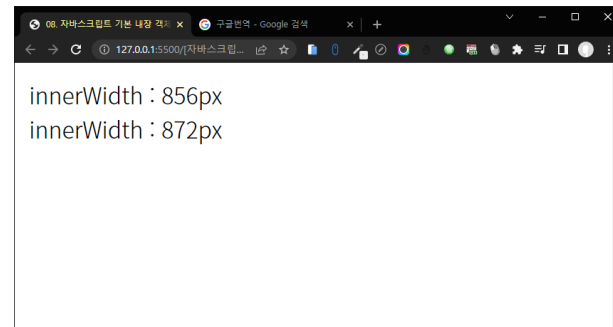
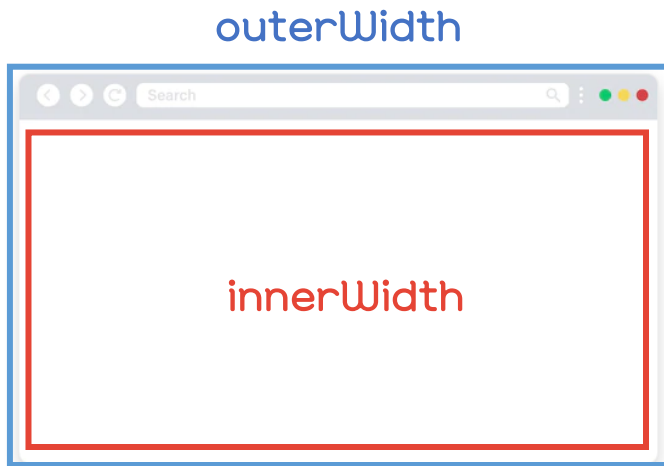
속성 이름	설명
closed	창(윈도우) 열려있는지 닫혔는지 체크하는 불 표현식(닫히면 true, 열리면 false)
innerWidth / innerHeight	창의 콘텐츠 영역 너비와 높이를 반환(스크롤바, 주소 표시줄, 제목 표시줄은 제외)
length	현재 창의 iframe 개수를 숫자로 반환
outerWidth / outerHeight	스크롤바, 주소 표시줄, 제목 표시줄을 포함한 창의 모든 콘텐츠 영역 너비와 높이를 반환
pageXOffset / pageYOffset	현재 문서가 창(윈도우)의 왼쪽에서 또는 위쪽에서 얼마나 스크롤 되었는지 픽셀로 반환
screenLeft / screenTop	창(윈도우)이 왼쪽 또는 위를 기준으로 얼마나 떨어져(offset) 있는지 좌표를 반환
screenX / screenY	창(윈도우)이 왼쪽 또는 위를 기준으로 얼마나 떨어져(offset) 있는지 좌표를 반환
self	현재 창(윈도우)를 반환
top	현재 창(윈도우)의 최상위 브라우저 창을 반환

▲ 모두 알 필요는 당장 없고 예제 제작을 하면서 window 객체 속성을 다루기 때문에 필요한 것만 그때 그때 보시면 됩니다.
속성이기 때문에 괄호가 없습니다. innerWidth() 라고 하면 안됩니다. 괄호가 있는건 메서드입니다.

01) window 객체의 속성과 메서드

{ innerWidth / outerWidth }

Small Mission) 브라우저(window)의 너비 값을 알아내기



스크롤바의 너비가 중요하지는 않지만 스크롤바의 너비는 16픽셀입니다. $872 - 856 = 16$
브라우저 너비가 늘어나건 줄어들건 스크롤바의 너비는 같습니다. 물론 대부분의 브라우저의 스크롤바의 너비가 같지만 브라우저 종류에 따라 다를 수도 있습니다.

```
body {  
  height: 2000px;  
}
```

▲ 스크롤바를 만들기 위해서 강제로 body의 높이를 키움

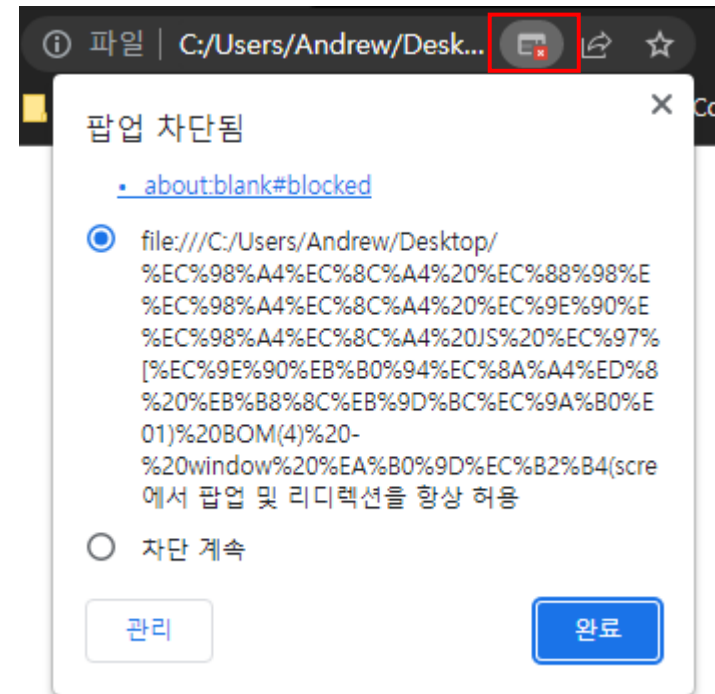
```
<script>  
  /* 브라우저(window)의 너비 값을 알아내기 */  
  
  // 객체명.속성명 ex)window.innerWidth  
  // innerWidth : 스크롤바를 제외한 브라우저 너비  
  document.write('innerWidth : ' + window.innerWidth + 'px' + '<br>');  
  
  // outerWidth : 스크롤바를 포함한 브라우저 너비  
  document.write('innerWidth : ' + window.outerWidth + 'px');  
</script>
```

01) window 객체의 속성과 메서드

{ window 객체의 메서드 }

메서드 이름	설명
close()	창을 닫는 메서드 (매개 변수 없음)
moveBy(x,y)	창을 상대적으로 이동 시키기
moveTo(x,y)	창을 절대적으로 이동 시키기
open(url, name, specs, replace)	새창 또는 새탭으로 창을 열기
print()	인쇄창을 띄우기
resizeBy(w,h)	창 사이즈를 상대적으로 변경시키기
resizeTo(w,h)	창 사이즈를 절대적으로 변경시키기
scrollBy(x,y)	창의 스크롤바 위치를 상대적으로 이동시키기
scrollTo(x,y)	창의 스크롤바 위치를 절대적으로 이동시키기

▲ 모두 알 필요는 당장 없고 예제 제작을 하면서 window 객체 메서드를 다루기 때문에 필요한 것만 그때 그때 보시면 됩니다. 메서드이기 때문에 괄호가 있어야 합니다.



▲ 윈도우 객체를 연습할 때 새 창을 띄우는 경우 팝업 차단 을 해제해야 합니다.

01) window 객체의 속성과 메서드 - open() close()

{ 팝업창을 띄우는 window 객체의 open() 메서드 }

```
window.open('주소', '_blank', '옵션=값, 옵션=값');
```

Small Mission) open 메서드로 팝업창 띄우기

팝업 차단하는 경우가 대부분이어서 요즘은 거의 사용하지 않고, div를 모달(modal)로 띄웁니다.
하지만 윈도우 속성과 메서드 공부 차원에서 한번쯤 해보도록 합니다. 나중에 DOM을 다루게 되면 div 모달을 띄우고 닫는 방법으로 하게 됩니다.

```
<button onclick="openWin()">Open Window</button>
```

```
<script>
```

```
/* open 메서드로 팝업창 띄우기 */
```

```
// open(url, name, specs, replace);
```

```
// window.open('주소', '_blank', '옵션=값, 옵션=값');
```

```
function openWin() {
```

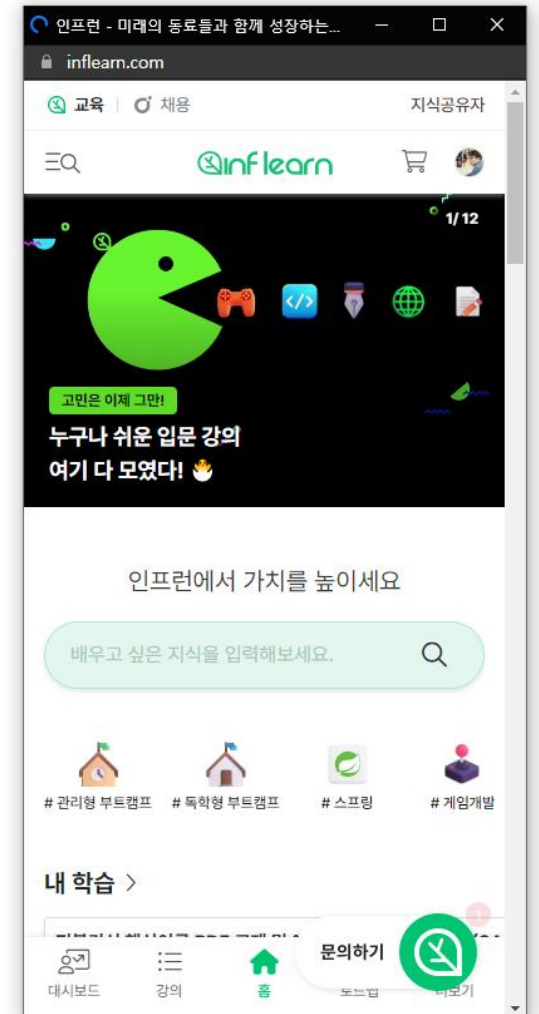
```
    window.open('https://www.inflearn.com/', '_blank', 'width=400, height=750');
```

```
}
```

```
</script>
```

html 문서가 열리면서 바로 팝업창이 뜨게 할 경우는 이 부분만 넣으면 바로 실행됩니다.

Open Window



01) window 객체의 속성과 메서드 - open() close()

{ 팝업창을 닫는 window 객체의 close() 메서드 }

Small Mission 01)

팝업창을 띄우고 3초 후에 팝업창이 자동으로 닫히게 만들기 그리고 자동으로 닫히지 않고 팝업창 내에 닫기를 클릭하면 닫히게 응용하기

스몰 미션 제작 노트

- ① 팝업창이 될 html 간단하게 만듭니다. ex) notice.html
- ② notice.html 문서가 열리면서 자동으로 띄우고 3초 후에 자동으로 닫히기
- ③ 응용해서 팝업창 열기 버튼을 누르면 notice.html이 뜨고 팝업창 닫기 버튼을 누르면 notice.html 닫기

```
<script>
// 코드의 편의상 창 여는 window.open을 변수에 담기
var newWin = window.open('notice.html', '_blank', 'width=400, height=400');

// 3초 후에 팝업창 자동으로 닫기
setTimeout(function(){
    newWin.close();
}, 3000);
</script>
```

`<button onclick="notice();">모달 열기</button>` → 버튼을 클릭해서 팝업창 함수 호출

```
<script>
// 팝업창이 자동으로 뜨지 않게 하기 위해서 함수에 넣기
function notice() {
    var newWin = window.open('notice.html', '_blank', 'width=400, height=400');
}
</script>
```

```
<div>
<h1>공지사항</h1>
<p>...
</p>
<button onclick="window.close();">모달 닫기</button>
</div>
```

해당 html 파일 끝, 윈도우(window)를 닫아라(close)

notice.html

Small Mission 02)

브라우저 현재 탭에서 5초가 지난 후에 자동으로 현재 탭 닫기

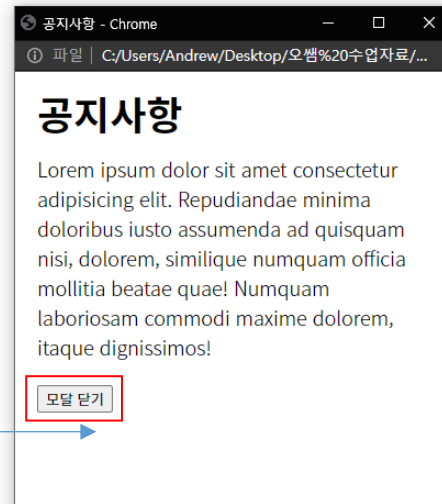
```
<script>
var num = 5;
// 5, 4, 3, 2, 1 시간 경과 보여주기

setInterval(function(){
    document.write('<h1>' + num + '</h1>');
    num--;
}, 1000);

setTimeout(function(){
    window.self.close();
}, 5010);
</script>
```

현재 탭 닫기

모달 열기





02) window 객체의 속성과 메서드 - 위치 관련

{ 위치 관련 window 객체 메서드 }

- `moveBy(x, y)` : 창(window)의 위치를 상대적으로 이동
- `moveTo(x, y)` : 창(window)의 위치를 절대적으로 이동

URL 또는 파일명이 없는 이유는 빈 페이지로 연
습하기 위해서 입니다.

```
<script>
// 브라우저에 따라 다르지만 위치가 지정되지 않으면 왼쪽 상단에 위치
var newWin = window.open('', '', 'width=400, height=400');

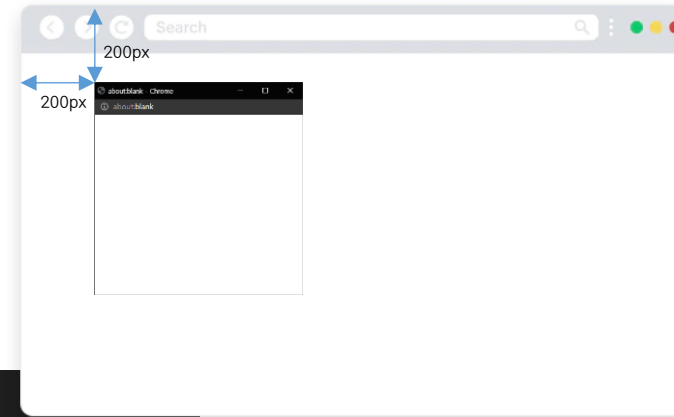
// 화면 기준으로 왼쪽으로 200픽셀, 상단으로 200픽셀 떨어져서 위치
newWin.moveTo(200, 200);
// 1초 마다 30픽셀씩 우측 아래로 이동
setInterval(function(){
    newWin.moveBy(30, 30);
}, 1000);
</script>
```

부모 창(window)의 절대적인 위치

시간이 지날 때 마다 상대적인 위치

해당 예시는 현실적으로 사용할 일은 거의 없지만, 학습 차원에서 한번쯤 해보시기 바랍니다.

Small Mission) 팝업창을 원하는 좌표에 띄우고, 1초마다 30픽셀 씩 이동하기



HELPFUL TIPS

콜백함수를 만들 때 일단 구조 먼저 만들고 중괄호에서 엔터치고 실행구문을 작성하시면 실수하지 않습니다.

```
setInterval(function(){}, 1000);

setInterval(function(){
    // 실행구문 작성
}, 1000);
```



03) window 객체의 속성과 메서드 - 크기 관련

{ 크기 관련 window 객체 메서드 }

- `resizeBy(w, h)` : 창(window)의 크기를 상대적으로 변경
- `resizeTo(w, h)` : 창(window)의 크기를 절대적으로 변경

해당 예시는 현실적으로 사용할 일은 거의 없지만, 학습 차원에서 한번쯤 해보시기 바랍니다.

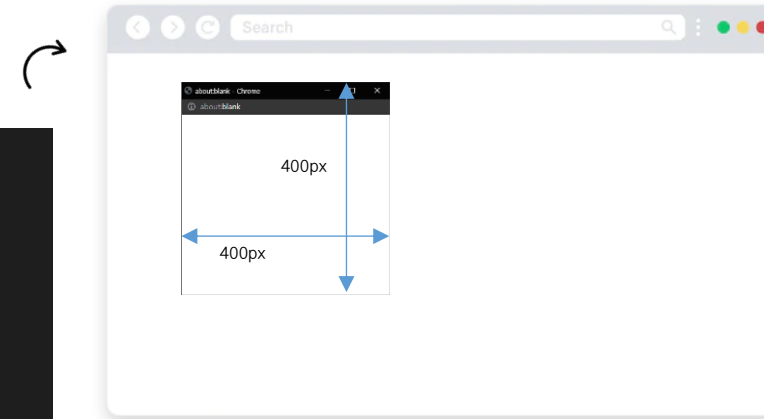
Small Mission)

팝업창을 원하는 좌표에 띄우고, 창의 너비와 높이를 강제로 변경하고, 1초마다 창의 너비와 높이를 30픽셀 씩 변경하기

```
<script>
// 브라우저에 따라 다르지만 위치가 지정되지 않으면 왼쪽 상단에 위치
var newWin = window.open('', '', 'width=400, height=400');

// 화면 기준으로 왼쪽으로 200픽셀, 상단으로 200픽셀 떨어져서 위치
newWin.moveTo(200, 200);
// 최초의 창 크기를 강제로 변경
newWin.resizeTo(400, 400);

// 1초 마다 30픽셀씩 창의 너비와 높이를 변경
setInterval(function(){
    newWin.resizeBy(30, 30);
}, 1000);
</script>
```



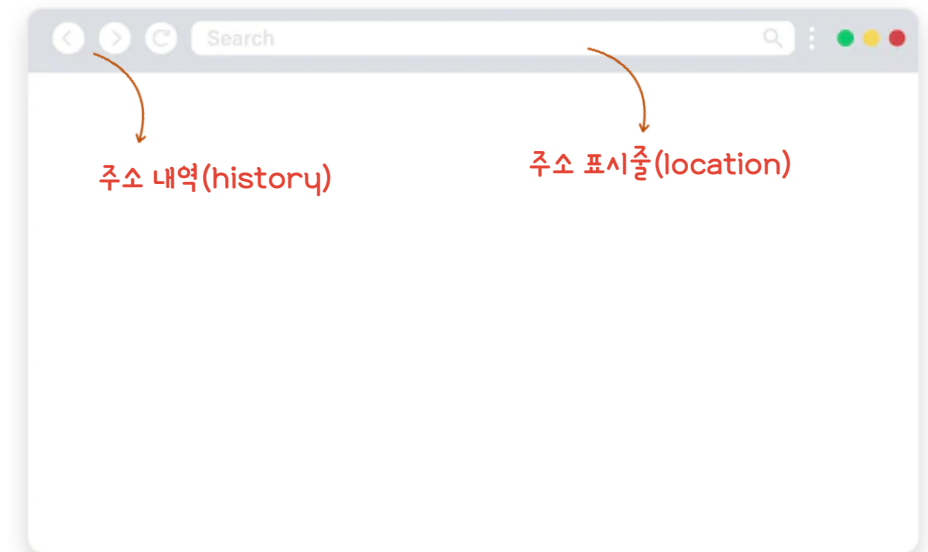
04) window 객체의 속성과 메서드 - 주소 표시줄

{ location 객체의 속성과 메서드 }

속성 이름	설명
has	# 다음에 오는 문자열, 즉 앵커 이름을 표시
host	호스트명과 포트번호 표시
hostname	호스트명을 표시
href	완전한 형태의 URL 주소 표시
pathname	문서의 경로 표시
port	포트번호 표시
protocol	프로토콜 종류 표시
serch	검색 엔진을 실행할 때 나타나는 ? 이후의 문자 표시

▲ 모두 알 필요는 당장 없고 예제 제작을 하면서 필요한 것만 그때 그때 보시면 됩니다.

메서드 이름	설명
assign()	주소 표시줄의 주소를 매개변수의 url 주소로 변경
reload()	문서를 다시 읽음(새로 고침)
replace("URL 주소")	문서를 URL 로 대체하고 이전 페이지로 돌아갈 수 없게 설정 (넷스케이프 전용)



- `window.location.속성` ex) `location.href`;
 - `window.location.속성 = 값` ex) `location.href = 'http://...'`;
 - `window.location.메소드` ex) `location.reload()`;
- `window` 생략 가능

04) window 객체의 속성과 메서드 - 주소 표시줄

{ location 객체의 속성 : href }

href 속성은 현재 페이지의 전체 URL을 설정하거나 반환하는 속성

- URL 값 반환 : `location.href`;
- URL 값 변경 : `location.href = 값`;

Small Mission)

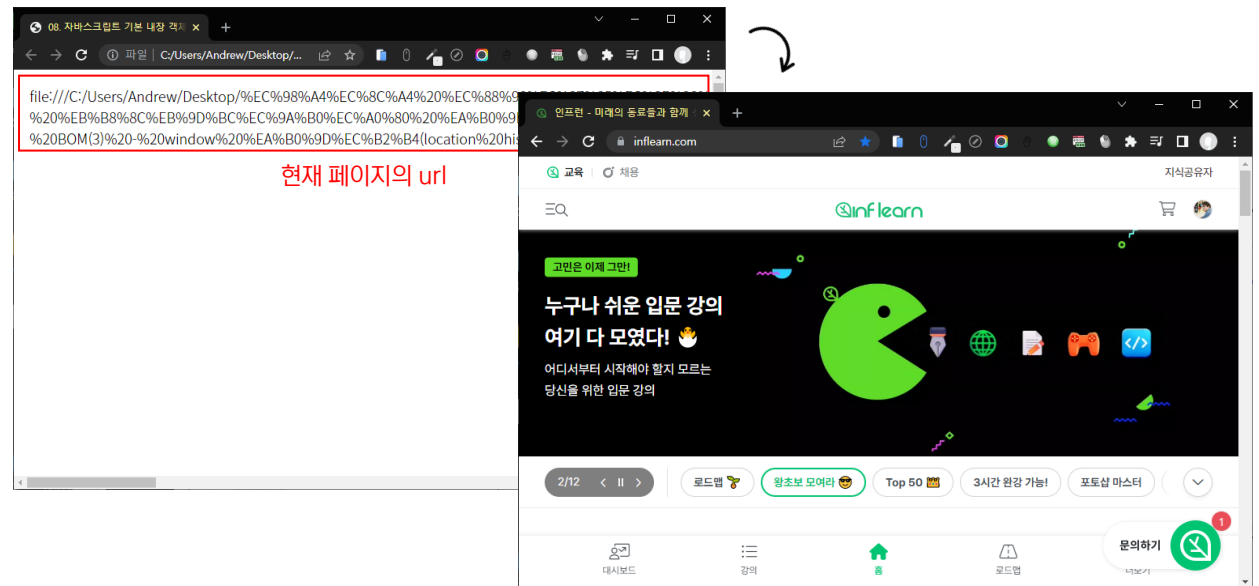
href 속성을 사용해서 현재 페이지의 URL 속성값 받아오기 & 속성값 변경하기

스몰 미션 제작 노트

- ① 현재 페이지의 url 문서에 출력하기
- ② 5초 후에 현재 페이지의 url 값을 인프런(<http://www.inflern.com>)으로 변경해서 이동하기

```
<script>
// 현재 페이지의 url 문서에 출력하기
var url = location.href;
document.write(url);

// 5초 후에 현재 페이지의 url 값을 인프런(http://www.inflern.com)으로 변경해서 이동하기
setTimeout(function(){
  location.href = 'http://www.inflern.com';
}, 5000)
</script>
```



04) window 객체의 속성과 메서드 - 주소 표시줄

{ location 객체의 메서드 : `location.assign()` }

- 주소 표시줄의 주소를 매개변수의 url 주소로 변경

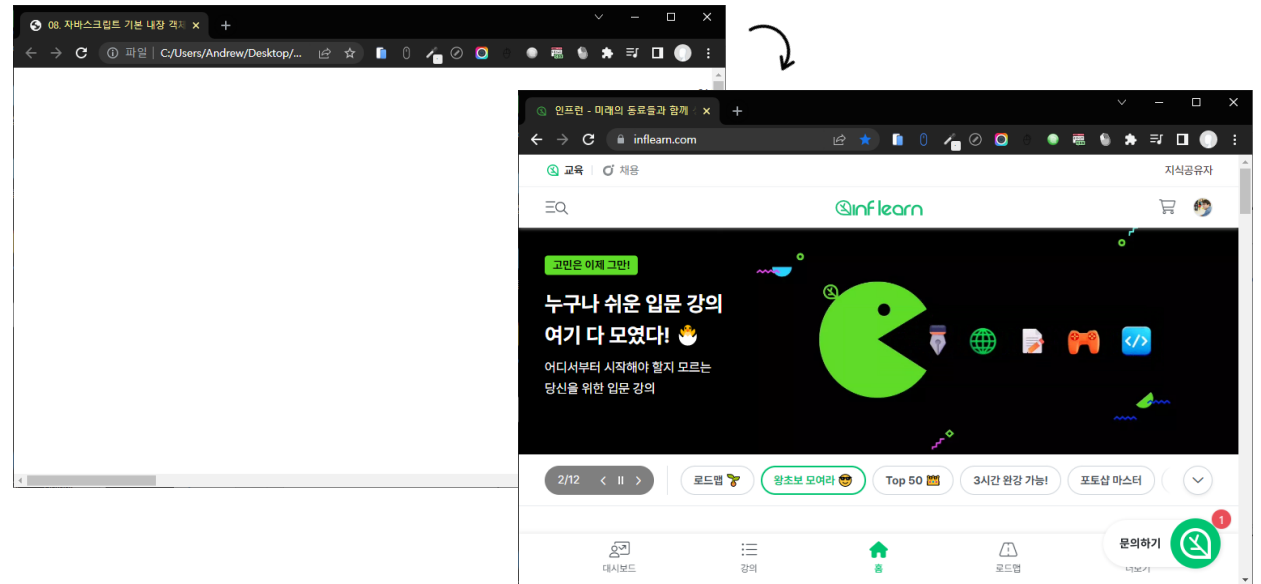
Small Mission)

assign 메서드를 사용해서 현재 페이지의 URL을 5초 후에 인프론으로 이동시키기

```
<script>
/* assign 속성을 사용해서 현재 페이지의 URL을 5초 후에
인프론으로 이동시키기 */
setTimeout(function(){
    location.assign('http://www.infllearn.com');
}, 5000);
</script>
```

아래 3개는 새 URL로 이동하는 완전히 동일한 기능입니다.

- `location.assign(url);`
- `location = url;`
- `location.href = url;`





04) window 객체의 속성과 메서드 - 주소 표시줄

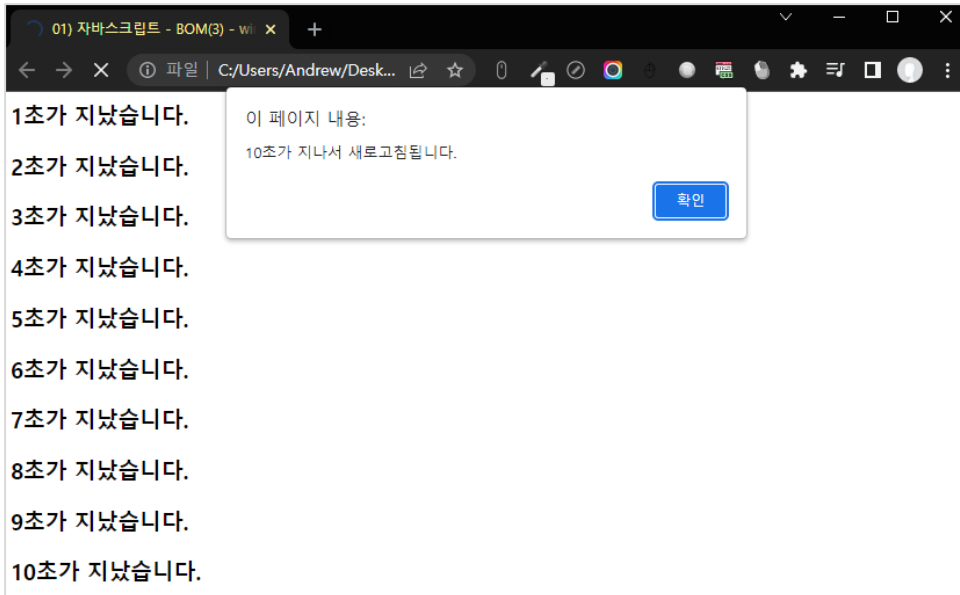
{ location 객체의 메서드 : `location.reload()` }

- 문서를 다시 읽음(새로 고침)

매개변수 없음

Small Mission)

`reload()` 메서드를 사용해서 10초 후 새로 고침 하고 새로 고침 되었다고 경고창 출력(새로 고침 전에 n초가 지났습니다.. 라고 문서에 1초 단위로 출력하기)



```
<script>
// n초가 지났습니다.. 라고 문서에 1초 단위로 출력하기
var num = 1;

setInterval(function(){
    document.write('<h3>' + num + '초가 지났습니다. </h3>');
    num++;
}, 1000);

// reload() 메서드를 사용해서 10초 후 새로 고침 하고 새로 고침
// 되었다고 경고창 출력
setTimeout(function(){
    location.reload();
    alert('10초가 지나서 새로고침됩니다. ');
}, 10010);
</script>
```

정확히 10000으로 하는 경우에는 '10초가 지났습니다.' 라는 마지막 초 단위 출력을 못하고 경고창이 뜨기 때문에 아주 약간 시간을 더 준 것 입니다.

HELPFUL TIPS

- 지속적으로 이벤트가 일어나면 `setInterval()`
- 한번 이벤트가 일어나고 종료되면 `setTimeout()`



05) window 객체의 속성과 메서드 - 주소 내역(1)

Small Mission)

history 메서드로 방문한 이전, 다음 페이지로 이동하기(주소 내역 개수도 확인하기)

스몰 미션 제작 노트

- ① 페이지 이동을 위해서 `history-mission` 이라는 폴더를 만들고 그 안에 html 페이지를 2개 만듭니다. 1개는 클릭해서 이동하고 이동한 페이지에서는 뒤로 가기
- ② `history-start.html` → `history-link.html`

```
<div>
  <a href="history-link-01.html">history-link-01.html로 이동하기</a>
  <p>
    <button>뒤로가기</button>
    <button>주소 내역 개수 확인</button>
  </p>
</div>
```

history-start.html

```
<div>
  <h2>여기는 history-link.html 입니다.</h2>
  <a href="history-start.html"></a>
</div>
```

history-link.html

{ history 객체의 속성 : `history.length`; }

- 주소 내역 개수 반환(주소의 개수를 알아내기)

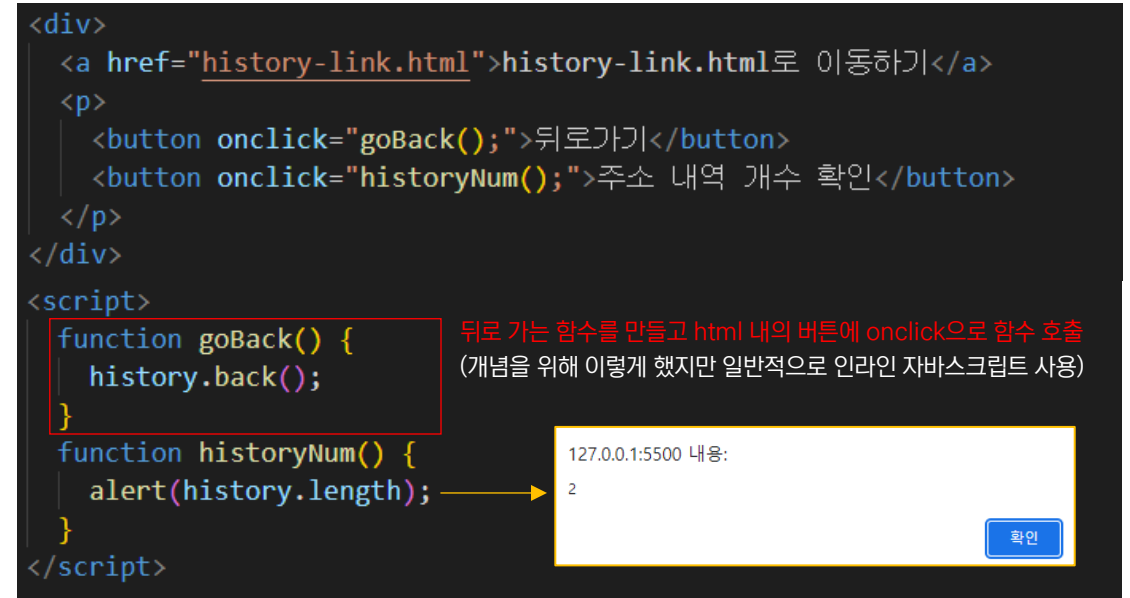
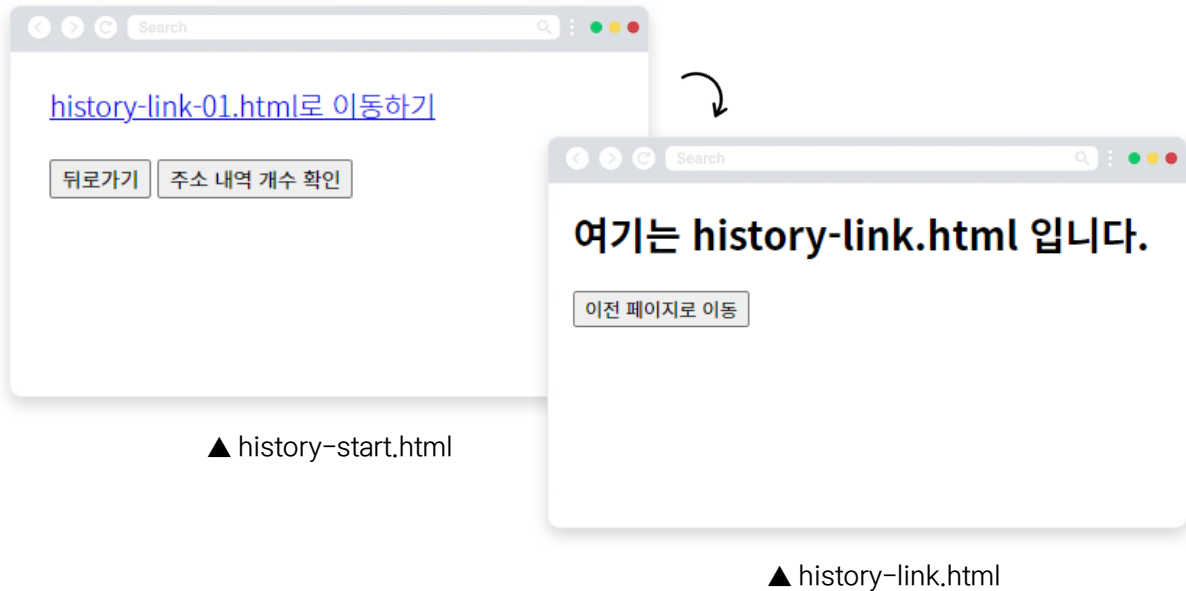
{ history 객체의 메서드 : `go()` `back()` `forward()` }

메서드 이름	설명
<code>go()</code>	현재 브라우저의 방문기록(history) 매개 변수 개수만 이동하기 음수는 이전, 양수는 다음으로 보내기
<code>back()</code>	현재 브라우저의 방문기록(history) 개수에서 이전 목록으로 보내기
<code>forward()</code>	현재 브라우저의 방문기록(history) 개수에서 다음 목록으로 보내기

HOW TO USE

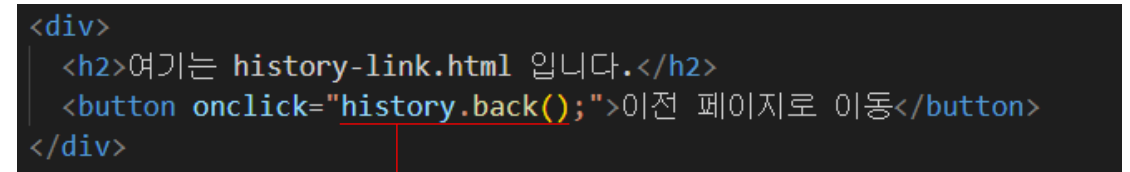
- `history.go();`
- `history.back();`
- `history.forward();`

05) window 객체의 속성과 메서드 - 주소 내역(2)



HELPFUL TIPS

- `history.back()` : 1단계 뒤로 가기 → `history.back(-1)` 과 같음
`history.go(-1)` 과 같음
- `history.back(-2)` : 2단계 뒤로 가기
- `history.forward()` : 1단계 앞으로 가기 → `history.forward(1)` 과 같음
`history.go(1)` 과 같음
- `history.forward(2)` : 2단계 앞으로 가기



보통 인라인 자바스크립트라고 하는 방식으로 `history.back()`을 onclick에 넣으면 위에 처럼 함수를 만들고 호출하지 않고 바로 작동합니다. 일반적인 사용방법입니다. 같은 방식으로 onclick에 `history.forward()`로 사용합니다.



05) window 객체의 속성과 메서드 - 인라인 자바스크립트(Inline JavaScript)

script에 함수를 만들고 함수를 호출하지 않고 html 태그 내에 작성 하는 간편한 자바스크립트

{ 인라인 자바스크립트로 이전 페이지로 이동하기 }

- `history.go(-1);` // 이전 페이지... 즉 한단계 뒤로 돌아가기
- `history.go(-2);` // 두단계 이전의 페이지로 이동
- `history.back();` // `history.go(-1)`과 동일한 기능
- `history.forward();` // `history.go(1)`과 동일하며 앞으로 이동

```
<button onclick="history.back()">뒤로 가기</button>
```

```
<a href="javascript:history.back();">a 태그로 뒤로가기</a>
```

```
<a href="javascript:location.reload()">새로고침</a>
```

```
<a href="javascript>window.close()">창닫기</a>
```

▲ a 태그를 사용할 경우 위의 형식으로 해야 합니다.

{ 인라인 자바스크립트로 외부 파일 링크하기 }

a 태그의 경우 href 와 target 속성이 있어서 편하게 외부 파일을 링크할 수 있지만 a 태그를 제외하고는 div, button 등 href 속성이 없는 태그는 그렇게 할 수 없습니다. 이럴 때 인라인 자바스크립트 외부 파일을 링크합니다.

▼ `location.href` 또는 `window.open` 모두 사용할 수 있습니다.(쌍 따옴표 외 따옴표를 잘 구분해서 사용해야 합니다.)

```
<button onclick="location.href='http://www.infllearn.com'">
  인라인 자바스크립트로 외부파일 링크(location.href)
</button>

<div onclick="window.open('http://www.infllearn.com', '_self')">
  인라인 자바스크립트로 외부파일 링크(window.open)
</div>
```

새 탭으로 외부 파일을 띄울 때는 `window.open`을 사용합니다. '_self'라는 부분을 넣지 않으면 기본적으로 새탭으로 외부 파일이 열립니다. 만약 `window.open`을 사용해서 현재 탭에서 띄울 때는 '_self'를 넣어주어야 합니다.

06) window 객체의 속성과 메서드 - screen & navigator

모니터 해상도 관련(screen) / 사용자의 브라우저 정보(navigator)

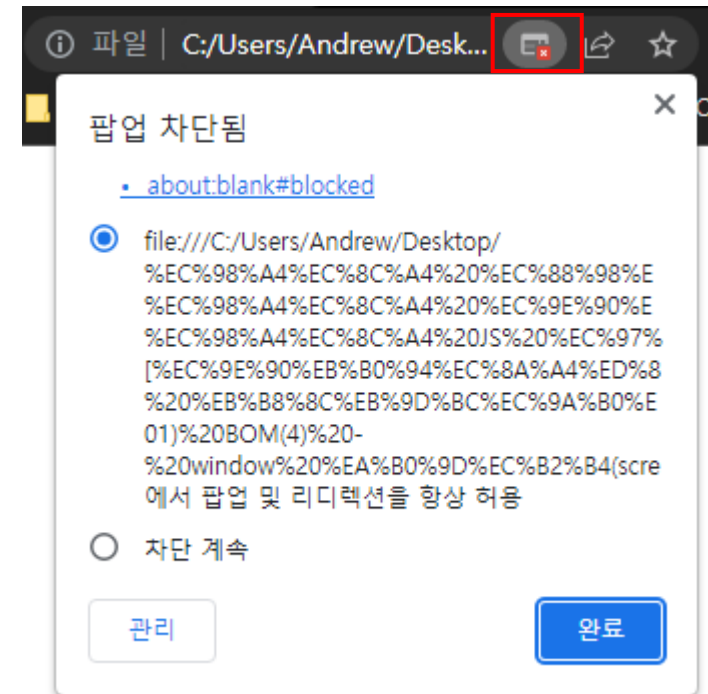
{ screen 객체의 속성 → 매개변수 없음 }

속성 이름	설명
availHeight	화면 높이 반환 (Windows 작업표시줄 제외).
availWidth	화면 너비 반환 (Windows 작업표시줄 제외).
colorDepth	이미지 표시 위한 색상표의 비트 심도 반환.
height	화면 전체 높이 반환. (작업표시줄 포함)
width	화면 전체 너비 반환. (작업표시줄 포함)
pixelDepth	화면의 색상 해상도 (픽셀 당 비트 수) 반환.

▲ 모두 알 필요는 당장 없고 예제 제작을 하면서 필요한 것만 그때 그때 보시면 됩니다.

HOW TO USE

- screen.height;
- screen.width;



▲ 윈도우 객체를 연습할 때 새 창을 띄우는 경우 팝업 차단
을 해제해야 합니다.



06) window 객체의 속성과 메서드 - screen

모니터 해상도 관련(screen)

Small Mission)

새 창을 모바일 사이즈 400 x 750으로 띄운 후 3초 후에 풀스크린(Full Screen)으로 변경하기

스몰 미션 제작 노트

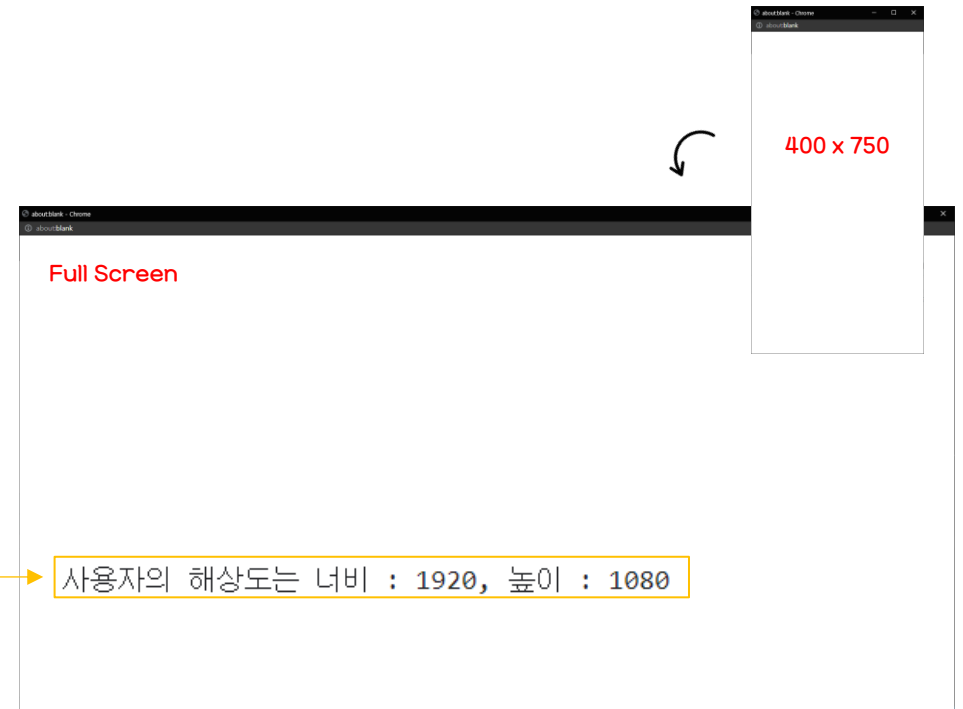
- ① 새 창으로 400 x 750 으로 띄우기
- ② 3초 후에 새 창을 사용자 모니터 해상도에 맞춰 풀스크린으로 변경하기

```
<script>
// 새 창을 모바일 사이즈로 띄울 변수 선언
var popopWin = window.open('', '', 'width=400, height=750');

// 사용자가 사용하고 있는 해상도 화면(screen)의 너비와 높이 가져오기
var userWidth = screen.width;
var userHeight = screen.height;
console.log('사용자의 해상도는 ' + '너비 : ' + userWidth + ', 높이 : ' + userHeight);

// 3초 후에 새 창을 사용자 모니터 해상도에 맞춰 풀스크린으로 변경하기
setTimeout(function(){
  popopWin.resizeTo(userWidth, userHeight);
}, 3000);
</script>
```

강제로 리사이즈하는 메서드로 받아온 매개변수인 사용자의 해상도 1920과 1080을 반환해서 풀 스크린을 만들



06) window 객체의 속성과 메서드 - navigator

사용자의 브라우저 정보(navigator)

{ navigator 객체의 속성 → 매개변수 없음 }

속성 이름	설명
appCodeName	브라우저의 코드명을 반환합니다.
appName	브라우저의 이름을 반환합니다.
appVersion	브라우저의 버전을 반환합니다.
cookieEnabled	브라우저의 쿠키 사용 가능 여부를 반환합니다.
language	브라우저에서 사용되는 언어를 반환합니다.
onLine	브라우저가 온라인인지 여부를 반환합니다.
platform	브라우저가 실행되는 플랫폼 정보를 반환합니다.
product	브라우저에서 사용되는 엔진 이름을 반환합니다.
userAgent	브라우저와 운영체제 정보를 반환합니다.
javaEnabled()	브라우저가 java를 사용할 수 있는지 체크하는 불 표현식(true, false) - 모바일 접속 체크에 필요

HOW TO USE

- navigator.appCodeName;
- navigator.appName;
- navigator.appVersion;
- navigator.language;
- navigator.userAgent;

▲ 모두 알 필요는 당장 없고 예제 제작을 하면서 필요한 것만 그때 그때 보시면 됩니다.



06) window 객체의 속성과 메서드 - navigator

Small Mission)

현재 사용자의 브라우저(navigator)의 정보를 확인하기

```
<script>
/* 현재 사용자의 브라우저(navigator)의 정보를 확인하기 */

// 브라우저 코드명
var print1 = '브라우저 코드명 : ' + navigator.appCodeName;
console.log(print1);

// 브라우저 이름
var print2 = '브라우저 이름 : ' + navigator.appName;
console.log(print2);

// 브라우저 버전
var print3 = '브라우저 버전 : ' + navigator.appVersion;
console.log(print3);

// 브라우저 언어
var print4 = '브라우저 언어 : ' + navigator.language;
console.log(print4);

// 브라우저 에이전트
var print5 = '브라우저 에이전트 : ' + navigator.userAgent;
console.log(print5);
</script>
```

```
<script>
/* 현재 사용자의 브라우저(navigator)의 정보를 확인하기 */

// 변수 하나로 여러개 출력하기(복합 대입 연산자)
var printText= '';

printText += '<p>브라우저 코드명 : ' + navigator.appCodeName + '</p>';
printText += '<p>브라우저 이름 : ' + navigator.appName + '</p>';
printText += '<p>브라우저 버전 : ' + navigator.appVersion + '</p>';
printText += '<p>브라우저 언어 : ' + navigator.language + '</p>';
printText += '<p>브라우저 에이전트 : ' + navigator.userAgent + '</p>';

// 문서에 출력
document.write(printText);
</script>
```

▲ 복합 대입 연산자를 사용해서
짧은 코드로 효율적으로 출력

브라우저 코드명 : Mozilla
브라우저 이름 : Netscape
브라우저 버전 : 5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/109.0.0.0 Safari/537.36
브라우저 언어 : ko-KR
브라우저 에이전트 : Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/109.0.0.0 Safari/537.36

브라우저 코드명 : Mozilla
브라우저 이름 : Netscape
브라우저 버전 : 5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/109.0.0.0 Safari/537.36
브라우저 언어 : ko-KR
브라우저 에이전트 : Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/109.0.0.0 Safari/537.36

06) window 객체의 속성과 메서드 - navigator

{ 모바일 기기 접속 확인 : **userAgent** }

HELPFUL TIPS

- 크롬 브라우저에서 모바일로 변경하는 방법은 아래 블로그에서 확인하세요.
- 코딩웍스 블로그 :

현재 접속한 운영체제는 iPhone OS라는 모바일

Mozilla/5.0 (iPhone; CPU iPhone OS 13_2_3 like Mac OS X) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/13.0.3 Mobile/15E148 Safari/604.1

Mozilla/5.0 (iPhone; CPU iPhone OS 13_2_3 like Mac OS X) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/13.0.3 Mobile/15E148 Safari/604.1

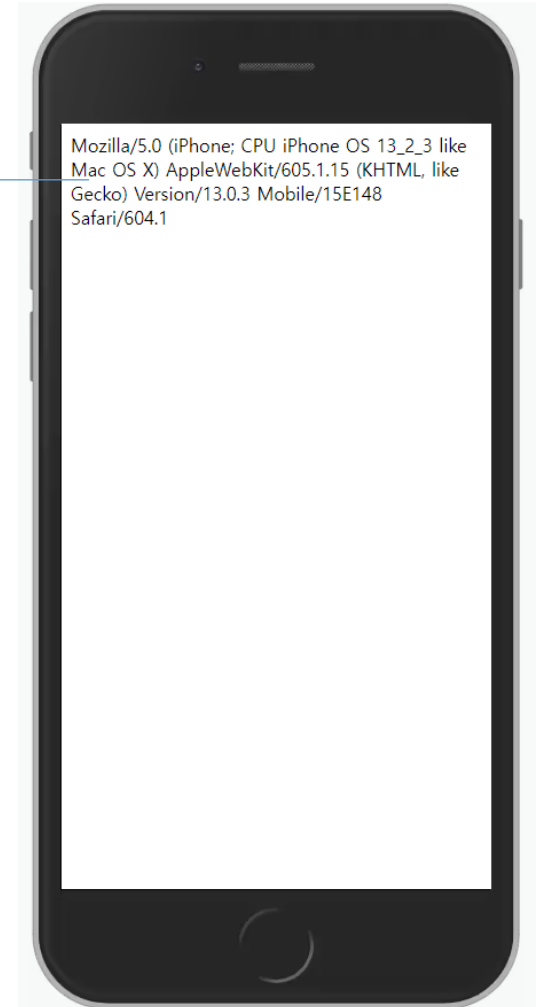
```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>모바일 기기 접속 확인 : userAgent</title>
</head>
```

모바일 접속 여부 체크를
위해 meta 태그 필수

```
<script>
  /* 현재 접속 기기 확인 */
  document.write(navigator.userAgent);
</script>
```

Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/109.0.0.0 Safari/537.36

현재 접속한 컴퓨터 운영체제를 보여줍니다. 곧, PC로 접속함



06) window 객체의 속성과 메서드 - navigator(모바일 기기 접속을 체크 01)

Small Mission 01)

userAgent 를 사용해서 PC 또는 모바일 기기 접속을 체크해서 모바일 접속한 경우 '모바일로 접속하셨습니다.' 라고 경고창 출력

스몰 미션 제작 노트

① 모바일 기기 배열 선언

Android, iPhone, iPad, BlackBerry, Windows Phone, Samsung, LG, Window CE, SonyEricsson, Nokia

② for in 문으로 모바일 기기 배열에 navigator.userAgent와 같은 단어가 있는지(match) 체크해서 있다면(true) 경고창 출력

```
<script>
//모바일 기기 배열 선언
// Android, iPhone, iPad, BlackBerry, Windows Phone, Samsung, LG, Window CE,
// SonyEricsson, Nokia
var mobileCheck = ['Android', 'iPhone', 'iPad', 'BlackBerry', 'Windows Phone', 'Samsung',
'LG', 'Window CE', 'SonyEricsson', 'Nokia'];

// for in 문으로 모바일 기기 배열에 navigator.userAgent와 같은 단어가 있는지(match) 체크해서
// 있다면(true) 경고창 출력
for(var i in mobileCheck) {
  if(navigator.userAgent.match(mobileCheck[i])) {
    alert('모바일로 접속하셨습니다.');
```

mobileCheck[i] 라는 의미는 mobileCheck[] 라는 배열에 i 라는 변수에 담긴 값을 반복해서 개수 만큼 넣어서 문자열이 match 되는지 확인합니다.

```
  }
}
</script>
```

▼ PC로 접속한 경우는 모바일이 아니므로 경고창 출력 없음

127.0.0.1:5500 내용:
모바일로 접속하셨습니다.

확인

모바일 기기 접속 확인



06) window 객체의 속성과 메서드 - navigator(모바일 기기 접속을 체크 02)

Small Mission 02)

모바일 체크해서 경고창이 아니라 PC라면 pc.html로 보내고 모바일 이라면 mobile.html로 보내기(redirect 하기)

스몰 미션 제작 노트

① 모바일 기기 배열 선언

Android, iPhone, iPad, BlackBerry, Windows Phone, Samsung, LG, Window CE, SonyEricsson, Nokia

② for in 문으로 모바일 기기 배열에 navigator.userAgent와 같은 단어가 있는지(match) 체크합니다.

③ 체크해서해서 있다면(true) mobile.html로 보내기 / 체크해서 없다면(false) pc.html로 보내기

<h2>Mobile 버전 웹사이트로 이동했습니다.</h2>

▲ mobile.html

<h2>PC 버전 웹사이트로 이동했습니다.</h2>

▲ pc.html

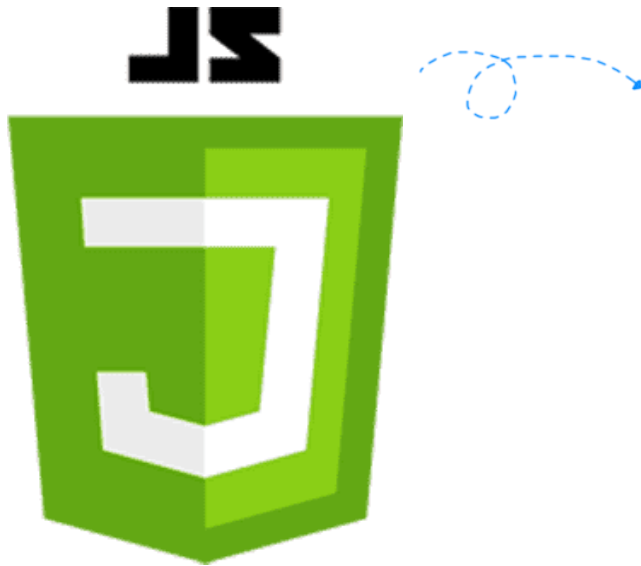
```
// for in 문으로 모바일 기기 배열에 navigator.userAgent와 같은 단어가 있는지(match) 체크합니다.
for(var i in mobileCheck) {
    // 체크해서해서 있다면(true) mobile.html로 보내기
    if(navigator.userAgent.match(mobileCheck[i])) {
        location.href = 'mobile.html';
    }
}
// 체크해서 없다면(false) pc.html로 보내기
location.href = 'pc.html';
```

```
setTimeout(function(){
    location.href = 'https://www.inflearn.com/';
}, 3000);
```

일정 시간이 지난 후 외부 파일로 링크하는 경우 setTimeout과 함께 사용하면 됩니다. ex) 3초 후에 인프런 웹사이트로 다시 보내기(redirect)

JAVASCRIPT 핵심이론 커리큘럼

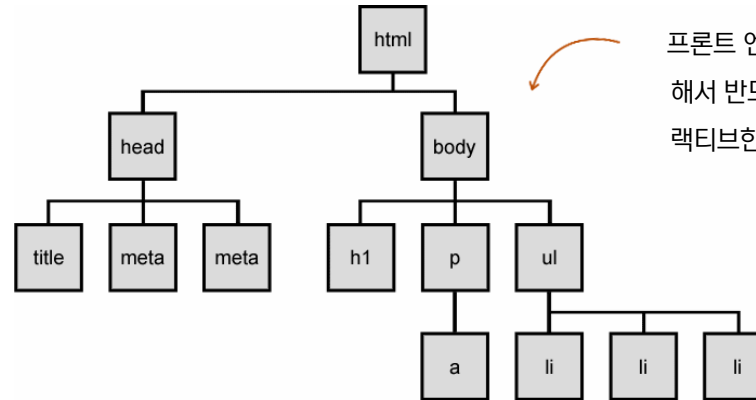
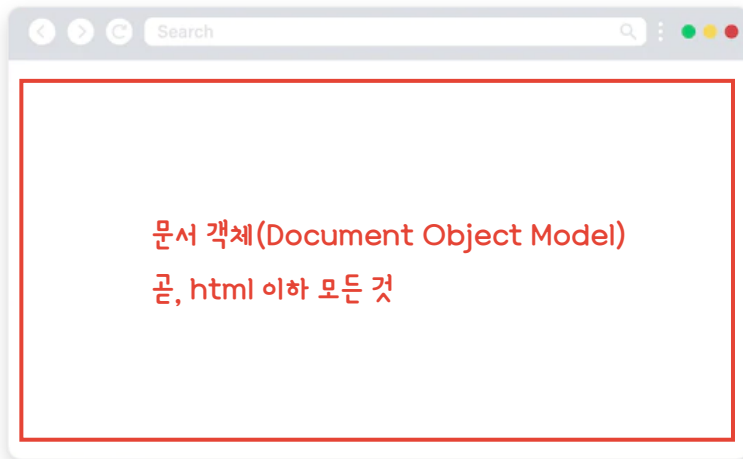
[웹 퍼블리셔를 위한 자바스크립트] 10. DOM(Document Object Model) : 문서 객체



1. DOM 문서 객체란?
2. DOM 문서 객체 가져오는 메서드
`getElementById`, `getElementsByTagName`, `getElementsByClassName`,
`querySelector`, `querySelectorAll`
3. DOM 문서 객체 준비 구문 메서드(`onload`)
4. 문서 객체의 속성과 메서드
5. 이벤트 객체의 속성과 메서드
6. 마우스 & 키보드 이벤트 객체의 속성과 메서드

문서 객체(DOM) 란? DOM은 BOM(Browser Object Model) 중의 하나입니다. DOM은 워낙 내용이 많고 활용도가 많고 자바스크립트에서 가장 중요한 부분이기 때문에 따로 빼서 학습합니다. 곧, HTML 문서 내에 태그 요소를 핸들링합니다.

{ DOM(Document Object Model) }



프론트 엔드 개발자 또는 웹 퍼블리셔라면 자바스크립트 언어에서 DOM에 대해서 반드시 알고 있어야 합니다. 브라우저에서 웹 페이지를 디자인하고 인터랙티브한 기능을 구현하고자 한다면 자바스크립트의 DOM을 반드시 잘 알고 활용할 수 있어야 합니다.

{ 문서 객체를 가져오는 메서드 }

▲ HTML DOM은 HTML 문서를 조작하고 접근하는 표준화된 방법을 정의합니다. 모든 HTML 요소는 HTML DOM를 통해 접근할 수 있습니다.

메서드 이름	설명
<code>getElementById('아이디명')</code>	태그의 id 값으로 문서 객체를 선택함
<code>getElementsByTagName('태그명')</code>	태그 자체를 문서 객체로 선택함
<code>getElementsByClassName('클래스명')</code>	태그의 class 값을 문서 객체로 선택함

▲ 매우 중요하므로 모두 잘 알고 있어야 합니다.

01) get??() 메서드 get으로 시작하는 메서드는 값을 가지고 오는 메서드. 곧, html 내에 태그를 선택하는 메서드

```
{ getElementById('아이디명') }
```

Small Mission)

body 내에 h1 태그 3개를 getElementById로 선택해서 스타일 변경하기

스몰 미션 제작 노트

문서객체(DOM)를 id로 선택해서 변수에 담고, 선택된 문서 객체 스타일 변경

```
<h1 id="heading1">This is Heading Text 1</h1>
<h1 id="heading2">This is Heading Text 2</h1>
<h1 id="heading3">This is Heading Text 3</h1>
```

HTML

HELPFUL TIPS

- 앞으로 `document` 와 `getElementById`를 매우 자주 쓰므로 Emmet 단축키를 사용하면 편합니다.
- `doc` 와 `gbd`를 치면 Emmet 단축키가 나옵니다. `Tab` 또는 `Enter`를 쳐서 사용하세요.

```
var heading.doc
```

abc document

```
var heading1 = document.gbd
```

getElement... (method) Docume...

This is Heading Text 1

This is Heading Text 2

This is Heading Text 3

해당 html 문서에 아이디를 사용했다면 객체 선택 후 변수에 담는 코드가 필요 없이 아이디 이름을 바로 객체 변수 이름으로 바로 사용할 수 있음.

```
<script>
/* body 내에 h1 태그 3개를 getElementById로 선택해서 스타일 변경하기 */

// 문서객체(DOM)를 id로 선택해서 변수에 담기
var heading1 = document.getElementById('heading1');
var heading2 = document.getElementById('heading2');
var heading3 = document.getElementById('heading3');

// 선택된 문서객체 스타일 변경
heading1.style.color = 'orange';
heading2.style.color = 'rgb(63,72,204)';
heading3.style.color = '#eb4d4b';
heading3.style.backgroundColor = 'black';
</script>
```

아이디를 사용했으므로 불필요한 코드

'해당 html 문서 내 id 값이 heading3인 것'이라는 뜻
아이디지만 #을 사용하면 안됩니다.

CSS에서는 `background-color` 라고 쓰지만 자바스크립트는 `카멜 표기법`을 사용

- 문서 객체의 CSS 스타일 사용할 때 아래와 같은 형식으로 사용합니다.
- 대상.style.속성 = 값 ex) `heading1.style.color = 'orange'`
- 숫자는 따옴표가 필요 없지만 `orange`는 문자열이기 때문에 반드시 따옴표로 묶어야 합니다.
- 자바스크립트에서 단어가 중첩된 CSS 속성은 .css 파일에서 쓰는 CSS 속성과 사용하는 방식이 다릅니다.

01) get??() 메서드

{ **getElementsByTagName('태그명')** }  태그는 문서에서 여러 개 사용되기 때문에 복수인 배열로 선택합니다. 그래서 s 가 붙습니다.

Small Mission)

body 내에 h1 태그 3개를 getElementsByTagName으로 Tag를 선택해서 스타일 변경하기

```
<h1>This is Heading Text 1</h1>
<h1>This is Heading Text 2</h1>
<h1>This is Heading Text 3</h1>
```

HTML

HELPFUL TIPS

- 앞으로 **getElementsByTagName**을 매우 자주 쓰므로 Emmet 단축키를 사용
- gbt**를 치면 Emmet 단축키가 나옵니다. Tab 또는 Enter를 쳐서 사용하세요.

```
var heading = document.gbt
```

```
getElementsByT... (method) D...
getElementsByTagNameNS
getElementById
getElementsByClassName
getElementsByTagName
```

color 속성을 사용할 수 없다는 오류가 나옵니다.
그래서 배열을 사용해서 선택해야 합니다.

✖ Uncaught TypeError: Cannot set properties of undefined (setting 'color')

```
<script>
// 문서객체(DOM)를 Tag로 선택해서 변수에 담기
var heading = document.getElementsByTagName('h1');

// 선택된 문서 객체 스타일 변경
heading.style.color = 'crimson';
</script>
```

```
<script>
// 문서객체(DOM)를 Tag로 선택해서 변수에 담기
var heading = document.getElementsByTagName('h1');

// 배열로 선택된 문서 객체 스타일 변경
heading[0].style.color = 'crimson';
heading[1].style.color = 'yellowgreen';
heading[2].style.color = 'royalblue';
</script>
```

h1은 문서에서 3개 이상의 복수이므로 배열로 선택합니다. 출현 순서상 0, 1, 2

This is Heading Text 1

This is Heading Text 2

This is Heading Text 3

01) get??() 메서드

{ `getElementsByClassName('클래스명')` } 클래스명도 문서에서 여러 개 사용되기 때문에 복수인 배열로 선택합니다. 그래서 s 가 붙습니다.

Small Mission)

h1 태그 3개를 `getElementsByClassName` 으로 선택해서 스타일 변경하기

```
<h1 Class="heading">This is Heading Text 1</h1>
<h1 Class="heading">This is Heading Text 2</h1>
<h1 Class="heading">This is Heading Text 3</h1>
```

HTML

```
<script>
// 문서객체(DOM)를 class로 선택해서 변수에 담기
var heading = document.getElementsByClassName('heading');

// 배열로 선택된 문서 객체 스타일 변경
heading[0].style.color = 'crimson';
heading[1].style.color = 'yellowgreen';
heading[2].style.color = 'royalblue';
</script>
```

클래스명이지만 콤마(.)을 사용하면 안됩니다.

.heading 이라는 클래스는 문서에서 3개 이상의 복
수이므로 배열로 선택합니다. 출현 순서상 0, 1, 2

HELPFUL TIPS

- 앞으로 `getElementsByClassName` 도 자주 쓰므로 Emmet 단축키를 사용
- `gbc`를 치면 Emmet 단축키가 나옵니다. `Tab` 또는 `Enter`를 쳐서 사용하세요.

```
var heading = document.gbc
```

`getElementsByCl...` (method)...

This is Heading Text 1

This is Heading Text 2

This is Heading Text 3

01) query??() 메서드 CSS 선택자를 매개변수로 선택합니다.

{ **querySelector('CSS 선택자')** }  같은 이름의 CSS 선택자가 여러 개 있으면 첫번째 요소를 선택합니다.

Small Mission)

h1 태그가 여러 개 있어도 특정 부모요소 내에 있는 h1 태그 **하나만** 선택하기

```
<div class="frame">
  <h1>This is Heading Text 1</h1>
  <h1>This is Heading Text 2</h1>
  <h1>This is Heading Text 3</h1>
</div>
<h1>This is Heading Text 4</h1>
<h1>This is Heading Text 5</h1>
```

HTML

HELPFUL TIPS

- 앞으로 **querySelector** 도 자주 쓰므로 Emmet 단축키를 사용
- qs**를 치면 Emmet 단축키가 나옵니다. **Tab** 또는 **Enter**를 쳐서 사용하세요.

```
var heading = document.qs
```

- querySel... (method) ParentN...
- querySelectorAll
- queryCommandState
- queryCommandSupported

```
<script>
  // 문서객체(DOM) 선택 후 변수에 담기
  var heading = document.querySelector('.frame h1');

  // 배열로 선택된 문서 객체 스타일 변경
  heading.style.color = 'crimson';
</script>
```

따옴표 안에 우리가 알고 있는 CSS 선택자 방식을 모두 사용할 수 있습니다.
ex) .frame h1:nth-child(2)

.frame 밑에 h1이 3개 있지만 첫번째 요소를 선택합니다.

```
heading[0].style.color = 'crimson';
```

querySelector를 사용한 경우 배열로 선택할 수 없으므로 오류가 납니다.

✖ Uncaught TypeError: Cannot read properties of undefined (reading 'style')

This is Heading Text 1
This is Heading Text 2
This is Heading Text 3
This is Heading Text 4
This is Heading Text 5

01) query??() 메서드

{ **querySelectorAll('CSS 선택자')** } → 같은 이름의 CSS 선택자가 여러 개 있는 경우 배열로 선택

Small Mission)

h1 태그가 여러 개 있어도 특정 부모요소 내에 있는 h1 태그들 **모두** 선택하기

```
<div class="frame">
  <h1>This is Heading Text 1</h1>
  <h1>This is Heading Text 2</h1>
  <h1>This is Heading Text 3</h1>
</div>
<h1>This is Heading Text 4</h1>
<h1>This is Heading Text 5</h1>
```

HTML

```
<script>
  // 문서객체(DOM) 선택 후 변수에 담기
  var heading = document.querySelectorAll('.frame h1');

  // 배열로 선택된 문서 객체 스타일 변경
  heading[0].style.color = 'crimson';
  heading[1].style.color = 'yellowgreen';
  heading[2].style.color = 'royalblue';
</script>
```

.frame 밑에 h1이 3개 모두 선택하기 위해 배열로 선택

This is Heading Text 1

This is Heading Text 2

This is Heading Text 3

This is Heading Text 4

This is Heading Text 5

HELPFUL TIPS

- 앞으로 **querySelectorAll** 도 자주 쓰므로 Emmet 단축키를 사용
- qsa**를 치면 Emmet 단축키가 나옵니다. **Tab** 또는 **Enter**를 쳐서 사용하세요.

```
var heading = document.qsa
```

```
querySelect... (method) Pare...
queryCommandState
queryCommandSupported
```

02) DOM 문서 객체 준비 구문 메서드

onload란 문서가 준비되었다.. 를 의미합니다.

(문서가 준비되었다는 말은 body 안에 html 요소를 모두 읽어온 상태라는 의미)

```
window.onload = function(){  
    // 실행할 구문  
};
```

▲ 자바스크립트 방식 onload

```
document.addEventListener('DOMContentLoaded', function(){  
    // 실행할 구문  
});
```

▲ 자바스크립트 방식 DOMContentLoaded

```
$(function(){  
    // 실행할 구문  
});
```

▲ 제이쿼리 방식 onload

→ 브라우저가 html 코드를 읽을 때 맨 위에서 부터 아래로 읽어 내려가는 방식입니다. 있어 보이는 말로 FIFO(First In First Out)이라고 합니다. (참고) 모든 것은 window의 하위 객체이므로 window를 생략해도 상관없음

▼ 아래 2가지 방식은 모두 동일하므로 원하는 방식을 사용하면 됩니다.

```
document.addEventListener('DOMContentLoaded', function(){  
    // 실행구문  
})  
  
window.onload = function(){  
    // 실행 구문  
}
```

▼ 위에서 아래로 읽어 내려오는 방식이기 때문에 브라우저가 h1 태그를 읽은 상태에서 script를 다음으로 읽기 때문에 오류없이 정상적으로 출력됩니다.

```
<!DOCTYPE html>  
<html lang="ko">  
<head>  
  <meta charset="UTF-8">  
  <title>10. 자바스크립트 - 문서 객체(DOM)</title>  
  <script>  
    var heading = document.getElementsByTagName('h1');  
    heading[0].style.color = 'crimson';  
  </script>  
</head>  
<body>  
  
  <h1>This is Heading Text</h1>  
  
</body>  
</html>
```

▲ 위의 경우 브라우저가 body 안에 h1 태그를 읽어 오기 전에 script가 실행되기 때문에 오류가 출력됩니다. 그래서 스크립트를 head 사이에 넣을 경우 onload 를 사용해야 합니다.

```
<head>  
  <meta charset="UTF-8">  
  <title>10. 자바스크립트 - 문서 객체(DOM)</title>  
  <script>  
    window.onload = function() {  
      var heading = document.getElementsByTagName('h1');  
      heading[0].style.color = 'crimson';  
    }  
  </script>  
</head>  
  
  <body>  
    <h1>This is Heading Text</h1>  
  </body>  
</html>
```

실무에서는 따로 js 파일을 만들어 head 사이에 링크해서 사용합니다.

head 사이에 스크립트가 들어가지만 onload에 의해 body 내용을 읽고 난 다음 콜백 함수가 실행되기 때문에 정상적으로 작동합니다.

03) DOM 문서 객체의 생성 메서드

문서 객체의 생성이란 html 요소(elements)를 만드는 것을 말합니다.
(html에 직접 태그를 만드는 것을 정적 생성, JavaScript로 태그를 만드는 것을 동적 생성)
※ 문서 객체 생성 메서드도 엄청 많습니다. 예를 통해서 자주 사용하는 것들을 알아가세요.

- `createElement('태그명')` → 태그 요소를 생성하는 메서드
- `appendChild()` → 생성한 태그 요소를 자식 요소로 삽입하는 메서드
- `innerHTML = '텍스트'` → 텍스트를 추가하는 메서드

▶ html 태그가 몇 개라면 모르겠지만 예시처럼 엄청 많은 경우 직접 html 치는 것은 너무 생산적이지 못합니다. 그래서 자바스크립트로 html 요소를 동적(자동)으로 생성합니다.

Small Mission)

body에 h2 엘리먼트를 자바스크립트로 동적으로 추가하기

동적으로 생성된 H2 엘리먼트

```
<script>
// 엘리먼트 동적 생성
var tagH2 = document.createElement('h2'); → h2이라는 엘리먼트 생성 후 변수에 담기

// 동적으로 생성한 엘리먼트 body 출력
document.body.appendChild(tagH2); → document 내 body라는 속성에 자식 요소로 tagH2라는 변수를 추가 (현재 사용된 body는 속성입니다.)

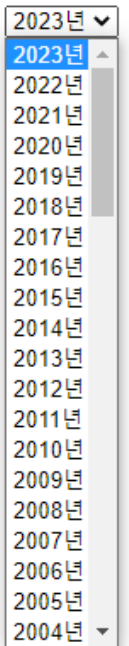
// 동적으로 생성한 엘리먼트에 텍스트 넣기
tagH2.innerHTML = '동적으로 생성된 H2 엘리먼트'; → h2 내에 텍스트가 없으면 소스코드에는 있지만 브라우저에서는 보이지 않음
</script>
```

REFERENCE

[참고1] Element가 html 태그와 같은 개념은 아닙니다. 같다고 생각해도 크게 지장은 없지만 정확한 개념을 알아두는게 좋습니다. 엘리먼트는 html 태그 중에 문서에 쓰인 것을 말합니다.

ex) <hr>이라는 태그가 문서에 사용되어야 비로소 엘리먼트라고 부릅니다. 사용되지 않으면 그냥 태그라고 부릅니다.

[참고2] append : 추가하다, 덧붙이다, 첨부하다 / child : 자손



03) DOM 문서 객체의 속성 관련 메서드 → 태그의 속성 값을 가져오거나 속성의 값을 변경하는 메서드

{ **getAttribute('속성명')** } → 태그의 속성 값을 가져오는 메서드

{ **setAttribute('속성명', 값)** } → 태그의 속성 값을 변경하는 메서드

Small Mission)

body에 img 엘리먼트를 자바스크립트로 동적으로 추가하고 img 속성 값 변경하기, img의 속성 값 가져오기

HELPFUL TIPS

- 앞으로 **appendChild** 도 자주 쓰므로 Emmet 단축키를 사용
- apc**를 치면 Emmet 단축키가 나옵니다. **Tab** 또는 **Enter**를 쳐서 사용하세요.

```
document.body.apc
```

```
appendCh... (method) Node.app...  
ariaPlaceholder  
ariaColSpan
```



▲ 동적으로 생성된 img 최초 사이즈



▲ 너비 400px 로 변경

```
<script>  
  // 엘리먼트 동적 생성  
  var tagImg = document.createElement('img');  
  
  // 동적으로 생성한 엘리먼트 body 출력  
  document.body.appendChild(tagImg);  
  
  // 동적으로 생성한 엘리먼트에 속성 값 변경  
  tagImg.setAttribute('src', 'sample-01.jpg');  
  tagImg.setAttribute('width', 400);  
  
  // 동적으로 생성한 엘리먼트에 속성 값 가져오기  
  alert(tagImg.getAttribute('width'));  
  alert(tagImg.getAttribute('src'));  
  alert(tagImg.getAttribute('height'));  
</script>
```

→ img라는 엘리먼트 생성 후 변수에 담기
이걸 문서 객체를 선택한다고 합니다.

→ document 내 body라는 속성에 자식 요소로 tagImg 라는 변수를 추가 (현재 사용된 body 는 속성입니다.)

→ 생성된 img 엘리먼트에 src와 width 속성과 값을 입력(문자열인 경우 반드시 따옴표, 숫자인 경우 따옴표 없이 사용)

현재 img에 height 속성 자체가 없기 때문에 값을 가져올 수 없음

03) DOM 문서 객체의 속성 관련 메서드

Small Mission)

1. 버튼을 클릭했을 때 이미지의 경로를 가져오기(getAttribute)
2. 버튼을 클릭했을 때 이미지의 경로를 세팅하기(setAttribute)
3. 마우스 오버하면 이미지가 변경되고 마우스 아웃하면 이미지가 원래로 돌아오기

```
<div class="frame">
  
  <p>
    <button onclick="imageSrc()">이미지 경로 가져오기</button>
    <button onclick="imageChange()">이미지 경로 변경하기</button>
  </p>
</div>
```

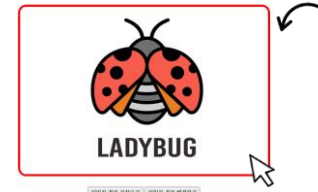
HTML

```
<script>
// 문서 객체 선택
var image = document.querySelector('.image');

// 이미지 경로 가져오기
function imageSrc() {
  alert(image.getAttribute('src'));
}

// 이미지 경로 변경하기
function imageChange() {
  image.setAttribute('src', 'ladybug.png');
}
</script>
```

JS



```
// 마우스 오버/아웃 이미지 변경
function changeImage() {
  image.src = 'ladybug.png';
  image.title = '무당벌레';
  image.style.border = '5px solid red';
  image.style.borderRadius = '20px';
}

function originImage() {
  image.src = 'bee.png';
  image.style.border = 'none';
  image.style.borderRadius = 0;
}

// 마우스 이벤트
image.addEventListener('mouseover', changeImage);
image.addEventListener('mouseout', originImage);
```

마우스 오버 & 아웃



이미지 경로 가져오기 이미지 경로 변경하기



이미지 경로 가져오기 이미지 경로 변경하기

03) DOM 문서 객체의 텍스트 컨트롤 속성(1)

- { innerHTML = '내용' } → 내용에 있는 텍스트는 텍스트로 읽고 태그를 태그로 인식
- { innerText = '내용' } → 내용에 있는 태그를 태그로 인식하지 않고 그냥 문자로 인식
- { textContent = '내용' } → 내용에 있는 태그를 태그로 인식하지 않고 그냥 문자로 인식

Small Mission)

body에 h1 엘리먼트 3개를 만들고 h1 내에 텍스트를 자바스크립트로 동적으로 innerHTML, innerText, textContent를 사용해서 넣기

```
<h1 id="heading1"></h1>
<h1 id="heading2"></h1>
<h1 id="heading3"></h1>
```

HTML

a 태그 링크1

a 태그 링크2

a 태그 링크3

```
<script>
// 엘리먼트 아이디 변수에 담기
var heading1 = document.getElementById('heading1'); → img라는 엘리먼트 생성 후 변수
var heading1 = document.getElementById('heading1'); → 에 담기 (문서 객체를 선택)
var heading1 = document.getElementById('heading1');

// 엘리먼트 내에 a태그 넣기
heading1.innerHTML = 'a 태그 <a href="#">링크1</a>';
heading2.innerText = 'a 태그 <a href="#">링크2</a>';
heading3.textContent = 'a 태그 <a href="#">링크3</a>';
</script>
```

03) DOM 문서 객체의 텍스트 컨트롤 속성(2)

- { `innerHTML` = '내용' } → 내용에 있는 텍스트는 텍스트로 읽고 태그를 태그로 인식
- { `innerText` = '내용' } → 내용에 있는 태그를 태그로 인식하지 않고 그냥 문자로 인식
- { `textContent` = '내용' } → 내용에 있는 태그를 태그로 인식하지 않고 그냥 문자로 인식

Small Mission)

body에 h3 엘리먼트 1개를 만들고 h3 내에 span 태그를 안보이게 하고 `innerHTML`, `innerText`, `textContent`를 사용해서 콘솔창에서 비교하기

```
<h3 id="heading">
  환영합니다. <span class="user-name">코딩웍스</span> 님
</h3>
```

HTML

```
.user-name {
  display: none;
}
```

CSS

환영합니다. 님

```
<script>
  // 엘리먼트 아이디 변수에 담기(문서 객체 선택)
  var heading = document.getElementById('heading');

  // 변수 콘솔창에 출력
  console.log(heading.innerHTML); → 환영합니다. <span class="user-name">코딩웍스</span> 님
  console.log(heading.innerText); → 환영합니다. 님
  console.log(heading.textContent); → 환영합니다. 코딩웍스 님
</script>
```

03) DOM 문서 객체의 자손 컨트롤 메서드(1)

Small Mission)

body에 아래 html 을 만들고 자손 컨트롤 메서드를 사용해서 요소를 제거, 추가, 변경하세요.

```
<div class="frame">
  <h1 class="heading">This is Heading</h1>
  <p class="desc">
    Lorem ipsum dolor sit amet consectetur, adipisicing elit. Alias
    repellat qui quam illo eius fuga explicabo vero sequi est nulla
    eos incidunt commodi atque ipsam repudiandae facere quibusdam,
    esse odio?
  </p>
</div>
```

HTML

This is Heading

Lorem ipsum dolor sit amet consectetur, adipisicing elit. Alias repellat qui quam illo eius fuga explicabo vero sequi est nulla eos incidunt commodi atque ipsam repudiandae facere quibusdam, esse odio?

◀ 최초 상태

This is Heading

◀ removeChild() 메서드로 p 태그 제거

This is Heading

◀ appendChild() 메서드로 hr 태그 추가

h1 태그가 b 태그로 변경되었습니다.

◀ replaceChild() 메서드로 h1을 b로 변경

{ appendChild(요소) } → 자손 요소 추가 메서드

{ removeChild(요소) } → 자손 요소 제거 메서드

{ replaceChild(신규요소, 기존요소) } → 자손 요소 변경 메서드

```
<script>
  // 엘리먼트 아이디 변수에 담기(문서 객체 선택)
  var frame = document.querySelector('.frame');
  var heading = document.querySelector('.heading');
  var desc = document.querySelector('.desc');
  // removeChild(요소) : removeChild() 메서드로 p 태그 제거
  frame.removeChild(desc);
  // removeChild(요소) : hr 태그 추가
  var hrTag = document.createElement('hr');
  frame.appendChild(hrTag);
  // replaceChild(신규요소, 기존요소) : h1을 b로 변경
  var bTag = document.createElement('b');
  bTag.innerText = 'h1 태그가 b 태그로 변경되었습니다.';
  frame.replaceChild(bTag, heading);
</script>
```

→ 문서 객체 생성

→ 문서 객체 생성

→ 문서 객체 생성

03) DOM 문서 객체의 속성 & 메서드를 활용한 스몰 미션

Step #01

Small Mission)

select 태그를 정적으로 생성한 후 현재 년도부터 1900년까지 option 태그를 동적으로 생성해서 출생년도 select 선택상자 만들기

스몰 미션 제작 노트

- ① html 구조 작성 (select 태그만 작성, id는 birthYear)
- ② 문서 객체 선택
- ③ 옵션 태그에 답을 변수 선언
- ④ 올해 년도 가져와서 변수에 담기
- ⑤ 반복문으로 option 태그 반복
- ⑥ alert()로 결과 중간 체크
- ⑦ 자손으로 select 안에 option 태그 넣기

▲ 해당 스몰미션은 39페이지의 [for 문 사용해서 2023년~1950년까지 select로 body에 태그를 출력] 하는 스몰미션과 거의 같습니다. **다른 점은..** 문서 객체를 생성하고, 현재의 년도를 시스템에서 자동으로 가져오고, 자손 요소에 텍스트를 동적으로 넣는게 다릅니다. 해당 예제처럼 **html 내부에 있는 문서 객체를 사용하면 HTML 구조 위치, CSS 디자인이 용이해집니다.**

```
<select id="birthYear"></select>
```

HTML

```
<script>
// 문서 객체 선택
var birthYear = document.getElementById('birthYear');

// 옵션 태그에 답을 변수 선언
var birthOutput = ''; → 변수 값이 없이 만드는 이유는 for 반복문을 통해서만
                        들어지기 때문에 변수 값이 고정되어 있으면 안됩니다.

// 올해 년도 가져와서 변수에 담기
var today = new Date(); → today에는 년,월,일,시,분,초까지 모두 들어가 있음
var todayYear = today.getFullYear(); → today에 있는 요소 중에 년도만 추출
alert(todayYear);
```

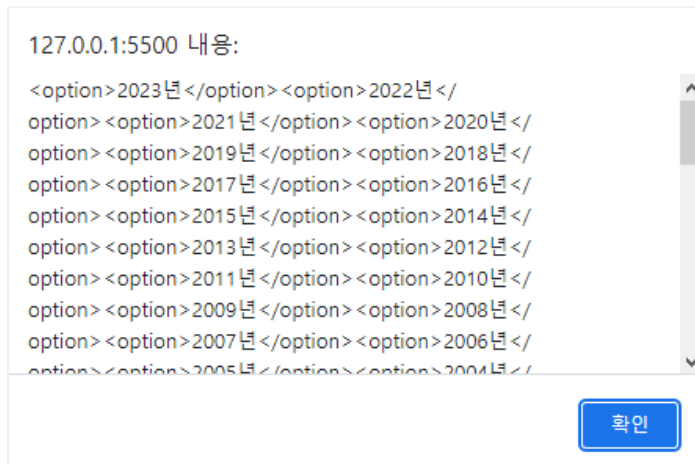
▲ 자바스크립트가 익숙해지기 전까지는 중간에 변수에 담은 값이 정상적인지 확인하는 것이 중요

03) DOM 문서 객체의 속성 & 메서드를 활용한 스몰 미션

Step #02

Small Mission)

select 태그를 정적으로 생성한 후 현재 년도부터 1900년까지 option 태그를 동적으로 생성해서 출생년도 select 선택상자 만들기



▲ 경고창에 데이터가 보여지고 있지만 모든 값들이 정상적으로 출력되고 있습니다.

최종적으로 경고창에 출력되고 있는 텍스트를 포함한 option 태그들(birthOutput)을 html에 있는 select 태그인 birthYear에 html 태그도 인식하는 텍스트 컨트롤 속성(innerHTML)으로 넣습니다.

option 태그를 동적으로 만든 것이기 때문에 모든 작업이 완료되어도 html 소스 코드에서는 여전히 select 태그만 있습니다. 곧, 수많은 option 태그가 없는 깔끔한 html을 만들 수 있습니다.

```
<select id="birthYear"></select>
```

아직 for 반복문으로 option이라는 태그가 만들어지기 전이라 반복문이 실행되기 전까지는 현재 상태로 <select></select> 만 브라우저에 출력됩니다.

```
// 반복문으로 option 태그 반복
for(i = todayYear; i >= 1900; i--) {
    birthOutput += '<option>' + i + '년</option>'
}

// 콘솔창에 결과 중간 체크
alert(birthOutput);

// 자손으로 select 안에 option 태그 넣기
birthYear.innerHTML = birthOutput;
</script>
```

- 초기 값은 todayYear 곧, 2023
- 1900년이 되면 반복문 빠져나오기
- 2023부터 시작해서 숫자가 작아지는 것이니까 마이너스로

경고창에 출력되고 있는 동일한 데이터

04) DOM 문서 객체 이벤트(Event) - 고전 이벤트(예전부터 사용되던 이벤트 방식)

이벤트(event)란 사용자가 명령할 때 작동하게 하는 시점

{ 문서객체.온이벤트타입명 = 이벤트핸들러; }

heading.onclick = function() {...}

이벤트 타입이란 마우스를 클릭
하거나, 오버하거나, 떠나거나 등
마우스 작동 방식(타입)을 말함

이벤트 핸들러는 명령을 담은
코드의 집합 곧, 콜백 함수를
작성합니다.

<h1 id="heading">This is Heading</h1>

HTML

This is Heading



This is Heading

Small Mission) body에 h1을 만들고 h1을 클릭했을 때 CSS 스타일을 변경(글자 색상, 배경 색상)

```
<script>
/* body에 h1을 만들고 h1을 클릭했을 때 css 스타일을 변경(글자 색상, 배경 색상) */

// 문서 객체 선택
var heading = document.getElementById('heading');

// 이벤트 만들기
heading.onclick = function() {
  heading.style.color = 'white';
  heading.style.backgroundColor = 'black';
}
</script>
```

```
// 이벤트 만들기
heading.onclick = function() {
  this.style.color = 'white';
  this.style.backgroundColor = 'black';
}
```

콜백 함수 내에 heading 이라는 객체 이름을 반복해서 사용해도 되지만 좋은 방법은 this를 사용하는 것입니다. this란 객체가 콜백 함수 내부에서 자신을 호출하는 것이기 때문에 객체 이름이 변경되더라도 this는 수정할 필요가 없습니다. 곧, this == heading

```
<script>
$('#heading').click(function(){
  $(this).css({'color': 'white', 'background-color': 'black'})
})
</script>
```

참고로 제이쿼리 방식은 문서 객체 선택과 이벤트가 하나로 통합되어 있어서 사용하기 편합니다.

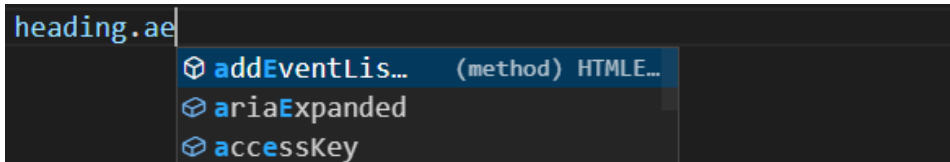
04) DOM 문서 객체 이벤트(Event) - 이벤트 메서드 문서 객체에 이벤트를 연결하는 메서드(문서에 뭔가를 누르면 이벤트를 작동시키는 방법)

```
{ 문서객체.addEventListener('이벤트타입', 콜백함수); }
```

↓
괄호 안에 이벤트 종류가 뭔지 듣고 추가(작동)
해 그리고 콜백 함수 실행해.. 라고 이해하세요.
addEventListener()는 메서드입니다. 그
래서 바로 괄호가 나옵니다.

HELPFUL TIPS

- 앞으로 `addEventListener` 도 자주 쓰므로 Emmet 단축키를 사용
- `ae`를 치면 Emmet 단축키가 나옵니다. `Tab` 또는 `Enter`를 쳐서 사용하세요.



고전 이벤트 방식 또는 이벤트 메서드 방식 중에 제작자가 편한 방식을 사용하시면 됩니다.

Small Mission) body에 h1을 만들고 h1을 클릭했을 때 CSS 스타일을 변경(글자 색상, 배경 색상)

```
<script>
  /* body에 h1을 만들고 h1을 클릭했을 때 css 스타일을 변경(글자 색상, 배경 색상) */

  // 문서 객체 선택
  var heading = document.getElementById('heading');

  // 이벤트 메서드로 이벤트 만들기
  heading.addEventListener('click', function(){
    this.style.color = 'white'; 이벤트명은 반드시 따옴표로 묶습니다.
    this.style.backgroundColor = 'black';
  });
</script>
```

```
<h1 id="heading">This is Heading</h1>
```

HTML

This is Heading



This is Heading

04) DOM 문서 객체 이벤트(Event) - 이벤트 메서드(이벤트 만들고 이벤트 제거하기)

{문서객체.removeEventListener('이벤트타입', 콜백함수); }

문서 객체에 설정된 이벤트 제거

자주 사용되지는 않지만 특정 이벤트를 더 이상 작동시키지 말아야 하는 경우에 사용합니다.

- 마우스가 올라간다.. mouseenter
- 마우스가 떠난다.. mouseleave

Small Mission)

- body에 h1을 이벤트 메서드 방식으로 마우스 오버하면 CSS 스타일을 변경(글자 색상, 배경 색상)되고 마우스가 떠나면 스타일이 없는 원래로 돌아오게 합니다.
- 일정 시간(5초)이 지나면 해당 마우스 이벤트를 작동 시키지 않습니다. 
- ※ 이벤트를 CSS 스타일이 변화하는 함수를 만들어서 재활용합니다.

```
<h1 id="heading">This is Heading</h1>
```

HTML

This is Heading



This is Heading

```
<script>
// 문서 객체 선택
var heading = document.getElementById('heading');

// CSS 변화 스타일 함수 만들기
function cssChange() {
  this.style.color = 'white';
  this.style.backgroundColor = 'black';
}

// 이벤트 연결
heading.addEventListener('mouseenter', cssChange);
heading.addEventListener('mouseleave', function(){
  this.style.color = '';
  this.style.backgroundColor = '';
});

// 5초 후 이벤트 제거
setTimeout(function(){
  heading.removeEventListener('mouseenter', cssChange);
}, 5000);
</script>
```

자주 사용될 수 있으니 특정 함수로 만들어 재활용

04) DOM 문서 객체 이벤트(Event) - 인라인 이벤트 인라인 이벤트란 html 태그에 스크립트를 작성하는 이벤트 (114페이지에 있는 인라인 자바스크립트와 같은 방식)

Small Mission)

- body에 h1을 2개 만듭니다.
- 첫번째 h1은 인라인 이벤트로 CSS를 변경
- 두번째 h1은 CSS 변경 함수를 만들고 인라인 이벤트로 호출합니다.

```
<h1>인라인 이벤트로 CSS를 변경</h1>
```

```
<h1>CSS 변경 함수를 만들고 인라인 이벤트로 호출</h1>
```

HTML

▲ 인라인 이벤트 작성 전

```
<h1 onclick="this.style.color='crimson';">인라인 이벤트로 CSS를 변경</h1> ①
```

```
<h1 onclick="cssChange(this);">CSS 변경 함수를 만들고 인라인 이벤트로 호출</h1> ②
```

▲ 인라인 이벤트 작성 후

인라인 이벤트로 CSS를 변경

CSS 변경 함수를 만들고 인라인 이벤트로 호출

인라인 이벤트로 CSS를 변경

CSS 변경 함수를 만들고 인라인 이벤트로 호출



- ① 첫번째 h1의 인라인 이벤트의 경우 CSS 스타일을 태그에 직접 작성합니다. **여기에서 this는 필수는 아닙니다.** 해당 경우에 텍스트 색상만 바꾸는거라 간단하지만 배경색 등 여러가지 CSS 변화가 있으면 html 내부가 많이 길어집니다. 간단한 경우에는 직접 CSS 스타일을 해당 경우처럼 적어주면 됩니다.

```
<script>
// 인라인 이벤트로 호출할 함수
function cssChange(e) {
  e.style.color = 'crimson';
  e.style.backgroundColor = 'black';
}
</script>
```

- ② 두번째 h1의 인라인 이벤트의 경우 CSS 스타일을 함수로 만들고 필요할 때 마다 호출해서 사용합니다. 이렇게 하면 html 내부가 보기 좋게 간단해 집니다.
- 어떤 html 요소에 해당 함수를 사용할지 모르기 때문에 cssChange라는 함수에 매개변수를 줍니다
 - 인라인 이벤트에서 CSS 스타일 함수를 호출할 때 **this**라고 매개변수를 반드시 받아야 합니다.

04) DOM 문서 객체 이벤트(Event) - 이벤트 타입의 종류 이벤트 타입이란 마우스 올리고 떠나고 클릭하고 키보드를 누르고 등 사용자의 마우스와 키보드 반응 형태를 말합니다.

자바스크립트 이벤트 타입 분류 4가지

- ① 마우스 이벤트
- ② 키 이벤트
- ③ 폼 이벤트
- ④ 로드 및 기타 이벤트

▲ 자바스크립트 이벤트 타입은 정말 많습니다. 이 많은 것을 알 필요도 없고 다 사용하지도 않습니다.

다 알고 해보려는 노력은 매우 비 생산적입니다.

예제를 통해서 자주 사용하는 이벤트 타입을 익히시기 바랍니다.

마우스 이벤트

이벤트	설명
click	요소에 마우스를 클릭했을 때 이벤트가 발생
dblclick	요소에 마우스를 더블클릭했을 때 이벤트가 발생
mouseover	요소에 마우스를 오버했을 때 이벤트가 발생
mouseout	요소에 마우스를 아웃했을 때 이벤트가 발생
mousedown	요소에 마우스를 눌렀을 때 이벤트가 발생
mouseup	요소에 마우스를 떼었을 때 이벤트가 발생
mousemove	요소에 마우스를 움직였을 때 이벤트가 발생

키 이벤트

이벤트	설명
keydown	키를 눌렀을 때 이벤트가 발생
keyup	키를 떼었을 때 이벤트가 발생
keypress	키를 누른 상태에서 이벤트가 발생

로드 및 기타 이벤트

이벤트	설명
load	페이지의 로딩이 완료되었을 때 이벤트 발생
abort	이미지의 로딩이 중단되었을 때 이벤트 발생
unload	페이지가 다른 곳으로 이동될 때 이벤트 발생
resize	요소에 사이즈가 변경되었을 때 이벤트 발생
scroll	스크롤바를 움직였을 때 이벤트 발생

폼 이벤트

이벤트	설명
focus	요소에 포커스가 이동되었을 때 이벤트 발생
blur	요소에 포커스가 벗어났을 때 이벤트 발생
change	요소에 값이 변경 되었을 때 이벤트 발생
submit	submit 버튼을 눌렀을 때 이벤트 발생
reset	reset 버튼을 눌렀을 때 이벤트 발생
select	input이나 textarea 요소 안의 텍스트를 선택했을 때 발생

04) DOM 문서 객체 이벤트(Event) - 이벤트 객체의 속성과 메서드 이벤트 타입 매개변수가 곧, 이벤트 객체 그래서 보통 event 또는 줄여서 e를 사용

이벤트 객체의 매개변수 타입

문서객체.**on이벤트타입명** = function(이벤트 객체 매개변수) {...});

```
btn.onclick = function(e) {...}
```

문서객체.**addEventListener**('이벤트타입', function(이벤트 객체 매개변수) {...});

```
btn.addEventListener('click', function(e) {...})
```

Small Mission)

- 이벤트 객체의 타입을 type 속성을 활용해서 콘솔창에서 확인하기
- 2가지 방식으로 확인(고전 이벤트 방식, 이벤트 메서드 방식)

```
<button id="btn">Click Me!!</button>
```

HTML



```
<script>
// 문서 객체 생성
var btn = document.getElementById('btn');

// 고전 이벤트 방식
btn.onclick = function(e){
  console.log(e.type);
}

//이벤트 메서드 방식
btn.addEventListener('mouseenter', function(e){
  console.log(e.type);
});
</script>
```

▲ 마우스가 클릭되면 콘솔창에 click 이라고 이벤트 객체 매개변수 타입을 출력
마우스가 올라가면 콘솔창에 mouseenter 이라고 객체 이벤트 매개변수 타입을 출력

type : 현재 발생한 이벤트의 종류를 나타내는 문자열(click, load 등)

자바스크립트 주요 구문 무지성으로 연습하기(1)

- 머리가 아니라 몸이 기억할 정도로 일정 기간 연습하세요. 그럼 확실히 자신감이 생깁니다.
- 컴퓨터 앞에 앉으면 1~6번을 한 사이클 연습합니다. 이렇게 며칠 정도 하시면 됩니다.

for 반복문, 함수구문, 매개변수 사용한 함수 구문, 문서로딩 구문, for of 반복문, forEach 반복문

- 1) for 반복문 → for 반복문으로 1부터 10까지 합 콘솔출력
- 2) 함수와 for 반복문 → 함수를 사용해 반복문으로 1부터 10까지, 20까지, 30까지 합을 콘솔출력
- 3) 매개변수 사용한 함수와 for 반복문 → 함수를 매개변수 사용해 반복문으로 1부터 10까지, 20까지, 30까지 합을 콘솔출력
- 4) 문서로딩 구문 with 화살표 함수 → 문서로딩(2가지 방법 만들기(DOMContentLoaded, window.onload) / 화살표 함수로 2가지 다시 만들기
- 5) for of 반복문, forEach 반복문 → 문서 로딩 하고 문서 객체 선택 후 for of 반복문과 for each 반복문으로 body 내 여러 개 h1 선택하기

```
<div class='frame'>
  <h1>This is h1 text 1</h1>
  <h1>This is h1 text 2</h1>
  <h1>This is h1 text 3</h1>
</div>
```

HTML


This is h1 text 1
This is h1 text 2
This is h1 text 3

대부분의 프로그래밍은 코드를 반드시 이해해야 합니다. 하지만 암기의 영역도 있습니다. 대표적으로 반복문, 함수 구문은 이해하려 하지 말고 그냥 암기해야 합니다. 그러기 위해서 몸이 기억하도록 무지성으로 반복해주어야 합니다.

자바스크립트 주요 구문 무지성으로 연습 항목(2)

01) for 반복문으로 1부터 10까지 합 콘솔 출력

```
let output = 0
for (let i = 1; i <= 10; i++) {
  output += i
}
console.log(output)
```

2) 함수 사용해 반복문으로 1부터 10, 20, 30까지 합을 콘솔 출력

```
const sum = function(){
  let output = 0
  for (let i = 1; i <= limit; i++) {
    output += i
  }
  console.log(output)
}
let limit = 10
sum()

limit = 20
sum()

limit = 30
sum()
```

03) 매개변수 사용한 함수와 for 반복문

```
const sum = function(limit){
  let output = 0
  for (let i = 1; i <= limit; i++) {
    output += i
  }
  return output
}

console.log(`1~10의 합은 ${sum(10)}`)
console.log(`1~20의 합은 ${sum(20)}`)
console.log(`1~30의 합은 ${sum(30)}`)
```

04-1) 문서로딩(2가지 방법 만들기) / 화살표 함수로 2가지 다시 만들기
DOMContentLoaded, window.onload

```
document.addEventListener('DOMContentLoaded', function(){
  // 실행구문
})

window.onload = function(){
  // 실행구문
}
```


자바스크립트 주요 구문 무지성으로 연습 항목(3)

04-2) 문서로딩(2가지 방법 만들기) / 화살표 함수로 2가지 다시 만들기
DOMContentLoaded, window.onload

```
document.addEventListener('DOMContentLoaded', ()=>{  
  // 실행구문  
})  
  
window.onload = ()=>{  
  // 실행구문  
}
```

05) 문서 로딩 하고 객체 선택 후 for of와 for each 반복문으로
body 내 여러 개 h1 선택하기

```
<div class='frame'>  
  <h1>This is h1 text 1</h1>  
  <h1>This is h1 text 2</h1>  
  <h1>This is h1 text 3</h1>  
</div>
```

HTML

```
document.addEventListener('DOMContentLoaded', ()=>{  
  const h1s = document.querySelectorAll('.frame h1')  
  for (const el of h1s) {  
    el.style.color = 'crimson'  
  }  
})
```

▲ for of 반복문으로 복수의 HTML 엘리먼트 선택

```
window.onload = ()=>{  
  const h1s = document.querySelectorAll('.frame h1')  
  h1s.forEach((num, key)=>{  
    num.style.color = 'red'  
  })  
}
```

▲ forEach 반복문으로 복수의 HTML 엘리먼트 선택

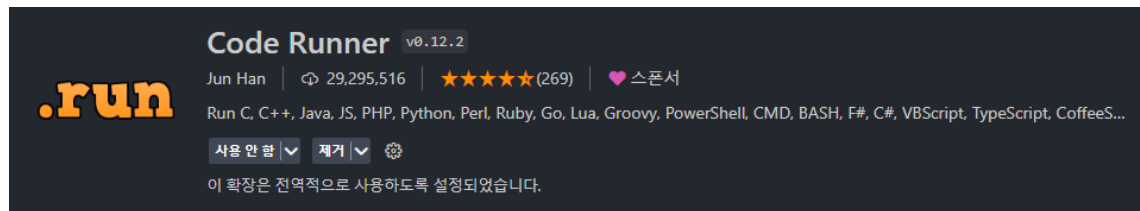
This is h1 text 1
This is h1 text 2
This is h1 text 3

```
forEach(function(num, key){...})  
forEach(num, key)=>{...}
```

자바스크립트 주요 구문 무지성으로 연습 방법

- 1) Code Runner 익스텐션 설치
- 2) 파일명.js 파일 만들어 비주얼스튜디오코드에서 파일 열기
- 3) 첨부된 js 파일 오픈해서 연습하기

첨부된 파일에 자바스크립트 주요 구문 무지성으로 연습 방법.js 파일을 활용



▲ 비주얼스튜디오코드 내에서 콘솔 확인하는 익스텐션
코드 러너 실행 단축키 : Ctrl + Alt + N

```
81 /**
82  * #####
83  * 1) 변수 output = 0로 시작해 반복문으로 1부터 10까지 합 콘솔출력
84  *
85  * 2) 함수를 사용해 반복문으로 10까지 합을 콘솔출력
86  *
87  * 3) 함수를 매개변수 사용해 반복문으로 10,20,30까지 합을 콘솔출력
88  *
89  * 4) 화살표 함수로 DOMContentLoaded 만들기(2가지 방법으로)
90  *
91  * 5) 문서 window.onload 후 for of와 for each body 내 여러개 h1 선택
92  * #####
93  * console.log()
94  */
95
96 // 여기서 부터 연습하고 연습 후 삭제합니다.
97 let output = 0
98 for (let i = 1; i <= 10; i++) {
99     output += i
100 }
101 console.log(output)
102
```

문제 출력 디버그 콘솔 Code

[Running] node "d:\인프런 강의 목록(최종 원본)\[3주과정] 오프라인 강의 수업 커리큘럼 및 수업자료\웹 퍼블리셔를 위한 JS 핵심이론 및 실전에제 교재 및 완성본\수강생 참고자료 다운로드 (코딩웍스 JS 교재)\[자바스크립트] 핵심이론 및 실전에제 완성본(Files)\자바스크립트 주요 구문 무지성으로 연습 방법.js"

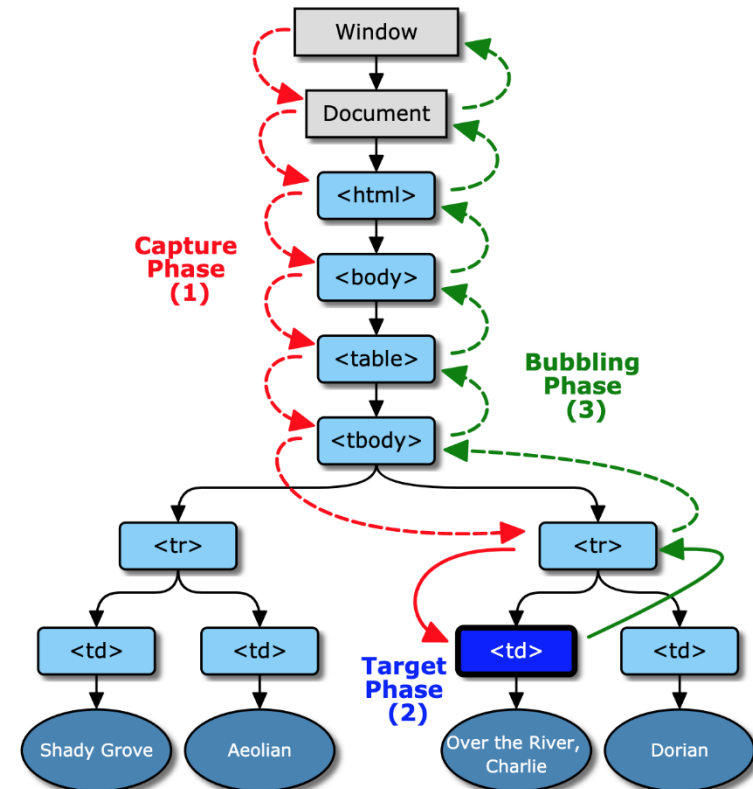
55

[Done] exited with code=0 in 0.275 seconds

04) DOM 문서 객체 이벤트(Event) - 이벤트 객체의 속성과 메서드

이벤트 객체의 공통 속성과 메서드

종류	설명
type	현재 발생한 이벤트의 종류를 나타내는 문자열(click, load 등)
currentTarget	현재 이벤트 리스너를 실행하고 있는 DOM 객체
defaultPrevented	이벤트의 디폴트 행동이 취소되었는지를 나타내는 true/false 값
preventDefault()	이벤트의 디폴트 행동을 취소시키는 메서드
bubbles	이벤트가 상위 요소로 전파되는 단계(true, false로 반환)
capturing	이벤트가 하위 요소로 전파되는 단계
target	이벤트가 실제 타겟 요소에 전달되는 단계
stopPropagation()	이벤트가 버블링 또는 캡처링 되는 것을 방지하는 메서드



04) DOM 문서 객체 이벤트(Event) - 이벤트 객체의 속성과 메서드

{ **stopPropagation();** } → 이벤트가 상위 요소로 전달 되는 버블링, 하위 요소로 전달되는 캡처링을 방지하는 메서드, **propagation**: 변식, 증식

```
<div id="frame">
  <a class="link" href="#none">a tag link</a>
</div>
```

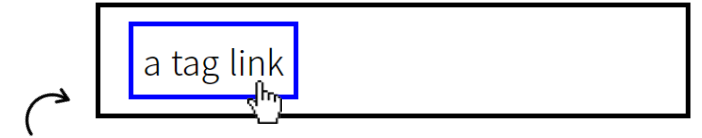
HTML

▼ link 객체를 클릭한 경우 : link 객체의 보더 색상이 변경되고 버블링 현상으로 frame 객체의 보더 색상도 변경



▲ frame 객체를 클릭한 경우 : frame 객체의 보더 색상만 변경

▼ link 객체를 클릭해도 stopPropagation() 메서드가 버블링을 방지해서 link 객체에만 이벤트가 전달됩니다.



```
<script>
  // 문서 객체 생성
  var frame = document.getElementById('frame');
  var link = document.querySelector('#frame a');

  // 이벤트 연결
  frame.onclick = function() {
    this.style.borderColor = 'red';
  }
  link.onclick = function() {
    this.style.borderColor = 'blue';
  }
</script>
```

function(e)에 있는 e 매개변수를 stopPropagation() 메서드가 e를 매개변수 객체로 사용해야 합니다.

```
link.onclick = function(e) {
  e.stopPropagation();
  this.style.borderColor = 'blue';
}
```

04) DOM 문서 객체 이벤트(Event) - 이벤트 객체의 속성과 메서드

{ **preventDefault();** } → 요소가 가지고 있는 기본 이벤트를 제거하는 메서드
prevent : 막다, 방지하다

Small Mission)

a 태그의 기본 이벤트를 preventDefault() 메서드로 제거하고 confirm 창에서 확인 누르면 href 주소로 이동하고, 취소를 누르면 href 주소로 이동하지 않게 하기

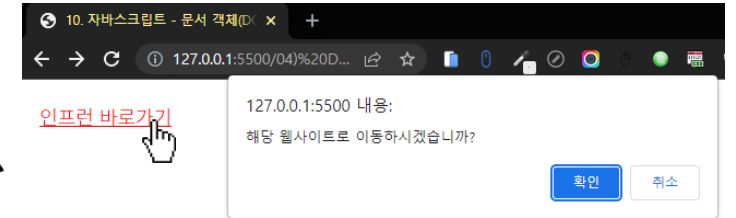
스몰 미션 제작 노트

- ① a 태그 문서 객체 변수에 담기
- ② 변수에 담은 문서 객체에 href 기본 이벤트 제거하기
- ③ 문서 객체의 href 속성을 가져와서 변수에 담기
- ④ 해당 url로 이동할지 안할지 confirm으로 물어보기(확인 누르면 이동하고 취소 누르면 이동 안함)

```
<a class="link" href="http://www.infllearn.com">인프런 바로가기</a>
```

HTML

당연하게도 a 태그의 기본 이벤트를 막지 않으면 a태그를 클릭하는 즉시 이벤트가 발생하고 href 속성 내에 있는 값(url)로 이동합니다.



```
<script>
// a 태그 문서 객체 변수에 담기
var link = document.querySelector('.link');

// 변수에 담은 문서 객체에 href 기본 이벤트 제거하기
link.addEventListener('click', function(e){
  e.preventDefault();
  // 문서 객체의 href 속성을 가져와서 변수에 담기
  var url = this.getAttribute('href');
  // href 속성의 값을 제대로 가져오는지 확인
  console.log(url);
  // 해당 url로 이동할지 안할지 confirm으로 물어보기
  var goToUrl = confirm('해당 웹사이트로 이동하시겠습니까?');
  // 확인 누르면 이동하고 취소 누르면 이동 안함
  if(goToUrl == true) {
    location.href = url;
  }
});
</script>
```

- 참(true)을 표현할 경우 goToUrl 만 있어도 됨
- url로 보내는 2가지 방법 모두 정상적으로 작동

location.assign() 메서드 : 주소 표시줄 매개변수의 url 주소로 변경

```
if(goToUrl) {
  location.assign(url);
}
```

04) DOM 문서 객체 이벤트(Event) - 이벤트 객체의 속성과 메서드

{ **마우스 & 키보드 이벤트 객체 속성과 메서드** } → 마우스 & 키보드 이벤트 객체란 click, mouseenter, mouseleave, keydown, keypress, keyup 등 마우스와 키보드 이벤트 때 발생하는 객체 (예제에서 사용되는 것만 중심으로 학습하세요.)

clientX

마우스 이벤트 촉발되었을 때 현재 윈도우 기준으로 마우스커서의 가로 좌표 반환

clientY

마우스 이벤트 트리거되었을 때 현재 윈도우 기준으로 마우스커서의 수직 좌표 반환

MovementX

마지막 mousemove 이벤트의 위치를 기준으로 마우스커서의 수평 좌표 반환

MovementY

마지막 mousemove 이벤트의 위치를 기준으로 마우스커서의 수직 좌표 반환

offsetX

대상 요소의 가장자리 위치를 기준으로 마우스 포인터의 가로 좌표 반환

offsetY

대상 요소의 가장자리 위치를 기준으로 마우스 포인터의 세로 좌표 반환

pageX

마우스 이벤트가 촉발되었을 때 문서 기준으로 마우스커서의 수평 좌표 반환

pageY

마우스 이벤트가 촉발되었을 때 문서 기준으로 마우스커서의 세로 좌표 반환

keydown

사용자가 키를 누를 때 이벤트 발생

keypress

사용자가 키 누를 때 이벤트 발생

keyup

사용자가 키 놓을 때 이벤트 발생

keyCode

onkeypress 이벤트를 촉발한 키의 유니코드 문자코드 또는 onkeydown 또는 onkeyup 이벤트를 촉발한 키의 유니코드 키코드를 반환

altKey : 키 이벤트 촉발되었을 때 "ALT"키가 눌러 졌는지 여부 반환

code : 이벤트를 촉발한 키의 코드 반환

ctrlKey : 키 이벤트가 촉발될 때 "CTRL"키가 눌러 졌는지 여부 반환

location : 키보드 또는 장치에 있는 키의 위치 반환

repeat : 키가 반복적으로 눌리는지아닌지 여부 반환

shiftKey : 키 이벤트가 촉발되었을 때 "SHIFT"키가 눌렸는지 여부 반환

04) DOM 문서 객체 이벤트(Event) - 마우스 이벤트 객체의 속성과 메서드

{ pageX } { pageY } → 마우스 이동 시 마우스 포인터의 좌표 값을 문서(body) 기준으로 확인하는 마우스 이벤트 속성

Small Mission)

문서(body)에서 마우스 이동하면서 마우스 포인터의 좌표 값 x, y를 pageX와 pageY 속성을 사용해서 문서 객체에 출력하기

스몰 미션 제작 노트

- ① x, y 좌표 값을 출력할 문서 객체 생성(span 태그 2개)
- ② body에서 마우스 이동 시 마우스 포인터 좌표 값 변화를 생성한 문서 객체 span에 반환

```
<div class="frame">
  <p>
    <b>x 좌표 값 : </b><span id="textX"></span>
  </p>
  <p>
    <b>y 좌표 값 : </b><span id="textY"></span>
  </p>
</div>
```

HTML

X 좌표 값 : 323
Y 좌표 값 : 296

마우스 포인터 이동시
x, y 좌표 값이 변함

```
<script>
  // x, y 좌표 값을 출력할 문서 객체 생성(span 태그 2개)
  var textX = document.getElementById('textX');
  var textY = document.getElementById('textY');

  // body에서 마우스 이동 시 마우스 좌표 값 변화를 생성한 문서 객체 span에 반환
  document.onmousemove = function(e) {
    var x = e.pageX;
    var y = e.pageY;

    textX.innerText = x;
    textY.innerText = y;
  }
</script>
```

04) DOM 문서 객체 이벤트(Event) - 마우스 이벤트 객체의 속성과 메서드

{ offsetX } { offsetY } → 마우스 이동 시 마우스 포인터의 좌표 값을 특정 대상 요소의 가장자리 기준으로 확인하는 마우스 이벤트 속성

Small Mission)

부모요소 내에서 자식요소가 마우스 움직일 때 offsetX와 offsetY 속성을 사용해서 자식요소가 마우스를 따라 다니게 하기

스몰 미션 제작 노트

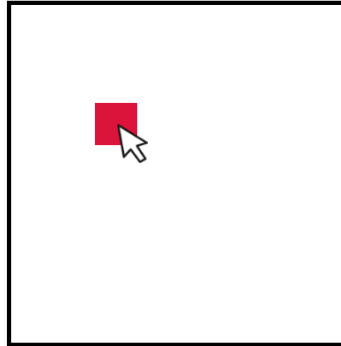
- ① 부모요소와 자식요소 문서 객체 생성 후 변수에 담기
- ② 부모요소 내에서 자식요소가 마우스 움직일 때(onmousemove) 따라다니기

```
<div class="frame">
  <div class="dot"></div>
</div>
```

HTML

```
.frame {
  border: 5px solid #000; width: 400px; height: 400px;
  position: relative;
}
.dot {
  background-color: crimson; width: 50px; height: 50px;
  position: absolute;
}
```

CSS



다 만들고 나면
pageX와 pageY로
테스트 해보세요.

```
.dot {
  background-color: crimson; width: 50px; height: 50px;
  position: absolute;
  top: 50%; left: 50%;
  transform: translate(-50%, -50%);
  transition: 0.35s ease-out;
}
```

추가 CSS

```
<script>
// 부모요소와 자식요소 문서 객체 생성 후 변수에 담기
var frame = document.querySelector('.frame');
var dot = document.querySelector('.dot');

// 부모요소 내에서 자식요소가 마우스 움직일 때(onmousemove) 따라다니기
frame.onmousemove = function(e){
  var x = e.offsetX;
  var y = e.offsetY;

  console.log(x);
  console.log(y);

  dot.style.left = x + 'px';
  dot.style.top = y + 'px';
}
</script>
```

- CSS에 부모 자식 관계가 만들어져 있기 때문에 **pageX**와 **pageY**를 사용해도 괜찮습니다. 오히려 부드럽게 움직이는 건 더 낫습니다.
- 다만 CSS에 부모 자식 관계가 만들어져 있지 않다면 offsetX, offsetY로 부모요소 내에서 마우스 따라다니게 할 수 있습니다.

부모 요소의 x, y 좌표 값을 dot 객체의 스타일(style) 중에 왼쪽(left) 값, 상단(top) 값에 넣기. 단, 좌표이므로 px 단위가 필요함

04) DOM 문서 객체 이벤트(Event) - 키보드 이벤트 객체의 속성과 메서드

{ **keyCode 속성** } → 키보드를 눌렀을 때 유니코드를 반환하는 키보드 속성

- **keydown** : 키보드를 누를 때 실행(누르고 있는 경우에는 계속 실행)
- **keypress** : 키보드를 누를 때 실행(누르고 있는 경우에는 계속 실행)
- **keyup** : 키보드를 누르고 떼었을 때 실행(가장 일반적으로 사용)

keydown과 keypress의 차이점은 keypress의 경우는 한글 입력, 방향키, del 키와 같은 즉시 문자 입력이 직접 반영 안되는 키는 반응하지 않음

[고전 방식 키보드 이벤트 주기]

```
문서객체 onkeyup = function(e){  
    // 실행할 구문  
}
```

고전 방식 마우스이벤트 종류 : onclick, onmouseenter, onmouseleave, onmousemove, onkeyup 등

[이벤트 메서드 방식 키보드 이벤트 주기]

```
문서객체 addEventListener('keyup', function(e) {  
    // 실행할 구문  
});
```

이벤트 메서드 방식 마우스이벤트 종류 : 'click', 'mouseenter', 'mouseleave', 'mousemove', 'keyup' 등 ※ 반드시 따옴표 안에 이벤트 핸들러 이름 입력

Small Mission)

input에 입력되는 텍스트의 키코드(keyCode) 값 알아내기

```
<div class="frame">  
  <input type="text" id="inputBox">  
  <p>키코드 : <span id="result"></span></p>  
</div>
```

HTML

```
<script>  
  // 문서 객체 변수에 담기  
  var inputBox = document.getElementById('inputBox');  
  var result = document.getElementById('result');  
  
  // 고전 이벤트 방식 : 발생한 키보드 이벤트를 #result에 출력하기  
  inputBox.onkeyup = function(e){  
    var code = e.keyCode;  
    result.innerText = code;  
  };  
  
  // 이벤트 메서드 방식 : 발생한 키보드 이벤트를 #result에 출력하기  
  inputBox.addEventListener('keyup', function(e){  
    var code = e.keyCode;  
    result.innerText = code;  
  });  
</script>
```

이벤트 방식은 한가지만 사용하지 마시고 2가지 모두 기회가 될 때 마다 자주 연습해보세요.

키보드 a와 ㄱ 에 대한 키코드 값 ←

a	ㄱ
키코드 : 65	키코드 : 91

04) DOM 문서 객체 이벤트(Event) - 키보드 이벤트 객체의 속성과 메서드

{ **shiftKey 속성** } → 키보드를 눌렀을 shift가 눌렸는지 true, false로 반환

Small Mission)

input에 입력되는 키보드가 shift가 눌렸는지 shiftKey 속성으로 알아내기

```
<div class="frame">
  <input type="text" id="inputBox">
  <p>shift 키 눌렀는지 여부 : <span id="result"></span></p>
</div>
```

HTML

```
<script>
  // 문서 객체 변수에 담기
  var inputBox = document.getElementById('inputBox');
  var result = document.getElementById('result');

  // 발생한 키보드 이벤트를 #result에 출력하기
  inputBox.onkeydown = function(e){
    var code = e.shiftKey;
    result.innerText = code;
  };
</script>
```



shift 키 눌렀는지 여부 : false shift 키 눌렀는지 여부 : true

{ **deltaY 속성** } → 마우스 휠의 세로 스크롤 정도를 반환 (۫축 곧, 위아래로)

Small Mission)

마우스 휠을 내렸을 때 세로스크롤 정도를 deltaY 속성으로 알아내기

```
<div class="frame">
  <p>마우스 휠 내진 정도 : <span id="result"></span></p>
</div>
```

HTML

```
.frame { position: fixed; }
#result { color: red; }
```

CSS

```
<script>
  // 문서 객체 변수에 담기
  var result = document.getElementById('result');

  // 마우스 휠 내진 정도 확인
  document.onwheel = function(e){
    var top = e.deltaY;
    result.textContent = top;
  };
</script>
```



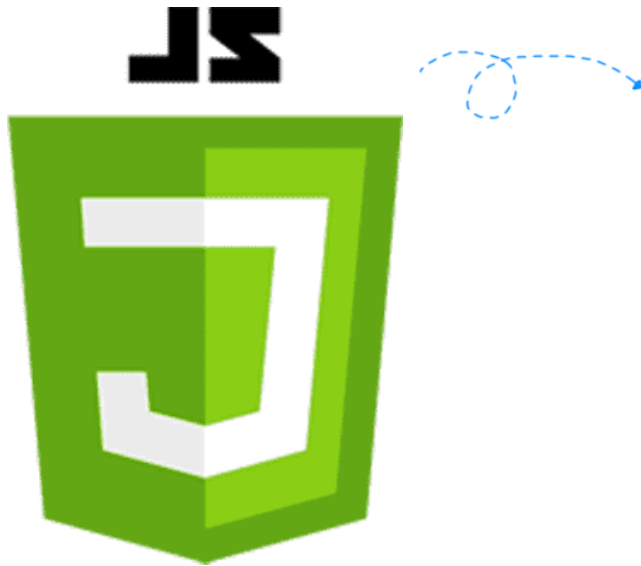
마우스 휠 내진 정도 : 100

마우스 휠 내진 정도 : -100

JAVASCRIPT 핵심이론 커리큘럼

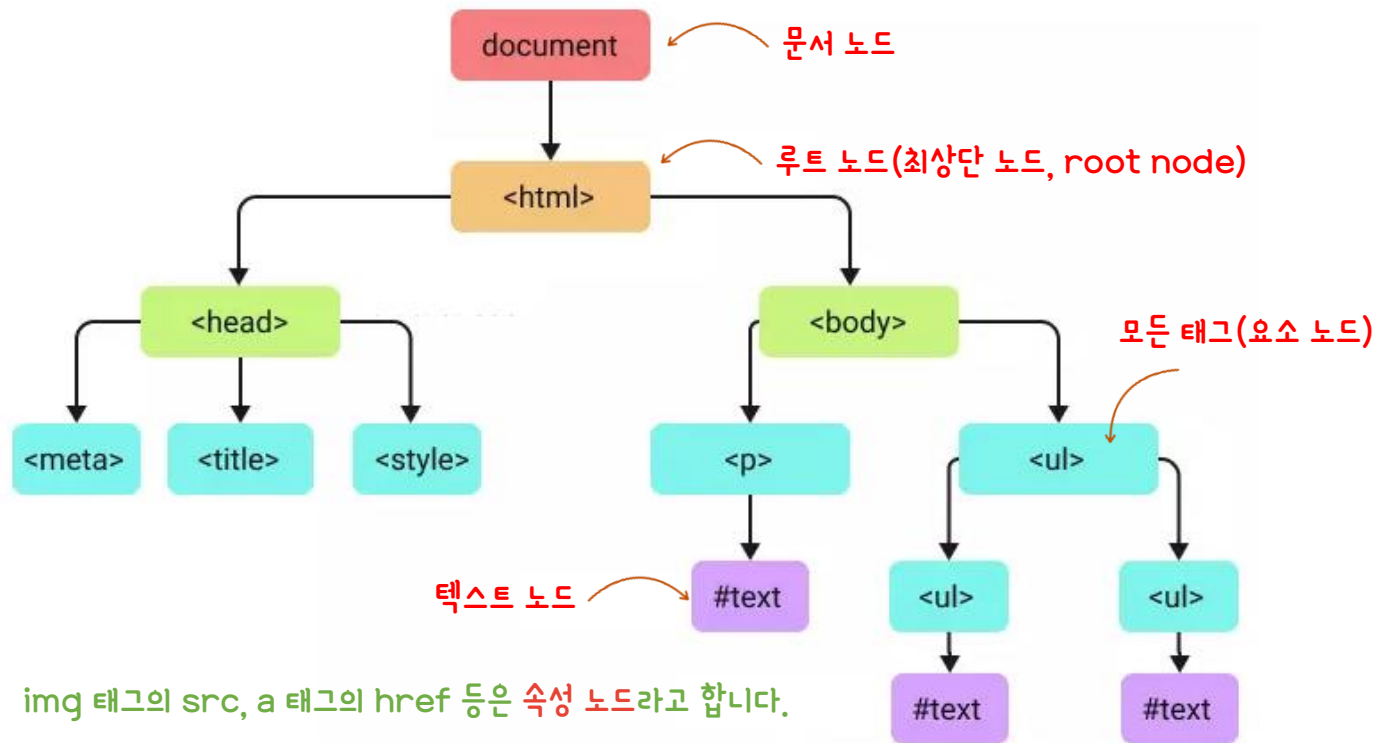
[웹 퍼블리셔를 위한 자바스크립트] 11. HTML DOM Node

1. HTML DOM Node 란?
2. HTML DOM Node 간의 탐색



HTML DOM Node 란? Node란 자바스크립트에서 html 모든 요소를 말합니다. 쉽게 노드(node)란 부모요소 형제요소 자식요소를 뜻 합니다. DOM을 이용해서 html의 모든 요소에 접근합니다. 앞으로 노드 탐색에서는 요소를 노드(Node)라고 부릅니다.

{ HTML DOM Node }



img 태그의 src, a 태그의 href 등은 속성 노드라고 합니다.

node : (줄기의) 마디 곧, 단계를 의미

```
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>11. HTML DOM Node </title>
</head>
<body>
  <div class="container">
    <section>
      <div class="heading">This is Heading</div>
      <div class="frame">
        <p>
          Lorem ipsum dolor sit
        </p>
      </div>
    </section>
  </div>
</body>
</html>
```

형제요소이면서 head의 자식요소

body의 자식요소 이면서 section의 부모요소

frame의 자식요소 이면서 텍스트의 부모요소

1) HTML DOM Node - 노드 탐색

Node 탐색의 경우 자바스크립트와 제이쿼리 모두 중요합니다. 사용하는 형식이 다를 뿐 거의 같습니다. 나중에 제이쿼리를 학습하실 때 지금 공부하신 자바스크립트 노드 탐색을 기본으로 하기 때문에 학습하시기 쉬워집니다. 자바스크립트에서 Node 탐색은 속성, 제이쿼리에서 Node 탐색은 메서드

{ Node 탐색 속성 }

속성 명	기능 설명
parentNode	부모 노드를 선택
childNodes	자식 노드를 선택(텍스트 노드 포함)
children	자식 노드를 선택(엘리먼트 노드만 선택)
firstChild	첫번째 자식 노드 선택
lastChild	마지막 자식 노드 선택
previousSibling	이전 형제 노드 선택
nextSibling	다음 형제 노드 선택

▲ 자바스크립트 노드 탐색 속성은 매우 자주 사용되므로 모두 잘 알아야 합니다.

메소드	설명
.siblings()	선택한 요소의 형제(sibling) 요소 중에서 지정한 선택자에 해당하는 요소를 모두 선택
.next()	선택한 요소의 바로 다음에 위치한 형제 요소를 선택
.nextAll()	선택한 요소의 다음에 위치한 형제 요소를 모두 선택
.nextUntil()	선택한 요소의 형제 요소 중에서 지정한 선택자에 해당하는 요소 바로 이전까지의 요소를 모두 선택
.prev()	선택한 요소의 바로 이전에 위치한 형제 요소를 선택
.prevAll()	선택한 요소의 이전에 위치한 형제 요소를 모두 선택
.prevUntil()	선택한 요소의 형제 요소 중에서 지정한 선택자에 해당하는 요소 바로 다음까지의 요소를 모두 선택

▲ 제이쿼리 노드 탐색 메서드

JS 학습에서 당연히 자바스크립트를 기본으로 하지만 제이쿼리가 사용하기는 훨씬 편하고 여러 면에서 효율적입니다. 그렇지만 자바스크립트 기본기가 있으면 제이쿼리 학습이 훨씬 쉬워집니다. 제이쿼리 학습도 나중에 꼭 필요합니다.

2) HTML DOM Node - 노드 탐색

{ parentNode : 부모 노드 탐색 속성 }

Small Mission)

부모요소 내 버튼을 클릭했을 때 부모요소의 CSS 디자인 변경하기

```
<div class="frame">
  <h1>This is Heading</h1>
  <p>
    Lorem ipsum dolor sit amet consectetur adipisicing elit. Sit aperiam
    corporis dolore expedita a corrupti vel consequatur eum saepe aliquid!
  </p>
  <button id="trigger">Click Me</button>
</div>
```

HTML

```
.frame {
  border: 1px solid #000;
  padding: 20px;
  border-radius: 5px;
  width: 500px;
}
```

CSS

▲ 기본 모양을 위한 CSS

```
trigger.addEventListener('click', function(){
  frame.style.backgroundColor = 'skyblue';
  frame.style.borderColor = 'transparent';
  frame.style.boxShadow = '0 0 20px rgba(0, 0, 0, 0.2)';
});
```

▲ this.parentNode 대신에 직접 문서 객체의 이름인 frame을 선택해도 됩니다. 만약 이렇게 frame이름을 직접선택하면 frame 개꼇을 반드시 선언해야 합니다. 하지만 this.parentNode로 선택하면 frame 객체를 선언하지 않아도 됩니다.

```
<script>
  // 객체 선언 후 변수에 담기
  var frame = document.querySelector('.frame');
  var trigger = document.getElementById('trigger');

  // 버튼 누르면 부모요소(parentNode) CSS 디자인 바꾸기
  trigger.addEventListener('click', function(){
    this.parentNode.style.backgroundColor = 'skyblue';
    this.parentNode.style.borderColor = 'transparent';
    this.parentNode.style.boxShadow = '0 0 20px rgba(0, 0, 0, 0.2)';
  });
</script>
```

this.parentNode로 요소 탐색을 했기 때문에 여기에서는 필요 없는 객체 선택입니다. 그래서 Node 탐색 속성을 사용하는 것이 좋습니다.



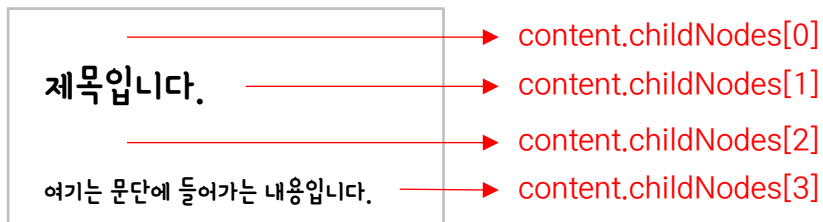
2) HTML DOM Node - 노드 탐색

{ **childNodes[n]** : 자손 노드 탐색 속성 }

보통 n이라고 되어 있는 것은 정해져 있지 않다.. 를 의미합니다. n분의 1로 밥값을 낸다 라고 생각하면 됩니다. 그래서 여기에 [n]이라고 배열 형태로 적힌 부분은 자손 노드가 여러 개 일 수 있기 때문입니다. 자손 노드와 자식 노드는 다릅니다.

전 페이지의 parentNode는 [n]이 없는 이유는 부모 노드는 한 개 일 수 밖에 없기 때문입니다.

[Note] html 자체에서 엔터 또는 공백은 전혀 브라우저 출력에 영향을 주지 않습니다. 하지만 자바스크립트에서 노드 탐색할 때는 엔터와 공백도 하나의 노드로 선택되니 주의하세요.



▲ 실작업에서 노드 선택에서 엔터와 공백을 고려할 수 없습니다. 그냥 참고 정도로만 아세요.

```
// 자손 노드 중 h1 선택 후 디자인 변경
frame.childNodes[1].style.color = 'crimson';
```

This is Heading

Lorem ipsum dolor sit amet consectetur adipisicing elit. Sit aperiam corporis dolore expedita a corrupti vel consequatur eum saepe aliquid!

Small Mission)

해당 html 구조에서 문서 객체의 자식 노드 개수를 확인하고 콘솔창에 출력

```
<div class="frame"> ① frame.childNodes[0]
  ②<h1>This is Heading</h1> ③
  ④<p>
    Lorem ipsum dolor sit amet consectetur adipisicing elit. Sit aperiam
    corporis dolore expedita a corrupti vel consequatur eum saepe aliquid!
  </p> ⑤ frame.childNodes[5]
</div>
```

HTML

```
<script>
  // 문서 객체 선언 후 변수에 담기
  var frame = document.querySelector('.frame');

  // 자손 개수 확인해서 콘솔창에서 확인
  var num = frame.childNodes.length;
  console.log('자식 노드의 개수는 : ' + num);
</script>
```

자식 노드의 개수는 : 5

▲ HTML 구조상 자식 요소는 h1과 p 2개지만 자바스크립트 노드는 5개입니다.

사실 노드의 개수를 체크는 큰 의미는 없습니다. 다만, 공백과 엔터가 노드의 개수에 포함된다고 정도 알고 있으면 됩니다.

2) HTML DOM Node - 노드 탐색

[참고] 공백과 엔터를 없애는 Remove Line Breaks

```
<div class="frame">
  <h1>This is Heading</h1>
  <p>
    Lorem ipsum dolor sit amet consectetur adipisicing elit. Sit aperiam
    corporis dolore expedita a corrupti vel consequatur eum saepe aliquid!
  </p>
</div>
```

```
<div class="frame"><h1>This is Heading</h1><p>Lorem ipsum dolor sit amet
consectetur adipisicing elit. Sit aperiam corporis dolore expedita a
corrupti vel consequatur eum saepe aliquid!</p></div>
```

자식 노드의 개수는 : 2

▲ 사람이 일일이 수동으로 줄 바꿈 제거를 할 수는 없습니다. 아래 웹사이트와 같은 Remove Line Breaks를 통해서 쉽게 할 수 있습니다.

줄 바꿈 제거를 하게 되면 자식 노드의 개수는 2가 됩니다.

<https://codebeautify.org/remove-line-breaks>

Remove Line Breaks

Add to Fav New Save & Share

Input Text

Sample

```
<div class="frame">
  <h1>This is Heading</h1>
  <p>
    Lorem ipsum dolor sit amet consectetur adipisicing elit. Sit aperiam corporis
    dolore expedita a corrupti vel consequatur eum saepe aliquid!
  </p>
</div>
```

Size : 226 B, 222 Characters

☒ Auto

Remove Line Breaks

File..

Load URL

Output Text

```
<div class="frame"><h1>This is Heading</h1><p>Lorem ipsum dolor sit amet
consectetur adipisicing elit. Sit aperiam corporis dolore expedita a
corrupti vel consequatur eum saepe aliquid!</p></div>
```

Size : 217 B, 217 Characters

Copy To Clipboard

Download



2) HTML DOM Node - 노드 탐색

{ **firstChild** : 첫번째 자손 노드 탐색 속성 }

{ **lastChild** : 마지막 자손 노드 탐색 속성 }

```
<div class="frame">
  <h1>This is Heading</h1>
  <p>
    Lorem ipsum dolor sit amet consectetur adipisicing elit. Sit aperiam
    corporis dolore expedita a corrupti vel consequatur eum saepe aliquid!
  </p>
</div>
```

HTML

Small Mission)

해당 html 구조에서 firstChild와 lastChild 속성을 사용해서 h1 앞에 텍스트 요소(텍스트 노드)를 넣고 p태그 다음에 텍스트 요소를 넣기

```
<script>
  // 문서 객체 선언 후 변수에 담기
  var frame = document.querySelector('.frame');

  // 앞에 텍스트 요소(텍스트 노드) 넣기
  frame.firstChild.textContent = '처음에 추가된 텍스트 요소';
  frame.lastChild.textContent = '마지막에 추가된 텍스트 요소';
</script>
```

This is Heading

Lorem ipsum dolor sit amet consectetur adipisicing elit. Sit aperiam corporis dolore expedita a corrupti vel consequatur eum saepe aliquid!

처음에 추가된 텍스트 요소

This is Heading

Lorem ipsum dolor sit amet consectetur adipisicing elit. Sit aperiam corporis dolore expedita a corrupti vel consequatur eum saepe aliquid!

마지막에 추가된 텍스트 요소



2) HTML DOM Node - 노드 탐색

{ **firstElementChild** : (공백을 뺀)첫번째 자손 노드 탐색 속성 }

{ **lastElementChild** : (공백을 뺀)마지막 자손 노드 탐색 속성 }



엔터와 공백을 제외하고 직접적으로 요소를 선택하는 방식.
그래서 Element가 붙어 있습니다.

```
<div class="frame">
  <h1>This is Heading</h1>
  <p>
    Lorem ipsum dolor sit amet consectetur adipisicing elit. Sit aperiam
    corporis dolore expedita a corrupti vel consequatur eum saepe aliquid!
  </p>
</div>
```

HTML

Small Mission)

하위 요소를 직접 선택해서 디자인 변경하기

```
<script>
  // 문서 객체 선언 후 변수에 담기
  var frame = document.querySelector('.frame');

  // 엔터와 공백을 제외하고 직접적으로 요소를 선택
  frame.firstElementChild.style.color = 'crimson';
  frame.lastElementChild.style.color = 'yellowgreen';
</script>
```

This is Heading

Lorem ipsum dolor sit amet consectetur adipisicing elit. Sit aperiam corporis dolore expedita a corrupti vel consequatur eum saepe aliquid!

This is Heading

Lorem ipsum dolor sit amet consectetur adipisicing elit. Sit aperiam corporis dolore expedita a corrupti vel consequatur eum saepe aliquid!



2) HTML DOM Node - 노드 탐색

{ **nextElementSibling** : (공백을 뺀)다음 형제 노드 탐색 속성 }

{ **previousElementSibling** : (공백을 뺀)이전 형제 노드 탐색 속성 }

```
<div class="frame">
  <h1>This is H1</h1>
  <div class="content">
    <h2>This is H2</h2>
    <p>
      Lorem ipsum dolor sit amet consectetur adipisicing elit. Officia quos
      ex rem, aut ut error?
    </p>
  </div>
</div>
```

HTML

```
console.log(frame);
console.log(content);
```

▲ 문서 객체를 콘솔창에서 보면 html
구조 전체가 입력되어 있습니다.

```
▼ <div class="frame">
  <h1>This is H1</h1>
  ▶ <div class="content">...</div>
</div>
```

```
▼ <div class="content">
  <h2>This is H2</h2>
  ▶ <p>...</p>
</div>
```

Small Mission)

형제 요소 중 다음 요소, 이전 요소를 선택해서 디자인 변경하기

```
<script>
  // 문서 객체 선언 후 변수에 담기
  var frame = document.querySelector('.frame');
  var content = document.querySelector('.content');
  var h2Tag = document.querySelector('.content h2');

  // 엔터와 공백을 제외하고 형제 요소 중 다음 요소, 이전 요소 디자인 변경하기

  // 1) .content 밑에 p태그를 선택하기
  h2Tag.nextElementSibling.style.border = '2px solid crimson';

  // 2) .frame 바로 밑에 h1태그 선택하기
  content.previousElementSibling.style.border = '2px dashed royalblue';
</script>
```

This is H1

This is H2

Lorem ipsum dolor sit amet consectetur adipisicing elit.
Officia quos ex rem, aut ut error?

This is H1

This is H2

Lorem ipsum dolor sit amet consectetur adipisicing elit.
Officia quos ex rem, aut ut error?

2) HTML DOM Node - 노드 탐색(Node List) 배열 형태로 선택하는 방식이 번거롭기 때문에 for 반복문을 사용해서 좀 더 효율적으로 노드를 선택합니다.

{ 배열 방식으로 노드를 선택한 예시 }

```
<div class="frame">
  <h1>This is H1</h1>
  <h1>This is H1</h1>
  <h1>This is H1</h1>
</div>
```

HTML

```
<script>
// 문서 객체 선언 후 변수에 담기
var h1Tag = document.querySelectorAll('.frame h1');

// 배열을 통한 노드 선택
h1Tag[0].style.color = 'crimson';
h1Tag[1].style.color = 'yellowgreen';
h1Tag[2].style.color = 'royalblue';
</script>
```

This is H1
This is H1
This is H1

This is H1
This is H1
This is H1

Small Mission)

왼쪽에 배열로 노드 선택한 것을 for 반복문을 사용해서 노드를 선택해서 디자인하기

```
<script>
// 문서 객체 선언 후 변수에 담기
var h1Tag = document.querySelectorAll('.frame h1');

// for 반복문을 사용해서 노드를 선택

// 변경할 글자 색상 배경
var colorSet = ['crimson', 'yellowgreen', 'royalblue'];

for(i = 0; i < h1Tag.length; i++) {
  h1Tag[i].style.color = colorSet[i];
}
h1Tag[0]은 html 구조의 첫번째 h1 태그 colorSet[0]은 배열의 첫번째 값인 'crimson'
</script>
```

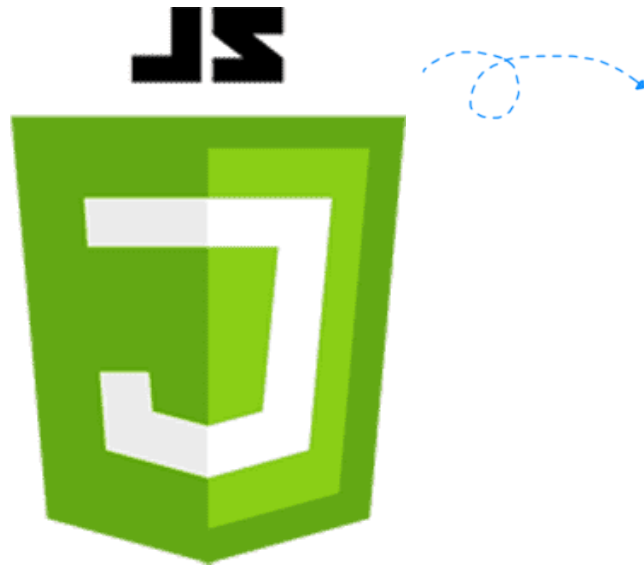
```
console.log(h1Tag.length);
```

3

▲ length의 개수는 3입니다.

JAVASCRIPT 핵심이론 커리큘럼

[웹 퍼블리셔를 위한 자바스크립트] 12. 자바스크립트 - 문서 객체 속성



1. 문서 객체 - 폼(Form)

2. 문서 객체 - 클래스 속성

className 속성, classList의 length 속성, add() 메서드, remove() 메서드, toggle() 메서드, contains() 메서드

3. 문서 객체 - 영역 속성

문서 객체 영역 속성 정리, clientWidth / clientHeight, offsetWidth / offsetHeight

4. 문서 객체 - dataset 속성

1) 문서 객체 속성 - Form(폼 요소의 name 속성 선택) 폼 요소 선택은 name을 통해서 선택할 수도 있습니다.

`{ document.폼네임.요소네임 }` → 가장 일반적인 사용방법

`{ document.forms['폼네임'].elements['요소네임'] }`

`{ document.forms['폼네임']['요소네임'] }`

Small Mission)

폼 요소를 선택하는 3가지 방법으로 input 태그의 디자인 변경하기

```
<form name="testForm">
  <input type="text" name="testInput">
</form>
```

HTML

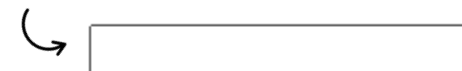
```
<script>
```

```
// 1) document.폼네임.요소네임
var testForm1 = document.testForm.testInput;
testForm1.style.border = '2px solid crimson';
```

```
// 2) document.forms['폼네임'].elements['요소네임']
var testForm2 = document.forms['testForm'].elements['testInput'];
testForm2.style.backgroundColor = 'yellowgreen';
```

```
// document.forms['폼네임']['요소네임']
var testForm3 = document.forms['testForm']['testInput'];
testForm3.style.padding = '20px';
```

```
</script>
```



1) 문서 객체 속성 - Form(폼 요소 관련 속성과 메서드)

종류	설명
checked	input 태그의 checkbox나 radio의 체크 여부(체크되면 true, 해제되면 false)
defaultValue	초기 설정값 반환
disabled	요소가 비활성 상태 체크(활성화면 false, 비활성상태이면 true)
length	요소의 개수 확인
value	input, select, textarea 요소의 value 값을 확인
blur()	요소에서 포커스 가는 것을 없앴
focus()	요소에 포커스를 자동으로 가게 함
submit()	form 내 모든 요소 값들을 전송
reset()	form 내 모든 요소 값들을 초기화

[input 요소에서 value 란?]

- 사용자가 직접 키보드로 입력하는 값을 value 라고 합니다.
- 대부분 비워 놓지만 미리 value를 value라는 속성의 값으로 넣어 놓을 수 있습니다.

```
<input type="text" name="testInput" value="http://">
```

→ http://

Small Mission)

input에 입력하는 내용 곧, value 값이 다른 요소에 자동으로 입력되게 하기

```
<form name="testForm">
  <input type="text" name="testInput">
</form>
<div id="print"></div>
```

HTML

```
<form name="testForm">
  <input type="text" name="testInput">
</form>
<div id="print"></div>
```

input 요소에 입력하는 텍스트 내용이 아래 print라는 문서 객체에 그대로 표시됨

코딩웍스 온라인 클래스

코딩웍스 온라인 클래스

```
<script>
  // 문서 객체 선택
  var userInput = document.testForm.testInput;
  var print = document.getElementById('print');
```

// 키보드 이벤트

```
userInput.onkeyup = function(){
  var inputValue = this.value;
  print.textContent = inputValue;
}
```

```
</script>
```

→ this.value란 해당 input 요소가 this가 되고 사용자가 입력하는 값이 value가 됩니다. 곧, this.value란 사용자가 입력하는 입력 값을 말합니다.

1) 문서 객체 속성 - Form(스몰미션 - 주민번호 앞자리 체크)

Small Mission)

사용자에게 주민번호 뒷번호 첫번째 숫자를 넣었을 때 1,3 이면 라디오 버튼에 남자에 체크, 2,4 이면 라디오 버튼에 여자 체크(단, 1~4 사이의 숫자가 아닌 경우 '1~4사이의 숫자를 입력하세요'라고 경고창 출력

여기에 숫자를 입력하면 입력된 숫자에 따라 경고창 또는 라디오 버튼의 남자 여자에 알맞게 체크

901011

-

xxxxxxx

☐ 남자
☐ 여자

901011

-

1

xxxxxxx

☒ 남자
☐ 여자

스몰 미션 제작 노트

- ① 사용자에게 주민번호 뒷자리 첫번째 숫자 키보드로 입력 받기
문서 객체 선택은 3개(주민번호 뒷번호 첫째 자리, 남자 라디오 버튼, 여자 라디오 버튼)
- ② 처음 if 조건문으로 1~4 사이의 숫자인지 아닌지 체크 아니라면 정상 숫자 입력하라고 경고창 출력
- ③ 중첩 if 조건문에서 1 또는 3이면 남자, 2 또는 4이면 여자를 구분하고 라디오 버튼에 checked 속성으로 true, false를 반환

```
<form name="residentCheck" class="residentCheck">
  <input type="text" name="firstNum" maxlength="6" value="901011" disabled>
  -
  <input type="text" name="genderNum" maxlength="1">
  xxxxxxx
  <input type="radio" name="gender" id="male"><label for="male">남자</label>
  <input type="radio" name="gender" id="female"><label for="female">여자</label>
</form>
```

HTML

```
<script>
  // 문서 객체 선택
  var genderNum = document.residentCheck.genderNum;
  var male = document.getElementById('male');
  var female = document.getElementById('female');

  // keyup 이벤트
  genderNum.onkeyup = function(){
    var inputValue = this.value;
    // 1~4 사이 숫자인지 체크
    if(inputValue >= 1 && inputValue <= 4) {
      // 1,3인지 2,4인지 체크해서 라디오 버튼에 true, false 반환
      if(inputValue == 1 || inputValue == 3){
        male.checked = true;
        female.checked = false;
      }
      else {
        male.checked = false;
        female.checked = true;
      }
    }
    else if(inputValue != '') {
      alert('1~4사이의 숫자를 입력하세요');
    }
  }
</script>
```

상호작용이 필요한 3개의 객체

사용자가 입력하는 주민번호 뒷번호 첫 글자의 값

조건문이 참이면 male이라는 라디오버튼에 checked 속성을 참(true)을 리턴 하므로 남자 라디오 버튼이 체크됩니다. false면 반대로 체크가 풀립니다.

사용자가 숫자를 넣고 지울 경우 경고창을 출력합니다. 이것 방지하기 위해서 사용자 입력 값이 공백이 아니라면.. 이라고 해줍니다.

1) 문서 객체 속성 - Form(스몰미션 - 입력 정보 출력)

Small Mission)

정보를 입력하고 버튼을 클릭하면 입력된 정보를 문서에 출력하기

```
/* Custom CSS */
.frame {
  width: 300px;
  box-shadow: 0 0 20px rgba(0, 0, 0, 0.3);
  border-radius: 10px;
  padding: 25px;
}
#myForm .field {
  margin: 10px 0; display: flex;
}
#myForm .field label { flex: 1; }
#myForm .field input {
  flex: 5;
  border: 1px solid #ddd;
  border-radius: 3px;
  padding: 5px;
}
#myForm button {
  width: 100%; padding: 5px;
  border: none;
  background-color: #b2bec3;
  color: #fff; font-size: 1.1em;
  cursor: pointer;
}
```

```
<div class="frame">
  <h2>정보입력 후 입력된 정보출력</h2>
  <form id="myForm">
    <div class="field">
      <label for="userName">이름</label>
      <input type="text" id="userName" name="userName">
    </div>
    <div class="field">
      <label for="userAddr">주소</label>
      <input type="text" id="userAddr" name="userAddr">
    </div>
    <button>기본 정보 입력</button>
  </form>
</div>
```

<input type="submit" value="기본 정보 입력">

document.createElement(nodename)

HTML element를 동적으로 만드는 메서드
노드네임 == HTML 엘리먼트 이름(태그명)

정보입력 후 입력된 정보출력

이름

주소

기본 정보 입력



정보입력 후 입력된 정보출력

이름

주소

기본 정보 입력

홍길동,경기도 남양주시 별내동

```
// 문서 객체 선택
var myForm = document.getElementById('myForm');

// 함수 선언
myForm.onsubmit = function(){
  var userInfo = document.createElement('p');
  userInfo.textContent = this.userName.value + ',' + this.userAddr.value;
  myForm.appendChild(userInfo);

  // 입력 후 입력된 내용 비우기
  this.userName.value = '';
  this.userAddr.value = '';

  return false;
}
```

폼 전송하기 전에 작업 수행하게 하는 이벤트

myForm 요소의 맨 마지막에 userInfo를 자식 요소로 추가

JS

1) 문서 객체 속성 - Form(폼 유효성 관련 속성과 메서드)

`checkValidity()` → 입력 요소에 유효한 데이터가 있으면 true, 아니면 false

`setCustomValidity()` → 입력 요소의 `validationMessage` 속성을 설정
유효성이 거짓일 경우 브라우저가 표시할 메시지를 포함

Small Mission)

1~100 사이의 숫자를 입력하지 않으면 유효성 메시지 띄우기

```
<form name="numberForm">
  <p>숫자를 1~100 사이로 입력하세요.</p>
  <input type="number" name="numberBox" id="numberBox" min="1" max="100" required>
  <button type="button" id="btnCheck">확인</button>
</form>
<div id="printMsg"></div>
```

HTML

숫자를 1~100 사이로 입력하세요.

1~100 사이의 숫자를 입력하고 확인을 누르면 유효한 값이라고 출력, 아니라면 유효한 값이 아니라고 출력하기

숫자를 1~100 사이로 입력하세요.

이 입력란을 작성하세요.

아무것도 입력하지 않고 확인을 누르면 나오는 브라우저가 기본적으로 출력하는 메시지로 브라우저에 따라 다릅니다.

폼 요소를 다루는 개발자에게는 매우 중요한 파트이지만 퍼블리셔에게는 유효성 검사의 개념 정도면 일단 오케이~!!

숫자를 1~100 사이로 입력하세요.

값은 100 이하여야 합니다.

숫자를 1~100 사이로 입력하세요.

유효한 데이터입니다.

```
<script>
  // 문서 객체 선택
  var numberBox = document.getElementById('numberBox');
  var btnCheck = document.getElementById('btnCheck');
  var printMsg = document.getElementById('printMsg');

  // 클릭 이벤트
  btnCheck.addEventListener('click', function(){
    // 만약 numberBox 가 유효한 데이터라면..
    if(numberBox.checkValidity()) {
      printMsg.innerHTML = '유효한 데이터입니다.';
    }
    else {
      printMsg.innerHTML = numberBox.validationMessage;
    }
  });
</script>
```

2) 문서 객체(클래스 속성) - className 속성 ↪ className 속성은 문서 객체의 클래스명을 설정(원래 없던 클래스명 추가 변경)

{ 클래스명 설정 문법 }

객체선택.className = '클래스명';

{ 클래스명 반환 문법 }

var 변수명 = 객체선택.className;

Small Mission)

h2와 h1 태그에 .active 클래스네임 추가하기

```
<h1 class="heading">This is H1</h1>
<h2>This is H2</h2>
```

HTML

```
.heading {
  font-style: italic;
  text-decoration: underline;
  color: crimson;
}
.active {
  background-color: #000;
  color: #fff;
  border-radius: 10px;
  padding: 5px;
}
```

CSS

This is H1

This is H2

This is H1

This is H2

▲ h2에 .active 클래스네임 추가

This is H1

This is H2

▲ h1에 .active 클래스네임 추가되면서
기존의 .heading이라는 클래스네임을 덮
어써버린 경우

```
<h1 class="heading active">This is H1</h1>
```

▲ 위에 처럼 .heading이라는 클래스네임을 유지하면서 .active라는
클래스네임을 추가해야 하기 때문에 className 속성을 사용합니다.

```
<script>
  // 문서 객체 선택
  var h1Tag = document.querySelector('h1');
  var h2Tag = document.querySelector('h2');

  // 1) h2 클래스네임 추가
  h2Tag.className = 'active';

  // 2) h1 클래스네임 변경
  h1Tag.className = 'active';
</script>
```

2) 문서 객체(클래스 속성) - classList.length 속성 클래스 개수를 확인

classList.add() 메서드, classList.remove() 메서드

 클래스 이름을 추가

 클래스 이름을 제거

Small Mission)

h1와 h2 태그가 가지고 있는 클래스 네임 개수를 classList.length 속성으로 확인하기

```
<h1 class="style01 style02">This is H1</h1>
<h2 class="style01 style02 style03">This is H2</h2>
```

```
<script>
// 문서 객체 선택
var h1Tag = document.querySelector('h1');
var h2Tag = document.querySelector('h2');

// 클래스 네임 개수를 classList.length 속성으로 확인하기
console.log(h1Tag.classList.length); // 2
console.log(h2Tag.classList.length); // 3
</script>
```

Small Mission)

h1 태그에 마우스 올리면 .active 클래스 추가하고 마우스 나가면 .active 클래스 제거하기

```
<h1>This is H1</h1>
.active {
  background-color: crimson;
  color: #fff;
  border-radius: 10px;
  padding: 5px 15px;
}
```

This is H1



```
<script>
// 문서 객체 선택
var h1Tag = document.querySelector('h1');

h1Tag.addEventListener('mouseenter', function(){
  this.classList.add('active');
});
h1Tag.addEventListener('mouseleave', function(){
  this.classList.remove('active');
});
</script>
```

문서객체.classList.add('클래스명', '클래스명')

문서객체.classList.remove('클래스명', '클래스명')

2) 문서 객체(클래스 속성) - `classList.toggle()` 메서드 클래스를 추가하고 제거하는 기능을 번갈아서 실행

`classList.contains()` 메서드 문서 객체가 해당 클래스네임을 가지고 있는지 유무에 따라 있으면 true, 없으면 false를 반환

Small Mission)

h1 태그 클릭하면 .active 클래스 추가하고 다시 클릭하면 .active 클래스 제거하기

```
<h1>This is H1</h1>
```

HTML

```
.active {
  background-color: crimson;
  color: #fff;
  border-radius: 10px;
  padding: 5px 15px;
}
```

CSS

```
<script>
// 문서 객체 선택
var h1Tag = document.querySelector('h1');

// toggle() 메서드로 클래스 추가 제거하기
h1Tag.addEventListener('click', function(){
  this.classList.toggle('active');
});
</script>
```

This is H1



문서객체.classList.toggle('클래스명')

▲ toggle() 메서드에서는 한 개의 클래스명만 사용 가능

Small Mission)

h1, h2 태그를 클릭하면 .active 클래스 있는지 여부를 contains() 메서드로 확인

```
<h1 class="active">This is H1</h1>
<h2>This is H2</h2>
```

HTML

```
<script>
// 문서 객체 선택
var h1Tag = document.querySelector('h1');
var h2Tag = document.querySelector('h2');

// toggle() 메서드로 클래스 추가 제거하기
function classChk() {
  var hasClass = this.classList.contains('active');
  alert(hasClass);
}

h1Tag.addEventListener('click', classChk);
h2Tag.addEventListener('click', classChk);
</script>
```

true
false

2) 문서 객체(클래스 속성) - 스몰미션

Small Mission)

CSS 클래스 속성을 변경하는 버튼 4개를 만들고 눌러서 결과 확인하기

(클래스 추가 버튼, 클래스 제거 버튼, 클래스 토글 버튼, 클래스 유무 체크 버튼)

스몰 미션 제작 노트

- ① html 작성하고 .active 클래스에 간단한 CSS 디자인 속성 넣기
- ② 문서 객체 선언(버튼 4개 + 클래스가 변화될 h1 요소 1개)
- ③ 클래스 추가 버튼, 클래스 제거 버튼, 클래스 토글 버튼은 h1 요소를 변화시키고 클래스 유무 체크 버튼은 경고창에 true, false를 출력

```
<h1 class="active">Good to see you! This is CodingWorks</h1>
<div class="btns">
  <button class="btnAdd">.active 클래스 추가</button>
  <button class="btnRemove">.active 클래스 제거</button>
  <button class="btnToggle">.active 클래스 토글</button>
  <button class="btnContains">.active 클래스 유무 체크</button>
</div>
```

HTML

```
h1 {
  text-align: center;
  padding: 5px 15px;
}
.btns {
  text-align: center;
}
.btns button {
  cursor: pointer;
}
.active {
  background-color: crimson;
  color: #fff;
  font-weight: normal;
}
```

CSS

Good to see you! This is CodingWorks



Good to see you! This is CodingWorks

.active 클래스 추가 .active 클래스 제거 .active 클래스 토글 .active 클래스 유무 체크

.active 클래스 추가 .active 클래스 제거 .active 클래스 토글 .active 클래스 유무 체크

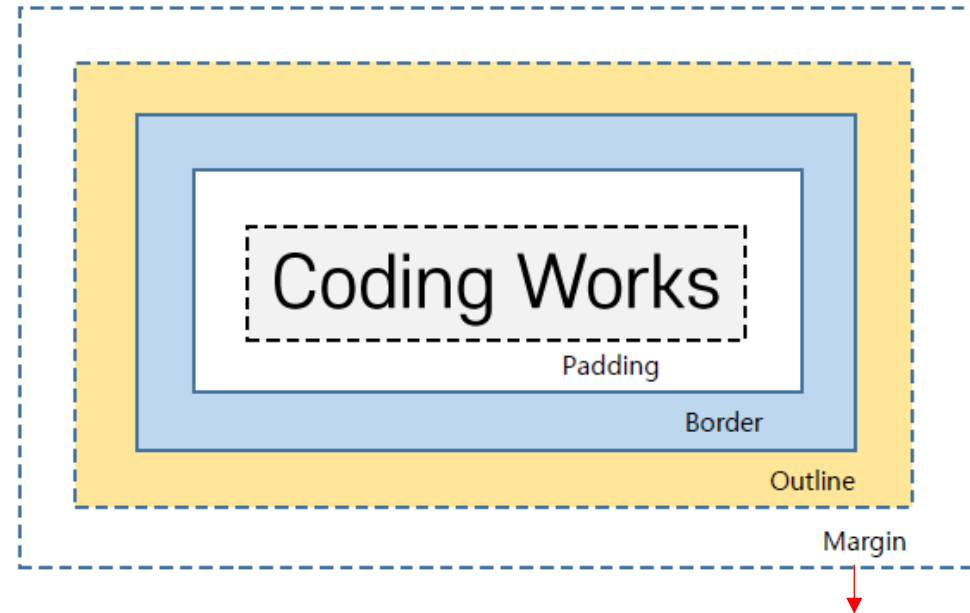
```
<script>
  // 문서 객체 선택
  var h1Tag = document.querySelector('h1');
  var btnAdd = document.querySelector('.btnAdd');
  var btnRemove = document.querySelector('.btnRemove');
  var btnToggle = document.querySelector('.btnToggle');
  var btnContains = document.querySelector('.btnContains');

  /*
   클래스 추가 버튼, 클래스 제거 버튼, 클래스 토글 버튼은
   h1 요소를 변화시키고 클래스 유무 체크 버튼은
   경고창에 true, false를 출력
  */
  btnAdd.addEventListener('click', function(){
    h1Tag.classList.add('active');
  });
  btnRemove.addEventListener('click', function(){
    h1Tag.classList.remove('active');
  });
  btnToggle.addEventListener('click', function(){
    h1Tag.classList.toggle('active');
  });

  btnContains.onclick = function(){
    var hasCheck = h1Tag.classList.contains('active');
    alert(hasCheck);
  }
</script>
```

3) 문서 객체(영역 속성) - 문서 객체의 영역 속성 정리

속성 명	기능 설명
clientWidth	패딩을 포함한 너비
clientHeight	패딩을 포함한 높이
offsetWidth	보더, 패딩, 스크롤바를 포함한 너비
offsetHeight	보더, 패딩, 스크롤바를 포함한 높이
scrollWidth	overflow로 볼 수 없는 높이까지 패딩을 포함한 너비
scrollHeight	overflow로 볼 수 없는 높이까지 패딩을 포함한 높이



마진은 요소의 높이나 너비에 포함되지 않습니다.

3) 문서 객체(영역 속성) - clientWidth / clientHeight 패딩을 포함한 너비와 높이

Small Mission)

.frame의 패딩을 포함한 너비와 높이를 알아내기

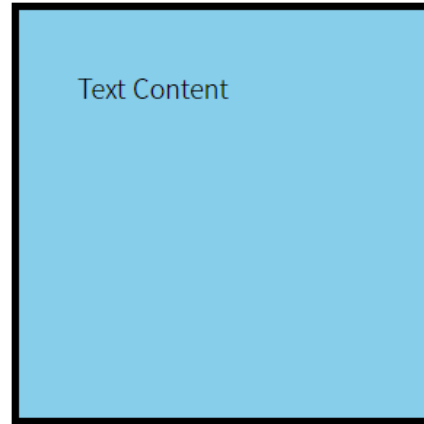
```
<div class="frame">Text Content</div>
<div class="print">
  <p>
    clientWidth : <span id="result01"></span>
  </p>
  <p>
    clientHeight : <span id="result02"></span>
  </p>
</div>
```

HTML

```
.frame {
  width: 200px;
  height: 200px;
  border: 5px solid #000;
  padding: 40px;
  background-color: skyblue;
}
```

CSS

▲ 현재 .frame의 너비와 높이는 200 + 80(좌우 패딩) = 280px



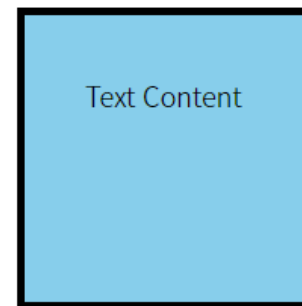
clientWidth : 280

clientHeight : 280

box-sizing: border-box 인경우는 패딩과 보더를 최초의 너비와 높이에 포함하므로 clientWidth, clientHeight 도 달라집니다.

```
<script>
  // 문서 객체 선택
  var frame = document.querySelector('.frame');
  var result01 = document.getElementById('result01');
  var result02 = document.getElementById('result02');

  // 문서 객체의 너비와 높이 알아내기
  result01.textContent = frame.clientWidth;
  result02.textContent = frame.clientHeight;
</script>
```



clientWidth : 190

clientHeight : 190

box-sizing: border-box를 주면 200픽셀에서 좌우 보더 합 10픽셀을 제외하고 190픽셀을 출력

3) 문서 객체(영역 속성) - offsetWidth / offsetHeight ↪ 보더, 패딩, 스크롤바를 포함한 너비와 높이

Small Mission)

.frame의 패딩과 보더를 포함한 너비와 높이를 알아내기

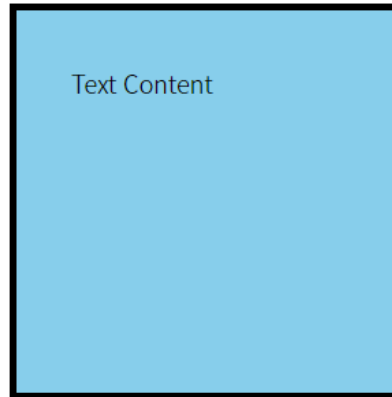
```
<div class="frame">Text Content</div>
<div class="print">
  <p>
    offsetWidth : <span id="result01"></span>
  </p>
  <p>
    offsetHeight : <span id="result02"></span>
  </p>
</div>
```

HTML

```
.frame {
  width: 200px;
  height: 200px;
  border: 5px solid #000;
  padding: 40px;
  background-color: skyblue;
}
```

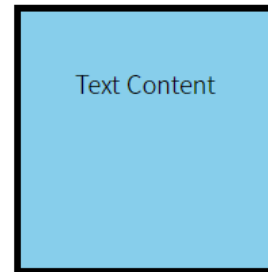
CSS

▲ 현재 .frame의 너비와 높이는 200 + 80(좌우 패딩) = 280px



offsetWidth : 290

offsetHeight : 290



offsetWidth : 200

offsetHeight : 200

→ box-sizing: border-box를 주면 200픽셀에서 좌우 보더 합 10픽셀까지 포함하고 200픽셀을 출력

```
<script>
  // 문서 객체 선택
  var frame = document.querySelector('.frame');
  var result01 = document.getElementById('result01');
  var result02 = document.getElementById('result02');

  // 문서 객체의 너비와 높이 알아내기
  result01.textContent = frame.offsetWidth;
  result02.textContent = frame.offsetHeight;
</script>
```

4) 문서 객체(dataset 속성)

보통 data 로 시작하는 사용자 정의 속성으로 부릅니다. 사용자 정의 속성은 data- 로 시작합니다. 규약은 아니지만 그로벌한 약속 정도로 이해하시면 됩니다. data-속성성격="값" 이런 구조로 만드시면 됩니다.

ex) 숫자를 표현하는 사용자 정의 속성이라면 data-count, 주문 관련이라면 data-order 이렇게 네이밍하세요.

Small Mission)

h1 태그를 클릭할 때 h1 태그가 가지고 있는 사용자 정의 속성의 값을 콘솔창에 출력하기

```
<div class="frame">
  <h1 data-num="에스프레소(더블)">주문명 : 에스프레소(더블)</h1>
  <h1 data-num="아이스 아메리카노">주문명 : 아이스 아메리카노</h1>
  <h1 data-num="카라멜 마키야토">주문명 : 카라멜 마키야토</h1>
</div>
```

HTML

주문명 : 에스프레소(더블) → 에스프레소(더블)이(가) 주문되었습니다.

주문명 : 아이스 아메리카노 → 아이스 아메리카노이(가) 주문되었습니다.

주문명 : 카라멜 마키야토 → 카라멜 마키야토이(가) 주문되었습니다.

```
<script>
// 문서 객체 선택
var h1Tag = document.querySelectorAll('.frame h1')

// h1을 각각 클릭해서 data 속성의 값을 출력하기
for(var i = 0; i < h1Tag.length; i++) {
  h1Tag[i].addEventListener('click', function(){
    var dataValue = this.getAttribute('data-num');
    console.log(dataValue + '이(가) 주문되었습니다.');
```

현재 예제는 크게 중요한 건 아니고 태그 요소에 data로 시작하는 사용자 정의 속성을 만들고 그 사용자 정의 속성이 가지고 있는 값을 getAttribute로 가져와서 활용할 수 있다는 개념이 중요합니다.

4) 문서 객체(dataset 속성) ↪ dataset 속성은 사용자 정의 속성에 쉽게 접근을 위해 만들어진 속성

```
{ 문서객체.dataset.속성명; }
```

```
{ 문서객체.dataset[속성명]; }
```

▼ data- 다음에 영문이 1개인 경우

```
<a href="#" data-count="5">
```

문서객체.dataset.count;

▼ data- 다음에 영문이 2개인 경우

```
<a href="#" data-order-count="5">
```

문서객체.dataset.orderCount;

Small Mission) 커피 주문 프로그램 만들기(버튼을 클릭하면 data 속성이 가지고 있는 메뉴와 가격을 아래에 출력하기

```
<div class="frame">
  <h3>커피주문 : 메뉴를 선택하면 가격이 나옵니다.</h3>
  <div class="btns-order">
    <button data-coffee-menu="에스프레소" data-price="3700">에스프레소(싱글)</button>
    <button data-coffee-menu="에스프레소(더블)" data-price="4200">에스프레소(더블)</button>
    <button data-coffee-menu="고구마 라떼" data-price="5800">고구마 라떼</button>
    <button data-coffee-menu="카페 라떼" data-price="6300">카페 라떼</button>
  </div>
  <p><span id="orderName">선택 메뉴 없음</span> : <span id="orderPrice">0</span>원(VAT 포함)</p>
</div>
```

커피주문 : 메뉴를 선택하면 가격이 나옵니다.

에스프레소(싱글) 에스프레소(더블) 고구마 라떼 카페 라떼

선택 메뉴 없음 : 0원(VAT 포함)

커피주문 : 메뉴를 선택하면 가격이 나옵니다.

에스프레소(싱글) 에스프레소(더블) 고구마 라떼 카페 라떼

에스프레소 : 3700원(VAT 포함)

```
// 문서 객체 선택(버튼, 메뉴출력, 가격출력)
var btnOrder = document.querySelectorAll('.btns-order button');
var orderName = document.getElementById('orderName');
var orderPrice = document.getElementById('orderPrice');

// 클릭 이벤트 반복문으로 data 속성의 값을 출력하기
for(i = 0; i < btnOrder.length; i++) {
  btnOrder[i].addEventListener('click', function(){
    // 눌린 버튼이 가지고 있는 data 속성의 값을 새로운 변수에 담기
    var selectMenu = this.dataset.coffeeMenu;
    var selectPrice = this.dataset.price;
    // 새로운 변수를 출력하기
    orderName.textContent = selectMenu;
    orderPrice.textContent = selectPrice;
  });
}
```

4) 문서 객체(dataset 속성) : 스몰미션 - 티켓 구매 버튼 자동으로 활성화 하기(1)

Small Mission)

data 속성을 이용해서 현재 날짜와 시간에 맞춰 자동으로 티켓 구매 버튼 활성화 하기(HTML+CSS)

```
<div class="events-items">
  <h1>Events Reservation</h1>
  <div class="events-item disabled" data-dday="2024-09-01">
    <h2>piano performance</h2>
    <p>
      Lorem ipsum dolor sit amet consectetur adipisicing elit. Enim molestias quo aut iusto
      aspernatur dignissimos nulla? Quod ullam culpa impedit dolorem, voluptatibus perspici
      odit exercitationem.
    </p>
    <a href="#none">buy tickets</a>
  </div>
  <div class="events-item disabled" data-dday="2024-09-29">
    <h2>musical performance</h2>
    <p>
      Lorem, ipsum dolor sit amet consectetur adipisicing elit. Ea voluptatum harum fuga delectus, labore
      facere ex autem voluptatem commodi explicabo quisquam dicta praesentium, quaerat, debitis exercitationem
      corporis veritatis nulla quod?
    </p>
    <a href="#none">buy tickets</a>
  </div>
</div>
```

HTML

```
/* Custom CSS */
.events-items { width: 80%; }
.events-items h1 {
  font-size: 3em; text-align: center;
}
.events-item {
  background-color: rgba(0, 0, 0, 0.3);
  padding: 30px 20px; border-radius: 15px; margin: 20px 0;
}
.events-item h2 {
  margin: 0; text-transform: capitalize; font-weight: 500;
}
.events-item p {
  margin-bottom: 30px;
}
```

```
.events-item a {
  background-color: #fff;
  color: #333;
  border-radius: 10px;
  padding: 10px 20px;
  font-weight: 500;
  text-transform: capitalize;
}
.events-item.disabled a {
  opacity: 0.3;
  pointer-events: none;
  cursor: none;
}
```

CSS

4) 문서 객체(dataset 속성) : 스몰미션 - 티켓 구매 버튼 자동으로 활성화 하기(2)

Small Mission)

data 속성을 이용해서 현재 날짜와 시간에 맞춰 자동으로 티켓 구매 버튼 활성화 하기(JavaScript)

```
// 문서객체 선택
let eventsItem = document.querySelectorAll('.events-item');
// forEach 반복문
eventsItem.forEach(el => {
  let dday = new Date(el.dataset.dday)
  let today = new Date()
  if(dday < today) {
    el.classList.remove('disabled')
  }
});
```

→ .events-item이 2개 이상이므로 반복문을 사용해서 객체 선택

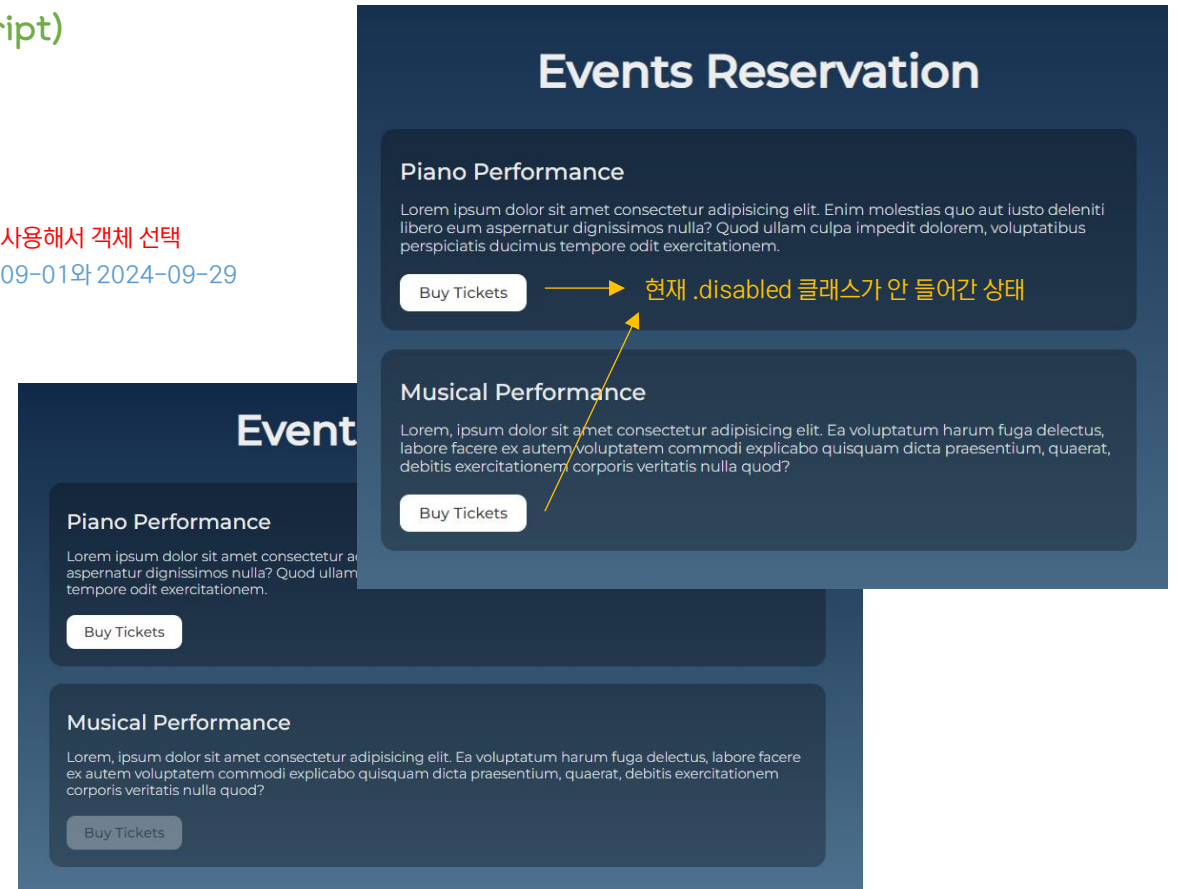
HTML에 있는 data-dday 속성에 들어간 값들. 곧, 2024-09-01과 2024-09-29

시스템의 현재 날짜. 곧, 제작 당일 날짜 2024-09-27 이 들어감

첫번째 .events-item의 data-dday 속성의 값이 2024-09-01이고, 현재 날짜의 값이 2024-09-27입니다. 곧, 2024-09-01 < 2024-09-27 이므로 .disabled 클래스가 사라짐

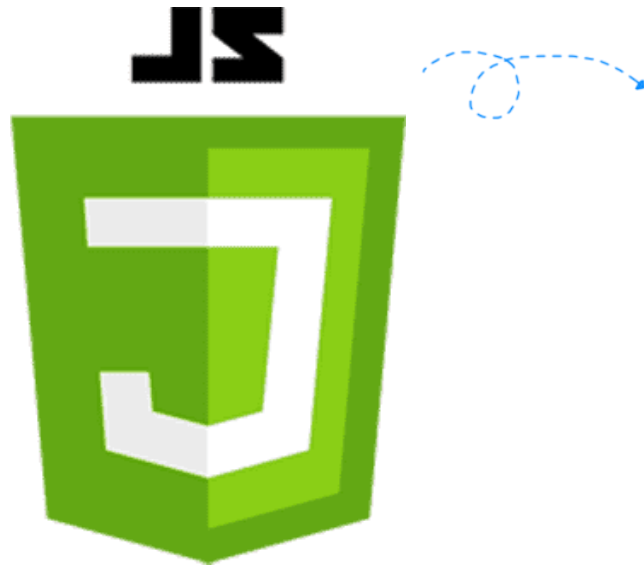
▼ 아래 콘솔창에서 보면 화면을 제작하고 있는 현재 날짜가 2024년 9월 27일이므로 첫번째 .events-item은 날짜가 지났으므로 .disabled 클래스가 삭제(remove)되면서 버튼을 누를 수 있고, 두번째는 아직 날짜가 오지 않았으므로 disabled 클래스가 유지된 상태로 버튼 클릭 안되는 상태

```
<div class="events-items">
  <h1>Events Reservation</h1>
  <div class="events-item" data-dday="2024-09-01">...</div>
  <div class="events-item disabled" data-dday="2024-09-29">...</div>
</div>
```



▲ data-dday의 값에 설정된 날짜가 되면 .disabled 클래스가 사라지면서 클릭 가능

JAVASCRIPT 핵심이론 커리큘럼



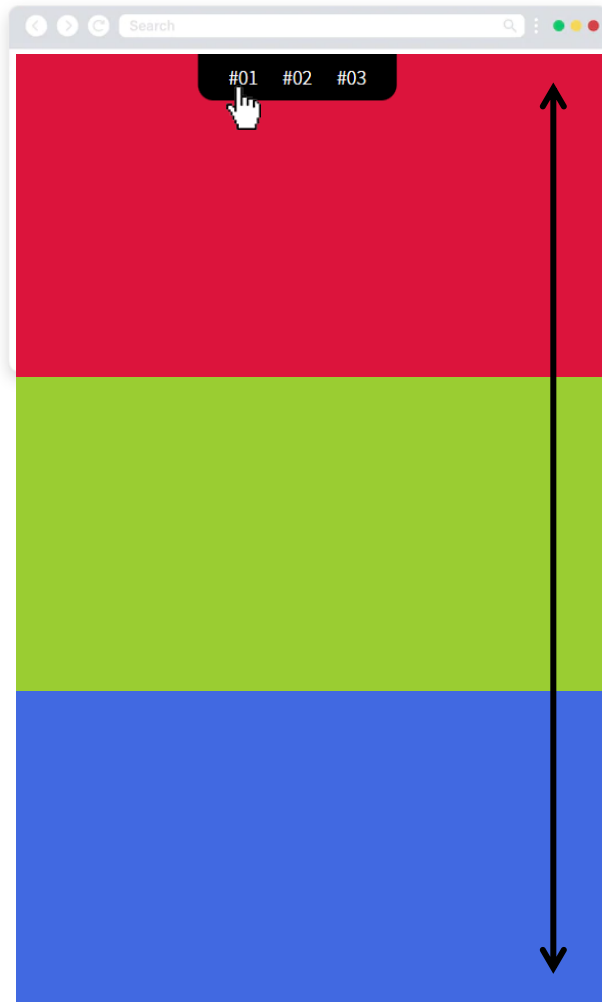
[웹 퍼블리셔를 위한 자바스크립트] 13. 자바스크립트 실전 예제

1. 마우스 클릭해서 부드럽게 스크롤되며 이동시키기
2. 디지털 시계 만들기(방법 1)
3. 디지털 시계 만들기(방법 2)
4. 모달(Modal) 띄우기(1)
5. 모달(Modal) 띄우기(2)
6. 스톱워치(StopWatch) 만들기(분:초:밀리초)
7. 스톱워치(StopWatch) 만들기(시:분:초)
8. 가위 바위 보(Rock Paper Scissors) 게임 만들기
9. 계산기 만들기 with Grid(인라인 자바스크립트 방식)
10. 계산기 만들기 with Grid(함수 매개변수 호출 방식)



자바스크립트와 제이쿼리로 뭔가를 만들 때 제작자에 따라 제작하는 방법과 코드는 매우 다양한 합니다. 실전 예제의 제작 방식과 코드가 정답이라고 생각하지 마시고 다른 제작자의 코드도 참고하시기 바랍니다.

자바스크립트 실전 예제(1-1) - 마우스 클릭해서 부드럽게 스크롤되며 이동시키기



← 객체1.offsetTop

← 객체2.offsetTop

← 객체3.offsetTop

Small Mission)

네비게이션을 클릭하면 해당 div의 상단을 부드럽게 스크롤해서 위치시키기

스몰 미션 제작 노트

- ① 문서 객체 선언
- ② .content의 상단의 값을 반환해서 변수에 담기
- ③ 클릭 이벤트로 스크롤 시키기

window.scroll() 메서드

브라우저 스크롤을 특정 위치로 이동시키는 메서드

offsetTop 속성

요소의 최상단의 거리 위치를 반환하는 속성

```
<link rel="stylesheet" href="style.css">
```

```
<div class="container">
  <div class="menu">
    <span>#01</span>
    <span>#02</span>
    <span>#03</span>
  </div>
  <section>
    <div class="content">
      <h1>This is #01</h1>
    </div>
    <div class="content">
      <h1>This is #02</h1>
    </div>
    <div class="content">
      <h1>This is #03</h1>
    </div>
  </section>
</div>
```

HTML

```
<script src="custom.js"></script>
```


자바스크립트 실전 예제(1-2) - 마우스 클릭해서 부드럽게 스크롤되며 이동시키기

```
/* Custom CSS */
.menu {
  position: fixed;
  top: 0;
  left: 50%;
  transform: translateX(-50%);
  border-radius: 0 0 15px 15px;
  padding: 10px 20px;
  background-color: #000;
}
.menu span {
  cursor: pointer; margin: 10px; color: #fff;
}
.content {
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
  color: #fff;
}
.content:nth-child(1) {
  background-color: crimson;
}
.content:nth-child(2) {
  background-color: yellowgreen;
}
.content:nth-child(3) {
  background-color: royalblue;
}
```

CSS

```
// 문서 객체 선언
var navTrigger = document.querySelectorAll('.menu span');
var scrollContent = document.querySelectorAll('section .content');

// .content의 상단의 값을 반환해서 변수에 담기
var showContent1 = scrollContent[0].offsetTop;
var showContent2 = scrollContent[1].offsetTop;
var showContent3 = scrollContent[2].offsetTop;

// 클릭 이벤트로 스크롤 시키기
navTrigger[0].onclick = function(){
  window.scroll({
    top: showContent1,
    behavior: 'smooth'
  });
}
navTrigger[1].onclick = function(){
  window.scroll({
    top: showContent2,
    behavior: 'smooth'
  });
}
navTrigger[2].onclick = function(){
  window.scroll({
    top: showContent3,
    behavior: 'smooth'
  });
}
```

• 클릭이 되는 문서 객체와 스크롤이 되는 문서 객체 선택 후 변수에 담기

• querySelectorAll()로 선택되었기 때문에 배열로 선택됨.

→ 브라우저 스크롤 시키는 메서드

• window.scroll(키, 키값)
• window.scroll(상단위치, 부드럽게 이동)

JS

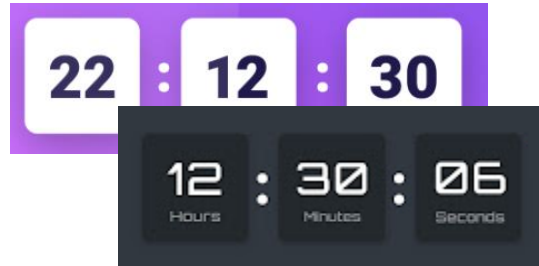
자바스크립트 실전 예제(2-1) - 디지털 시계 만들기(방법 1)

Small Mission)

디지털 시계를 만들고 1초마다 갱신되게 만들기

스몰 미션 제작 노트

- ① 문서 객체 선언
- ② 시계 함수 만들기(Date 객체 선언, Date 객체의 메서드 변수에 담기)
- ③ setInterval(함수, 경과시간) 호출



02:37:40

String 객체 메서드 padStart()로 숫자 앞에 0 붙이기

- String().padStart(자릿수, 특정문자열)
 - String(1).padStart(2, '0'); // 01
- 1~9이라는 숫자를 2자리로 만들고 앞에 0을 붙입니다.

```
<link rel="stylesheet" href="style.css">

<div class="digital-clock">
  <div class="time hour">00</div>
  <div class="divider">:</div>
  <div class="time min">00</div>
  <div class="divider">:</div>
  <div class="time sec">00</div>
</div>

<script src="custom.js"></script>
```

▲ 현재 html 구조로 만들면 html 구조는 다소 복잡하고 CSS가 복잡해지지만 독특한 디자인을 개별적으로 할 수 있습니다.

```
/* Custom CSS */
body {
  font-weight: 500;
  height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
}

.digital-clock {
  display: flex;
  font-size: 4em;
}
```

```
// 문서 객체 선언
var hour = document.querySelector('.hour');
var min = document.querySelector('.min');
var sec = document.querySelector('.sec');

// 시계 함수 만들기
function clock() {
  // Date 객체 선언
  var now = new Date();
  // Date 객체의 메서드 변수에 담기
  hour.innerHTML = String(now.getHours()).padStart(2, '0');
  min.innerHTML = String(now.getMinutes()).padStart(2, '0');
  sec.innerHTML = String(now.getSeconds()).padStart(2, '0');
}

// setInterval(함수, 경과시간);
setInterval(clock, 1000);
```

JS

자바스크립트 실전 예제(2-2) - 디지털 시계 만들기(방법 2)

3:17:2

Small Mission)

디지털 시계를 만들고 1초마다 갱신되게 만들기

```
<link rel="stylesheet" href="style.css">
```

```
<h1 class="" id="clock"></h1>
```

```
<script src="custom.js"></script>
```

HTML

```
/* Custom CSS */
body {
  font-weight: 500;
  height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
}
#clock {
  font-size: 4em;
}
```

CSS

```
// 문서 객체 선언
var clock = document.querySelector('#clock');
// 시계 함수 만들기
function getClock() {
  var date = new Date();
  clock.innerHTML = `${date.getHours()}:${date.getMinutes()}:${date.getSeconds()}`;
}
// 1초 마다 시계 함수 반복하기
setInterval(getClock, 1000);
```

JS

03:26:02

date.getHours()를 String 객체의 메서드 padStart()를 사용해서 숫자가 0~9 사이가 나오면 앞에 0을 붙입니다.

```
// 문서 객체 선언
var clock = document.querySelector('#clock');
// 시계 함수 만들기
function getClock() {
  var date = new Date();
  var hours = String(date.getHours()).padStart(2, '0');
  var min = String(date.getMinutes()).padStart(2, '0');
  var sec = String(date.getSeconds()).padStart(2, '0');
  clock.innerHTML = `${hours}:${min}:${sec}`;
}
// 1초 마다 시계 함수 반복하기
setInterval(getClock, 1000);
```

JS

자바스크립트 실전 예제(3-1) - 모달(Modal) 띄우기(1)

Small Mission) 모달창 띄우기(심플하게 띄우기)

```
<button id="openModal">Open Modal</button>
```

HTML

```
<div id="modal">
```

```
<h2>This is Modal</h2>
```

```
<p>
```

```
  Lorem ipsum dolor sit, amet consectetur adi  
  adipisci ea sunt ex consectetur? Laudantium  
  at impedit iure, consequatur quaerat verita  
  dolores cumque! Voluptates, non dolorum.
```

```
</p>
```

```
<button id="closeModal">Close Modal</button>
```

```
</div>
```

```
<script src="custom.js"></script>
```

```
/* Custom CSS */
```

```
#modal {
```

```
  position: absolute;
```

```
  top: 50%;
```

```
  left: 50%;
```

```
  transform: translate(-50%, -50%);
```

```
  width: 500px;
```

```
  padding: 20px;
```

```
  background-color: #f5f6fa;
```

```
  border: 1px solid #ddd;
```

```
  box-shadow: rgba(149, 157, 165, 0.2) 0px 8px 24px;
```

```
  border-radius: 10px;
```

```
  display: none;
```

```
}
```

```
#modal button {
```

```
  background-color: #3498db;
```

```
  color: #fff;
```

```
  border: none;
```

```
  padding: 5px 10px;
```

```
  cursor: pointer;
```

```
}
```

CSS

```
// 문서 객체 선택
```

```
var openModal = document.getElementById('openModal');
```

```
var closeModal = document.getElementById('closeModal');
```

```
var modal = document.getElementById('modal');
```

```
// 클릭 이벤트
```

```
openModal.addEventListener('click', function(){  
  modal.style.display = 'block';  
});
```

```
closeModal.addEventListener('click', function(){  
  modal.style.display = 'none';  
});
```

JS

Open Modal

This is Modal

Lorem ipsum dolor sit, amet consectetur adipisicing elit. Suscipit adipisci ea sunt ex consectetur? Laudantium, repudiandae, blanditiis at impedit iure, consequatur quaerat veritatis quos reiciendis dolores cumque! Voluptates, non dolorum.

Close Modal

자바스크립트 실전 예제(3-2) - 모달(Modal) 띄우기(2)

Small Mission) 모달창 띄우기(CSS 클래스 추가 제거로 애니메이션 모달 띄우기)

```
<button id="openModal">Open Modal</button>

<div id="modal">
  <h2>This is Modal</h2>
  <p>
    Lorem ipsum dolor sit, amet consectetur ad
    adipisci ea sunt ex consectetur? Laudantium
    at impedit iure, consequatur quaerat verit
    dolores cumque! Voluptates, non dolorum.
  </p>
  <button id="closeModal">Close Modal</button>
</div>

<script src="custom.js"></script>
```

```
/* Custom CSS */
#modal {
  position: absolute;
  top: 50%;
  left: 50%;          추가 변경 CSS
  transform: translate(-50%, -70%);
  transition: 0.5s;
  opacity: 0;
  visibility: hidden;
  width: 500px;
  padding: 20px;
  background-color: #f5f6fa;
  border: 1px solid #ddd;
  box-shadow: rgba(149, 157, 165, 0.2) 0px 8px 24px;
  border-radius: 10px;
}

/* active class add & remove */
#modal.active {
  transform: translate(-50%, -50%);
  opacity: 1;
  visibility: visible;
}
```

- opacity: 0만 주면 안보일 때도 모달창이 선택되기 때문에 visibility: hidden으로 선택도 되지 않게 해줘야 합니다.
- transform: translate(-50%, -70%)을 X좌표, Y좌표를 조정하면서 상하로 좌우로 모달이 나타나는 애니메이션을 바꿀 수 있습니다.

```
// 문서 객체 선택
var openModal = document.getElementById('openModal');
var closeModal = document.getElementById('closeModal');
var modal = document.getElementById('modal');

// 클릭 이벤트
openModal.addEventListener('click', function(){
  modal.classList.add('active'); 변경된 JS
});
closeModal.addEventListener('click', function(){
  modal.classList.remove('active'); 변경된 JS
});
```

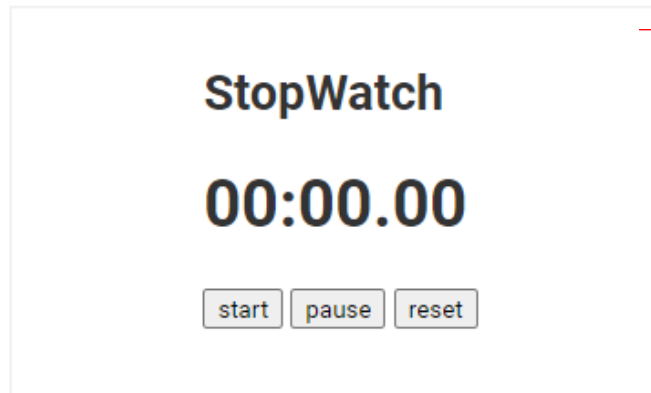


자바스크립트 실전 예제(04-1) - 스톱워치(StopWatch) 만들기(분:초:밀리초)

Small Mission) 자바스크립트 [분:초:밀리초] 스톱워치(StopWatch) 만들기(1) - HTML & CSS

스몰 미션 제작 순서

HTML 구조 만들기 → 자바스크립트 → CSS 배치 및 디자인 하기



→ 자바스크립트 코딩이 목적이니 왼쪽처럼 CSS 디자인이 없는 상태로 기능을 모두 완성합니다.

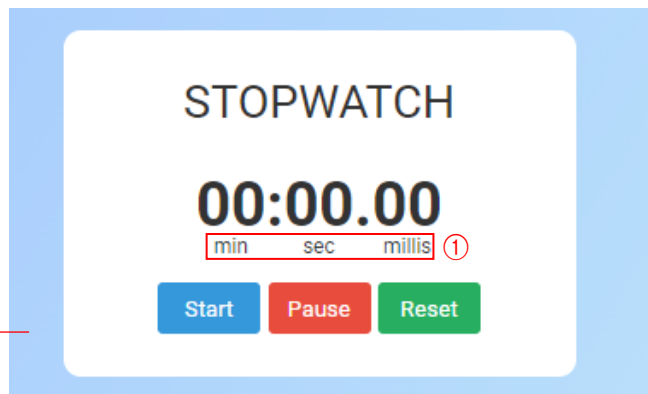
① 깔끔한 HTML 구조를 위해 CSS 가상클래스 :before :after로 만듦

```
.stopwatch .display span {  
  position: relative;  
}  
.stopwatch .display span:after {  
  content: attr(data-alt);  
  position: absolute;  
  top: 90%;  
  left: 50%;  
  transform: translateX(-50%);  
  font-weight: normal;  
  font-size: 12px;  
}
```

CSS

```
<div class="stopwatch">  
  <h2>StopWatch</h2>  
  <h1 class="display">  
    <span id="showMinutes" data-alt="min">00</span>:<span id="showSeconds"  
      data-alt="sec">00</span>.<span id="showMilli" data-alt="millis">00</span>  
  </h1>  
  <div class="buttons">  
    <button id="btnStart">start</button>  
    <button id="btnPause">pause</button>  
    <button id="btnReset">reset</button>  
  </div>  
</div>
```

HTML



CSS 디자인은 완성본
CSS 파일을 참고하세요.

자바스크립트 실전 예제(04-2) - 스톱워치(StopWatch) 만들기(분:초:밀리초)

Small Mission) 자바스크립트 [분:초:밀리초] 스톱워치(StopWatch) 만들기(2) - JavaScript

```
// 문서 객체 선택 후 변수에 담기
var minutes = 0;
var seconds = 0;
var milli = 0;
var stopWatch; // setInterval 함수를 담기 위한 변수

var showMinutes = document.getElementById('showMinutes');
var showSeconds = document.getElementById('showSeconds');
var showMilli = document.getElementById('showMilli');
var btnStart = document.getElementById('btnStart');
var btnPause = document.getElementById('btnPause');
var btnReset = document.getElementById('btnReset');
```

자바스크립트에서 사용되는 문서 객체를 선택하고 변수에 담기

```
// 시작 버튼
btnStart.onclick = function() {
    // start 버튼을 두번 이상 누르는 경우 id가 겹치는 경우 방지를 위해
    clearInterval(stopWatch);
    stopWatch = setInterval(operateStopwatch, 10);
}

// 일시정지 버튼
btnPause.onclick = function() {
    clearInterval(stopWatch);
}
```

스톱워치 기능을 하는 함수 operateTimer라는 함수를 지속적으로 반복하는 setInterval을 10밀리초 단위로 계속 반복

스톱워치 기능을 정지

```
// 초기화 버튼
btnReset.onclick = function() {
    clearInterval(stopWatch);
    minutes = 0;
    seconds = 0;
    milli = 0;
    showMinutes.textContent = '00';
    showSeconds.textContent = '00';
    showMilli.textContent = '00';
}
```

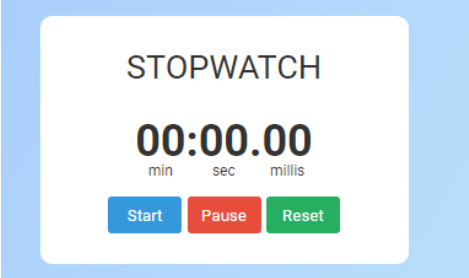
스톱워치 기능을 정지하면서
실제 기능하는 분,초,밀리초를 0으로 다시 초기화
브라우저에 보여지는 분,초,밀리초를 00으로 초기화

```
// 10ms 마다 시간에 대한 숫자 증가
function operateStopwatch() {
    milli++;
    showMilli.textContent = milli > 9 ? milli : '0' + milli;
    if(milli > 99) {
        seconds++;
        showSeconds.textContent = seconds > 9 ? seconds : '0' + seconds;
        milli = 0;
        showMilli.textContent = '00';
    }
    if(seconds > 59) {
        minutes++;
        showMinutes.textContent = minutes > 9 ? minutes : '0' + minutes;
        seconds = 0;
        showSeconds.textContent = '00';
    }
}
```

밀리초를 계속 증가시키기

밀리초를 증가시켜서 0~99 곧 100이 되면 1초를 증가시키고 조건문

초를 증가시켜서 0~59 곧 60이 되면 1분을 증가시키는 조건문

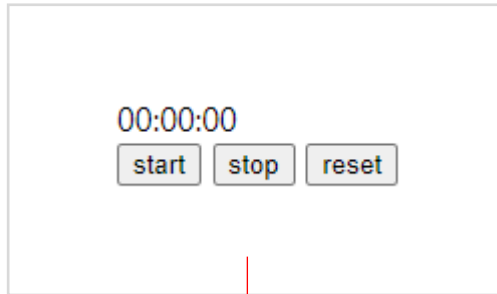


자바스크립트 실전 예제(05-1) - 스톱워치(StopWatch) 만들기(시:분:초)

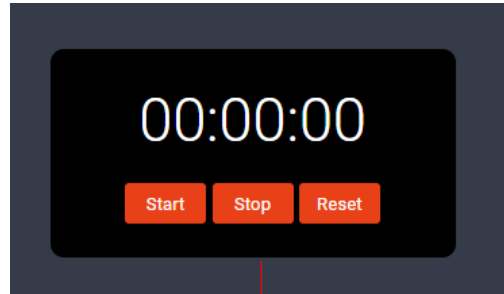
Small Mission) 자바스크립트 [시:분:초] 스톱워치(StopWatch) 만들기(1) - HTML & CSS

스몰 미션 제작 순서

HTML 구조 만들기 → 자바스크립트 → CSS 배치 및 디자인 하기



자바스크립트 코딩이 목적이니 왼쪽처럼 CSS 디자인이 없는 상태로 기능을 모두 완성합니다.



CSS 디자인은 완성본 CSS 파일을 참고하세요.

```
<div class="stopwatch">
  <div class="time">00:00:00</div>
  <div class="controls">
    <button id="startBtn">start</button>
    <button id="stopBtn">stop</button>
    <button id="resetBtn">reset</button>
  </div>
</div>
```

HTML

```
/* Custom CSS */
.stopwatch {
  background-color: #000;
  border-radius: 10px;
  width: 300px;
  padding: 25px;
  text-align: center;
}
.stopwatch h2 {
  text-transform: uppercase;
  font-weight: normal;
  color: #fff;
  margin-bottom: 10px;
}
.time {
  color: #fff;
  font-size: 2.5em;
  margin-bottom: 20px;
}
```

```
body {
  font-weight: 300;
  font-size: 18px;
  margin: 0;
  color: #333;
  height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
  background-color: #353b48;
}
```

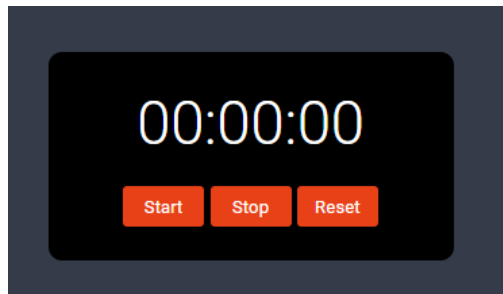
```
.stopwatch .controls button {
  padding: 7px 10px;
  width: 60px;
  border: none;
  cursor: pointer;
  text-transform: capitalize;
  color: #fff;
  border-radius: 3px;
  background-color: #e84118;
  text-transform: capitalize;
}
.stopwatch .controls button:hover {
  opacity: 0.7
}
```

CSS

자바스크립트 실전 예제(05-2) - 스톱워치(StopWatch) 만들기(시:분:초)

Small Mission) 자바스크립트 [시:분:초] 스톱워치(StopWatch) 만들기(2) - JavaScript

```
// 문서 객체 선택 후 변수에 담기
var timeDisplay = document.querySelector('.stopwatch .time');
var startBtn = document.getElementById('startBtn');
var stopBtn = document.getElementById('stopBtn');
var resetBtn = document.getElementById('resetBtn');
```



```
// 초기화
var seconds = 0;
var interval = null;
```

초를 초기화(시작,정지,초기화 함수 만들기 전에 seconds에 숫자를 넣어 미리보기
interval 변수를 null로 하지 않아도
var interval; 이라고 변수만 선언 해도 됩니다.

```
// 이벤트 주기
startBtn.addEventListener('click', start)
stopBtn.addEventListener('click', stop)
resetBtn.addEventListener('click', reset)
```

시작,종지,초기화 함수를 불러오는 이벤트

```
// 업데이트 타이머
function timer() {
    seconds++;
```

3600을 수식으로 표현 가능(1시간 => 3600초)

// 시분초 포맷 설정

```
var hrs = Math.floor(seconds / (60 * 60));
```

```
var mins = Math.floor((seconds - (hrs * 3600)) / 60);
```

```
var secs = seconds % 60;
```

// 0~9사이 숫자인 경우 앞에 0 붙이기

```
if(secs < 10) secs = '0' + secs;
```

```
if(mins < 10) mins = '0' + mins;
```

```
if(hrs < 10) hrs = '0' + hrs;
```

한자리 자릿수일 경우를 대비해서
0~9사이 숫자 앞에 0 붙이기

```
timeDisplay.textContent = `${hrs}:${mins}:${secs}`;
```

```
}
```

```
/* 아래 시작, 정지, 초기화 버튼을 만들기 전에  
timer() 함수를 실행해서 체크 */
```

```
// timer(); → 아래 함수를 만들기 전에 정상적으로  
timer() 함수가 작동하는지 체크
```

```
function start() {
    if(interval) return;
    interval = setInterval(timer, 1000);
}
```

```
function stop() {
    clearInterval(interval);
    interval = null;
}
```

```
function reset() {
    stop();
    seconds = 0;
    timeDisplay.textContent = '00:00:00'
}
```


자바스크립트 실전 예제(06-1) - 가위 바위 보(Rock Paper Scissors) 게임 만들기

Small Mission) 가위 바위 보(Rock Paper Scissors) 게임 만들기(1) - HTML & CSS

스몰 미션 제작 순서

HTML 구조 만들기 → 자바스크립트 → CSS 배치 및 디자인 하기

Rock Paper Scissors Game

Play :

Computer :

Let's Do This!

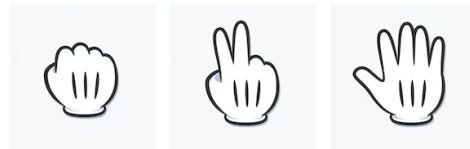


Rock Paper Scissors Game

Player : Scissors

Computer : Paper

You Win!



HTML 완성하고 JS로 기능을 모두 완성 후에
CSS 디자인하시기 바랍니다.

```
<div class="game">
  <h1 class="heading">Rock Paper Scissor
  <div class="gameText">
    <h2 id="playerText">Play : </h2>
    <h2 id="computerText">Computer : </h2>
    <h2 id="resultText">Let's do this!</h2>
  </div>
  <div class="choiceBtns">
    <button data-value="rock"></button>
    <button data-value="paper"></button>
    <button data-value="scissors"></button>
  </div>
</div>
```

```
/* Custom CSS */
.game {
  width: 600px;
}
.game .heading {
  text-align: center; background-color: #000;
  color: #fff; padding: 5px;
}
.gameText h2 {
  text-transform: capitalize;
}
.choiceBtns button {
  border: none;
  background-color: transparent;
  cursor: pointer;
  transition: 0.35s;
}
.choiceBtns button:active {
  transform: scale(0.5);
}
#resultText {
  color: crimson;
}
```

CSS

HTML

자바스크립트 실전 예제(06-2) - 가위 바위 보(Rock Paper Scissors) 게임 만들기

Small Mission)

가위 바위 보(Rock Paper Scissors) 게임 만들기(2) - JavaScript

```
// 문서 객체 선택 후 변수 담기 & 초기 변수 생성
var playerText = document.querySelector('#playerText');
var computerText = document.querySelector('#computerText');
var resultText = document.querySelector('#resultText');
var choiceBtns = document.querySelectorAll('.choiceBtns button');

var player;
var computer;
var result;

// 클릭 이벤트
choiceBtns.forEach(choiceBtns => choiceBtns.addEventListener('click', () => {
  // player = choiceBtns.textContent;
  player = choiceBtns.getAttribute('data-value');
  computerTurn();
  playerText.textContent = `Player : ${player}`;
  computerText.textContent = `Computer : ${computer}`;
  resultText.textContent = checkWinner();
}));
```

HTML에 사용자 정의 속성 data-value의 값을 불러와서 player 변수에 넣기

```
<button data-value="rock"></button>
```

```
// 컴퓨터의 숫자에 rock, paper, scissors 넣기
function computerTurn() {
  var randomNum = Math.floor(Math.random() * 3) + 1;
  switch(randomNum) {
    case 1:
      computer = 'rock';
      break;
    case 2:
      computer = 'paper';
      break;
    case 3:
      computer = 'scissors';
      break;
  }
}
```

1을 더하는 건 아래 switch 문에서 편의상 1,2,3으로 만들기 위해

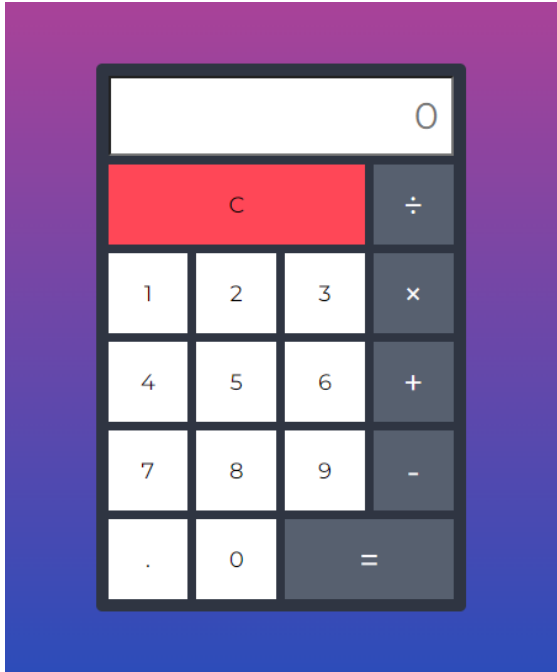
```
> Math.random()
< 0.4026675640276476
> Math.random()
< 0.33294493502967804
> Math.random() * 3
< 0.8209027482197147
```

```
> Math.floor(Math.random() * 3)
< 0
> Math.floor(Math.random() * 3)
< 1
> Math.floor(Math.random() * 3)
< 2
```

```
// 컴퓨터 숫자와 매칭된 텍스트와 플레이어 텍스트 비교하기
function checkWinner() {
  if(player == computer) {
    return 'Draw';
  }
  else if(computer == 'rock') {
    return (player == 'paper') ? 'You Win!' : 'You Lose!';
  }
  else if(computer == 'paper') {
    return (player == 'scissors') ? 'You Win!' : 'You Lose!';
  }
  else if(computer == 'scissors') {
    return (player == 'rock') ? 'You Win!' : 'You Lose!';
  }
}
```

자바스크립트 실전 예제(07) - 계산기 만들기 with Grid(인라인 자바스크립트 방식)

Small Mission) 계산기 만들기 - HTML & CSS & JavaScript



▲ HTML 완성하고 JS로 기능을 모두 완성 후에 CSS 디자인하시기 바랍니다.

```
<form name="calc" class="calculator">
  <input type="text" class="display" name="output" placeholder="0" readonly>
  <input type="button" class="btn-clear" value="C" onclick="document.calc.output.value=''>
  <input type="button" class="btn-operator" value="÷" onclick="document.calc.output.value+='/ '>

  <input type="button" class="number" value="1" onclick="document.calc.output.value+='1'>
  <input type="button" class="number" value="2" onclick="document.calc.output.value+='2'>
  <input type="button" class="number" value="3" onclick="document.calc.output.value+='3'>
  <input type="button" class="btn-operator" value="×" onclick="document.calc.output.value+='* '>

  <input type="button" class="number" value="4" onclick="document.calc.output.value+='4'>
  <input type="button" class="number" value="5" onclick="document.calc.output.value+='5'>
  <input type="button" class="number" value="6" onclick="document.calc.output.value+='6'>
  <input type="button" class="btn-operator" value="+" onclick="document.calc.output.value+='+'>

  <input type="button" class="number" value="7" onclick="document.calc.output.value+='7'>
  <input type="button" class="number" value="8" onclick="document.calc.output.value+='8'>
  <input type="button" class="number" value="9" onclick="document.calc.output.value+='9'>
  <input type="button" class="btn-operator" value="-" onclick="document.calc.output.value+='-'>

  <input type="button" class="dot" value="." onclick="document.calc.output.value+='.'>
  <input type="button" class="number" value="0" onclick="document.calc.output.value+='0'>
  <input type="button" class="btn-operator result" value="=" onclick="document.calc.output.value=eval(document.calc.output.value)">
</form>
```

HTML

처음에 0을 보여줌/편집할 수 없도록 읽기 전용으로

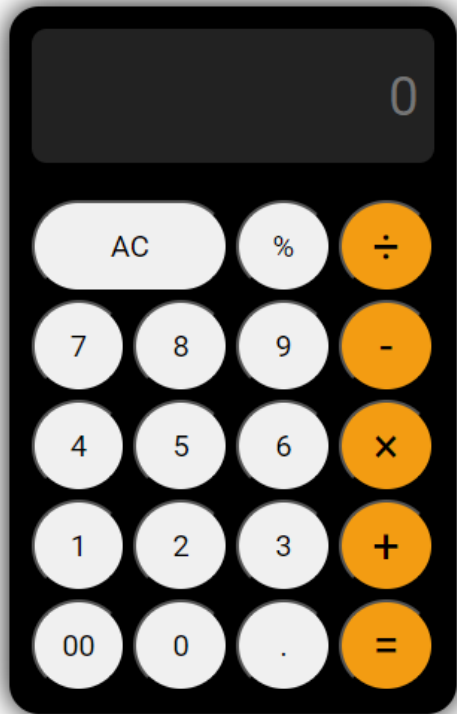
value=""는 다음표 안에 아무것도 없게 초기화
value+="는 +=이라는 대입연산자를
사용해서 숫자를 찍은 것에 더해서 나
누기(/)를 한다 라는 의미

문서객체.폼name.출력name.value ==> document.calc.output.value

- eval() 함수는 문자열로 이루어진 스크립트 코드를 실행하는 함수
- eval(출력하는 곳의 계산된 값을 다시 출력하는 곳에 출력)

자바스크립트 실전 예제(08) - 계산기 만들기 with Grid(함수 매개변수 호출 방식)

Small Mission) 계산기 만들기 - HTML & CSS & JavaScript



▲ HTML 완성하고 JS로 기능을 모두 완성 후에 CSS 디자인하시기 바랍니다.

```
<div class="calculator">
  <input type="text" placeholder="0" id="screen">

  <button class="allclear" onclick="allclear();">AC</button>
  <button class="percent" onclick="calc('%')">%</button>
  <button class="operator" class="first" onclick="calc('/')">÷</button>

  <button class="number" onclick="calc('7')">7</button>
  <button class="number" onclick="calc('8')">8</button>
  <button class="number" onclick="calc('9')">9</button>
  <button class="operator" onclick="calc('-')">-</button>

  <button class="number" onclick="calc('4')">4</button>
  <button class="number" onclick="calc('5')">5</button>
  <button class="number" onclick="calc('6')">6</button>
  <button class="operator" onclick="calc('*')">×</button>

  <button class="number" onclick="calc('1')">1</button>
  <button class="number" onclick="calc('2')">2</button>
  <button class="number" onclick="calc('3')">3</button>
  <button class="operator" onclick="calc('+')">+</button>

  <button class="number" onclick="calc('00')">00</button>
  <button class="number" onclick="calc('0')">0</button>
  <button class="number" onclick="calc('.')">.</button>
  <button class="operator" onclick="result()">=</button>
</div>
```

→ 클릭하면 calc() 함수를 불러오고 해당 함수의 매개 변수에 7을 넣어라 라는 의미

계산 결과를 출력할 문서 객체를 선택하고 변수에 담기

```
let display = document.getElementById('screen');

function allclear() {
  display.value = '';
}

function calc(n) {
  display.value += n;
}

function result() {
  display.value = eval(display.value);
}
```

계산 결과를 출력하는 함수

JS